

Ordo

Практична мова керованих процесів



Як перетворювати людські інструкції на чіткі виконувані
процеси для AI-моделей

Версія 0.12 · M72 · EXECUTION_TRACE

Практичне видання

Передмова



Ця книга народилася з дуже практичної проблеми: AI-моделі вже можуть допомагати з аналізом, документацією, кодом, тестами, перевітками і прийняттям рішень, але складні інструкції для них швидко перетворюються на хаотичні довгі prompt-и.

Спочатку здається, що достатньо написати детальніше. Потім доводиться додавати винятки. Потім — правила. Потім — приклади. Потім — заборони. Потім — перевірки. Потім — окремі уточнення, чого модель робити не повинна. У якийсь момент prompt стає схожим не на інструкцію, а на великий документ, який модель читає щоразу трохи по-різному.

Ordo з'являється як відповідь на цю проблему.

Ідея Ordo проста: якщо ми хочемо, щоб AI-модель виконувала складний процес, то цей процес треба описувати не як суцільний текст, а як керовану програму виконання. У ній мають бути мета, контракт, стан, вузли, шляхи, перевірки, результати, трасування, тести й механізм покращення.

Ця книга не є академічним стандартом і не є фінальною специфікацією. Це практичний вступ: з прикладами, аналогіями, простими формулюваннями і поступовим переходом від звичайного prompt-а до мови керування AI-моделлю.

Головна мета книги — допомогти читачеві зрозуміти не тільки синтаксис Ordo, а спосіб мислення за ним.

Зміст

Передмова	2
Як проблема перетворилася на ідею: чому промптів виявилось недостатньо	6
Для кого ця книга	8
Як читати цю книгу	8
Вступ. Головна ідея Ordo	9
Розділ 1. Чому звичайних prompt-ів недостатньо	11
Розділ 2. Що таке Ordo простими словами	14
Розділ 3. Ordo як мова керування моделлю	23
Розділ 4. Intent, Contract, Context	30
Розділ 5. State: пам'ять процесу	42
Розділ 6. Entry і Node: як модель ставить питання	52
Розділ 6.1. Атомарність кроків у процесах з мовними моделями	59
Розділ 6.2. Мікропромпти як модульна архітектура процесу	78
Розділ 7. Output: що модель має створити	92
Розділ 8. Пишемо першу Ordo Source-програму	100
Розділ 9. Компілюємо Ordo Source в Semantic JSON IR	109
Розділ 10. Чому Semantic JSON IR зараз найкращий	122
Розділ 11. Gate — це не порада, а контрольна точка	132
Розділ 12. Негативні перевірки: ASSERT.NOT → ASSERTION	143
Розділ 13. Status semantics	153
Розділ 14. Чому Ordo-програми потрібно дебажити	163
Розділ 15. Debug mode і діагностичний trace	171
Розділ 16. Тести, fixtures і expected behavior	183
Розділ 17. Regression suite і coverage	193
Розділ 18. Feedback & Improvement Loop	206
Розділ 19. Чому не все треба формалізувати	215
Розділ 20. Як правильно використовувати FREEFORM	224

Зміст - продовження

Розділ 21. FREEFORM.COVERAGE	235
Розділ 22. Ordo Core	241
Розділ 23. Ordo Profiles	249
Розділ 24. Domain Packs	257
Розділ 25. Бібліотеки Ordo: include, import і повторне використання готових рішень	268
Розділ 26. Чому all-in-one більше не головний формат	278
Розділ 27. Documentation Runtime	284
Розділ 28. Rendered artifact validation	291
Розділ 29. Старий playbook як markdown-база знань	300
Розділ 30. Міграція playbook-а в Ordo	306
Розділ 31. Guided intake як Ordo-програма	314
Розділ 32. Package generation	322
Розділ 33. Що лишилось у FREEFORM	329
Розділ 34. Мінімальні вимоги до автора	335
Розділ 35. Як не зламати playbook	342
Розділ 36. Як оцінювати покриття Ordo	348
Розділ 37. Ordo без runtime	355
Розділ 38. Ordo з helper-runner	360
Розділ 39. Ordo як native model language	368
Chapter 40. Process Rail: як Ordo тримає ШІ на маршруті	372
Розділ 41. Contract → Artifact Coverage і Go/No-Go	373
Розділ 42. Зовнішній аудит pre-release пакета	378
Розділ 43. Final handoff: як передавати Ordo на зовнішню перевірку	381
Розділ 44. Прийом зовнішнього feedback після review	382
Розділ 45. Feedback decision planning	384
Chapter 46 — Publication Readiness Decision	385
Розділ 47. Two-tier rendering model	386

Зміст - продовження

Розділ 48. Runtime source of truth і чесність CLI	388
Розділ 49. Runtime Guided Intake Entry Protocol	390
Розділ 50. Runtime Mode start files standard	392
Розділ 51. Package build profiles: dev / runtime / evidence	393
Розділ 52. Runtime Checkpoint Discipline	395
Розділ 53. CLI-enforced Runtime Package	396
Розділ 54. Multi-target Runtime Compilation Layer	398
Розділ 55. Ordo-code Runtime View	400
Розділ 56. Session-trace як proof program	402
Розділ 57. Scenario testing і PathWalk	404
Розділ 58. Restore-session: безпечне повернення назад	409
Розділ 59. PathWalk Benchmark Readiness	411
Розділ 60. Model Benchmark Protocol	417
Розділ 61. Real Module Testcase Generation	420
Розділ 62. Human Review Scenario Cards	425
Розділ 63. Visual Graph Generator як companion utility	427
Розділ 64. Спільний маршрут companion utilities	429
Розділ 73. Як APF почав створювати сам себе	431
Розділ 74. EXECUTION_TRACE — повна історія виконання процесу	438
Розділ 75. Перевірка, replay і практичне використання EXECUTION_TRACE	440
Додаток А. Словник Ordo	442
Додаток В. Каталог ор-кодів	454
Додаток С. Приклади програм	465
Додаток D. Чеклисти	473
Додаток Е. Антипатерни	481
Додаток F. Практичний довідник ор-кодів і YAML-атрибутів	490

Як проблема перетворилася на ідею: чому промптів виявилось недостатньо



Ідея Ordo не з'явилася раптово. Спочатку могло здаватися, що ми просто вирішили створити нову мову для опису процесів. Але якщо чесно подивитися на цей шлях, то видно інше: мова виникла не з бажання вигадати ще один формат, а з послідовності практичних проблем.

На самому початку стояв технологічний зсув. Поява сучасних AI-моделей відкрила дуже привабливу можливість: вбудовувати штучний інтелект у реальні процеси компаній і прискорювати роботу людей. Особливо це стосувалося тих ролей, де багато роботи складається з аналізу, структурування, написання текстів, створення задач, підготовки документації, узгодження вимог і підтримки процесів у синхронному стані.

Одним із таких прикладів стала робота аналітика в компанії, яка розробляє програмне забезпечення. Аналітик має багато складної, але частково повторюваної роботи: зрозуміти запит, уточнити деталі, оформити вимоги, створити задачі, підготувати документацію, перевірити логіку, не загубити відкриті питання, синхронізувати зміни з командою. Це робота, де людина залишається головою процесу, але значну частину рутинного оформлення теоретично можна було б передати AI-моделі.

Початкова ідея виглядала просто:

є проблема: аналітики працюють повільно;
є нова технологія: AI-моделі;
отже, можна спробувати прискорити аналітиків за допомогою AI.

На цьому етапі рішення здавалося очевидним: потрібно написати якісні промпти й інструкції. У цих інструкціях можна описати стандартний процес роботи аналітика: які питання ставити, що перевіряти, як оформлювати результат, коли повертатися до уточнень, як створювати документи або задачі.

Роль AI у такій моделі була зрозумілою: модель виконує рутинну частину роботи, а аналітик контролює сенс, приймає рішення і підтверджує результат. Умовно кажучи, аналітик залишається головою, а AI стає руками.

Але після багатьох спроб стало видно іншу проблему. AI справді може допомагати. Він добре пише, добре переформулює, може структурувати текст, пропонувати варіанти, робити чернетки. Але що складнішим стає процес, то гірше працює звичайна текстова інструкція.

Коли інструкція коротка, модель ще може її втримати. Але коли процес має багато гілок, умов, повернень назад, винятків, проміжних рішень і перевірок, поведінка стає нестабільною. Якщо користувач відволікає модель додатковим питанням, просить повернутися до попереднього кроку або змінює якусь деталь, модель може втратити

місце, де вона перебувала. Вона починає пропускати важливі частини, змішувати різні етапи, забувати відкриті питання або робити вигляд, що процес уже завершено, хоча насправді це не так.

Ми намагалися розв'язати цю проблему довгими й детальнішими інструкціями. Але це не допомогло. Навпаки, інструкції почали розростатися до сотень кілобайт. У них було все більше правил, уточнень, винятків, спеціальних випадків. Проте чим більше ставало тексту, тим важче було моделі стабільно виконувати процес.

У якийсь момент стало зрозуміло: проблема не тільки в якості пром프트. Проблема в самій формі інструкції.

Людська мова дуже гнучка. Це її сила, але для виконуваного процесу це одночасно і слабкість. Одну й ту саму дію можна описати десятками різних способів. Наприклад, просту функцію додавання двох чисел можна пояснити так:

```
Поверни суму двох чисел.  
Склади перший аргумент із другим.  
Результатом має бути число, яке дорівнює сумі переданих параметрів.  
Функція має взяти два значення і повернути їх додавання.
```

Усі ці фрази означають приблизно одне й те саме. Для людини це нормально. Але для складного процесу така свобода створює проблему: текст треба щоразу інтерпретувати. А якщо процес великий, то інтерпретація починає хитатися.

З кодом AI-моделі часто працюють краще саме тому, що код має жорсткішу форму. У коді менше місця для розмитості. Якщо є функція, умова, змінна, виклик або повернення значення, вони мають конкретну структуру. Модель може помилитися і в коді, але сам формат змушує її триматися певних правил.

Тоді й виникла ключова думка: можливо, процеси для AI не треба описувати тільки людською мовою. Можливо, їм потрібна проміжна форма — більш технічна, структурована, стабільна, але все ще зрозуміла людині.

Не повноцінна мова програмування в класичному сенсі. І не просто довгий пром프트. А мова опису процесів, у якій можна явно сказати:

```
ось намір;  
ось контракт;  
ось стан;  
ось питання;  
ось можливі гілки;  
ось умови переходу;  
ось перевірки;  
ось результат;  
ось шаблон;  
ось точка, де потрібне підтвердження людини.
```

Так поступово з'явилася ідея Ordo.

Її корінь був не в бажанні ускладнити роботу, а навпаки — у бажанні зменшити хаос. Ми хотіли залишити людині природне мислення, але дати AI-моделі достатньо чітку структуру, щоб вона не губилася в довгих процесах.

Тому шлях до Ordo можна описати як ланцюг:

технологічний прорив AI
 → бажання прискорити роботу аналітиків
 → спроба описати процеси через промпти
 → виявлення нестабільності довгих текстових інструкцій
 → розуміння, що потрібна більш формальна форма
 → ідея мови керованих процесів

Саме так Ordo почала формуватися як практична мова для AI-процесів. Її завдання — не замінити людину і не перетворити будь-яку роботу на код. Її завдання — зробити складний процес достатньо явним, щоб людина могла його контролювати, а AI-модель могла стабільно його виконувати.

У цьому сенсі Ordo виникла не як абстрактна мова, а як відповідь на дуже конкретну проблему: звичайні промпти добре допомагають у коротких задачах, але для довгих керованих процесів їм бракує форми, пам'яті, меж і дисципліни.

Ordo стала спробою дати цим процесам форму.

Для кого ця книга



Ця книга для людей, які вже використовують AI-моделі не тільки для простих відповідей, а для складних робочих процесів.

Вона буде корисною для аналітиків, які описують playbook-и, правила перевірки, сценарії збору даних і пакети документів. Вона також буде корисною для розробників, які хочуть перетворити AI-інструкції на щось більш стабільне, тестоване і придатне для підтримки.

Окрема аудиторія цієї книги — автори внутрішніх стандартів, QA-фахівці, product managers, технічні письменники і всі, хто стикався з ситуацією, коли модель “майже все зрозуміла”, але пропустила один критичний крок.

Для читання не потрібно бути програмістом. Але буде корисно мати базове розуміння того, що таке інструкція, перевірка, стан процесу, результат і помилка виконання.

Ця книга також для тих, хто хоче в майбутньому публікувати Ordo-програми, бібліотеки, domain packs або playbook-и як відкриту документацію на GitHub.

Як читати цю книгу



Книгу можна читати послідовно, від першого розділу до додатків. Це найкращий спосіб, якщо ви вперше знайомитеся з Ordo.

Перші розділи пояснюють, чому звичайних prompt-ів недостатньо і чому Ordo варто сприймати як мову керування поведінкою моделі. Далі вводяться базові поняття: intent, contract, context, state, node, gate, output. Після цього книга переходить до debug, testing, FREEFORM, Core, Profiles, Domain Packs, бібліотек і великих playbook-ів.

Якщо ви вже працюєте зі складними інструкціями, можна читати вибірково. Наприклад, розділи про gates, debug mode, regression suite або feedback loop можна використовувати як практичні чеклисти для покращення існуючих playbook-ів.

У книзі є маленькі приклади, типові помилки і міні-вправи. Їх не обов'язково виконувати, але саме вони допомагають перетворити Ordo з теорії на практичний інструмент.

Позначки з маскотом Nebu використовуються як візуальні підказки: увага, ідея, місце для роздумів або цікаве спостереження.

Вступ. Головна ідея Ordo



Головна ідея Ordo полягає в тому, що складна інструкція для AI-моделі має бути не просто текстом, а виконуваним контрактом.

Звичайний prompt часто описує бажаний результат. Ordo описує шлях до результату: що потрібно з'ясувати, які рішення можна приймати, де треба зупинитися, які перевірки пройти, що вважати готовим результатом і як пояснити виконання.

Базова формула Ordo виглядає так:

```
intent → contract → context → state → path → steps → gates → result → handoff
```

Ця формула важлива, бо вона змінює сам підхід до роботи з моделлю. Ми більше не просто просимо модель “зробити добре”. Ми визначаємо процес, у межах якого модель має працювати.

Ordo не намагається зробити AI-модель звичайною детермінованою програмою. Модель усе одно залишається семантичним виконавцем: вона розуміє текст, узагальнює, формулює, порівнює, пропонує. Але Ordo задає межі цієї свободи.

У короткій формі:

```
Ordo — це мова керованого виконання інструкцій для AI-моделей.
```

Або ще простіше:

```
Ordo перетворює людський намір на процес, який модель може виконувати, перевіряти, пояснювати і покращувати.
```

М30 уточнення: Ordo як Process Rail

Поточна головна ідея Ordo: це не CLI-first runtime і не жорсткий wizard. Ordo дає III Process Rail — опорну структуру, яка тримає стан, маршрут, gates, повернення назад і результат, але залишає III активним мислячим виконавцем.

III мислить і спілкується.
Process Rail тримає процес.
Semantic JSON IR зберігає машинозчитувану форму рейки.
CLI перевіряє детерміновані частини.

Розділ 1. Чому звичайних prompt-ів недостатньо



Навіщо це потрібно

Prompt-и добре працюють для простих задач. Якщо потрібно перекласти коротке речення, придумати заголовок або швидко пояснити поняття, звичайного prompt-а часто достатньо.

Проблеми починаються тоді, коли задача стає процесом.

Наприклад, потрібно не просто “підготувати документ”, а спочатку зібрати дані, уточнити контракт, обрати сценарій, поставити правильні питання, не переходити до фінального результату без підтвердження, виконати self-check і підготувати handoff. У такій задачі важливий не тільки результат, а й шлях до нього.

Звичайний prompt погано тримає такий шлях.

Просте пояснення

Prompt — це текстове прохання до моделі. Він може бути коротким або довгим, простим або детальним. Але навіть дуже довгий prompt залишається суцільним текстом.

У суцільному тексті модель може:

- пропустити частину правил;
- переплутати приклад і обов’язкову вимогу;
- виконати крок раніше, ніж треба;
- вважати непідтверджене припущення підтвердженим фактом;
- змішати статуси;
- створити фінальний результат до проходження gate;
- не пояснити, чому обрала саме цей шлях.

Це не означає, що модель “погана”. Це означає, що ми намагаємося керувати складним процесом через формат, який для цього не призначений.

Великий prompt не дорівнює точний процес

Поширена помилка — думати, що якщо prompt став довшим, він автоматично став надійнішим.

Насправді довгий prompt часто створює нові проблеми. У ньому з’являються дублікати, винятки, суперечливі формулювання, приклади без чіткої сили правила, старі інструкції, які вже не відповідають новій логіці.

Модель читає це як текст і намагається допомогти. Але вона не завжди бачить, яка частина є контрактом, яка — поясненням, яка — прикладом, а яка — заборонаю.

Ordo пропонує інший підхід: не робити prompt нескінченним, а розкласти інструкцію на керовані частини.

Що саме ламається у звичайних prompt-ах

У складних задачах найчастіше ламаються такі місця:

1. Контракт
Модель починає працювати до того, як зрозуміла, що саме треба зробити.
2. Порядок дій
Модель переходить до фінального результату до проміжних підтверджень.
3. Розгалуження
Модель обирає неправильний сценарій і не пояснює чому.
4. Стан процесу
Модель забуває, що вже підтверджено, а що ще ні.
5. Перевірки
Модель сприймає gate як рекомендацію, а не як блокуючу умову.
6. Заборони
Модель робить те, що “виглядає корисним”, хоча це було заборонено.
7. Пояснюваність
Після помилки незрозуміло, на якому кроці вона виникла.

Саме тому для складних інструкцій потрібна не просто краща фраза, а інша структура керування.

Ordo-зсув мислення

Ordo змінює головне питання.

Замість:

Який prompt дати моделі?

ми питаємо:

Який процес має виконати модель?

Це невелика зміна формулювання, але дуже велика зміна підходу.

Prompt орієнтований на відповідь.

Ordo орієнтована на виконання.

У prompt-і ми часто описуємо бажаний результат. В Ordo ми описуємо процес, у якому цей результат дозволено створити.

Базова формула

У найпростішому вигляді Ordo-процес можна описати так:

intent → contract → state → path → steps → gates → result → handoff

Це означає:

intent	– що користувач хоче;
contract	– що саме модель має право робити;
state	– що вже відомо і підтверджено;
path	– який сценарій виконання обрано;
steps	– які кроки треба пройти;
gates	– які перевірки мають зупинити процес;

```
result    — що можна видати як результат;
handoff   — що потрібно передати людині або наступному процесу.
```

Ця формула і є першою відповіддю на питання, чому звичайних prompt-ів недостатньо.

Типова помилка

Типова помилка — намагатися виправити нестабільну інструкцію ще одним абзацом у кінці prompt-a.

Наприклад:

```
І ще обов'язково не забудь перевірити все перед фіналом.
```

Це краще, ніж нічого. Але для складного процесу цього мало.

В Ordo така вимога має стати gate:

```
GATE.CHECK(method=self_verification) self_validation_passed
ASSERTION(polarity=not) final_output_before_validation
```

Тоді це вже не прохання “не забудь”, а частина процесу.

Міні-вправа

Візьміть будь-який довгий prompt, який ви реально використовували або могли б використовувати.

Спробуйте знайти в ньому:

- головну мету;
- очікуваний результат;
- правила;
- дані, які треба зібрати;
- рішення, які модель може прийняти сама;
- рішення, які має підтвердити людина;
- місця, де модель повинна зупинитися;
- перевірки перед фінальним результатом;
- заборонені дії.

Якщо ці речі розкидані по тексті і не мають чіткої структури, це вже кандидат на Ordo.

Короткий підсумок

Звичайні prompt-и корисні, але вони не є надійним способом керування складними процесами.

Коли задача має багато кроків, розгалужень, перевірок, статусів і заборон, потрібна структура, яка описує не тільки відповідь, а й виконання.

Ordo виникає саме для цього: щоб перетворювати людські інструкції на керований, пояснюваний і перевірюваний процес для AI-моделі.

Розділ 2. Що таке Ordo простими словами

2.1. Ordo — це не ще один “формат prompt-а”

Коли люди вперше чують про Ordo, легко подумати, що це просто ще один спосіб оформлювати prompt-и: трохи YAML, трохи JSON, кілька назв на кшталт intent, gate, state, result.

Але це не зовсім так.

Prompt — це звернення до моделі.

Ordo — це опис процесу, який модель має виконати.

Prompt часто звучить так:

```
Зроби мені результат.
```

Ordo звучить інакше:

```
Ось мета.  
Ось контракт.  
Ось що вже відомо.  
Ось які питання можна ставити.  
Ось у якому порядку рухатися.  
Ось що заборонено.  
Ось де потрібно зупинитися.  
Ось які перевірки пройти.  
Ось який результат можна видати.
```

Тобто Ordo не просто просить модель щось зробити. Ordo задає рамки виконання.

Можна сказати так:

```
Prompt — це прохання.  
Ordo — це інструкційна програма.
```

Або ще простіше:

```
Prompt каже моделі: “зроби”.  
Ordo каже моделі: “виконай процес”.
```

2.2. Побутова аналогія: рецепт і кухня

Уявімо, що ви просите людину приготувати борщ.

Можна сказати:

```
Приготуй борщ.
```

Якщо людина досвідчена, вона, можливо, зробить усе добре. Але результат залежить від її пам'яті, звичок, смаку і припущень.

Можна дати рецепт:

```
1. Підготуй інгредієнти.  
2. Звари бульйон.  
3. Додай буряк.  
4. Додай капусту.  
5. Перевір сіль.  
6. Подай зі сметаною.
```

Це вже краще. Але навіть рецепт може бути нечітким. Наприклад, що робити, якщо немає буряка? Чи можна замінити інгредієнт? Коли страва вважається готовою? Хто підтверджує смак?

Ordo — це не просто рецепт. Це рецепт із правилами виконання:

- якщо інгредієнта немає — не вигадуй заміну, а запитай;
- не переходь до подачі, поки не перевірено готовність;
- якщо страва пересолена — не видавай як finished;
- якщо користувач просив вегетаріанський варіант — не використовуй м'ясний бульйон;
- у кінці покажи, які перевірки пройдені.

Саме так Ordo працює з AI-моделлю.

Вона не просто описує бажаний результат. Вона описує, як модель має рухатися до результату.

2.3. Ordo як карта маршруту

Ще одна проста аналогія — навігатор.

Prompt часто схожий на фразу:

Доїдь до Львова.

Модель може вибрати маршрут сама. Іноді це добре. Але якщо вам важливо не заїжджати на платні дороги, не їхати через певну область, зробити зупинку, перевірити паливо і не завершити маршрут без підтвердження, одного речення вже недостатньо.

Ordo — це маршрут із контрольними точками:

```
start
→ обрати напрям
→ перевірити обмеження
→ пройти checkpoint
→ якщо дорога закрита — перейти на альтернативний шлях
→ не завершувати маршрут без arrival confirmation
```

У термінах Ordo це виглядає так:

```
ENTRY.DEF
→ NODE.DEF
→ STATE.UPDATE
→ GATE.CHECK
→ PATH.SELECT
→ OUTPUT.DEF
→ HANDOFF
```

Тобто Ordo дає моделі не тільки “куди їхати”, а й “як саме рухатися”.

2.4. Головні частини Ordo

У найпростішому вигляді Ordo складається з кількох базових частин.

Intent

Intent — це мета.

Наприклад:

```
intent:
  goal: create_summary
```

Або:

```
intent:
  goal: guide_user_through_history_event_intake
```

Intent відповідає на питання:

Навіщо взагалі запускається цей процес?

Contract

Contract — це домовленість про результат і правила.

Наприклад:

```
contract:
  output:
    format: bullet_list
    max_items: 3
  rules:
    - do_not_invent_facts
    - use_only_input_text
```

Contract відповідає на питання:

Що вважається правильним результатом?
Що заборонено?
Які обмеження треба виконати?

Context

Context — це вхідні дані, документи, джерела або попередні рішення.

Наприклад:

```
context:
  source_text: "$USER_INPUT.text"
  playbook: "history_event_playbook_ordo_v0_10"
```

Context відповідає на питання:

Звідки модель бере дані?

State

State — це поточний стан процесу.

Наприклад:

```
state:
  current_node: N1
  selected_path: null
  confirmed_contracts: []
  open_questions: []
```

State відповідає на питання:

Що вже відомо?
Де ми зараз?
Що ще не підтверджено?

Path

Path — це обраний маршрут виконання.

Наприклад:

```
path:
  selected: Path_1
  reason: "internal source row change"
```

Path відповідає на питання:

Яким сценарієм ідемо?

Step

Step — це конкретна дія.

Наприклад:

```
step:
  id: S1
  do: read_input_text
```

Step відповідає на питання:

Що модель робить зараз?

Gate

Gate — це контрольна точка.

Наприклад:

```
gate:
  id: G1
  method: self_verification
  trust_class: model_judgment
  check: no_unsupported_facts
  on_fail: block_or_repair
```

Gate відповідає на питання:

Чи можна йти далі?

Result

Result — це те, що створюється.

Наприклад:

```
result:
  type: summary
  status: ready
```

Result відповідає на питання:

Що отримує користувач?

Handoff

Handoff — це передача результату.

Наприклад:

```
handoff:
  include:
    - result
    - status
    - gate_report
```

Handoff відповідає на питання:

Що саме віддати користувачу або передати далі?

2.5. Ordo не змушує все робити машинним кодом

Важливо: Ordo не каже, що всю людську інструкцію потрібно перетворити на короткі ор-коди.

Це була б помилка.

У реальних процесах є багато речей, які краще залишити людською мовою:

- доменні пояснення;
- приклади;
- стилістичні вимоги;
- довгі шаблони документів;
- застереження;
- історичні примітки;
- пояснення для аналітика.

Для цього в Ordo є controlled FREEFORM.

Але різниця в тому, що FREEFORM не є хаотичним текстом. Він має роль, прив'язку і причину.

Наприклад:

```
{
  "op": "FREEFORM.ADD",
  "id": "F1",
  "role": "domain_guidance",
  "binding": "Path_5",
  "reason": "domain explanation cannot be safely reduced to core opcodes",
  "text": "For Path 5, the first received state creates only a baseline snapshot by default."
}
```

Тобто Ordo не викидає людський текст. Вона ставить його на правильне місце.

2.6. Ordo має кілька рівнів читання

Ordo можна читати на різних рівнях.

Людський рівень

Це пояснення, яке читає аналітик або автор playbook-а.

Наприклад:

Спочатку потрібно визначити, звідки система має зрозуміти, що історична подія сталася.

Ordo Source

Це структурований опис, який ще зручний для людини.

Наприклад:

```
entry:
  id: START_HISTORY_EVENT_INTAKE
  first_node: N1
```

Semantic JSON IR

Це compiled representation, краще придатне для моделі або runtime.

Наприклад:

```
{
  "op": "ENTRY.DEF",
  "id": "START_HISTORY_EVENT_INTAKE",
  "first_node": "N1"
}
```

Compact IR

Це майбутній компактний машинний формат.

Наприклад:

```
["ENTRY", "START_HISTORY_EVENT_INTAKE", "N1"]
```

Усі ці рівні описують одне й те саме, але для різних читачів.

Людині зручніше читати пояснення й YAML.

Моделі або runtime зручніше працювати з JSON IR.

2.7. Ordo не є заміною моделі

Ordo не виконує роботу замість AI-моделі.

Модель усе ще потрібна для:

- розуміння людського тексту;
- формулювання відповідей;
- заповнення документів;
- узагальнення;
- пояснення;
- роботи з неоднозначностями;
- пропозицій і покращень.

Але Ordo обмежує і направляє модель.

Можна сказати так:

```
Модель генерує зміст.
Ordo керує виконанням.
```

Або:

```
Модель – це двигун.
Ordo – це маршрут, правила руху і контрольні пункти.
```

Без двигуна маршрут нікуди не поїде. Але без маршруту двигун може поїхати не туди.

2.8. Ordo не є жорсткою бюрократією

Ще одне можливе непорозуміння: Ordo може здатися надто формальною.

Навіщо стільки структури? Чому не можна просто написати нормальний prompt?

Можна. Для простих задач — краще так і робити.

Ordo потрібна там, де ціна помилки вища:

- складні документи;
- багато правил;
- багато кроків;
- треба ставити питання;
- треба відрізнати candidate від confirmed;
- треба не вигадувати;
- треба створити пакет артефактів;
- треба перевірити результат;
- треба мати пояснюваний trace.

Ordo — це не про формальність заради формальності.

Ordo — це про керованість.

2.9. Малий приклад: звичайне завдання

Користувач пише:

Підсумуй цей текст українською у максимум 3 пункти.
Не додавай фактів, яких немає в тексті.

Звичайна модель може просто відповісти.

Ordo-модель спочатку бачить структуру:

```
intent:
  goal: summarize_text
  language: uk

contract:
  output:
    format: bullet_list
    max_items: 3
  rules:
    - use_only_input_text
    - no_unsupported_facts

gates:
  - input_text_present
  - max_3_items
  - no_unsupported_facts
  - output_language_uk
```

Для користувача результат може виглядати майже так само, як звичайна відповідь.

Але внутрішньо процес більш контрольований.

2.10. Великий приклад: playbook

Користувач пише:

Хочу створити нову історичну подію.

Звичайна модель може почати пропонувати alias або source.

Ordo-модель бачить:

```
{
  "op": "ENTRY.DEF",
  "id": "START_HISTORY_EVENT_INTAKE",
  "first_node": "N1"
}
```

Потім:

```
{
  "op": "NODE.DEF",
  "id": "N1",
  "question": "Звідки система має зрозуміти, що історична подія сталася?",
  "answers": ["A", "B", "C", "D", "E", "F"]
}
```

І тільки після відповіді користувача вона рухається далі.

Це і є різниця між “прочитати playbook” і “виконувати playbook”.

2.11. Ordo як дисципліна мислення

Навіть якщо у вас немає Ordo-runtime, Ordo вже корисна як спосіб мислення.

Вона змушує автора інструкції запитати себе:

- Яка мета?
- Який контракт?
- Який стан процесу?
- Які дозволені відповіді?
- Які gates?
- Які outputs?
- Що не можна робити?
- Що лишається у FREEFORM?
- Що потрібно підтвердити людиною?

Ці питання самі по собі роблять інструкції кращими.

Навіть якщо модель не підтримує Ordo нативно, Ordo-native playbook значно краще організовує роботу, ніж великий хаотичний all-in-one prompt.

2.12. Короткий підсумок розділу

Ordo — це мова керування виконанням AI-моделі.

Вона не є просто YAML або JSON.

Вона описує:

- мету;
- контракт;
- контекст;
- стан;
- дерево рішень;
- кроки;
- перевірки;
- результати;
- передачу в роботу;
- контрольований freeform.

Найкоротше визначення:

Ordo – це спосіб перетворити людську інструкцію на керований execution contract для AI-моделі.

Prompt просить модель щось зробити.

Ordo задає процес, у межах якого модель має це зробити.

Міні-вправа

Візьміть просту задачу:

Підготуй коротку відповідь клієнту про затримку доставки.

Спробуйте розкласти її в Ordo-стилі:

1. Який intent?
2. Який contract результату?
3. Який context потрібен?
4. Який state треба вести?
5. Які gates потрібні?
6. Який output має бути?
7. Що не можна вгадувати?

Навіть така проста вправа показує головне: Ordo починається не з формату, а з мислення про контрольоване виконання.

Ordo після Process Rail reframing

Найпростіше пояснення: Ordo дозволяє III вести складний процес людською мовою, але не дає йому загубити маршрут. Для цього Ordo описує Process Rail: що вже зібрано, який крок активний, які gates ще не пройдені, що робити після уточнення або зміни попередньої відповіді.

Розділ 3. Ordo як мова керування моделлю

3.1. Не просто формат, а спосіб керування поведінкою

Коли люди вперше бачать Ordo, вони часто питають:

То це YAML? JSON? Набір ор-кодів? Новий формат prompt-ів?

Правильна відповідь: Ordo може використовувати YAML, JSON або компактний IR, але сама Ordo не зводиться до жодного з цих форматів.

YAML — це спосіб записати Ordo Source так, щоб людині було зручно читати.

JSON — це спосіб передати структуровану інструкцію моделі або runtime.

Compact IR — це майбутній спосіб зробити інструкції коротшими й ближчими до машинного виконання.

Але Ordo як ідея — це не формат. Це мова керування поведінкою моделі.

Її задача — не просто красиво записати інструкцію, а зробити так, щоб модель:

- знала, з чого почати;
- розуміла, який крок виконує зараз;
- не перескакувала вперед;
- зберігала state;
- відрізняла підтверджене від припущеного;
- виконувала gates;
- зупинялася, якщо результат ще не можна фіналізувати;
- могла пояснити, чому виконала саме цей шлях.

Тому Ordo ближча не до “формату документа”, а до “мови процесу”.

3.2. Чим Ordo відрізняється від звичайної інструкції

Звичайна інструкція часто виглядає так:

Спочатку визнач тип задачі. Потім постав уточнювальні питання. Якщо даних достатньо, створи документ. Не вигадуй фактів. Якщо є ризики, покажи їх. Перед фіналом перевір результат.

Це зрозумілий текст. Але в ньому не видно, де саме початок, які відповіді дозволені, що вважається достатніми даними, яка перевірка обов’язкова, а яка бажана, і що робити при помилці.

Ordo намагається зробити ці речі явними.

Наприклад:

```
entry:
  id: START
  next_node: N1
nodes:
  - id: N1
    question: "Який тип задачі потрібно виконати?"
    answers:
      A: document_generation
      B: analysis
      C: qa_check
      D: not_sure
```

```

state:
  task_type: unknown
  data_sufficiency: unknown
  confirmed_contracts: []
  assumptions: []

gates:
- id: G1
  rule: task_type_must_be_known
  on_fail: ask_clarifying_question

- id: G2
  rule: no_unsupported_facts
  on_fail: remove_or_mark_unsupported

```

Це вже не просто “порада моделі”. Це структура, за якою модель має рухатися.

3.3. Ordo задає дозволений простір дій

Один із головних принципів Ordo: модель не повинна мати необмежену свободу там, де потрібен контроль.

У звичайному prompt-і модель часто сама вирішує:

- яке питання поставити;
- чи достатньо даних;
- чи можна створювати фінальний результат;
- чи треба питати підтвердження;
- чи можна зробити припущення;
- які правила важливіші.

Іноді це корисно. Але у складних процесах це небезпечно.

Ordo задає дозволений простір дій:

```

зараз дозволено тільки поставити питання N1;
після відповіді А перейти до N2A;
після відповіді В перейти до N2B;
не дозволено генерувати output до проходження G_CONTRACTS_CONFIRMED;
не дозволено називати proposed value confirmed;
не дозволено ховати hard gate у FREEFORM.

```

Це дуже схоже на різницю між розмовою і процесом.

У розмові модель може імпровізувати.

У процесі модель має виконувати правила.

Ordo потрібна саме для другого випадку.

3.4. Модель як виконавець, а не автор процесу

У звичайній роботі з AI модель часто стає не тільки виконавцем, а й автором процесу. Користувач дає приблизну задачу, а модель сама вирішує, як її розкласти.

Це зручно для творчих або відкритих задач.

Але якщо процес уже визначений, модель не має його вигадувати заново.

Наприклад, якщо в playbook-у сказано, що guided intake має починатися з вибору типу входу, модель не повинна починати з alias, QA або source row.

В Ordo це формалізується так:

```
{
  "op": "ENTRY.DEF",
  "id": "START_HISTORY_EVENT_INTAKE",
  "first_node": "N1_ENTRY_SOURCE"
}
```

І далі:

```
{
  "op": "NODE.DEF",
  "id": "N1_ENTRY_SOURCE",
  "allowed_answers": ["A", "B", "C", "D", "E", "F"],
  "next_policy": "follow_answer_transition_only"
}
```

Тобто модель не проектує процес на льоту. Вона виконує вже описаний процес.

Це одна з найважливіших ідей Ordo.

3.5. Людська мова залишається важливою

Може здатися, що Ordo хоче замінити людську мову на технічні структури. Насправді ні.

Людська мова потрібна для опису сенсу.

Наприклад, доменне правило може звучати так:

Не можна вважати приклад готовим контрактом, якщо аналітик або фактичні дані не підтвердили, що саме цей контракт використовується в поточній події.

Це важливий зміст. Його можна частково формалізувати:

```
{
  "op": "GATE.DEF",
  "id": "G_EXAMPLE_NOT_CONTRACT",
  "assert": "EXAMPLES_ARE_NOT_PROMOTED_TO_CONFIRMED_CONTRACTS",
  "on_fail": "BLOCK_AND_REQUEST_EVIDENCE"
}
```

Але пояснення, чому це важливо, може залишатися людською мовою в controlled FREEFORM:

```
{
  "op": "FREEFORM.ADD",
  "id": "F_EXAMPLE_WARNING",
  "role": "domain_explanation",
  "binding": "GATE.G_EXAMPLE_NOT_CONTRACT",
  "text": "Приклади в playbook показують форму, але не стають контрактом для нової події без підтвердження."
}
```

Ordo не намагається стиснути весь сенс у короткі коди. Вона відділяє те, що має бути виконане як правило, від того, що пояснює правило.

3.6. Ordo як проміжний шар між людиною і моделлю

У майбутньому робота може виглядати так:

людський опис
↓

```

Ordo Source
↓
Semantic JSON IR
↓
модель або Ordo-runtime
↓
validated result

```

Людина не обов'язково має писати IR вручну. Вона може описати процес людською мовою. Потім compiler або сама модель допоможе перетворити цей опис на Ordo Source.

А вже Ordo Source може бути скомпільований у Semantic JSON IR, який краще підходить для виконання.

Тобто Ordo — це не тільки “мова для людей” і не тільки “мова для машин”. Це міст між ними.

Людині потрібен зрозумілий опис.

Моделі потрібна структурована інструкція.

Runtime-у потрібні вузли, gates, state і outputs.

Ordo намагається об'єднати ці потреби в одному підході.

3.7. Ordo не обов'язково потребує окремого runtime вже зараз

Ідеальний майбутній варіант — це Ordo-runtime, який сам читає IR, веде state, визначає current node, запускає gates і дає моделі тільки поточну semantic task.

Але Ordo можна використовувати вже зараз без runtime.

Для цього достатньо дати моделі Ordo-native package і сказати:

```

Працюй за execution contract.
Перед кожною відповіддю визнач current node, selected docs, state update і gate.
Не використовуй playbook як суцільний markdown.
Не переходь до наступного кроку без дозволеного transition.

```

Це ще не повна автоматизація, але це вже краще, ніж просто прикріпити великий all-in-one документ.

З часом частину цієї роботи можна винести з моделі в helper-runner:

```

IR читає helper;
state зберігає helper;
next node визначає helper;
модель формує людське питання або документ;
validation перевіряє helper або окремий модуль.

```

У такій архітектурі модель стає менш відповідальною за механіку процесу і більше відповідає за зміст.

3.8. Ordo як “програмування без коду” для AI-поведінки

Ordo можна назвати програмуванням поведінки AI-моделі.

Але це не програмування в сенсі Java, Python або JavaScript. Тут ми не описуємо, як комп'ютер має рахувати числа або викликати функції.

Ми описуємо, як модель має поводитися:

- коли питати;
- коли мовчати;
- коли генерувати;
- коли перевіряти;
- коли зупинятися;
- коли просити підтвердження;
- коли використовувати FREEFORM;
- коли відмовитися від фіналізації.

Це особливий тип програмування.

У класичному коді ми керуємо машиною.

В Ordo ми керуємо моделлю, яка працює з мовою, змістом і невизначеністю.

Тому Ordo не може бути такою ж жорсткою, як звичайна мова програмування. Вона має залишати місце для людського сенсу, доменного тексту і controlled FREEFORM.

Але вона має бути достатньо чіткою, щоб модель не перетворювала складний процес на імпровізацію.

3.9. Які задачі добре підходять для Ordo

Ordo особливо корисна, коли задача має структуру.

Наприклад:

- створення аналітичного пакета;
- guided intake;
- QA-сценарії;
- compliance review;
- юридичний аналіз із gates;
- генерація документів за шаблонами;
- перевірка артефактів;
- міграція playbook-a;
- робота з великими knowledge base;
- агентні процеси з human approval.

Ordo менш потрібна для дуже простих задач:

- переписати одне речення;
- дати коротке пояснення;
- перекласти одну фразу;
- придумати 10 назв;
- відповісти на просте питання.

Хоча навіть просту задачу можна описати в Ordo, не завжди це варто робити.

Головне правило:

Якщо помилка процесу дорожча за вартість формалізації – використовуйте Ordo.

3.10. Типові помилки в розумінні Ordo

Помилка 1. “Ordo — це просто JSON”

Ні. JSON — лише один із форматів представлення. Ordo — це модель виконання.

Помилка 2. “Ordo має формалізувати абсолютно все”

Ні. Якщо формалізація спотворює сенс, використовується controlled FREEFORM.

Помилка 3. “Ordo потрібна тільки для History Event Playbook”

Ні. History Event — це перший великий приклад. Але конструкції Ordo загальні.

Помилка 4. “Ordo замінює людину”

Ні. Ordo навпаки краще фіксує, де потрібне людське підтвердження.

Помилка 5. “Ordo завжди має бути складною”

Ні. Для простої задачі Ordo може бути дуже маленькою. Складність має відповідати ризику задачі.

3.11. Короткий підсумок розділу

Ordo — це мова керування поведінкою AI-моделі.

Вона не зводиться до YAML, JSON або ор-кодів. Вона описує процес виконання:

```
entry → node → state → gate → output → validation → handoff
```

Ordo дозволяє:

- обмежити дозволені дії моделі;
- вести state;
- розділити підтверджені дані й припущення;
- виконувати gates;
- використовувати controlled FREEFORM;
- перевіряти rendered artifacts;
- рухатися від людського опису до model-oriented IR.

Головна зміна мислення така:

```
ми не просто просимо модель відповісти;
ми задаємо процес, у межах якого вона має діяти.
```

Міні-вправа

Візьміть будь-який ваш процес, який ви зараз описуєте prompt-ом.

Наприклад:

```
Підготуй задачу для Jira на основі мого опису.
```

Спробуйте відповісти:

1. Який entry point процесу?
2. Які 2-3 перші node-и потрібні?
3. Які відповіді користувача треба зберігати у state?
4. Який gate має зупинити модель перед фінальним результатом?
5. Який output має бути створений?
6. Що в цьому процесі може лишитися у FREEFORM?

Якщо ви можете відповісти на ці питання, ви вже почали проектувати Ordo-програму.

Керування моделлю не означає заміну моделі

Ordo не намагається прибрати мислення ІІІ. Навпаки, ІІІ залишається активним: питає, пояснює, радить, інтерпретує відповіді й пропонує рішення. Ordo керує тим, щоб ця свобода не зламала процес: Process Rail тримає state, gates, allowed deviations і resume rules.

Розділ 4. Intent, Contract, Context

У попередніх розділах ми говорили про Ordo як про мову керування поведінкою моделі. Тепер починаємо розбирати її базові будівельні блоки.

Перші три поняття, з яких варто починати майже будь-яку Ordo-програму, — це:

```
Intent
Contract
Context
```

Їх легко переплутати, бо у звичайному prompt-і вони часто змішані в одному реченні. Але для Ordo важливо розділяти їх.

Коротко:

```
Intent — що ми хочемо зробити.
Contract — яким має бути правильний результат і які правила не можна порушити.
Context — з чим саме модель працює: вхідні дані, джерела, попередні рішення, документи.
```

Ці три речі створюють стартову рамку. Якщо вони не розділені, модель часто починає виконувати завдання занадто вільно. Якщо вони визначені чітко, модель має менше простору для випадкових припущень.

4.1. Чому ці три поняття важливі

Уявімо простий запит:

```
Підготуй короткий звіт по цьому тексту.
```

На вигляд усе зрозуміло. Але якщо придивитися, тут багато невизначеного.

Що означає “короткий”?

Це 3 речення? 5 пунктів? Одна сторінка?

Що означає “звіт”?

Це підсумок? Аналітична записка? Таблиця? Рекомендації?

Чи можна моделі додавати власні висновки?

Чи має вона посилатися тільки на текст?

Чи потрібно показати відкриті питання?

Чи треба писати українською?

У звичайному prompt-і ці питання часто не проговорені. Модель сама добудовує відсутні частини. Іноді це зручно. Але якщо процес критичний, така свобода небезпечна.

Ordo пропонує почати з трьох уточнень:

```
intent:
goal: create_short_report
```

```
contract:
  output:
    format: bullet_report
    max_items: 5
    language: uk
  rules:
    - use_only_input_text
    - mark_missing_information
context:
  input_text: "$USER_INPUT.text"
```

Тепер завдання вже набагато чіткіше.

Модель не просто “готує звіт”. Вона створює короткий звіт у визначеному форматі, за визначеними правилами, на основі конкретного вхідного тексту.

4.2. Intent: намір або мета

Intent відповідає на питання:

Що ми хочемо досягти?

Це не повний опис результату і не набір правил. Це головна мета.

Приклади intent-ів:

```
intent:
  goal: summarize_text
intent:
  goal: classify_user_request
intent:
  goal: create_jira_task
intent:
  goal: guide_analyst_through_history_event_intake
intent:
  goal: generate_qa_package
```

Добрий intent має бути достатньо коротким. Якщо він перетворюється на великий абзац із правилами, значить частина змісту має переїхати в contract, context, state, path або gates.

Погано:

```
intent:
  goal: "Створи повний пакет, але не вигадуй значення, перевір source row, зроби QA, automation, validation, не забудь approvals і не створи архів якщо щось не підтверджено"
```

Чому погано?

Бо це вже не intent. Це змішання мети, правил, gates, outputs і hard stops.

Краще:

```
intent:
  goal: create_history_event_analysis_package
```

А решту рознести:

```

contract:
  final_result:
    type: compact_analysis_package
    required_outputs: 11

rules:
  - contract_first
  - no_final_archive_before_confirmed_contracts

gates:
  - path_selected
  - mandatory_contracts_confirmed
  - pre_archive_approvals_passed

```

Intent — це компас, а не карта.

Він показує напрям, але не описує всі дороги й перевірки.

4.3. Типові помилки з Intent

Помилка 1. Робити intent занадто широким

Наприклад:

```

intent:
  goal: help_user

```

Це майже нічого не означає. Допомогти можна сотнями способів. Модель мусить сама здогадуватися, що саме робити.

Краще:

```

intent:
  goal: classify_support_request_and_select_output_documents

```

Тут уже зрозуміло, що треба не просто “допомогти”, а класифікувати звернення і вибрати документи.

Помилка 2. Вкладати в intent правила

Наприклад:

```

intent:
  goal: summarize_text_without_adding_facts_and_return_3_bullets_in_ukrainian

```

Це працює для маленьких задач, але погано масштабується.

Краще:

```

intent:
  goal: summarize_text

contract:
  output:
    format: bullet_list
    max_items: 3
    language: uk
  rules:
    - no_unsupported_facts

```


Помилка 3. Плутати intent і output

Наприклад:

```
intent:
  goal: README.md
```

README.md — це output, а не intent.

Краще:

```
intent:
  goal: explain_package_usage

outputs:
  - id: README
    file: README.md
```

Intent каже, навіщо створюється результат. Output каже, що саме буде створено.

4.4. Contract: що вважається правильним результатом

Contract відповідає на питання:

Яким має бути результат, щоб ми могли вважати його правильним?

Це одне з найважливіших понять Ordo.

Звичайний prompt часто описує бажання користувача. Contract описує межі допустимого результату.

Наприклад, prompt:

Підсумуй текст.

Contract:

```
contract:
  output:
    format: bullet_list
    max_items: 3
    language: uk
  rules:
    - use_only_input_text
    - do_not_add_external_facts
    - if_input_is_insufficient_return_insufficient_data_status
```

Тут модель уже знає:

- формат результату;
- максимальну кількість пунктів;
- мову;
- джерело фактів;
- що робити, якщо даних недостатньо.

Contract перетворює “зроби добре” на “зроби ось так і не порушуй ось ці правила”.

4.5. Contract не обов'язково має бути вигаданим з нуля

У реальних проектах contract часто вже існує.

Наприклад:

- існуючий формат JSON;
- API contract;
- database schema;
- шаблон документа;
- приклад реального HistoryEvent;
- існуючий ChangeRecord;
- корпоративний стандарт Jira-задачі;
- вимоги QA runner-a.

У такому випадку модель не має придумувати “кращий” contract. Вона має знайти і використати існуючий.

Це один із головних принципів Ordo:

Спочатку перевірити, чи contract уже існує.
Тільки якщо його немає – запропонувати новий як proposed_contract.

Приклад:

```
contract_resolution:
  search_order:
    - current_playbook_rules
    - factual_examples
    - source_row
    - code_contract
    - analyst_confirmation
  if_found: use_exactly
  if_not_found: propose_contract_with_confirmation_required
```

Це дуже важливо для AI-моделей, бо вони часто вміють створити логічний contract, але не завжди знають, що в конкретному проекті вже існує інший.

4.6. Contract і “confirmed”



Не кожен contract одразу confirmed.

У Ordo варто розрізняти:

missing – контракту немає;
candidate – є можливий варіант;
proposed – модель запропонувала варіант;
requires_confirmation – потрібне підтвердження;
confirmed – контракт підтверджений;
blocked – без контракту не можна рухатися далі.

Наприклад:

```
history_event_values_contract:
  status: proposed
```

```

proposed_shape: flat_keys
proposed_keys:
  - old#value
  - new#value
evidence_source: analogy_with_existing_profile_events
final_package_allowed: false

```

Тут модель може показати пропозицію, але не має права вважати її підтвердженою.

Після підтвердження аналітиком:

```

history_event_values_contract:
  status: confirmed
  shape: flat_keys
  keys:
    - old#value
    - new#value
  evidence_source: analyst_confirmation
  final_package_allowed: true

```

Це здається бюрократією, але саме такі дрібні статуси захищають складні процеси від тихих помилок.

4.7. Contract може містити не тільки output

Часто contract помилково сприймають тільки як формат фінального результату.

Насправді він може описувати різні речі.

Наприклад:

```

contract:
  input:
    required:
      - source_row
      - event_alias
      - display_names

  output:
    type: compact_analysis_package
    files_count: 11

  rules:
    - no_invented_contracts
    - no_final_archive_before_approvals

  status_semantics:
    ready_for_first_run:
      runner_policy: execute

  forbidden:
    - nested_values_without_confirmation
    - delta_field_with_item_prefix

```

Тобто contract може включати:

```

- input contract;
- output contract;
- field contract;
- status contract;
- naming contract;
- evidence contract;
- approval contract;
- forbidden forms.

```

У складних Ordo-програмах contract — це не один маленький блок, а цілий шар правил.

4.8. Context: з чим модель працює

Context відповідає на питання:

Які дані, документи, приклади або правила модель має використати зараз?

Context — це не мета і не результат. Це матеріал для роботи.

Приклади context:

```
context:
  user_request: "$USER_INPUT.message"
  source_text: "$USER_INPUT.text"
context:
  documentation:
    - START_HERE_ORDO.md
    - 00_ORDO_EXECUTION_CONTRACT.md
    - 07_HISTORY_EVENT_PLAYBOOK_COMPILED_IR.json
context:
  source_row:
    type: companyProfile
    sub_type: EDR
    dms_id: "..."
context:
  examples:
    - factual_history_event_example
    - changerecord_example
```

Context може бути коротким або дуже великим. Але Ordo вимагає не просто “мати контекст”, а правильно його вибирати.

Для великих playbook-ів це особливо важливо.

Не треба щоразу змушувати модель тримати весь playbook у голові. Краще:

DOC.SPLIT → DOC.CATALOG → DOC.SELECT

Тобто:

розбити документацію;
створити каталог;
вибрати потрібні розділи для поточного кроку.

4.9. Чому context має бути обмеженим



Може здатися, що чим більше контексту, тим краще. Але це не завжди так.

Якщо модель бачить занадто багато документів одночасно, вона може:

- використати неактуальне правило;
- змішати різні гілки процесу;
- взяти приклад за contract;
- застосувати правило з іншого path;
- пропустити поточний gate;
- витратити увагу на нерелевантні секції.

Тому Ordo вводить ідею focused context.

Для кожного кроку треба вибирати тільки те, що потрібно саме зараз.

Наприклад, якщо модель перебуває на старті guided intake, їй не потрібні повні QA templates. Їй потрібні:

- entry definition;
- decision tree;
- правила першого питання;
- заборона питати alias/source row завчасно.

Якщо модель генерує QA package, тоді їй потрібні:

- QA template;
- manual QA gates;
- rendered artifact validation;
- automation status rules;
- runner preflight, якщо застосовно.

Це дисципліна контексту.

4.10. Intent, Contract і Context на одному прикладі

Візьмемо завдання:

Створи Jira-задачу для нової історичної події.

У prompt-підході модель може одразу написати задачу. Але в Ordo ми спочатку розділяємо.

Intent

```
intent:
  goal: create_jira_task_for_history_event
```

Contract

```
contract:
  output:
    file: "02_JIRA_TASK_<ALIAS>.md"
    format: jira_description
    required_sections:
      - Summary
      - Context
      - Acceptance criteria
      - Event Creation Logic
      - Data Mapping
      - Technical Deliverables
      - Out of scope
      - QA reference data
      - Test coverage
  rules:
    - do_not_replace_passport
    - do_not_duplicate_full_qa_runbook
    - use_confirmed_contracts_only
```

Context

```
context:
  confirmed_alias: "$STATE.alias"
  path: "$STATE.path"
  passport_draft: "$ARTIFACTS.passport"
  confirmed_values_contract: "$STATE.history_event_values_contract"
  qa_scope: "$STATE.qa_scope"
```

Тепер модель розуміє:

- що вона робить;
- яким має бути результат;
- з яких даних вона має його створити.

Без цього вона може створити красиву, але неправильну Jira-задачу.

4.11. Як ці поняття пов'язані з gates

Intent, Contract і Context самі по собі ще не гарантують виконання. Потрібні gates.

Наприклад:

```
gates:
  - id: G_INTENT_DEFINED
    assert: intent.goal is not empty

  - id: G_CONTRACT_CONFIRMED
    assert: mandatory_contracts.status == confirmed

  - id: G_CONTEXT_AVAILABLE
    assert: required_context_items are present
```

Якщо context відсутній, модель не має вигадувати дані.

Якщо contract не confirmed, модель не має створювати final package.

Якщо intent неясний, модель має поставити уточнення.

У Ordo gates перетворюють “було б добре” на контроль виконання.

4.12. Малий шаблон для старту будь-якої Ordo-програми



Коли ви проектуєте нову Ordo-програму, можна почати з такого мінімального шаблону:

```
ordo: "0.12-draft"
program: "example.program"

intent:
  goal: "<що потрібно зробити>"

contract:
  input:
    required: []
```

```

output:
  type: "<тип результату>"
  rules: []
  forbidden: []

context:
  sources: []
  user_input: "$USER_INPUT"

state:
  status: draft
  assumptions: []
  open_questions: []

gates:
  - id: G_INTENT_DEFINED
    check: intent_defined
  - id: G_CONTRACT_AVAILABLE
    check: mandatory_contract_available
  - id: G_CONTEXT_AVAILABLE
    check: required_context_available

```

Це ще не повна програма. Але це хороший старт.

4.13. Типові помилки

Помилка 1. Починати з output без intent

Погано:

```

outputs:
  - README.md
  - SUMMARY.json

```

Краще спочатку:

```

intent:
  goal: explain_and_validate_package

```

Output без intent не пояснює, навіщо створюється артефакт.

Помилка 2. Писати rules у context

Погано:

```

context:
  note: "Не вигадувати фактів"

```

Краще:

```

contract:
  rules:
    - no_unsupported_facts

```

Або як gate:

```

gates:
  - id: G_NO_UNSUPPORTED_FACTS
    check: no_unsupported_facts

```

Помилка 3. Класти джерело даних у contract

Погано:

```
contract:
  source_text: "$USER_INPUT.text"
```

Краще:

```
context:
  source_text: "$USER_INPUT.text"
```

Contract описує правила результату, а context — матеріал, з яким працює модель.

Помилка 4. Не позначати статус контракту

Погано:

```
contract:
  values:
    - old#value
    - new#value
```

Краще:

```
contract:
  values:
    status: confirmed
  keys:
    - old#value
    - new#value
  evidence_source: factual_history_event_example
```

Якщо status не вказаний, модель може сприйняти candidate як confirmed.

4.14. Короткий підсумок розділу

Intent, Contract і Context — це перші три опори Ordo.

Intent — що ми хочемо зробити.
 Contract — яким має бути правильний результат і які правила не можна порушити.
 Context — з якими даними, джерелами й документами модель працює.

Вони мають бути розділені.

Якщо intent занадто широкий, модель не знає, що саме робити.

Якщо contract нечіткий, модель може створити красивий, але неправильний результат.

Якщо context не обмежений, модель може застосувати не ті правила.

Добра Ordo-програма починається не з великого тексту, а з чіткої відповіді на три питання:

1. Яка мета?
2. Який контракт правильного результату?
3. Який контекст потрібен саме зараз?

Міні-вправа

Візьміть будь-який свій prompt і перепишіть його у три блоки.

Наприклад, prompt:

Підготуй короткий лист клієнту про те, що інцидент ще аналізується, але ми вже працюємо над ним.

Спробуйте заповнити:

```
intent:
  goal: "..."
```

```
contract:
  output:
    type: "..."
```

```
    tone: "..."
```

```
    language: "..."
```

```
  rules:
    - "..."
```

```
context:
  incident_summary: "..."
```

```
  known_facts: []
```

```
  unknown_facts: []
```

Після цього перевірте:

- чи intent не містить зайвих правил;
- чи contract описує результат;
- чи context містить тільки дані й джерела;
- чи є щось, що треба винести в gate.

Розділ 5. State: пам'ять процесу

5.1. Навіщо Ordo потрібен State

Коли людина веде складний процес, вона майже завжди тримає в голові проміжний стан.

Наприклад, аналітик пам'ятає:

- який тип задачі вже обрано;
- які відповіді користувач уже дав;
- які дані ще відсутні;
- які припущення поки не підтверджені;
- які документи вже створені;
- які gates ще не пройдені;
- чи можна переходити до фінального результату.

У короткій розмові це не проблема. Якщо користувач просить переписати одне речення, моделі майже не потрібна складна пам'ять процесу.

Але якщо задача складається з багатьох кроків, без явного стану модель починає губитися. Вона може забути, що вже було підтверджено. Може вдруге поставити те саме питання. Може вважати припущення підтвердженим фактом. Може перейти до генерації документа, хоча обов'язкові дані ще не зібрані.

State в Ordo — це явна пам'ять процесу.



Не пам'ять моделі “десь у контексті”, а структурований об'єкт, який показує:

```
що вже відомо;
що ще невідомо;
що підтверджено;
що тільки запропоновано;
що заблоковано;
який наступний дозволений крок.
```

Іншими словами, State відповідає на питання:

```
Де ми зараз у процесі?
```

5.2. Чим State відрізняється від Context

На перший погляд context і state можуть здатися схожими. Обидва містять дані. Але їхня роль різна.

Context — це те, з чим модель працює.

Наприклад:

```
вхідний текст;
source row;
документ користувача;
```

```
історичний приклад;
JSON payload;
витяг із playbook-a.
```

State — це те, що модель уже вирішила або зафіксувала в процесі виконання.

Наприклад:

```
обраний path;
поточний node;
відповіді користувача;
підтверджені контракти;
відкриті питання;
статуси документів;
результати gates.
```

Можна сказати так:

```
Context — це матеріал.
State — це поточне положення процесу.
```

Або ще простіше:

```
Context відповідає на питання: "З чим ми працюємо?"
State відповідає на питання: "Що ми вже зробили і що дозволено далі?"
```



5.3. Простий приклад State

Уявімо невеликий процес: модель має допомогти користувачу підготувати відповідь клієнту про інцидент.

Перший вузол питає тип звернення:

```
1. incident
2. change_request
```

Користувач відповідає:

```
incident
```

Модель має не просто перейти до наступного питання. Вона має оновити стан:

```
state:
  request_type: incident
  current_node: N2_INCIDENT_SEVERITY
  completed_nodes:
    - N1_REQUEST_TYPE
```

Далі модель питає критичність. Користувач відповідає:

```
high
```

State оновлюється:

```
state:
  request_type: incident
  severity: high
```

```
current_node: N3_INCIDENT_AREA
completed_nodes:
  - N1_REQUEST_TYPE
  - N2_INCIDENT_SEVERITY
```

Тепер модель знає не просто окрему відповідь. Вона знає шлях, яким іде процес.

Це важливо, бо наступні питання, outputs і gates залежать від уже зібраного стану.

5.4. STATE.SCHEMA

У простих задачах State можна вести вільно. Але в Ordo краще заздалегідь описувати схему стану.

Для цього використовується STATE.SCHEMA.

Приклад:

```
state_schema:
  current_node:
    type: string
    required: true

  request_type:
    type: enum
    values:
      - incident
      - change_request
      - unknown
    required: false

  severity:
    type: enum
    values:
      - high
      - normal
      - unknown
    required: false

  terminal_path:
    type: string
    required: false

  open_questions:
    type: list
    required: true
    default: []

  assumptions:
    type: list
    required: true
    default: []

  gates:
    type: object
    required: true
    default: {}
```

Схема стану потрібна для трьох речей.

По-перше, вона допомагає моделі не вигадувати випадкові ключі.

По-друге:

```
state:
  type_of_problem: incident
  problem_kind: incident
  incidentType: incident
```

Добре:

```
state:
  request_type: incident
```

По-друге, схема показує, які дані вже можуть бути у процесі.

По-третє, вона дозволяє gates перевіряти стан не за здогадками, а за конкретними полями.

5.5. State як захист від повторних питань

Без State модель може повторити питання, на яке користувач уже відповів.

Наприклад:

```
Користувач: Це incident.
Модель: Добре. А який тип звернення – incident чи change_request?
```

Це створює відчуття, що модель не веде процес, а просто реагує на останнє повідомлення.

У Ordo після кожної відповіді має бути STATE.UPDATE.

Наприклад:

```
{
  "op": "STATE.UPDATE",
  "path": "request_type",
  "value": "incident",
  "source": "USER_ANSWER.N1"
}
```

Після цього gate або node selection може сказати:

```
request_type already set → do not ask N1 again
```

Тобто State не тільки зберігає інформацію. Він керує тим, які питання ще дозволені.

5.6. State як захист від передчасного результату

Одна з найважливіших функцій State — не дати моделі створити фінальний результат занадто рано.

Наприклад, для складного аналітичного пакета можуть бути такі поля:

```
state:
  path_selected: false
  source_contract_confirmed: false
  alias_confirmed: false
  display_names_confirmed: false
  values_contract_confirmed: false
  qa_scope_confirmed: false
```

```
automation_status_confirmed: false
document_approvals:
  passport: pending
  jira: pending
  qa_package: pending
  automation_spec: pending
```

Поки ці поля не перейшли в потрібний стан, модель не має права створювати фінальний архів.



Gate може виглядати так:

```
{
  "op": "GATE.CHECK",
  "id": "G_FINAL_ARCHIVE_ALLOWED",
  "method": "human",
  "trust_class": "human_decision",
  "assert": "state.path_selected == true && state.source_contract_confirmed == true && state.document_approvals.passport == 'approved'",
  "on_fail": "BLOCK_FINAL_ARCHIVE"
}
```

У звичайному prompt-і таке правило можна написати текстом. Але модель може його пропустити.

В Ordo це має бути частиною виконуваного стану.

5.7. State і статуси

State часто містить статуси. Але статуси мають бути чітко визначені.

Наприклад:

```
state:
  package_status: draft
  automation_status: ready_for_first_run
  validation_status: passed_with_notes
```

Якщо статуси не мають семантики, модель може трактувати їх довільно.

Тому State має працювати разом із STATUS.SEMANTICS.

Наприклад:

```
status_semantics:
  draft:
    meaning: "робота ще не фіналізована"
    final_handoff_allowed: false

  confirmed:
    meaning: "явно підтверджено користувачем або evidence"
    can_be_used_as_contract: true

  proposed:
    meaning: "запропоновано моделлю, але не підтверджено"
    can_be_used_as_contract: false
```

```
blocked:
  meaning: "виконання зупинено через конкретну причину"
  next_action_required: true
```

Це дозволяє моделі не змішувати “запропоновано”, “погоджено”, “готово” і “заблоковано”.

5.8. Assumption ledger як частина State

Модель часто робить припущення.

Це нормально. Проблема починається тоді, коли припущення непомітно стає фактом.

Наприклад:

Модель припустила, що подія має source = companyProfile / EDR.
Потім записала це в паспорт як confirmed.

Так робити не можна.

В Ordo припущення мають бути видимими:

```
state:
  assumptions:
    - id: A1
      assumption: "source може бути companyProfile / EDR"
      evidence: "бізнесова назва схожа на EDR-фактографію"
      status: proposed
      required_confirmation: representative_source_row_or_analyst_confirmation
      final_package_allowed: false
```

Така форма не забороняє моделі думати. Вона забороняє їй видавати здогадку за підтверджений контракт.

ASSUMPTION.LEDGER — одна з найважливіших конструкцій для довіри до AI-процесу.

5.9. State і open questions

У складних задачах частина даних часто відсутня.

Погана модель заповнює їх сама.

Хороша Ordo-програма фіксує open questions:

```
state:
  open_questions:
    - id: Q1
      question: "Який точний alias події використовуємо?"
      blocks:
        - final_package_generation
    - id: Q2
      question: "Який фактичний контракт HistoryEvent.item.values?"
      blocks:
        - passport_finalization
        - qa_automation_spec
```

Open question — це не слабкість. Це ознака чесного процесу.

Якщо даних не вистачає, Ordo має показати, чого саме бракує, а не вигадати відповідь.

5.10. State і traceability

State також допомагає пояснити, звідки взялося кожне рішення.

Наприклад:

```
state:
  path:
    value: Path 1 candidate
    source: USER_ANSWER.N1
    confidence: medium
    reason: "користувач вказав, що зміна виникає у внутрішньому Mongo source row"
```

Або:

```
state:
  values_contract:
    status: confirmed
    source: HISTORY_EVENT_EXAMPLE
  keys:
    - old#value
    - new#value
```

Це важливо для перевірки. Коли користувач питає “чому ти так вирішив?”, модель має не вигадувати пояснення заднім числом, а показати trace зі State.

5.11. State у compiled IR

У Semantic JSON IR State зазвичай представлений через операції:

```
{
  "op": "STATE.INIT",
  "id": "ST1",
  "schema": "history_event_intake_state",
  "defaults": {
    "current_node": "N1",
    "path": "unknown",
    "open_questions": [],
    "assumptions": [],
    "gates": {}
  }
}
```

Після відповіді користувача:

```
{
  "op": "STATE.UPDATE",
  "id": "STU1",
  "set": {
    "request_type": "incident",
    "current_node": "N2_INCIDENT_SEVERITY"
  },
  "source_ref": "USER_ANSWER.N1"
}
```

Після gate:


```
{
  "op": "STATE.UPDATE",
  "id": "STU2",
  "set": {
    "gates.G1.status": "passed"
  },
  "source_ref": "GATE.G1"
}
```

Така форма зручна для runtime, тому що кожна зміна стану має джерело.

5.12. Типові помилки зі State

Помилка 1. Не вести State явно

Модель просто відповідає на кожне повідомлення, не фіксуючи, що вже відомо.

Наслідок: повторні питання, втрата контексту, передчасні висновки.

Помилка 2. Змішувати Context і State

Наприклад, зберігати весь source row у State.

Краще:

```
context:
  representative_source_row: "...".
state:
  source_row_contract:
    status: confirmed
    evidence_ref: representative_source_row
```

Помилка 3. Не розрізняти proposed і confirmed

Якщо модель запропонувала alias, це ще не confirmed alias.

State має показувати:

```
alias:
  value: LU_CHANGE_CAPITAL
  status: proposed
```

а не:

```
alias: LU_CHANGE_CAPITAL
```

Помилка 4. Не зберігати blockers

Якщо виконання заблоковано, причина має бути в State:

```
blockers:
  - id: B1
    reason: "source row contract not confirmed"
    blocks: final_archive
```

Помилка 5. Не очищати припущення перед фіналом

Перед фінальним результатом assumption ledger має бути або порожній, або містити тільки non-blocking notes.

Якщо mandatory contract досі proposed, фінальний пакет не можна створювати.

5.13. Практичний шаблон State для Ordo-процесу

Для багатьох Ordo-процесів можна починати з такої базової схеми:

```
state:
  current_node: START
  completed_nodes: []
  selected_path: unknown

  answers: {}

  contracts:
    required: []
    confirmed: []
    proposed: []
    missing: []

  assumptions: []
  open_questions: []
  blockers: []

  outputs:
    selected: []
    generated: []
    approved: []

  gates: {}

  handoff:
    status: draft
    allowed: false
```

Це не універсальний закон. Але це хороший стартовий шаблон.

Він змушує модель думати процесно:

```
де ми;
що вже зібрано;
що підтверджено;
що заблоковано;
що можна створювати;
чи дозволений handoff.
```

5.14. Короткий підсумок розділу

State — це пам'ять процесу в Ordo.

Він потрібен, щоб модель:

- не повторювала питання;
- не перескакувала кроки;
- не видавала припущення за підтвердження;
- не створювала фінальний результат занадто рано;
- могла пояснити, звідки взялося рішення;
- розуміла, які gates пройдені;
- бачила open questions і blockers.

Context — це матеріал, із яким працює модель.

State — це поточне положення процесу.

Без State Ordo перетворюється назад на довгий prompt. Зі State Ordo стає керованим процесом.

Міні-вправа

Візьміть будь-який процес, де модель має поставити кілька питань перед результатом.

Наприклад:

Підготувати Jira-задачу за описом користувача.

Спробуйте описати State:

```
state:
  current_node: START
  task_type: unknown
  business_goal: missing
  acceptance_criteria: missing
  out_of_scope: missing
  assumptions: []
  open_questions: []
  blockers: []
  handoff:
    status: draft
    allowed: false
```

Потім дайте відповідь на питання:

1. Які поля мають бути confirmed перед фінальною задачею?
 2. Які поля можуть бути proposed?
 3. Які open questions блокують handoff?
 4. Який gate не дозволить створити фінальний документ занадто рано?
-

Розділ 6. Entry і Node: як модель ставить питання

6.1. Навіщо Ordo потрібні Entry і Node

У звичайному діалозі з моделлю користувач часто очікує, що модель сама зрозуміє, з чого почати.

Наприклад, користувач пише:

Хочу створити нову історичну подію.

Модель може відреагувати по-різному. Вона може одразу запропонувати alias. Може попросити source row. Може почати формувати паспорт. Може перейти до QA. Може поставити десять питань одразу.

Для простого діалогу це не завжди критично. Але для playbook-а така свобода небезпечна.

Якщо процес має правильний порядок, модель не повинна сама вирішувати, з якого місця починати. Початок має бути явно визначений.



У Ordo для цього є ENTRY.DEF.

ENTRY.DEF відповідає на питання:

З якого місця починається цей процес?

А NODE.DEF відповідає на питання:

Який конкретний крок або вузол рішення зараз виконується?

Разом вони перетворюють діалог із моделлю на керований маршрут.

6.2. Entry — це двері в процес

ENTRY.DEF можна уявити як двері.

Коли користувач каже: “Починаємо”, модель не має бігти по всьому playbook-у. Вона має зайти через визначені двері.

Приклад:

```
{
  "op": "ENTRY.DEF",
  "id": "START_HISTORY_EVENT_INTAKE",
  "description": "Start guided intake for a new History Event",
  "first_node": "N1_INPUT_TYPE"
}
```

Це означає:

Процес створення історичної події завжди починається з вузла `N1_INPUT_TYPE`.

Не з `alias`. Не з `display names`. Не з QA. Не з `automation`. Не з повного шаблону паспорта.

А саме з першого питання дерева.

У цьому і є цінність `ENTRY.DEF`: він не дозволяє моделі починати “з того, що їй здається корисним”.

6.3. Node — це один керований крок

`NODE.DEF` описує один вузол процесу.

Вузол може бути питанням, вибором, перевіркою або проміжним рішенням. У навчальній версії найпростіше думати про `node` як про одне питання, яке модель ставить користувачу.

Приклад:

```
{
  "op": "NODE.DEF",
  "id": "N1_INPUT_TYPE",
  "kind": "question",
  "question": "Звідки система має зрозуміти, що історична подія сталася?",
  "answers": ["A", "B", "C", "D", "E", "F"],
  "state_write": "entry_input_type"
}
```

Такий вузол чітко задає:

```
яке питання ставити;
які відповіді дозволені;
куди записати відповідь;
що робити далі після відповіді.
```

Це важливо, бо модель більше не має вгадувати формат діалогу на ходу.

6.4. Чому не можна ставити всі питання одразу



Коли модель отримує складне завдання, вона часто намагається бути корисною і питає все одразу:

```
Надай alias, назву українською та англійською, source row, type/sub_type, field path, values contract, QA fixture, automation scope і runner status.
```

На вигляд це ефективно. Але для користувача це перевантаження. А для процесу — ризик.

Чому?

Тому що частина цих питань ще не має сенсу до вибору шляху.

Наприклад, поки ми не знаємо, чи це Path 1, Path 3, Path 4 або Path 5, неможливо правильно питати про конкретний source contract. Поки не підтверджено source або ChangeRecord, небезпечно питати про HistoryEvent.item.values. Поки не підтверджено контракт, рано переходити до QA automation.

Ordo вводить правило:

один вузол — одне головне питання

Це не означає, що в процесі буде мало питань. Це означає, що питання йдуть у правильному порядку.

6.5. Answer Registry: дозволені відповіді

У звичайному діалозі користувач може відповісти як завгодно. І це добре для природної мови, але погано для керованого процесу.

Якщо вузол очікує A, B, C, D, E, F, а користувач пише 5, модель має не вгадувати, що він мав на увазі, а перевірити відповідь.

Для цього потрібен ANSWER.REGISTRY.

Приклад:

```
{
  "op": "ANSWER.REGISTRY",
  "node": "N1_INPUT_TYPE",
  "allowed": [
    {
      "key": "A",
      "meaning": "internal_mongo_source_row_change"
    },
    {
      "key": "B",
      "meaning": "new_source_or_data_block_extension"
    },
    {
      "key": "C",
      "meaning": "existing_internal_changerecord"
    },
    {
      "key": "D",
      "meaning": "external_history_fact_or_change_payload"
    },
    {
      "key": "E",
      "meaning": "state_comparison_or_data_block_comparison"
    },
    {
      "key": "F",
      "meaning": "unknown_or_other"
    }
  ],
  "on_invalid": "ASK_VALID_OPTION_AGAIN"
}
```

Це означає, що відповідь користувача має бути перевірена до переходу далі.

Якщо відповідь не входить у allowed list, модель не повинна рухатися наступним вузлом.

6.6. State update після відповіді

Node не існує сам по собі. Його результат має потрапити в state.

Наприклад, користувач відповів:

A

Модель має записати:

```
state:
  entry_input_type: internal_mongo_source_row_change
  path_candidate: Path 1 or Path 2
```

Після цього можна перейти до наступного вузла.

Без state модель може “пам’ятати” відповідь у тексті діалогу, але це ненадійно. У довгому процесі відповідь може загубитися або бути переінтерпретована.

Ordo змушує модель явно оновлювати стан.

6.7. Node як спосіб не перескакувати вперед

У правильно описаному Ordo-процесі кожен node має межі.

Наприклад, вузол N1_INPUT_TYPE має право тільки визначити тип входу. Він не має права підтверджувати alias, не має права вгадувати type/sub_type, не має права створювати QA package.

Це можна описати так:

```
{
  "op": "NODE.DEF",
  "id": "N1_INPUT_TYPE",
  "allowed_actions": [
    "ASK_QUESTION",
    "VALIDATE_ANSWER",
    "STATE.UPDATE",
    "PATH.CANDIDATE.SET"
  ],
  "forbidden_actions": [
    "ALIAS.CONFIRM",
    "SOURCE_ROW.REQUEST_FULL",
    "VALUES.CONTRACT.DEFINE",
    "PACKAGE.GENERATE"
  ]
}
```

Це дуже важлива ідея.



Node не тільки каже, що робити. Він також каже, чого на цьому кроці робити не можна.

6.8. Приклад маленького дерева

Уявімо простий support-процес.

Користувач пише:

У мене проблема з сервісом.

Ordo-процес може починатися так:

```
entry:
  id: START_SUPPORT_TRIAGE
  first_node: N1_REQUEST_TYPE

nodes:
  N1_REQUEST_TYPE:
    question: "Що це за звернення?"
    answers:
      incident: "інцидент / збій"
      change_request: "запит на зміну"
    write_to: request_type

  N2_INCIDENT_SEVERITY:
    when: request_type == incident
    question: "Наскільки критична проблема?"
    answers:
      high: "блокує роботу"
      normal: "не блокує повністю"
    write_to: severity

  N3_INCIDENT_EVIDENCE:
    when: request_type == incident
    question: "Є кроки відтворення або приклад?"
    answers:
      present: "є"
      missing: "немає"
    write_to: reproduction_evidence
```

Це вже дерево рішень. Модель не імпровізує маршрут, а виконує його.

6.9. Node і користувацький досвід

Може здатися, що такий підхід робить діалог повільнішим. Але на практиці він часто робить його зручнішим.

Користувач не отримує величезну анкету. Він отримує одне питання за раз.

Модель не вантажить його технічними деталями раніше часу. Вона веде його по маршруту.

Це особливо важливо, коли користувач не знає внутрішніх термінів системи. Наприклад, аналітик не зобов'язаний знати, що таке Path 1 або Path 5. Він може просто відповісти на питання:

Звідки система має зрозуміти, що подія сталася?

А модель уже сама мапить відповідь на candidate path.

6.10. Типові помилки з Entry і Node

Перша помилка — не мати явного entry.

Тоді модель кожного разу починає по-різному.

Друга помилка — змішати кілька рішень в одному node.

Наприклад:

Оберіть Path, alias, source row і QA fixture.

Це поганий node. Він об'єднує кілька різних hard contracts.

Третя помилка — не мати allowed answers.

Якщо відповіді не визначені, модель починає вгадувати.

Четверта помилка — не записувати відповідь у state.

Тоді відповідь є в чаті, але її немає в execution state.

П'ята помилка — дозволити node робити більше, ніж треба.

Наприклад, стартовий node не має права створювати фінальний пакет.

6.11. Короткий підсумок розділу

ENTRY.DEF і NODE.DEF потрібні для того, щоб модель не починала процес довільно і не перескакувала між кроками.

ENTRY.DEF визначає точку входу.

NODE.DEF визначає конкретний вузол процесу.

ANSWER.REGISTRY задає дозволені відповіді.

STATE.UPDATE зберігає результат відповіді.

Головний принцип:

один вузол — одне головне рішення

Це робить діалог керованим, перевірюваним і зручним для користувача.

Міні-вправа

Візьміть будь-який складний процес, де модель має ставити користувачу питання.

Опишіть:

1. Який ENTRY має запускати процес?
2. Який перший NODE?
3. Яке одне головне питання він ставить?
4. Які відповіді дозволені?
5. Куди відповідь записується в STATE?
6. Які дії заборонені на цьому вузлі?

Якщо ви не можете відповісти на ці питання, ваш процес ще не готовий до стабільного виконання моделлю.

Розділ 6.1. Атомарність кроків у процесах з мовними моделями

Один крок - одна відповідальність, один результат і одна точка перевірки або підтвердження.

Процес, який виконує людина, програма або мовна модель, часто виглядає простим лише на рівні назви кроку. Наприклад, фраза «підготувати документ» може приховувати збір фактів, відновлення відсутніх даних, прийняття рішення, написання тексту, перевірку, погодження і зміну стану. Для людини така фраза іноді є зручною скороченою назвою. Для мовної моделі вона небезпечна: модель може виконати частину дій, інші домислити, ще інші пропустити - і все одно оголосити крок завершеним.

Атомарність потрібна не для штучного подрібнення процесу. Вона потрібна, щоб кожен крок мав чітку межу відповідальності, спостережуваний результат і локальну точку контролю. Тоді помилку можна знайти, виправити й повторити саме там, де вона виникла, а не запускати весь процес заново.

1. Формулювання принципу

Атомарний крок - це завершена операція з визначеними входами, однією основною відповідальністю, одним спостережуваним результатом і одним набором критеріїв завершення.

«Один крок» не обов'язково означає одну коротку команду або одну фразу. Крок може містити кілька механічних дій, якщо вони разом створюють один результат і мають один критерій готовності. Наприклад, відсортувати ключі структурованого документа, відформатувати відступи, зберегти файл і обчислити checksum можна як одну технічну операцію. Але не можна в цій самій операції непомітно дописувати відсутні вимоги або затверджувати нову версію.

Один крок

Одна завершена операція, яку можна запустити, завершити, повторити або відхилити як ціле.

Одна відповідальність

Крок відповідає за один тип зміни: зібрати факти, сформулювати рішення, створити артефакт, перевірити результат або застосувати перехід стану. Якщо крок одночасно визначає факти й приймає рішення, ці дві задачі починають впливати одна на одну.

Один результат

У кроку має бути один основний спостережуваний вихід. Це може бути evidence artifact, design decision, готовий документ, validation report або новий стан. Кілька файлів можна вважати одним результатом лише тоді, коли вони є неподільним технічним комплектом і не можуть мати різні статуси готовності.

Одна точка валідації

Крок має перевірятися за критеріями, які відповідають саме його результату. Перевірка фактичності даних відрізняється від перевірки повноти документа, а перевірка документа - від підтвердження користувачем.

Одна точка підтвердження

Підтвердження потрібне там, де змінюється рішення, домовленість або confirmed state. Воно не повинно підмінятися припущенням моделі. Водночас підтвердження не треба вимагати після кожної механічної дії, яка не змінює зміст.

2. Чому змішування відповідальностей небезпечне

Втрата даних між внутрішніми підзадачами

Коли один великий крок містить кілька прихованих операцій, проміжний результат часто не матеріалізується. Модель могла знайти суперечність, але не зафіксувати її перед переходом до генерації. Пізніше неможливо встановити, чи суперечність була втрачена, проігнорована або вирішена припущенням.

Приховані припущення

Якщо відсутнє значення потрібне для наступної дії, модель схильна підставити правдоподібне значення. Без окремого кроку реконструкції таке значення швидко починає виглядати як підтверджений факт.

Передчасний перехід

У складеному кроці перша успішна підумова може помилково відкрити наступний етап. Наприклад, дані отримані, але не пройшли перевірку; проте процес переходить до генерації, бо `input_received = true`.

Неможливість локалізувати помилку

Якщо крок одночасно збирає дані, приймає рішення і створює документ, статус failed не пояснює, де саме виникла проблема. А статус completed може приховати слабкий результат однієї з внутрішніх операцій.

Формальна перевірка замість змістовної

Великий крок легко перевірити лише за наявністю файлів, секцій або ідентифікаторів. Це створює false green: структура існує, але зміст не придатний до виконання.

Неможливість повторити одну операцію

Якщо фінальна перевірка знайшла відсутній rollback, правильно повернутися до формування rollback-секції. У монолітному кроці доводиться повторювати весь процес, включно з уже підтвердженими фактами.

Змішування фактів, припущень і рішень

Факти мають походити з evidence. Припущення повинні бути позначені. Рішення мають спиратися на зафіксований evidence artifact. Якщо все це виконується в одному контексті без проміжних виходів, межі стираються.

Надмірне стискання результату

Мовна модель прагне скорочувати повторювані або схожі дії. Кілька конкретних сценаріїв можуть перетворитися на фразу «перевірте відповідні випадки», хоча саме конкретика була головною цінністю.

Самоперевірка тим самим контекстом

Модель може оцінити не фактичний текст, а намір, який утримує в контексті. Тому post-render review повинен працювати з матеріалізованим артефактом, а не з планом його створення.

3. Базовий патерн

```
отримати або підготувати вхідні дані
→ створити один визначений результат
→ окремо перевірити результат
→ показати результат або статус
→ отримати підтвердження, якщо воно потрібне
→ перейти до наступної операції
```

Отримати або підготувати вхідні дані

Відповідальність: зібрати підтверджені дані та явно позначити прогалини.

Допустимі входи: джерела, попередньо підтверджені артефакти, відповіді користувача.

Очікуваний вихід: структурований input artifact.

Критерій завершення: відомо, що підтверджено, що суперечливе, а що відсутнє.

Не повинен робити: приймати бізнес-рішення або тихо реконструювати критичні значення.

Створити один визначений результат

Відповідальність: матеріалізувати один artifact або decision result.

Допустимі входи: лише зафіксовані та дозволені дані.

Очікуваний вихід: конкретний документ, запис, рішення або state proposal.

Критерій завершення: результат існує у формі, придатній до окремої перевірки.

Не повинен робити: самостійно оголошувати результат перевіреном або підтвердженням.

Окремо перевірити результат

Відповідальність: перевірити матеріалізований вихід за явними критеріями.

Допустимі входи: готовий artifact і validation rules.

Очікуваний вихід: evidence зі статусом passed, failed або needs_review.

Критерій завершення: кожен обов'язковий критерій має доказ.

Не повинен робити: непомітно переписувати результат і після цього ставити passed.

Показати результат або статус

Відповідальність: зробити стан процесу видимим людині або наступному вузлу.

Очікуваний вихід: artifact reference, diagnostic summary або статус етапу.

Не повинен робити: маскувати частковий успіх загальним completed.

Отримати підтвердження

Відповідальність: зафіксувати рішення там, де змінюється confirmed state.

Очікуваний вихід: approved, rejected або revision_requested.

Не повинен робити: трактувати мовчання або правдоподібність як згоду.

Перейти до наступної операції

Відповідальність: застосувати явний state transition після виконання всіх передумов.

Критерій завершення: completion criteria попереднього кроку виконані повністю.

Не повинен робити: переходити за одним частковим позитивним сигналом.

4. Патерн атомарного вузла

```
step:
  purpose:
  inputs:
  action:
  output:
  completion_criteria:
  validation:
  confirmation_required:
  next_step:
  failure_route:
```

purpose

Одне речення про те, навіщо існує крок. Якщо purpose містить кілька незалежних дієслів - це сигнал до декомпозиції.

inputs

Точний список дозволених входів. Тут важливо відрізнити confirmed data, proposed data і open gaps.

action

Операція, яка створює результат. Саме action змінює artifact або формує новий output.

output

Один основний спостережуваний результат, який можна передати на validation.

completion_criteria

Умови, за яких action справді завершена. Вони не повинні обмежуватися перевіркою, що файл існує.

validation

Окрема перевірка результату. Вона не повинна непомітно виправляти або регенерувати output. Якщо виправлення потрібне, validation повертає failure route.

confirmation_required

Явне правило, чи потрібне людське або системне підтвердження. Відсутність підтвердження не можна замінювати припущенням.

next_step

Наступний крок дозволений лише після виконання completion criteria, validation і, за потреби, confirmation.

failure_route

Маршрут повернення до найближчого кроку, який може виправити конкретну помилку. Він не повинен автоматично перезапускати весь процес.

5. Патерни правильної декомпозиції

Pattern card: Collect → Resolve → Confirm

Коли застосовувати: коли джерела містять прогалини, суперечності або неповні значення.

Яку проблему усуває: змішування фактів із реконструкцією та підтвердженням.

Мінімальна структура:

```
Collect confirmed facts and gaps
→ Resolve one gap as proposal
→ Confirm or reject proposal
```

Умовний приклад - змішаний крок підготовки документа.

Неправильно:

Проаналізуй заявку, віднови відсутні відомості, визнач правила обробки, створи інструкцію, перевір її та познач готовою.

Модель читає заявку, помічає відсутнє поле delivery_mode, припускає scheduled, формує правила, створює документ, сама його перевіряє і переводить процес у approved.

Проблема: неможливо визначити, чи scheduled було фактом, реконструкцією або припущенням.

Правильно:

Крок 1 – Витягти підтверджені дані із заявки
Результат: перелік фактів і перелік прогалин

Крок 2 – Опрацювати прогалину `delivery_mode`
Результат: `proposed value` або `explicit open gap`

Крок 3 – Показати реконструкцію для підтвердження
Результат: `confirmed value` або повернення до уточнення

Крок 4 – Сформулювати правила обробки
Результат: окремий `design artifact`

Крок 5 – Згенерувати інструкцію
Результат: готовий документ

Крок 6 – Незалежно перевірити документ
Результат: `passed` або `failed validation`

Крок 7 – Підтвердити документ
Результат: `approved state`

Критерій успіху: кожне реконструйоване значення має окремий статус і не використовується як `confirmed` до підтвердження.

Якщо відсутнє значення впливає на правила, реконструкція має бути окремим кроком. Якщо значення не впливає на результат, його можна залишити відкритою прогалиною.

Pattern card: Design → Materialize → Review

Коли застосовувати: коли спочатку треба визначити структуру або контракт, а потім створити фактичний артефакт.

Яку проблему усуває: підміну готового результату планом або `coverage-list`.

Мінімальна структура:

```
Design artifact
→ Materialized artifact
→ Post-render review evidence
```

Умовний приклад - план тестування помилково вважається готовою інструкцією.

Неправильно: визначено позитивний сценарій, порожнє значення, `no-change` і помилку, після чого створено `MANUAL_CHECKS.md`:

1. Підготуйте позитивні дані.
2. Виконайте оновлення.
3. Перевірте очікуваний результат.
4. За потреби виконайте відкат.

Файл існує і формально має секції, але виконавець сам повинен придумати запит, значення, перевірки й `rollback`.

Правильно: спочатку створити `execution specification`:

```
manual_case:
  id: CASE-01
  fixture_lookup:
    source: sample_records
    filter:
      external_key: "EXAMPLE-1042"
  projection:
```



```

    external_key: 1
    current_value: 1
  action:
    method: PATCH
    path: /api/example-records/EXAMPLE-1042
    body:
      current_value: "new-example-value"
  expected_result:
    current_value: "new-example-value"
    audit_status: "recorded"
  rollback:
    method: PATCH
    path: /api/example-records/EXAMPLE-1042
    body:
      current_value: "original-example-value"
  post_rollback:
    expected_value: "original-example-value"

```

Лише після completeness gate specification перетворюється на документ для людини.

Критерій успіху: виконавець може виконати інструкцію без конструювання суттєвих команд або очікувань.

Якщо після читання документа ще треба самостійно створювати суттєві команди або критерії, документ не є виконуваною інструкцією.

Pattern card: Generate → Validate → Approve

Коли застосовувати: для будь-якого результату, який має бути перевірений і підтверджений.

Яку проблему усуває: generate-and-self-approve та автоматичний перехід у confirmed state.

Мінімальна структура:

```

Generate draft
→ Validate final artifact
→ Request approval
→ Apply state transition

```

Умовний приклад - explicit state transition.

Неправильно:

```
draft → confirmed
```

одразу після генерації документа.

Правильно:

```

GENERATE_DOCUMENT
output: draft_document

VALIDATE_DOCUMENT
output: validation_evidence

REQUEST_CONFIRMATION
output: approved | rejected | revision_requested

APPLY_STATE_TRANSITION
precondition:
  validation_status == passed
  approval_status == approved

```

```
output:
  state == confirmed
```

Критерій успіху: confirmed state змінюється лише окремою операцією з явними передумовами.

Pattern card: One Artifact at a Time

Коли застосовувати: коли пакет містить документи з різними цілями та критеріями якості.

Яку проблему усуває: загальний completed, який приховує слабкий файл.

Умовний приклад - один крок генерує кілька документів.

Неправильно:

На основі підтверджених вимог створи специфікацію, інструкцію для користувача, checklist перевірки та підсумковий manifest.

Три файли змістовні, а checklist складається із загальних фраз. Загальний статус кроку - completed.

Правильно:

```
Artifact A: специфікація
→ generate
→ validate
→ approve

Artifact B: інструкція користувача
→ generate
→ validate
→ approve

Artifact C: checklist перевірки
→ generate
→ validate
→ approve

Artifact D: manifest
→ assemble from approved artifacts
→ cross-document validation
```

Критерій успіху: пакет збирається лише з артефактів, які мають індивідуальний ready status.

Якщо результати можуть мати різний статус готовності, вони не повинні створюватися як один неподільний результат.

Pattern card: Reconstruction as a Separate Branch

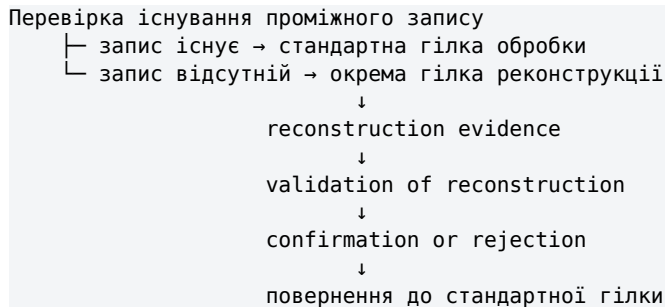
Коли застосовувати: коли процес змушений створити те, що у звичайній гілці лише читається.

Яку проблему усуває: приховану реконструкцію і змішування різних рівнів ризику.

Умовний приклад - реконструкція запису всередині звичайної обробки.

Неправильно: крок «Обробити запис і перейти до формування результату» непомітно реконструює відсутній запис із двох джерел, ставить `valid` і продовжує без `evidence`.

Правильно:



Критерій успіху: реконструйований об'єкт має `evidence`, `validation` і явний статус до повернення в основну гілку.

Якщо процес створює те, що зазвичай лише читається, це інший тип операції й окрема гілка.

Pattern card: Explicit State Transition

Коли застосовувати: коли змінюється `draft`, `approved`, `confirmed`, `released` або інший значущий стан.

Яку проблему усуває: state transition як побічний ефект генерації.

Критерій успіху: перехід має `preconditions`, окрему дію й окремий `output state`.

Pattern card: Independent Post-Render Review

Коли застосовувати: після матеріалізації документа, файлу, таблиці, PDF або іншого кінцевого представлення.

Яку проблему усуває: перевірку наміру замість фактичного результату.

Умовний приклад - самоперевірка за планом.

Неправильно: модель мала вимогу «кожен розділ має містити конкретний приклад», побачила приклади у власному плані й повернула `validation_status: passed`, хоча один розділ готового документа прикладу не має.

Правильно:

```

Generation:
створити final_document.md

Post-render review:
відкрити final_document.md
перерахувати required sections
для кожної секції знайти конкретний приклад
зафіксувати missing examples
повернути passed або failed
  
```

Критерій успіху: `validation evidence` посилається на фактичний артефакт, а не на `plan` або `prompt`.

Перевіряй артефакт після матеріалізації. Наявність вимоги в плані не доводить її наявність у результаті.

6. Антипатерни

Antipattern card: Combined Node

Ознаки: один вузол збирає дані, приймає рішення, генерує результат і переходить далі.

Чому виникає: автор хоче скоротити flow або зменшити кількість вузлів.

Наслідки: приховані підзадачі мають різний ступінь успіху, але один загальний статус.

Як виявити: у purpose або action більше одного незалежного дієслова; існує кілька виходів; різні частини потребують різних gates.

Чим замінити: Collect → Resolve → Confirm, Design → Materialize → Review.

Antipattern card: Hidden Reconstruction

Ознаки: відсутні дані відновлюються і відразу використовуються як confirmed.

Чому виникає: модель прагне завершити задачу без відкритої прогалини.

Наслідки: downstream logic залежить від непідтвердженого значення.

Як виявити: у trace немає reconstruction event, але в результаті з'явилося значення, якого не було у джерелах.

Чим замінити: Reconstruction as a Separate Branch.

Antipattern card: Design Equals Artifact

Ознаки: план, список сценаріїв або coverage помилково вважається готовою інструкцією.

Чому виникає: структура виглядає завершеною, а файл формально існує.

Наслідки: виконавець сам відновлює конкретні дії, inputs і expected results.

Як виявити: у документі багато «відповідний», «потрібний», «очікуваний», але мало конкретних значень.

Чим замінити: Design → Materialize → Review.

Antipattern card: Generate and Self-Approve

Ознаки: той самий крок створює результат і оголошує його готовим.

Чому виникає: економія на окремому review step.

Наслідки: модель оцінює свій намір, а не артефакт.

Як виявити: між generation і approved state немає validation evidence.

Чим замінити: Generate → Validate → Approve.

Antipattern card: Validation by Presence

Ознаки: перевіряється лише існування файлу, секції або ID.

Чому виникає: такі checks легко автоматизувати.

Наслідки: загальні фрази проходять gate, хоча результат не виконуваний.

Як виявити: validation не перевіряє конкретність, повноту або придатність до виконання.

Чим замінити: artifact-level completeness і post-render review.

Antipattern card: Automatic Transition on Partial Success

Умовний приклад - частковий успіх запускає наступний етап.

Неправильно: крок має отримати дані, перевірити їх і визначити режим. Дані отримано, але атрибут не пройшов перевірку; процес переходить до генерації через `input_received = true`.

Правильно:

```
input_status: received
validation_status: failed
decision_status: not_started
generation_status: blocked
overall_status: blocked_by_input_validation
```

Перехід дозволено лише за умовою:

```
input_status == received
AND validation_status == passed
AND required_gaps == 0
```

Чим замінити: NO_PARTIAL_SUCCESS_TRANSITION_GATE.

Перехід залежить від повного completion criteria, а не від першого позитивного сигналу.

Antipattern card: Package-Level Success Hides Artifact Failure

Ознаки: загальний пакет passed, хоча один artifact failed або не перевірений.

Наслідки: користувач отримує комплект, де слабкий компонент прихований агрегованим статусом.

Чим замінити: One Artifact at a Time і ARTIFACT_LEVEL_STATUS_GATE.

Antipattern card: Generic Instruction as Completed Work

Умовний приклад - semantic placeholders.

Неправильно:

```
Знайдіть відповідний запис.
Застосуйте потрібну зміну.
Переконайтеся, що результат правильний.
Виконайте відкат.
```

Слова відповідний, потрібну і правильний приховують незаповнені рішення.

Правильно:

Знайдіть запис у вигаданому наборі `sample_items` за фільтром `{ "sample_key": "ITEM-204" }`.

Змініть поле `display_label` з `"Old sample label"` на `"New sample label"`.

Після оновлення перевірте:
- `display_label == "New sample label"`;
- `change_status == "applied"`.

Для відкату поверніть:
`display_label = "Old sample label"`.

Після відкату повторіть `lookup` і перевірте початкове значення.

Чим замінити: `explicit input/output contract` і `executable specification`.

Якщо прикметник або займенник замінює конкретний об'єкт, значення чи критерій, це `semantic placeholder`.

Antipattern card: Evidence Collection Mixed with Decision

Умовний приклад - збір фактів змішано з прийняттям рішення.

Неправильно:

Збери інформацію про заявку та визнач, чи її можна автоматично схвалити.

Модель під час читання одразу класифікує неоднозначні значення як позитивні, щоб завершити рішення.

Правильно:

Крок А – Evidence collection

Вихід:

- confirmed facts;
- source references;
- contradictions;
- open gaps.

Крок В – Eligibility evaluation

Вхід:

- лише зафіксований evidence artifact.

Вихід:

- eligible;
- not eligible;
- needs review.

Крок С – Approval

Вхід:

- evaluation result.

Вихід:

- confirmed decision.

Якщо дія визначає, що є фактом, вона не повинна одночасно вирішувати, яке рішення бажано отримати.

7. Коли крок потрібно розділити

Розділіть крок, якщо виконується хоча б одна з умов:

- він створює більше одного незалежного результату;
- для різних частин потрібні різні критерії якості;
- одна частина може бути успішною, а інша - невдалою;
- одна частина потребує підтвердження, а інша - ні;
- помилку треба виправити без повторення всього процесу;
- змішуються факти, припущення й рішення;
- один контекст створює і перевіряє той самий результат;
- поєднані різні типи діяльності: аналіз, реконструкція, генерація, validation;
- крок може приховати невизначеність;
- результат однієї внутрішньої частини стає входом іншої без явної фіксації.

Практична перевірка: якщо для опису failure route доводиться писати «повторити крок», але насправді треба повторити лише одну його частину, крок неатомарний.

8. Коли об'єднання допустиме

Принцип атомарності не вимагає перетворювати кожен дрібну механічну дію на окремий вузол.

Об'єднання допустиме, якщо дії:

- мають одну відповідальність;
- створюють один результат;
- мають спільний критерій завершення;
- не містять нового бізнес-рішення;
- не змінюють confirmed state;
- не приховують реконструкцію;
- можуть бути безпечно повторені як одна операція;
- є механічним перетворенням підтвердженого джерела.

Умовний приклад допустимого об'єднання

Після підтвердження структурованого документа потрібно:

- відсортувати ключі;
- відформатувати відступи;
- зберегти файл;
- обчислити checksum.

Це можна виконати одним технічним кроком: зміст не змінюється, рішення не додаються, вихід один - release artifact.

Недопустиме об'єднання

У цьому самому кроці не можна:

- доповнювати відсутні вимоги;
- переписувати неоднозначні правила;
- видаляти секції як «зайві»;
- автоматично підтверджувати нову версію.

Механічні перетворення можна групувати. Семантичні рішення, реконструкцію та підтвердження - ні.

9. Валідація атомарності процесу

Нижче наведено методичні gates. Вони допомагають автору процесу перевірити структуру, але самі по собі не змінюють нормативний контракт мови.

SINGLE_RESPONSIBILITY_GATE

Перевіряє: чи має крок один тип відповідальності.

Проходить: purpose і action описують одну операцію.

Блокує: крок одночасно збирає, реконструює, вирішує й генерує.

Діагностика: STEP_ATOMICITY_001: step mixes evidence collection and decision making.

SINGLE_OUTPUT_GATE

Перевіряє: чи існує один основний результат.

Проходить: output має один artifact/status contract.

Блокує: незалежні результати можуть мати різну готовність.

Діагностика: STEP_ATOMICITY_002: multiple independently validatable outputs detected.

EXPLICIT_INPUT_OUTPUT_GATE

Перевіряє: чи визначені входи й вихід.

Блокує: використовуються semantic placeholders або незафіксовані проміжні дані.

Діагностика: STEP_ATOMICITY_003: output depends on implicit input.

NO_HIDDEN_RECONSTRUCTION_GATE

Перевіряє: чи відновлення відсутнього оформлене окремою гілкою.

Блокує: reconstructed value використано як confirmed без evidence.

Діагностика: STEP_ATOMICITY_004: reconstructed value has no proposal/confirmation state.

SEPARATE_GENERATION_VALIDATION_GATE

Перевіряє: чи validation працює з готовим результатом окремо від generation.

Блокує: той самий крок генерує, вивіряє і ставить passed.

Діагностика: STEP_ATOMICITY_005: generation and validation share one uncontrolled action.

EXPLICIT_CONFIRMATION_GATE

Перевіряє: чи підтвердження потрібних рішень зафіксоване явно.

Блокує: confirmed state отримано без approval evidence.

Діагностика: STEP_ATOMICITY_006: confirmed transition has no explicit approval.

NO_PARTIAL_SUCCESS_TRANSITION_GATE

Перевіряє: чи next step залежить від повних completion criteria.

Блокує: перехід запускається за однією успішною підумовою.

Діагностика: STEP_ATOMICITY_007: transition triggered by partial success.

FAILURE_LOCALIZATION_GATE

Перевіряє: чи failure route веде до найближчої операції, здатної виправити помилку.

Блокує: локальна помилка перезапускає весь процес.

Діагностика: STEP_ATOMICITY_008: failure route is broader than affected responsibility.

ARTIFACT_LEVEL_STATUS_GATE

Перевіряє: чи кожен незалежний artifact має власний статус.

Блокує: package-level passed приховує failed або unvalidated artifact.

Діагностика: STEP_ATOMICITY_009: aggregate status masks artifact failure.

10. Статуси та помилки

Один загальний статус недостатній для процесу, де різні етапи можуть мати різний стан.

```
input_status:
decision_status:
generation_status:
validation_status:
approval_status:
execution_status:
overall_status:
```

Умовний приклад різних статусів

Неправильно:

```
status: passed
```

При цьому документ створено, quality review не виконано, execution заблоковано, підтвердження відсутнє.

Правильно:

```
input_status: confirmed
design_status: completed
generation_status: completed
```

```
validation_status: not_started
approval_status: pending
execution_status: blocked
overall_status: awaiting_validation
```

Інший варіант:

```
generation_status: completed
validation_status: failed
approval_status: blocked
execution_status: not_started
overall_status: failed_quality_gate
```

Статус показує, де саме перебуває процес. Файл створено не означає файл якісний, а файл якісний не означає файл підтверджено.

Частковий успіх повинен бути видимим. `overall_status` агрегує, але не замінює деталізовані статуси.

11. Зв'язок із роботою мовних моделей

Мовні моделі мають кілька властивостей, які роблять атомарність особливо важливою.

Схильність стискати повторювані кроки

Коли багато сценаріїв мають схожу структуру, модель може об'єднати їх у загальну фразу. Проміжні artifacts і окремі gates зберігають конкретику.

Перетворення специфікації на опис

Модель може замінити точні значення словами «відповідний», «релевантний» або «очікуваний». Atomic output contract дозволяє перевірити, чи присутня безпечна конкретика.

Домислювання відсутнього

Модель часто прагне завершити задачу. Окрема reconstruction branch робить припущення видимим і не дозволяє перетворити його на факт без підтвердження.

Правдоподібне не означає підтверджене

Логічно ймовірне значення може бути неправильним для конкретного процесу. Тому evidence collection відокремлюється від decision evaluation.

Самооцінка за наміром

Модель пам'ятає, що хотіла додати секцію, і може вважати її присутньою. Independent post-render review примушує перевірити фактичний текст або файл.

Непомітний перехід до наступної задачі

Після генерації модель може почати виконувати наступний етап без explicit state transition. Completion criteria і next-step gate зупиняють такий перехід.

Один великий prompt втрачає локальні обмеження

Чим більше різних відповідальностей у prompt, тим вища ймовірність, що локальна умова загубиться серед глобальних інструкцій. Декомпозиція скорочує активний контекст кожного кроку.

12. Практичний шаблон проєктування процесу

1. Назвіть кінцеві результати.
2. Для кожного результату визначте окрему відповідальність.
3. Відділіть збір фактів від реконструкції.
4. Відділіть рішення від матеріалізації.
5. Відділіть generation від validation.
6. Додайте completion criteria.
7. Додайте локальний failure route.
8. Додайте confirmation point лише там, де змінюється рішення або стан.
9. Забороніть перехід за часткового успіху.
10. Збирайте сукупний результат лише з індивідуально готових частин.

Умовний приклад локального failure route

Неправильно: фінальна перевірка знайшла відсутній rollback, і процес повернувся на початок, повторно збираючи вже підтверджені дані.

Правильно:

```
POST_RENDER_REVIEW
  ↓ failed: rollback section missing
ROLLBACK_SPEC_RESOLUTION
  ↓
DOCUMENT_REGENERATION_FOR_AFFECTED_SECTION
  ↓
POST_RENDER_REVIEW
```

Не повторюються:

- збір фактів;
- підтвердження контракту;
- вибір сценаріїв;
- уже пройдені незалежні артефакти.

Failure route повертає до найближчого кроку, який може виправити помилку, а не до початку процесу.

13. Підсумкова таблиця

Ситуація	Що змішано	Основний ризик	Правильне розділення	Gate
Підготовка документа з відсутнім полем	facts, reconstruction, design, generation, approval	припущення стає фактом	Collect → Resolve → Confirm → Generate → Validate →	NO_HIDDEN_RE CONSTRUCTION _GATE

Ситуація	Що змішано	Основний ризик	Правильне розділення	Gate
План тестів вважається runbook	design і materialization	невиконуваний документ	Approve Design → execution specification → rendered guide → review	EXPLICIT_INPUT_OUTPUT_GATE
Відсутній запис реконструюється всередині обробки	read і create	немає evidence	separate reconstruction branch	NO_HIDDEN_RECONSTRUCTION_GATE
Один крок створює кілька документів	незалежні outputs	слабкий файл прихований пакетом	One Artifact at a Time	SINGLE_OUTPUT_GATE
Модель перевіряє власний план	intention і result	false green	Independent Post-Render Review	SEPARATE_GENERATION_VALIDATION_GATE
Дані отримані, але не валідні	partial success і transition	передчасна генерація	full completion criteria	NO_PARTIAL_SUCCESS_TRANSITION_GATE
Загальні фрази замість конкретики	design і execution	semantic placeholders	executable specification	EXPLICIT_INPUT_OUTPUT_GATE
Збір фактів одразу визначає eligibility	evidence і decision	упереджений evidence	Evidence → Evaluation → Approval	SINGLE_RESPONSIBILITY_GATE
Форматування, збереження, checksum	механічні дії одного output	низький ризик	допустиме об'єднання	SINGLE_OUTPUT_GATE
Локальна помилка перезапускає все	failure route і global restart	втрата часу та drift	nearest corrective step	FAILURE_LOCALIZATION_GATE
Один passed для всього	detailed statuses і aggregate status	маскування незавершеності	separate stage statuses	ARTIFACT_LEVEL_STATUS_GATE
Генерація автоматично підтверджує	generation і state transition	confirmed без approval	Generate → Validate → Approve → Transition	EXPLICIT_CONFIRMATION_GATE

Підсумкове правило

Не поєднуйте в одному кроці дії, які можуть мати різні результати, різні критерії якості, різні маршрути помилки або різні потреби в підтвердженні.

Практична формула:

Спочатку створіть один спостережуваний результат. Потім окремо перевірте його. Лише після цього переходьте далі.

Розділ 6.2. Мікропромпти як модульна архітектура процесу

Мікропромпт часто сприймають як коротшу версію великого prompt-а. У цій книзі він розглядається інакше: як **локальна одиниця поведінки**, яку можна окремо розвивати, тестувати, версіювати, замінювати та відкочувати без перебудови всього процесу.

Головна ідея проста:

Процес має еволюціонувати як композиція незалежних поведінкових модулів, а не як одна велика інструкція.

Це не нормативна вимога мови і не обов'язковий синтаксичний елемент. Це практична архітектурна рекомендація для процесів, у яких мовна модель виконує багато різних дій.

1. Від великої інструкції до композиції поведінки

Монолітний підхід

У монолітному підході весь процес описаний одним великим текстом:

```
Одна велика інструкція
↓
Будь-яка локальна зміна
↓
Потрібно перечитати і перевірити майже весь процес
↓
Зростає ризик випадково змінити інші частини поведінки
```

Такий текст може містити правила збору фактів, реконструкції, генерації, перевірки, підтвердження, маршрути помилок і формат фінальної відповіді. Навіть якщо змінюється лише одне правило, воно знаходиться всередині спільного контексту з усіма іншими правилами.

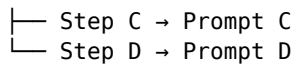
Мовна модель не виконує текст як ізольований набір функцій. Вона інтерпретує його як єдиний контекст. Через це локальна правка може:

- змінити пріоритет сусідніх інструкцій;
- послабити обмеження іншого кроку;
- додати нове припущення там, де його не було;
- змінити стиль або повноту результатів інших вузлів;
- створити суперечність між початком і кінцем prompt-а;
- збільшити ризик того, що частина правил загубиться в довгому контексті.

Модульний підхід

У модульному підході дерево процесу залишається окремою структурою, а кожен вузол посиляється на власну поведінкову інструкцію:

```
Decision Tree
|
├─ Step A → Prompt A
└─ Step B → Prompt B
```



Якщо потрібно покращити поведінку Step C, змінюється Prompt C. Prompt A, Prompt B і Prompt D не переписуються без окремої причини.

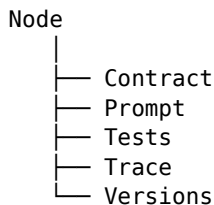
Це не означає, що інші частини процесу автоматично гарантовано не зміняться. Але область потенційного впливу стає меншою, видимішою і легшою для перевірки.

2. Що таке мікропромт у процесній архітектурі

Мікропромт - це не просто короткий текст. Це локальний поведінковий модуль, який:

- обслуговує одну визначену відповідальність;
- приймає конкретні входи;
- створює конкретний вихід;
- працює в межах контракту вузла;
- має власні критерії якості;
- може мати власні тести;
- може мати незалежну версію;
- залишає спостережуваний слід виконання.

Корисно розділяти сам вузол і мікропромт, який реалізує його поведінку:



Вузол визначає, **що має бути виконано**. Мікропромт описує, **як модель має виконати цю локальну поведінку** в межах дозволеного контракту.

Тому зміна тексту мікропромта не повинна непомітно змінювати:

- призначення вузла;
- тип його входу;
- форму його виходу;
- completion criteria;
- дозволені переходи;
- підтверджений стан;
- failure route.

Якщо змінюється хоча б один із цих пунктів, це вже не лише локальний рефакторинг мікропромта. Це зміна контракту або архітектури процесу.

3. Аналогія з програмною інженерією

Мікропромти близькі не до окремих абзаців документації, а до модулів програмної системи.

Функція

Функція має параметри, виконує одну операцію і повертає результат. Мікропромпт також має отримувати визначений контекст і створювати один очікуваний результат.

Модуль або бібліотека

Модуль приховує внутрішню реалізацію за стабільним інтерфейсом. Так само текст мікропромпта може змінюватися, поки зберігається контракт вузла.

Клас

Клас поєднує поведінку з визначеною відповідальністю. Мікропромпт теж має бути прив'язаний до конкретної ролі, а не намагатися керувати всім процесом одразу.

Dependency injection

Процес може посылатися не на жорстко вбудований текст, а на конкретну версію prompt-модуля. Це дозволяє підставляти іншу реалізацію без зміни дерева процесу.

Інтерфейс і ABI/API-контракт

Найважливіша аналогія - стабільний інтерфейс. Безпечна локальна еволюція можлива лише тоді, коли нова версія мікропромпта не порушує контракт взаємодії з сусідніми вузлами.

Наприклад:

```
node_contract:
  id: reconstruction_step
  input:
    confirmed_facts: list
    open_gaps: list
  output:
    proposed_values: list
    unresolved_gaps: list
    evidence_links: list
  completion_criteria:
    - every proposal has evidence
    - every gap is resolved or explicitly open
```

Реалізація prompt-а може змінюватися. Але якщо нова версія перестає повертати evidence_links, це вже порушення інтерфейсу.

Локальний рефакторинг

Рефакторинг покращує внутрішню реалізацію без зміни зовнішньої поведінки. Аналогічно можна:

- зробити інструкцію точнішою;
- прибрати двозначність;
- додати кращий порядок аналізу;
- уточнити формат локальної відповіді;
- додати захист від типової помилки;

але не змінювати контракт вузла без окремого рішення.

4. Основні переваги

4.1. Локальність змін

Покращення одного кроку відбувається в одному місці. Автору не потрібно шукати всі згадки про цю поведінку у великому prompt-i.

Критерій локальності:

Зміна поведінки одного вузла не вимагає редагування текстів інших вузлів, якщо їхні контракти не змінилися.

4.2. Менший regression scope

Монолітна інструкція створює широкий regression scope: навіть маленька правка може вплинути на весь контекст.

Моноліт
↓
одна локальна зміна
↓
майже повна повторна перевірка процесу

Мікропромпти дозволяють звужити область перевірки:

Мікропромт вузла C
↓
локальна зміна
↓
тести вузла C
↓
контрактні тести зв'язків C → наступні вузли
↓
обмежений end-to-end regression

Цифровий відсоток тут не є універсальним. Важливий сам принцип: що стабільніші контракти між вузлами, то меншу частину процесу потрібно повторно перевіряти після локальної зміни.

4.3. Незалежне версіонування

Кожен мікропромт може мати власну історію:

```
node:
  id: document_review
  contract_version: 1.0.0
  prompt:
    id: review_prompt
    version: 1.4.2
  approved_regression_suite: review-suite-7
  rollback_version: 1.4.1
```

Важливо розділяти:

11. версію контракту вузла;
12. версію тексту prompt-a;
13. версію тестів;

14. версію процесу, який використовує цей prompt.

Зміна prompt-а не завжди потребує зміни контракту. Але зміна контракту завжди потребує ширшої перевірки залежностей.

4.4. Точкові експерименти

Нову версію можна перевіряти лише на одному вузлі:

- A/B comparison двох версій;
- replay того самого входу;
- порівняння outputs;
- оцінка конкретної метрики;
- перевірка на локальному наборі edge cases.

4.5. Простіший review

Рецензент бачить:

- який вузол змінено;
- чому змінено;
- який контракт збережено;
- які тести додано;
- який regression score перевірено.

Це значно надійніше, ніж переглядати різницю у великій інструкції, де локальна зміна розчиняється серед десятків інших правил.

4.6. Простий rollback

Якщо нова версія погіршила результат, можна повернути попередню версію одного prompt-модуля без відкату всього процесу.

Rollback безпечний лише тоді, коли попередня версія сумісна з поточним контрактом вузла.

4.7. Кращий execution trace

Trace має фіксувати не тільки назву вузла, а й фактичну версію поведінки:

```
trace_event:
  node_id: reconstruction_step
  contract_version: 1.0.0
  prompt_id: reconstruction_prompt
  prompt_version: 1.4.2
  input_artifact: evidence-104
  output_artifact: proposal-105
  validation_status: passed
```

Це дозволяє:

- повторити запуск з тією самою версією;
- порівняти результати різних версій;

- локалізувати регресію;
- встановити, яка саме поведінкова реалізація вплинула на результат;
- відтворити історичне рішення.

4.8. Локальне накопичення знань

У моноліті нові знання часто додаються в загальний текст. Через деякий час важко зрозуміти, до якого кроку належить конкретне правило.

У модульній архітектурі досвід зберігається поруч із вузлом:

```
Node C
├── prompt.md
├── contract.yaml
├── examples/
├── tests/
├── regressions/
├── known_failures.md
└── changelog.md
```

Помилка в конкретному вузлі породжує:

- локальну зміну prompt-a;
- локальний regression test;
- локальну примітку про відому проблему;
- локальне пояснення рішення.

Знання не розмазується по великому документу.

5. Абстрактний приклад: реконструкція як локальний модуль

Уявімо процес із чотирьох кроків:

```
Збір фактів
→ Реконструкція
→ Генерація документа
→ Перевірка
```

Варіант 1. Монолітна інструкція

Уся поведінка описана одним prompt-ом. Після практичного використання виявилось, що правила реконструкції надто легко приймають правдоподібне значення за підтверджений факт.

Щоб виправити проблему, автор редагує великий текст. Після цього потрібно перевірити:

- чи не змінився збір фактів;
- чи не змінився формат reconstruction output;
- чи не почала генерація документа пропускати open gaps;
- чи не змінилась логіка review;
- чи зберігся порядок переходів.

Локальна проблема створила майже повний regression scope.

Варіант 2. Окремий мікропромт реконструкції

Процес зберігає стабільну структуру:

FACT_COLLECTION → RECONSTRUCTION → DOCUMENT_GENERATION → REVIEW

Вузол RECONSTRUCTION має окремий prompt. У ньому змінюють лише правила:

- кожне proposed value має посилання на evidence;
- правдоподібність не дорівнює confirmed;
- невпевнене значення залишається open gap;
- заборонено змінювати confirmed facts;
- output зберігає provenance кожної реконструкції.

Після зміни виконуються:

1. unit-like тести мікропромта реконструкції;
2. контрактний тест його output;
3. перевірка переходу RECONSTRUCTION → DOCUMENT_GENERATION;
4. короткий end-to-end regression для всього процесу.

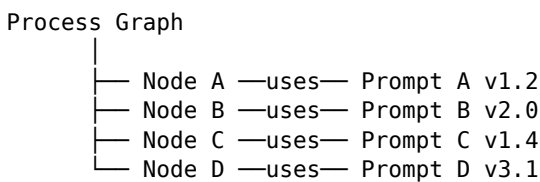
Інші prompt-модулі не переписуються.

Критерій успіху:

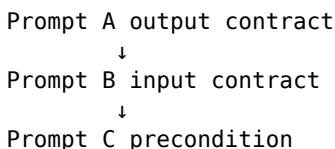
Нова версія зменшує помилки реконструкції, зберігає контракт вузла і не змінює поведінку сусідніх кроків поза визначеним regression score.

6. Prompt Dependency Graph

Окремі prompt-и не існують у вакуумі. Вони утворюють граф залежностей.



Але важливіші не лише зв'язки node → prompt. Треба бачити контрактні залежності:



Тому реєстр мікропромтів має відповідати щонайменше на питання:

- який вузол використовує prompt;
- яка версія активна;
- який контракт вона реалізує;
- які вузли залежать від її output;
- які тести захищають зміну;
- яка версія дозволена для rollback.

- у яких процесах вона використовується повторно.

Умовний реєстр:

```
prompt_registry:
- prompt_id: reconstruction_prompt
  version: 1.4.2
  implements_contract: reconstruction_contract@1.0.0
  used_by:
    - process_a.reconstruction
    - process_b.missing_data_resolution
  tests:
    - reconstruction_unit_suite
    - reconstruction_contract_suite
  rollback_to: 1.4.1
```

7. Контрактна сумісність і межа локальної зміни

Мікропромпти зменшують ризик, але не усувають його.

Локальне оновлення достатнє, якщо:

- призначення вузла не змінюється;
- input schema не змінюється;
- output schema не змінюється;
- semantics статусів не змінюється;
- completion criteria не змінюються;
- failure route не змінюється;
- prompt не отримує нове право змінювати confirmed state.

Локального оновлення недостатньо, якщо:

- вузол починає приймати нове бізнес-рішення;
- змінюється тип output;
- змінюється значення status-a;
- з'являється нова обов'язкова залежність;
- prompt починає виконувати роботу сусіднього вузла;
- змінюється порядок процесу;
- змінюється контракт між вузлами.

У такому випадку потрібні:

- нова версія контракту;
- аналіз Prompt Dependency Graph;
- ширший regression scope;
- оновлення trace semantics;
- можлива міграція процесів, які використовують старий контракт.

8. Патерни архітектури мікропромтів

Pattern: One Node - One Behavioral Module

Коли застосовувати: коли вузол має одну чітку відповідальність.

Структура:

```
Node contract
→ one micro-prompt
→ one observable output
→ local validation
```

Перевага: зміна поведінки легко локалізується.

Ризик: prompt може непомітно розростися і почати виконувати кілька задач.

Контроль: перевіряти його разом із критеріями атомарності кроку.

Pattern: Prompt Behind a Stable Contract

Коли застосовувати: коли поведінка часто вдосконалюється, але інтерфейс має залишатися стабільним.

```
Stable node contract
|
├── Prompt v1.2
├── Prompt v1.3
└── Prompt v1.4
```

Критерій: кожна версія проходить однаковий contract suite.

Pattern: Prompt Registry

Коли застосовувати: коли prompt-и повторно використовуються або мають кілька версій.

Реєстр відділяє посилання на поведінку від тексту процесу. Процес указує prompt_id і version policy, а сам prompt зберігається окремо.

Pattern: Contract Test Before End-to-End Test

Спочатку перевіряється локальна відповідність контракту, а лише потім весь процес.

```
prompt test
→ output schema test
→ semantic contract test
→ neighboring-node integration
→ selected end-to-end regression
```

Pattern: Traceable Prompt Resolution

Перед виконанням runtime або модель повинні зафіксувати, яку версію prompt-а фактично обрано.

Це захищає від ситуації, коли процес посиляється на latest, а історичний запуск неможливо відтворити.

Pattern: Local Failure Knowledge

Кожна знайдена помилка пов'язується з конкретним prompt-модулем, тестом і версією виправлення.

```
issue:
  id: ISSUE-17
  affected_prompt: reconstruction_prompt@1.4.1
  failure_class: unsupported_inference
  regression_test: REC-CASE-09
  fixed_in: 1.4.2
```

Pattern: Approved Promotion

Нова версія не стає активною одразу після редагування.

```
draft
→ local tests passed
→ contract tests passed
→ regression passed
→ review approved
→ promoted
```

9. Ризики та антипатерни

Antipattern: Micro-Prompt in Name Only

Prompt названий мікропромптом, але містить збір даних, рішення, генерацію, review і state transition.

Ознака: зміна однієї частини вимагає перевіряти все всередині prompt-a.

Виправлення: спочатку розділити відповідальності на рівні процесу.

Antipattern: Hidden Contract in Prompt Text

Контракт існує лише як кілька речень усередині prompt-a.

Ризик: нова версія випадково змінює output або статуси.

Виправлення: винести contract у явну структуру і тестувати окремо.

Antipattern: Floating Latest

Процес посилається на «останню» версію без фіксації фактичного resolution.

Ризик: два однакові запуски використовують різну поведінку, а replay стає недостовірним.

Виправлення: фіксувати resolved version в execution trace.

Antipattern: Shared Prompt with Divergent Responsibilities

Один prompt повторно використовується у вузлах, які мають схожі назви, але різні контракти.

Ризик: покращення для одного процесу погіршує інший.

Виправлення: спільне ядро плюс окремі adapters або різні prompt-и.

Antipattern: Prompt Change Without Regression Artifact

Текст змінено, але немає прикладу, тесту або trace, який показує, яку проблему виправлено.

Виправлення: кожне змістовне покращення має додавати або оновлювати локальний regression case.

Antipattern: Prompt Owns the State Transition

Мікропромпт не тільки створює output, а й сам вирішує, що процес тепер confirmed.

Ризик: локальна поведінка отримує право змінювати глобальний стан.

Виправлення: state transition залишається окремою операцією з передумовами.

Antipattern: Over-Fragmentation

Кожне речення винесене в окремий prompt, хоча дії утворюють одну механічну операцію.

Ризик: надмірна кількість залежностей, складне керування версіями і зайвий orchestration overhead.

Виправлення: межа prompt-модуля має збігатися з реальною межею відповідальності, а не з довжиною тексту.

10. Версіонування та життєвий цикл

Рекомендований мінімальний запис версії:

```
micro_prompt:
  id: document_review_prompt
  version: 2.3.0
  implements_contract: document_review@1.1.0
  status: approved
  created_from: 2.2.1
  change_reason: detect missing concrete examples
  regression_suite: review-regression-12
  rollback_to: 2.2.1
```

Практичне правило зміни версій:

- **patch** - уточнення формулювання без зміни контракту;
- **minor** - нова сумісна поведінка або додатковий локальний захист;
- **major** - несумісна зміна, яка потребує нового контракту або міграції залежностей.

Це не жорсткий стандарт, а корисна дисципліна. Головне - щоб історія дозволяла відповісти:

- що змінилося;
- навіщо;

- яка проблема це виправляє;
- які тести доводять покращення;
- як повернутися назад;
- які процеси використовують цю версію.

11. Тестування мікропромтів

Локальні приклади

Набір коротких inputs, що перевіряють основну поведінку вузла.

Edge cases

Випадки з прогалинами, суперечностями, неоднозначністю, відсутністю evidence або неправильним форматом.

Contract tests

Перевіряють не красу тексту, а дотримання інтерфейсу:

- усі обов'язкові поля присутні;
- статуси мають дозволені значення;
- output не містить заборонених state transitions;
- кожне рішення має evidence;
- open gaps не приховані.

Metamorphic tests

Перевіряють інваріанти. Наприклад, зміна порядку незалежних фактів не повинна змінювати рішення.

Replay tests

Той самий input виконується на старій і новій версіях. Результати порівнюються за визначеними критеріями.

Neighbor integration tests

Перевіряється, чи може наступний вузол коректно спожити output нової версії.

Selected end-to-end regression

Навіть за стабільного контракту потрібен обмежений наскрізний тест. Мовні моделі можуть створювати побічні зміни, які формальна схема не бачить.

12. Зв'язок з іншими принципами книги

Атомарність кроків

Мікропромт має сенс лише тоді, коли сам вузол атомарний. Якщо вузол змішує кілька відповідальностей, окремий prompt лише ховає моноліт під новою назвою.

Єдина відповідальність

Один prompt-модуль повинен відповідати за один тип поведінки і один основний результат.

Явні контракти

Контракт відділяє стабільну зовнішню поведінку від змінної внутрішньої реалізації.

Локальні failure routes

Помилка конкретного prompt-а має повертати до найближчого вузла, який може її виправити, а не перезапускати весь процес.

Execution trace

Trace фіксує node, prompt ID, prompt version, contract version, input, output і validation evidence.

Незалежна валідація

Prompt не повинен сам оголошувати власний результат правильним. Валідація залишається окремою операцією або окремим вузлом.

Артефактний підхід

Input і output мікропромпта краще оформлювати як спостережувані проміжні артефакти. Це робить поведінку тестованою і відтворюваною.

13. Практичний checklist

Перед виділенням окремого мікропромпта перевірте:

1. Чи має вузол одну відповідальність?
2. Чи існує один основний observable output?
3. Чи визначені input і output contracts?
4. Чи можна змінити внутрішню інструкцію без зміни процесу?
5. Чи має prompt власні тести?
6. Чи фіксується його версія в trace?
7. Чи існує локальний rollback?
8. Чи зрозумілий regression scope?
9. Чи не змінює prompt confirmed state самотійно?
10. Чи не виконує він приховану роботу сусіднього вузла?
11. Чи не використовується один prompt для несумісних контрактів?
12. Чи є причина для нового prompt-а сильнішою за просте бажання скоротити текст?

Перед локальним оновленням перевірте:

- контракт вузла не змінився;

- залежні inputs/outputs сумісні;
- додано regression case;
- локальні тести пройдені;
- contract tests пройдені;
- selected end-to-end regression пройдений;
- resolved version буде записана в execution trace;
- rollback version визначена.

14. Підсумкова таблиця

Ситуація	Ризик	Правильна дія	Область перевірки
Уточнено формулювання одного локального правила	Побічна зміна поведінки вузла	Нова patch-версія мікропромпта	Локальні та контрактні тести
Додано сумісний захист від edge case	Регресія на старих входах	Нова minor-версія і regression case	Вузол, сусідня інтеграція, selected end-to-end
Змінено output schema	Порушення наступних вузлів	Нова версія контракту	Dependency graph і широка інтеграційна перевірка
Один prompt використовується різними контрактами	Непередбачувані побічні ефекти	Розділити реалізації або додати adapters	Усі процеси-споживачі
Нова версія погіршила результат	Поширення дефекту	Локальний rollback	Перевірка сумісності rollback-версії
Trace не містить версії prompt-a	Неможливий достовірний replay	Фіксувати resolved prompt version	Trace validation
Prompt став виконувати кілька відповідальностей	Прихований моноліт	Декомпонувати вузол	Atomicity review

15. Підсумкове правило

Чим менша і чіткіша область відповідальності мікропромпта, тим простіше його покращувати, тестувати, відкочувати та повторно використовувати.

І ще одне архітектурне узагальнення:

Стабільний процес повинен залежати від контрактів поведінкових модулів, а не від випадкової форми великої інструкції.

Мікропромпти не усувають необхідність системного тестування. Вони роблять зміни локальнішими, залежності видимішими, regression score меншим, а еволюцію процесу - керованою.

Розділ 7. Output: що модель має створити

7.1. Навіщо Ordo окремо описує Output

У багатьох prompt-ах результат описаний нечітко.

Користувач пише:

Підготуй мені пакет документів.

Але що саме входить у пакет?

Один документ? Три файли? Markdown? JSON? YAML? PDF? Чернетка в чаті? Архів? Звіт про перевірку?

Якщо це не визначити, модель починає вирішувати сама. Іноді це працює. Але в складних процесах така свобода небезпечна.

У Ordo результат має бути описаний явно.



Для цього використовується OUTPUT.DEF.

OUTPUT.DEF відповідає на питання:

Що саме модель має створити?

Не в загальних словах, а як перевірюваний контракт.

7.2. Output — це не тільки текстова відповідь

У простому діалозі output — це відповідь у чаті.

Наприклад:

Ось короткий підсумок.

Але в Ordo output може бути різним:

документ;
набір документів;
JSON-звіт;
YAML-специфікація;
таблиця;
лист;
Jira-задача;
QA runbook;

```
automation spec;
архів;
короткий handoff-блок;
статус blocked із поясненням.
```

Тому output треба описувати як частину contract.

Приклад:

```
{
  "op": "OUTPUT.DEF",
  "id": "OUT_HISTORY_EVENT_COMPACT_PACKAGE",
  "kind": "file_set",
  "required_files": [
    "README.md",
    "SUMMARY.json",
    "VALIDATION_REPORT.json",
    "CONSISTENCY_CHECK_REPORT.json",
    "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md",
    "02_JIRA_TASK_<ALIAS>.md",
    "04_IMPLEMENTATION_PROMPT_<ALIAS>.md",
    "05_QA_PACKAGE_<ALIAS>.md",
    "07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md",
    "08_QA_AUTOMATION_SPEC_<ALIAS>.yaml",
    "09_QA_AUTOMATION_README_<ALIAS>.md"
  ]
}
```

Тепер модель не може вирішити, що достатньо одного короткого walkthrough.

Output contract каже: потрібен саме набір із визначених файлів.

7.3. Output має бути пов'язаний із terminal path

У процесі з деревом рішень різні terminal paths можуть мати різні outputs.

Наприклад, у support-процесі:

```
incident ready → INCIDENT_REPORT.md + CUSTOMER_REPLY.md
incident needs triage → INCIDENT_Triage_NOTE.md
change ready → CHANGE_BRIEF.md + ACCEPTANCE_CHECKLIST.md
change unclear → CHANGE_CLARIFICATION_NOTE.md
```

Це означає, що output не завжди один і той самий. Він залежить від обраного шляху.

В Ordo це можна описати так:

```
terminal_outputs:
  T_INCIDENT_READY:
    outputs:
      - INCIDENT_REPORT.md
      - CUSTOMER_REPLY.md

  T_INCIDENT_NEEDS_TRIAGE:
    outputs:
      - INCIDENT_Triage_NOTE.md

  T_CHANGE_READY:
    outputs:
      - CHANGE_BRIEF.md
      - ACCEPTANCE_CHECKLIST.md

  T_CHANGE_NEEDS_CLARIFICATION:
```

```
outputs:
  - CHANGE_CLARIFICATION_NOTE.md
```

Це захищає процес від помилки, коли модель створює неправильний документ для правильного шляху.

7.4. Output може бути дозволений або заблокований

Не кожен output можна створювати в будь-який момент.

Наприклад, фінальний archive не можна створювати, якщо contracts не підтверджені.

Тому output має мати умови готовності.

Приклад:

```
{
  "op": "OUTPUT.DEF",
  "id": "OUT_FINAL_ARCHIVE",
  "kind": "zip_archive",
  "allowed_when": [
    "path_confirmed",
    "mandatory_contracts_confirmed",
    "required_documents_approved",
    "validation_status_go"
  ],
  "blocked_when": [
    "unresolved_mandatory_contract",
    "proposed_contract_used_as_confirmed",
    "approval_missing",
    "validation_status_no_go"
  ]
}
```



Це важливо: output — це не тільки “що створити”, а й “коли це дозволено створити”.

7.5. Draft output і final output

У реальному житті модель часто має створити спочатку чернетку.

Наприклад:

```
спочатку draft passport у чаті;
потім користувач погоджує;
потім draft Jira task;
потім погодження;
потім QA package;
потім automation spec;
і тільки після approvals – фінальний archive.
```

Це означає, що є різні рівні output:

```
draft output;
review output;
approved output;
final output.
```

У Ordo це треба описувати явно.

Наприклад:

```
outputs:
  passport_draft:
    file: "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    status: draft
    requires_approval: true

  passport_final:
    file: "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    status: approved
    allowed_when:
      - passport_approval_passed
```

Без цього модель може переплутати чернетку з фінальним документом.

7.6. Output і Template

У багатьох Ordo-процесах output створюється не з нуля, а за шаблоном.

Наприклад:

```
Jira task має мати Summary, Context, Acceptance criteria, Out of scope, QA reference data.
```

Тоді одного OUTPUT.DEF недостатньо. Потрібен зв'язок із template.

Це вже конструкція профілю:

```
TEMPLATE.BIND
```

Приклад:

```
{
  "op": "TEMPLATE.BIND",
  "output": "02_JIRA_TASK_<ALIAS>.md",
  "template": "templates/TEMPLATE_JIRA_TASK.md",
  "required_sections": [
    "Summary",
    "Context",
    "Acceptance criteria",
    "Out of scope",
    "QA reference data",
    "Test coverage"
  ]
}
```

OUTPUT.DEF каже, що файл має існувати.

TEMPLATE.BIND каже, за якою структурою його створити.

7.7. Output має бути перевірюваним

Якщо output не можна перевірити, модель може сказати, що все готово, хоча це не так.

Поганий output contract:

Створи хороший QA package.

Що таке “хороший”? Для моделі це може бути короткий summary. Для тестувальника — повний покроковий runbook.

Кращий output contract:

```
output:
  file: "05_QA_PACKAGE_<ALIAS>.md"
  required_for_each_executable_tc:
    - goal
    - preconditions
    - source_lookup_before_action
    - preflight_restore
    - rest_action
    - source_lookup_after_action
    - changerecord_lookup_or_expected_absence
    - history_processing_step
    - history_event_lookup_or_expected_absence
    - change_errors_lookup
    - rollback
    - post_rollback_source_lookup
    - expected_result
    - diagnostics
```

Тепер output можна перевірити.

Модель не може обмежитися фразою “дивись загальний flow”.

7.8. Output може бути не створений — і це теж результат

Іноді правильний результат — не створити документ.

Наприклад, якщо mandatory contracts не підтверджені, правильна поведінка не “вигадати пакет”, а зупинитися.

Тоді output може бути таким:

```
{
  "op": "OUTPUT.DEF",
  "id": "OUT_BLOCKED_STATUS",
  "kind": "handoff_status",
  "status": "blocked_requires_confirmation",
  "include": [
    "missing_contracts",
    "open_questions",
    "next_required_action"
  ]
}
```



Це важлива ідея.

У Ordo результатом може бути не тільки готовий artifact, а й чесний blocked handoff.

Краще чесно сказати:

Пакет не можна фіналізувати, бо не підтверджено `HistoryEvent.item.values`.

ніж створити красивий, але неправильний документ.

7.9. Output і Handoff

Output — це те, що створюється.

Handoff — це те, як це передається далі.

Наприклад, output може бути набором файлів, а handoff — коротким повідомленням:

Статус: `ready_for_review`
 Створено: `passport draft`, `Jira draft`, `QA draft`
 Блокери: `automation runner contract not confirmed`
 Наступна дія: `погодити QA package`

У Ordo ці речі краще не змішувати.

`OUTPUT.DEF` описує artifact.

`HANDOFF.EMIT` описує передачу статусу користувачу або наступному процесу.

Приклад:

```
{
  "op": "HANDOFF.EMIT",
  "include": [
    "status",
    "created_outputs",
    "gate_report",
    "open_questions",
    "next_action"
  ]
}
```

7.10. Типові помилки з Output

Перша помилка — описати output занадто загально.

Зроби пакет.

Друга помилка — не сказати, скільки файлів має бути.

Третя помилка — не відрізнити draft від final.

Четверта помилка — не прив'язати output до template.

П'ята помилка — не описати readiness conditions.

Шоста помилка — вважати, що якщо модель щось написала, то output уже valid.

Сьома помилка — не передбачити blocked output.

У складних процесах правильне “не можу фіналізувати, бо...” часто цінніше за неправильне “готово”.

7.11. Короткий підсумок розділу

OUTPUT.DEF визначає, що саме модель має створити.

Output може бути:

```
відповіддю в чаті;
документом;
набором документів;
JSON-звітом;
YAML-специфікацією;
архівом;
blocked handoff status.
```

Хороший output contract має відповідати на питання:

```
що створюємо;
у якому форматі;
за яким шаблоном;
коли це дозволено;
що блокує створення;
як перевірити результат;
чи це draft, review або final.
```

Головний принцип:

```
результат має бути не просто красивим, а дозволеним, структурованим і перевірюваним.
```

Міні-вправа

Візьміть задачу:

```
Модель має підготувати документ для передачі розробнику.
```

Опишіть OUTPUT.DEF:

```
output:
  id: OUT_DEVELOPER_HANDOFF
  kind: document
  file_name: "IMPLEMENTATION_PROMPT.md"
  required_sections:
    - goal
    - context
    - files_to_check
    - required_changes
    - what_not_to_change
    - tests
    - acceptance_criteria
  status: draft
  allowed_when:
    - business_contract_confirmed
  blocked_when:
    - missing_acceptance_criteria
    - unresolved_scope
```

Потім дайте відповідь:

1. Які секції є mandatory?
 2. Що блокує створення final version?
 3. Який template потрібен?
 4. Як перевірити, що rendered document не порожній?
-

Розділ 8. Пишемо першу Ordo Source-програму

8.1. Навіщо починати з Ordo Source

До цього моменту ми говорили про Ordo як про спосіб мислення: є мета, контракт, стан, вузли, відповіді, результати й перевірки. Але рано чи пізно це потрібно записати у форму, з якою може працювати модель або майбутній runtime.

Для цього існує Ordo Source.



Ordo Source — це людсько-читабельний опис програми поведінки моделі. Його має розуміти людина, яка створює процес. Він не обов'язково є найкращим форматом для виконання моделлю, але він зручний для написання, обговорення, рев'ю і зміни.

Можна провести аналогію з класичним програмуванням:

```
людина пише source code
компілятор перетворює його у форму для виконання
машина виконує скомпільовану форму
```

В Ordo ідея схожа:

```
людина пише Ordo Source
Ordo compiler / translator перетворює його в Ordo IR
модель або runtime виконує Ordo IR
```

На ранніх етапах ми можемо не мати справжнього compiler-а. Але навіть ручне написання Ordo Source уже допомагає структурувати інструкцію набагато краще, ніж звичайний prompt.

8.2. Якою має бути перша програма

Першу Ordo-програму краще робити дуже простою.

Не треба одразу брати великий playbook із десятками gates і шаблонів. Почнемо з задачі, яку легко зрозуміти:

```
Підсумуй текст українською у максимум 3 пункти.
Не додавай фактів, яких немає у вхідному тексті.
Якщо тексту недостатньо, скажи про це.
```

Це хороший перший приклад, тому що тут уже є всі базові елементи Ordo:

```
мета;
вхідні дані;
контракт результату;
правила;
перевірки;
fallback-поведінка;
результат.
```

Звичайний prompt виглядав би так:

Підсумуй наведений текст українською у максимум 3 пункти. Не додавай фактів, яких немає у вхідному тексті. Якщо тексту недостатньо для змістовного підсумку, скажи про це.

Це нормально для простого використання. Але ми хочемо побачити, як той самий зміст виглядає як Ordo Source.

8.3. Мінімальна структура Ordo Source

Мінімальна Ordo Source-програма може мати такі частини:

```
ordo
program
intent
contract
context
state
path
result
handoff
```

Не кожна програма завжди має всі ці частини в повному вигляді, але для навчального прикладу краще показати повну форму.

Загальний каркас:

```
ordo: "0.12-draft"
program: "minimal.summary.uk"

intent:
  goal: "summarize_text"

contract:
  input: {}
  output: {}
  rules: []

context: {}

state: {}

path:
  steps: []
  gates: []

result: {}

handoff: {}
```

Це ще не програма, а тільки порожній скелет. Тепер заповнимо його.

8.4. Описуємо intent

intent відповідає на питання: “Що ми хочемо зробити?”

Для нашого прикладу:

```
intent:
  goal: "summarize_text"
  language: "uk"
```

Тут ми не пишемо довге пояснення. Ми фіксуємо головну дію: потрібно підсумувати текст.



Важливо: intent не має містити всі правила. Не треба туди записувати “не вигадувати факти”, “максимум 3 пункти”, “якщо тексту недостатньо”. Це вже contract і gates.

Типова помилка:

```
intent:
  goal: "summarize_text_in_ukrainian_max_3_items_without_facts_and_with_fallback"
```

Так робити не варто. Це перетворює intent на неконтрольовану фразу. Краще розділити:

```
intent:
  goal: "summarize_text"
```

А решту винести у contract.

8.5. Описуємо contract

contract визначає, що вважається правильним результатом.

Для нашого прикладу:

```
contract:
  input:
    text: required

  output:
    format: "bullet_list"
    max_items: 3
    language: "uk"

  rules:
    - id: R1
      rule: "use_only_input_text"
    - id: R2
      rule: "do_not_invent_facts"
    - id: R3
      rule: "if_input_insufficient_return_insufficient_data_status"
```

Тут уже видно, чим Ordo відрізняється від prompt-а. Ми не просто просимо “будь ласка, не вигадуй”. Ми створюємо явний contract rule.

Контракт має бути максимально конкретним. Якщо результат має бути списком — це пишеться. Якщо максимум 3 пункти — це пишеться. Якщо мова українська — це пишеться. Якщо модель не має права додавати зовнішні факти — це теж пишеться.

Контракт — це не рекомендація. Це умова правильності результату.

8.6. Описуємо context

context визначає, звідки модель бере дані.

У нашому прикладі джерело одне — текст користувача:

```
context:
  source:
    text: "$USER_INPUT.text"
```

Цей запис означає: не бери текст із пам'яті, не шукай додаткові факти, не додавай знання ззовні. Основне джерело — вхідний текст користувача.

У складніших програмах context може містити:

```
документи;
таблиці;
source row;
попередній стан;
приклади;
правила проєкту;
підтверджені контракти;
обмеження середовища.
```

Але для першої програми достатньо одного source.

8.7. Описуємо state

Навіть проста програма може мати стан.

Для summary-задачі нам достатньо такого стану:

```
state:
  status: "draft"
  assumptions: []
  unsupported_facts: []
```

Навіщо це потрібно?

status показує, що результат ще не фінальний до проходження gates.

assumptions фіксує припущення, якщо модель змушена щось припустити.

unsupported_facts фіксує факти, які модель не змогла підтвердити у вхідному тексті.

Для простого summary це може виглядати надлишково, але принцип важливий: Ordo-програма має знати, що вона вже збрала, що перевірила і що не може підтвердити.

8.8. Описуємо path і steps

path описує, як модель має дійти до результату.

У нашому прикладі шлях простий:

```

path:
  id: "summarize_or_report_insufficient_input"
  steps:
    - id: S1
      do: "read_input_text"
    - id: S2
      do: "extract_supported_points"
    - id: S3
      do: "select_up_to_3_key_points"
    - id: S4
      do: "render_ukrainian_bullet_summary"

```

Тут важливо, що ми не просимо модель одразу “написати підсумок”. Ми задаємо порядок:

```

спочатку прочитати;
потім виділити підтверджені пункти;
потім вибрати максимум 3;
потім оформити українською.

```

Це зменшує ризик того, що модель одразу почне генерувати красивий, але не перевірений текст.

8.9. Додаємо gates

Тепер додамо контрольні точки.

```

path:
  gates:
    - id: G1
      check: "input_text_present"
      on_fail: "return_insufficient_data"

    - id: G2
      check: "no_unsupported_facts"
      on_fail: "remove_or_mark_unsupported"

    - id: G3
      check: "max_3_items"
      on_fail: "compress_to_3_items"

    - id: G4
      check: "output_language_uk"
      on_fail: "rewrite_in_ukrainian"

```



Gate — це місце, де модель має зупинитися і перевірити умову.

G1 перевіряє, чи є текст.

G2 перевіряє, чи немає вигаданих фактів.

G3 перевіряє, чи не більше трьох пунктів.

G4 перевіряє мову результату.

Важливо: `on_fail` не завжди означає “помилка і кінець”. Іноді це `repair action`. Наприклад, якщо пунктів 5, модель може стиснути до 3. Якщо мова не українська, модель може переписати українською. Але якщо вхідного тексту немає, треба повернути `fallback`.

8.10. Описуємо `result` і `handoff`

`result` визначає, що саме повертається.

```
result:
  primary: "summary"
  fallback: "insufficient_data_message"
```

`handoff` визначає, що модель має показати наприкінці:

```
handoff:
  include:
    - "status"
    - "result"
    - "gate_report"
```

Це означає, що користувач має отримати не тільки сам підсумок, а й статус і короткий звіт по `gates`, якщо це потрібно.

У простому UI `gate report` можна приховати. Але для навчання та складних задач він дуже корисний.

8.11. Повна перша Ordo Source-програма

Тепер зберемо все разом.

```
ordo: "0.12-draft"
program: "minimal.summary.uk"

intent:
  goal: "summarize_text"
  language: "uk"

contract:
  input:
    text: required
  output:
    format: "bullet_list"
    max_items: 3
    language: "uk"
  rules:
    - id: R1
      rule: "use_only_input_text"
    - id: R2
      rule: "do_not_invent_facts"
    - id: R3
      rule: "if_input_insufficient_return_insufficient_data_status"

context:
  source:
    text: "$USER_INPUT.text"

state:
  status: "draft"
```

```

assumptions: []
unsupported_facts: []

path:
  id: "summarize_or_report_insufficient_input"
  steps:
    - id: S1
      do: "read_input_text"
    - id: S2
      do: "extract_supported_points"
    - id: S3
      do: "select_up_to_3_key_points"
    - id: S4
      do: "render_ukrainian_bullet_summary"

  gates:
    - id: G1
      check: "input_text_present"
      on_fail: "return_insufficient_data"
    - id: G2
      check: "no_unsupported_facts"
      on_fail: "remove_or_mark_unsupported"
    - id: G3
      check: "max_3_items"
      on_fail: "compress_to_3_items"
    - id: G4
      check: "output_language_uk"
      on_fail: "rewrite_in_ukrainian"

result:
  primary: "summary"
  fallback: "insufficient_data_message"

handoff:
  include:
    - "status"
    - "result"
    - "gate_report"

```

Це вже повноцінна маленька Ordo Source-програма.

8.12. Що тут важливо помітити

Перше: програма не дуже довга, але в ній уже є структура.

Друге: правила не заховані в одному реченні. Вони винесені в contract і gates.

Третє: модель має fallback-поведінку, якщо тексту недостатньо.

Четверте: результат має обмеження: формат, кількість пунктів, мова.

П'яте: є handoff, тобто визначено, що саме повертається користувачу.

У звичайному prompt-і це все теж можна написати, але Ordo робить структуру явною.

8.13. Як така програма виконується людиною через модель

Якщо ми ще не маємо Ordo-runtime, можна дати цю програму моделі як інструкцію і попросити виконувати її буквально:

Використовуй наведену Ordo Source-програму як execution contract.
 Не виконуй задачу як звичайний prompt.
 Спочатку перевір gates, потім поверни result і gate_report.

Це не ідеально, бо модель усе ще сама інтерпретує Ordo Source. Але навіть так поведінка стає дисциплінованішою.

У майбутньому runtime зможе читати Ordo Source або compiled IR і сам контролювати порядок виконання.

8.14. Типові помилки при написанні першої Ordo-програми

Помилка 1. Запхати все в intent

Погано:

```
intent:
  goal: "summarize text in 3 bullets without unsupported facts and fallback if insufficient"
```

Краще:

```
intent:
  goal: "summarize_text"

contract:
  output:
    max_items: 3
  rules:
    - do_not_invent_facts
```

Помилка 2. Не описати gates

Погано:

```
rules:
  - do_not_invent_facts
```

Але немає gate, який це перевіряє.

Краще:

```
gates:
  - id: G_NO_UNSUPPORTED_FACTS
    check: "no_unsupported_facts"
    on_fail: "remove_or_mark_unsupported"
```

Помилка 3. Не описати fallback

Якщо вхідних даних немає, модель не має вигадувати зміст. Тому fallback має бути явним.

Помилка 4. Змішати source і result

context — це звідки беремо дані.

result — це що повертаємо.

Їх не треба змішувати.

Помилка 5. Вважати Ordo Source фінальним машинним форматом

Ordo Source — це зручна людська форма. Для виконання краще мати compiled IR. Про це буде наступний розділ.

8.15. Короткий підсумок розділу

Ordo Source — це людсько-читабельний запис Ordo-програми.

Він потрібен, щоб автор міг описати процес не як суцільний prompt, а як структурований execution contract.

Перша проста Ordo-програма має містити:

```
intent;  
contract;  
context;  
state;  
path;  
steps;  
gates;  
result;  
handoff.
```

Головний принцип:

```
не писати “модель, будь ласка, зроби правильно”,  
а явно описувати, що означає “правильно”.
```

Міні-вправа

Спробуйте переписати звичайний prompt у Ordo Source.

Prompt:

```
Напиши короткий лист клієнту українською. Поясни, що ми отримали його звернення, розберемо  
ситуацію і повернемося з відповіддю. Тон має бути ввічливий, без обіцянок конкретного строку.
```

Опишіть:

```
1. intent;  
2. contract.output;  
3. contract.rules;  
4. context;  
5. gates;  
6. result;  
7. handoff.
```

Підказка: окремим gate варто зробити перевірку, що в листі немає конкретного строку відповіді.

Розділ 9. Компілюємо Ordo Source в Semantic JSON IR

9.1. Навіщо взагалі щось компілювати

У попередньому розділі ми написали першу Ordo Source-програму. Вона була схожа на YAML: її можна прочитати очима, зрозуміти логіку, обговорити з аналітиком або розробником, виправити руками.

Це зручно для людини.

Але для моделі або майбутнього runtime цього не завжди достатньо. Людсько-читабельна форма може містити зручні назви, коментарі, різні стилі запису, скорочення й пояснення. Для виконання краще мати більш однозначну форму.

Саме тому Ordo вводить ідею компіляції.



Компіляція в Ordo — це не обов'язково компіляція в машинний код, як у класичних мовах програмування. У першій практичній версії це радше перетворення людсько-читабельної Ordo Source-програми в структурований формат, який модель виконує стабільніше.

Найзручніший формат для цього на поточному етапі — Semantic JSON IR.

IR означає intermediate representation, тобто проміжне представлення.

Простіше:

Ordo Source — форма для людини.
Semantic JSON IR — форма для виконання моделлю або runtime.

Ordo Source відповідає на питання:

Як людині описати процес зрозуміло і контрольовано?

Semantic JSON IR відповідає на інше питання:

Як подати цей процес моделі так, щоб вона бачила чіткі ops, порядок виконання, gates, state і handoff?

9.2. Що таке Semantic JSON IR

Semantic JSON IR — це список або дерево інструкцій, де кожна інструкція має явний op.

Наприклад:

```
{
  "op": "INTENT.SET",
  "id": "I1",
  "goal": "summarize_text",
```

```
"lang": "uk"
}
```

Така інструкція каже не просто “десь у тексті є мета”, а прямо:

```
встановити intent програми;
id цієї інструкції – I1;
мета – summarize_text;
мова – українська.
```

Інший приклад:

```
{
  "op": "GATE.CHECK",
  "id": "G1",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNSUPPORTED_FACTS",
  "source": "RESULT.summary",
  "on_fail": "REPAIR.REMOVE_UNSUPPORTED"
}
```

Тут уже видно контрольну точку:

```
перевірити, що в результаті немає непідтверджених фактів;
перевіряти потрібно RESULT.summary;
якщо перевірка не пройдена – прибрати непідтверджене.
```

Це набагато точніше, ніж просто фраза в prompt-i:

```
Не вигадуй фактів.
```

Фраза може загубитися. GATE.CHECK складніше проігнорувати, особливо якщо runtime або дисциплінована модель веде execution trace.

9.3. Чому саме JSON

Можна запитати: чому не YAML, не XML, не власний компактний синтаксис?

Причина проста: JSON добре підходить для поточних моделей і програмних інструментів.

Його переваги:

- він структурований;
- його легко перевіряти програмно;
- його добре розуміють сучасні AI-моделі;
- його можна передавати між системами;
- його можна логувати;
- його можна валідовувати через schema;
- його можна виконувати покроково.

Ordo не прив’язана назавжди тільки до JSON. У майбутньому може з’явитися компактний orcode format або навіть нативний model-internal формат.

Але для поточного етапу Semantic JSON IR — найкращий компроміс між зрозумілістю, суворістю і практичністю.

9.4. Ordo Source і IR на одному прикладі

Візьмемо просту Ordo Source-програму для підсумку тексту.

```

ordo: "0.12-draft"
program: "minimal.summary.uk"

intent:
  goal: "summarize_text"
  language: "uk"

contract:
  input:
    text: required
  output:
    format: "bullet_list"
    max_items: 3
    language: "uk"
  rules:
    - id: R1
      rule: "use_only_input_text"
    - id: R2
      rule: "do_not_invent_facts"
    - id: R3
      rule: "if_input_insufficient_return_insufficient_data_status"

context:
  source:
    text: "$USER_INPUT.text"

state:
  status: "draft"
  unsupported_facts: []

path:
  id: "summarize_or_report_insufficient_input"
  steps:
    - id: S1
      do: "read_input_text"
    - id: S2
      do: "extract_supported_points"
    - id: S3
      do: "select_up_to_3_key_points"
    - id: S4
      do: "render_ukrainian_bullet_summary"

  gates:
    - id: G1
      method: "mechanical"
      trust_class: "deterministic"
      check: "input_text_present"
      on_fail: "return_insufficient_data"
    - id: G2
      method: "self_verification"
      trust_class: "model_judgment"
      check: "no_unsupported_facts"
      on_fail: "remove_or_mark_unsupported"
    - id: G3
      method: "mechanical"
      trust_class: "deterministic"
      check: "max_3_items"
      on_fail: "compress_to_3_items"
    - id: G4
      method: "mechanical"
      trust_class: "deterministic"
      check: "output_language_uk"
      on_fail: "rewrite_in_ukrainian"

result:

```

```

primary: "summary"
fallback: "insufficient_data_message"

handoff:
  include:
    - "status"
    - "result"
    - "gate_report"

```

Це нормально читається людиною.

Тепер подивимося на Semantic JSON IR для тієї ж програми.

```

{
  "ir_version": "ordo-ir/0.12-draft",
  "program_id": "minimal.summary.uk",
  "mode": "model_execution",
  "ops": [
    {
      "op": "INTENT.SET",
      "id": "I1",
      "goal": "summarize_text",
      "lang": "uk"
    },
    {
      "op": "CONTRACT.DEF",
      "id": "C1",
      "input": {
        "text": "required"
      },
      "output": {
        "type": "bullet_list",
        "max_items": 3,
        "lang": "uk"
      }
    },
    {
      "op": "RULE.ADD",
      "id": "R1",
      "kind": "SOURCE_BOUND_ONLY"
    },
    {
      "op": "RULE.ADD",
      "id": "R2",
      "kind": "NO_UNSUPPORTED_FACTS"
    },
    {
      "op": "RULE.ADD",
      "id": "R3",
      "kind": "INSUFFICIENT_INPUT_FALLBACK"
    },
    {
      "op": "CONTEXT.LOAD",
      "id": "CTX1",
      "slot": "input.text",
      "from": "$USER_INPUT.text"
    },
    {
      "op": "STATE.INIT",
      "id": "ST1",
      "status": "draft",
      "unsupported_facts": []
    },
    {
      "op": "PATH.SELECT",
      "id": "P1",
      "path": "summarize_or_report_insufficient_input"
    },
    {
      "op": "STEP.RUN",

```



```

    "id": "S1",
    "fn": "READ_INPUT_TEXT",
    "in": ["CTX1"],
    "out": ["ST1.input_read"]
  },
  {
    "op": "GATE.CHECK",
    "id": "G1",
    "method": "mechanical",
    "trust_class": "deterministic",
    "assert": "INPUT_TEXT_PRESENT",
    "on_fail": "RESULT.INSUFFICIENT_DATA"
  },
  {
    "op": "STEP.RUN",
    "id": "S2",
    "fn": "EXTRACT_SUPPORTED_POINTS",
    "in": ["CTX1"],
    "out": ["ST1.supported_points"]
  },
  {
    "op": "STEP.RUN",
    "id": "S3",
    "fn": "SELECT_TOP_N",
    "params": {
      "n": 3
    },
    "in": ["ST1.supported_points"],
    "out": ["ST1.selected_points"]
  },
  {
    "op": "STEP.RUN",
    "id": "S4",
    "fn": "RENDER_BULLETS",
    "params": {
      "lang": "uk"
    },
    "in": ["ST1.selected_points"],
    "out": ["RESULT.summary"]
  },
  {
    "op": "GATE.CHECK",
    "id": "G2",
    "method": "self_verification",
    "trust_class": "model_judgment",
    "assert": "NO_UNSUPPORTED_FACTS",
    "source": "RESULT.summary",
    "on_fail": "REPAIR.REMOVE_UNSUPPORTED"
  },
  {
    "op": "GATE.CHECK",
    "id": "G3",
    "method": "mechanical",
    "trust_class": "deterministic",
    "assert": "ITEM_COUNT_LE",
    "params": {
      "max": 3
    },
    "source": "RESULT.summary",
    "on_fail": "REPAIR.COMPRESS"
  },
  {
    "op": "GATE.CHECK",
    "id": "G4",
    "method": "mechanical",
    "trust_class": "deterministic",
    "assert": "LANG_IS",
    "params": {
      "lang": "uk"
    },
  },

```

```

    "source": "RESULT.summary",
    "on_fail": "REPAIR.REWRITE_LANG"
  },
  {
    "op": "HANDOFF.EMIT",
    "id": "H1",
    "include": [
      "status",
      "result",
      "gate_report"
    ]
  }
]
}

```

На перший погляд JSON довгий. Але він краще підходить для виконання.

У ньому чітко видно:

- порядок операцій;
- що є intent;
- що є contract;
- які rules додано;
- звідки береться context;
- який state ініціалізується;
- які steps виконуються;
- які gates перевіряються;
- що передається у handoff.

9.5. Що саме робить компілятор

У простому вигляді компілятор Ordo робить кілька речей.

По-перше, він знаходить основні блоки Ordo Source:

```

intent
contract
context
state
path
steps
gates
result
handoff

```

По-друге, він перетворює їх у явні operations:

```

INTENT.SET
CONTRACT.DEF
CONTEXT.LOAD
STATE.INIT
PATH.SELECT
STEP.RUN
GATE.CHECK
HANDOFF.EMIT

```

По-третє, він нормалізує назви.

Наприклад, у source може бути:

```

check: "no_unsupported_facts"

```

А в IR це стає:

```
{
  "assert": "NO_UNSUPPORTED_FACTS"
}
```

По-четверте, компілятор може виявити частини, які не вдалося безпечно формалізувати.

Наприклад:

Зберегти тон доброзичливий, але не надто офіційний, як у наших попередніх листах клієнтам.

Це можна частково формалізувати як `tone: polite`, але фраза “як у наших попередніх листах” вимагає контексту або стилістичного прикладу. Якщо такого формального представлення ще немає, компілятор може винести це в `FREEFORM.ADD`.

```
{
  "op": "FREEFORM.ADD",
  "id": "F1",
  "role": "style_guidance",
  "binding": "CONTRACT.C1",
  "text": "Зберегти тон доброзичливий, але не надто офіційний, як у наших попередніх листах клієнтам.",
  "compile_status": "not_structured",
  "reason": "style reference requires examples or domain-specific style profile"
}
```

Це чесніше, ніж удавати, що все повністю формалізовано.

9.6. Компіляція не має спотворювати сенс



Головне правило компіляції:

краще залишити частину в `controlled FREEFORM`, ніж перетворити її на неправильну структуровану інструкцію.

Наприклад, користувач пише:

Не роби документ сухим. Він має бути зрозумілий аналітику, але не втрачати технічну точність.

Можна формалізувати частину:

```
style:
  audience: analyst
  readability: high
  technical_accuracy: required
```

Але фразу “не сухим” важко точно формалізувати без стилістичного профілю. Її можна залишити як `controlled FREEFORM`:

```
{
  "op": "FREEFORM.ADD",
  "id": "F_STYLE_1",
  "role": "style_guidance",
  "binding": "OUTPUT.DEF",
  "text": "Документ не має бути сухим. Він має бути зрозумілий аналітику, але не втрачати технічну точність.",
}
```

```

"compile_status": "partially_structured",
"structured_parts": [
  "audience=analyst",
  "readability=high",
  "technical_accuracy=required"
],
"reason": "stylistic nuance is better preserved as controlled freeform"
}

```

Це і є чесна компіляція.

Ordo не намагається насильно стиснути весь людський сенс у маленький набір оркодів.

9.7. Source map: як не втратити зв'язок із людським текстом

Коли Ordo Source компілюється в IR, важливо не втратити зв'язок із першоджерелом.

Для цього потрібен source map.



Source map відповідає на питання:

з якої частини людської інструкції виникла ця IR-операція?

Наприклад:

```

{
  "op": "GATE.CHECK",
  "id": "G2",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNSUPPORTED_FACTS",
  "source_ref": "source.contract.rules.R2"
}

```

Це означає, що gate G2 походить із правила R2 у source-програмі.

У складних playbook-ах це дуже важливо. Якщо модель виконує gate, аналітик має бачити, звідки це правило взялося.

У майбутньому source map може дозволяти такі речі:

- показати користувачу, які частини інструкції були використані;
- знайти, чому модель поставила саме це питання;
- перевірити, чи не зникло правило під час компіляції;
- оновити source і регенерувати IR;
- порівняти різні версії playbook-а.

Без source map compiled IR може стати “чорною скринькою”. Ordo має уникати цього.

9.8. Execution trace: що модель реально виконала

Source map показує, звідки взялася інструкція.

Execution trace показує, що реально було виконано.

Наприклад:

```
{
  "trace": [
    {
      "op_id": "I1",
      "status": "passed",
      "note": "Intent set to summarize_text"
    },
    {
      "op_id": "CTX1",
      "status": "passed",
      "note": "Input text loaded"
    },
    {
      "op_id": "G1",
      "status": "passed",
      "note": "Input text is present"
    },
    {
      "op_id": "G2",
      "status": "passed",
      "note": "No unsupported facts detected"
    }
  ]
}
```

Для простого summary це може здаватися зайвим. Але для великого процесу trace дуже корисний.

Наприклад, якщо модель не створила фінальний архів, trace має показати чому:

```
{
  "op_id": "G_CONTRACTS_CONFIRMED",
  "status": "failed",
  "reason": "history_event_values_keys has status proposed",
  "action": "BLOCK_FINAL_ARCHIVE"
}
```

Це набагато краще, ніж загальна відповідь:

Поки не вистачає даних.

Ordo має робити причину зупинки видимою.

9.9. IR не обов'язково має бути зручним для людини

Важливий момент: Ordo Source і Semantic JSON IR мають різні цілі.

Ordo Source має бути зрозумілим людині.

IR має бути зручним для виконання.

Тому не треба вимагати, щоб IR був красивим для читання. Він може бути довшим, сухішим, формальнішим.

Це нормально.

У майбутньому може існувати ще компактніший формат:

```
[
  ["I", "summarize_text", "uk"],
  ["C", {"out": {"type": "bullets", "max": 3, "lang": "uk"}}],
  ["G", "NO_UNSUPPORTED_FACTS", "summary", "REPAIR.REMOVE_UNSUPPORTED"],
  ["H", ["status", "result", "gate_report"]]
]
```

Це майже не зручно для людини, але може бути зручно для runtime або моделі, спеціально навченої на Ordo.

Поточний Semantic JSON IR — проміжний практичний формат. Він ще достатньо читабельний, але вже достатньо структурований.

9.10. Що має бути в хорошому IR

Хороший Ordo IR має відповідати кільком правилам.

1. Кожна операція має мати op

Погано:

```
{
  "goal": "summarize_text"
}
```

Краще:

```
{
  "op": "INTENT.SET",
  "goal": "summarize_text"
}
```

op показує, що саме треба зробити.

2. Кожна важлива операція має мати id

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment"
}
```

Без id важко робити trace, source map і validation report.

3. Gates мають бути явними

Погано:

```
{
  "rules": ["do not invent facts"]
}
```

Краще:

```
{
  "op": "GATE.CHECK",
```

```

"method": "self_verification",
"trust_class": "model_judgment",
"assert": "NO_UNSUPPORTED_FACTS",
"on_fail": "REPAIR.REMOVE_UNSUPPORTED"
}

```

4. State має бути явним

Якщо модель має пам'ятати відповідь користувача, це має бути в state.

```

{
  "op": "STATE.UPDATE",
  "key": "request_type",
  "value": "incident"
}

```

5. Handoff має бути описаний

Модель має знати, що саме повертати користувачу.

```

{
  "op": "HANDOFF.EMIT",
  "include": ["status", "result", "gate_report"]
}

```

9.11. Типові помилки при компіляції

Помилка 1. Залишити важливий gate у звичайному тексті

Якщо правило блокує фінальний результат, воно має бути GATE.CHECK, а не просто реченням.

Помилка 2. Перетворити доменне пояснення на універсальне правило

Наприклад, HistoryEvent.item.values — це доменний контракт, а не правило Ordo Core.

Помилка 3. Втратити статуси

Якщо в source є candidate, confirmed, blocked, ready_for_first_run, IR має зберегти різницю між ними.

Помилка 4. Не винести неформалізоване в FREEFORM

Якщо частина інструкції не може бути безпечно структурована, її не можна просто загубити.

Помилка 5. Не зробити source map

Без source map важко довести, що IR справді відповідає source-програмі.

Помилка 6. Зробити IR занадто людським

IR не має бути есе. Якщо в ньому забагато вільного тексту, модель знову працюватиме з prompt-ом, а не з execution contract.

9.12. Як це виглядає в реальному playbook-у

У простому прикладі IR може містити 10–20 операцій.

У реальному playbook-у їх може бути десятки або сотні.

Наприклад, Ordo-native History Event Playbook має такі групи ops:

```
PROGRAM.DEF
PROFILE.USE
DOMAIN_PACK.LOAD
ENTRY.DEF
NODE.DEF
ANSWER.REGISTRY
STATE.SCHEMA
OUTPUT.DEF
TEMPLATE.BIND
GATE.DEF
APPROVAL.REQUIRE
DOC.SPLIT
DOC.CATALOG
DOC.SELECT
RENDER.VALIDATE
FREEFORM.ADD
HANDOFF.DEF
```

Це не означає, що модель має виводити всі ці ops користувачу. Користувачу достатньо бачити поточний вузол, вибрані документи, оновлення стану і наступне питання.

Але всередині процес має бути керованим.

9.13. Короткий підсумок розділу

Ordo Source потрібен для людей.

Semantic JSON IR потрібен для виконання.

Компіляція Ordo Source в IR робить процес більш явним:

```
- intent стає INTENT.SET;
- contract стає CONTRACT.DEF;
- context стає CONTEXT.LOAD;
- state стає STATE.INIT / STATE.UPDATE;
- steps стають STEP.RUN;
- gates стають GATE.CHECK;
- result i handoff стають HANDOFF.EMIT.
```

Хороший IR має:

```
- явні op;
- стабільні id;
- gates;
- state;
- source map;
- execution trace;
- controlled FREEFORM для того, що не можна безпечно формалізувати.
```

Головна думка:

```
компіляція не повинна робити інструкцію красивішою;
вона має робити її виконуванішою.
```


Міні-вправа

Візьміть просту Ordo Source-програму:

```
intent:
  goal: write_customer_reply

contract:
  output:
    type: email
    language: uk
  rules:
    - polite_tone
    - no_specific_deadline
    - acknowledge_request

context:
  customer_message: "$USER_INPUT.message"

result:
  primary: customer_reply
```

Спробуйте вручну перетворити її на Semantic JSON IR.

Мінімально вам потрібні ops:

```
INTENT.SET
CONTRACT.DEF
RULE.ADD
CONTEXT.LOAD
STATE.INIT
STEP.RUN
GATE.CHECK
HANDOFF.EMIT
```

Окремо подумайте: яке правило має бути GATE.CHECK, а яке може лишитися як style guidance?

Розділ 10. Чому Semantic JSON IR зараз найкращий

10.1. Навіщо взагалі потрібен IR

У попередньому розділі ми розібрали, що Ordo Source — це зручна для людини форма опису програми. Її можна читати, обговорювати, редагувати, погоджувати з аналітиком або розробником. Вона схожа на добре структурований YAML-документ: у ній видно intent, contract, state, gates, outputs і handoff.

Але Ordo Source — це ще не найкраща форма для виконання моделлю.

Людині зручно читати так:

```
intent:
  goal: summarize_text

contract:
  output:
    format: bullet_list
    max_items: 3
    language: uk

rules:
  - use_only_input_text
  - do_not_invent_facts
```

Моделі теж може це зрозуміти. Але коли процес стає великим, з'являються десятки вузлів, gate-ів, статусів, бібліотек, templates, domain rules і FREEFORM blocks, тоді людський YAML стає занадто вільним. Його можна прочитати по-різному. Частина полів може бути вкладена інакше. Частина правил може бути описана prose-текстом. Частина зв'язків може бути неявною.

Тому потрібен проміжний шар — IR.



IR означає Internal Representation, тобто внутрішнє представлення програми.

У контексті Ordo IR — це така форма програми, яка вже не призначена в першу чергу для комфортного читання людиною, а призначена для більш точного виконання моделлю або майбутнім Ordo-runtime.

Ordo Source відповідає на питання:

Як людині зручно описати процес?

Ordo IR відповідає на питання:

Як моделі або runtime краще виконувати цей процес?

Це дуже важлива різниця.

10.2. Чому не залишити тільки YAML

Може здатися, що YAML достатній. Він читається, у нього є структура, він набагато кращий за суцільний prompt. Для багатьох простих задач це справді так.

Але YAML лишається authoring-форматом. Він добре підходить для написання й обговорення інструкції. Проте для виконання він має кілька слабких місць.

По-перше, YAML не завжди задає строгий порядок виконання.

Наприклад:

```
steps:
  - read_input
  - extract_points
  - render_summary

gates:
  - input_present
  - no_unsupported_facts
```

Людині зрозуміло, що gates треба застосувати у правильні моменти. Але модель може не завжди зрозуміти, коли саме запускати input_present: до extract_points, після нього чи перед handoff.

По-друге, YAML часто допускає змішування рівнів. У ньому поруч можуть бути machine-readable поля й довгі пояснення для людини.

По-третє, YAML зручний для читання, але не завжди зручний для перевірки. Runtime-у або слабшій моделі краще мати список конкретних операцій:

```
[
  {"op": "INTENT.SET", "goal": "summarize_text"},
  {"op": "CONTRACT.DEF", "output": {"format": "bullet_list", "max_items": 3}},
  {"op": "STEP.RUN", "fn": "READ_INPUT_TEXT"},
  {"op": "GATE.CHECK",
   "method": "mechanical",
   "trust_class": "deterministic", "assert": "INPUT_TEXT_PRESENT"}
]
```

Тут уже видно не просто “що є в програмі”, а “що саме треба зробити”.

Саме тому в Ordo Source і IR мають бути різними шарами.

10.3. Чому саме Semantic JSON IR

На поточному етапі розвитку AI-моделей найкращим основним IR для Ordo є Semantic JSON IR.

Це означає: JSON-структура, у якій кожна операція має явний op, id, параметри, входи, виходи, прив'язки й очікувану поведінку.

Наприклад:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNUNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNUNSUPPORTED_FACTS",
```

```

"source": "RESULT.summary",
"on_fail": "REPAIR.REMOVE_UNSUPPORTED"
}

```

Це не просто JSON заради JSON. Тут важливо слово *Semantic*.

Semantic JSON IR означає, що структура не просто містить дані, а прямо несе значення виконання.

op каже, яку дію виконати.

id дає стабільний ідентифікатор.

assert каже, що перевіряється.

source каже, до чого застосовується перевірка.

on_fail каже, що робити при провалі.

Така форма добре підходить для моделей, тому що вона одночасно:

- достатньо структурована;
- достатньо зрозуміла;
- не занадто стисла;
- не потребує спеціального *binary/runtime* формату;
- легко перевіряється людиною;
- легко мапиться на майбутній *runtime*.

10.4. Чому не Compact Orcode IR одразу

У попередніх прикладах ми вже бачили компактний формат:

```

[
  ["I", "summarize_text", "uk"],
  ["C", {"out": {"type": "bullets", "max": 3}}],
  ["G", "NO_UNSUPPORTED_FACTS", "summary", "REPAIR.REMOVE_UNSUPPORTED"]
]

```

Він короткий. Він добре виглядає як майбутній *machine-native* формат. Але для поточних моделей він ризикований.

Проблема в тому, що компактність зменшує очевидність.



Людина або модель має пам'ятати, що означає I, C, G, який порядок аргументів, які значення допустимі, що робити з помилкою. Для спеціально навченого *runtime* це нормально. Для звичайної мовної моделі — ще ні.

Compact Orcode IR може стати хорошим майбутнім форматом, якщо:

```

модель спеціально навчена на Ordo;
або є Ordo execution layer;

```

або є compiler/runtime, який сам виконує opcodes;
або Compact IR використовується тільки як внутрішній формат, а не як основна інструкція для LLM.

Але на практичному етапі краще не поспішати.

Semantic JSON IR — це компроміс між людською зрозумілістю і машинною точністю.

10.5. Чому не просто природна мова

Природна мова дуже сильна. Саме вона дозволяє нам пояснити складні речі без формальних схем. Але природна мова має слабке місце: вона часто неоднозначна.

Наприклад:

Не створюй фінальний пакет, якщо контракти не підтверджені.

Людині це зрозуміло. Але для виконання виникають питання:

- які саме контракти mandatory;
- що означає confirmed;
- де це перевіряти;
- який статус ставити при провалі;
- чи можна створити draft package;
- чи треба питати користувача;
- чи це hard stop;
- чи це стосується всіх outputs або тільки archive.

У Semantic JSON IR це можна описати точніше:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_FINAL_ARCHIVE_BEFORE_CONTRACTS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ALL_MANDATORY_CONTRACTS_CONFIRMED",
  "scope": "final_archive",
  "on_fail": "BLOCK_FINAL_ARCHIVE",
  "failed_status": "no_go_requires_confirmation"
}
```

Тут уже менше простору для випадкового тлумачення.

Природна мова залишається важливою. Вона може бути в FREEFORM, поясненнях, descriptions, templates. Але hard rules краще виносити в структуровані ор-и.

10.6. Як виглядає одна й та сама інструкція в трьох формах

Візьмемо просте правило:

Перед фінальним результатом перевір, що немає вигаданих фактів.

У природній мові це зрозуміло, але не дуже формально.

В Ordo Source це може виглядати так:

```
gates:
- id: no_unsupported_facts
```

```
check: no_unsupported_facts
source: result
on_fail: remove_or_mark_unsupported
```

У Semantic JSON IR:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNSUPPORTED_FACTS",
  "source": "RESULT.primary",
  "on_fail": "REPAIR.REMOVE_OR_MARK_UNSUPPORTED"
}
```

У Compact Opcode IR:

```
["G", "NO_UNSUPPORTED_FACTS", "RESULT.primary", "REPAIR.REMOVE_OR_MARK_UNSUPPORTED"]
```

Усі три форми описують одну ідею. Але вони мають різні ролі.



Природна мова — для пояснення.

Ordo Source — для автора.

Semantic JSON IR — для виконання поточною моделлю або runtime.

Compact Opcode IR — для майбутнього оптимізованого виконання.

10.7. Основні переваги Semantic JSON IR

Semantic JSON IR має кілька сильних переваг.

1. Явний порядок виконання

IR може бути списком операцій. Це означає, що модель не просто бачить набір правил, а отримує послідовність:

```
спочатку завантажити документацію;
потім визначити entry;
потім поставити node question;
потім оновити state;
потім перевірити gate;
потім перейти до наступного node.
```

Це особливо важливо для playbook-ів.

2. Стабільні ідентифікатори

Кожна операція має id.

Це дозволяє посилалися на неї:

```
{
  "op": "APPROVAL.REQUIRE",
```

```
"id": "A_PASSPORT_APPROVAL",
"target": "OUTPUT.01_HISTORY_EVENT_PASSPORT"
}
```

Потім у validation report можна сказати:

```
{
  "approval_id": "A_PASSPORT_APPROVAL",
  "status": "passed"
}
```

Так з'являється traceability.

3. Просте логування

Якщо програма виконується покроково, runtime або модель може вести trace:

```
{
  "executed_op": "G_CONTRACTS_CONFIRMED",
  "status": "failed",
  "reason": "source_field_paths missing",
  "next_action": "ASK_CONFIRMATION"
}
```

Це набагато краще, ніж просто відповідь “потрібно уточнити деякі дані”.

4. Легше перевіряти gates

Коли gate описаний як структурований ор, його легше перевірити:

```
{
  "op": "ASSERT.NOT",
  "id": "A_NO_ITEM_PREFIX_IN_DELTA_FIELD",
  "target": "ChangeRecord.delta.field",
  "forbidden_pattern": "^item\\."
}
```

Це вже майже перевірка, яку можна автоматизувати.

5. Легше переносити між моделями

Один і той самий Semantic JSON IR можна дати різним моделям. Навіть якщо моделі мають різний стиль відповіді, структура завдання лишається однакою.

Це важливо для майбутнього стандарту Ordo.

10.8. Недоліки Semantic JSON IR

Semantic JSON IR не ідеальний.

Його головний недолік — він довший за compact format.

Наприклад, один gate у compact format може займати один рядок, а в Semantic JSON IR — десять рядків.

Але ця довжина купує зрозумілість.

Другий недолік — він потребує дисципліни. Якщо кожен автор буде вигадувати власні ор names, IR швидко стане хаотичним. Тому Ordo має мати стандартний каталог ор-кодів.

Третій недолік — JSON не дуже приємно писати вручну. Саме тому людина пише Ordo Source, а compiler або модель-помічник переводить його в IR.

Semantic JSON IR — це не обов’язково те, що аналітик має писати руками. Це те, що має отримувати модель для виконання.

10.9. Роль compiler-a

У зрілій Ordo-архітектурі між людиною і моделлю має бути compiler або translator.

Людина пише або говорить:

Я хочу, щоб модель вела аналітика через дерево, збирала контракти, не створювала фінальний пакет без підтверджень і перевіряла документи перед архівом.

Compiler перетворює це на Ordo Source:

```
entry:
  id: guided_intake

state:
  contracts_confirmed: false
  approvals: pending

gates:
  - no_final_archive_before_contracts
  - rendered_artifacts_validated
```

Потім компілює в Semantic JSON IR:

```
[
  {"op": "ENTRY.DEF", "id": "guided_intake"},
  {"op": "STATE.SCHEMA", "fields": {"contracts_confirmed": "boolean"}},
  {"op": "GATE.CHECK", "id": "G_NO_FINAL_ARCHIVE",
   "method": "mechanical",
   "trust_class": "deterministic", "assert": "ALL_CONTRACTS_CONFIRMED"}
]
```

У такій схемі модель не мусить сама витягувати весь процес із prose-тексту. Вона отримує вже структуровану карту виконання.

10.10. Чому це важливо для слабших моделей

Сильна модель часто може сама здогадатися, що мав на увазі автор. Слабша модель — ні.

Ordo має бути корисною не тільки для найсильнішої моделі. Навпаки, одна з ідей Ordo — зробити складні інструкції виконуваними навіть для моделей, які гірше тримають довгий контекст.

Semantic JSON IR допомагає слабшим моделям, бо:

- зменшує неоднозначність;
- явно задає порядок;
- розділяє операції;
- дає стабільні статуси;
- показує, що робити при fail;
- зменшує потребу “здогадуватись”

Слабкій моделі простіше виконати:

```
{ "op": "NODE.ASK", "id": "N1", "allowed_answers": ["A", "B", "C"] }
```

ніж довгий текст:

Спочатку, якщо користувач ще не визначив шлях, постав йому перше питання дерева, але не переходь до наступних кроків, доки не отримаєш відповідь...

10.11. Чому це важливо для майбутнього runtime

Semantic JSON IR також є містком до майбутнього Ordo-runtime.

Runtime може виконувати частину роботи без моделі:

- тримати state;
- визначати current node;
- перевіряти allowed answers;
- запускати gates;
- перевіряти status transitions;
- визначати, які docs треба підключити;
- блокувати фінальний archive;
- формувати validation report.

Модель тоді робить те, що в неї виходить найкраще:

- пояснює користувачу питання;
- інтерпретує відповідь;
- формує зміст документів;
- працює з domain prose;
- пише зрозумілий текст.

Тобто runtime контролює процес, а модель заповнює зміст.

Це важлива стратегічна ідея Ordo.

10.12. Практичне правило вибору формату

Для поточної версії Ordo можна зафіксувати таке правило:

Ordo Source – основний формат для людини.
 Semantic JSON IR – основний формат для виконання моделлю.
 Compact Opcode IR – майбутній формат для оптимізації або native runtime.
 FREEFORM – контрольований контейнер для того, що не можна безпечно формалізувати.

Якщо ви пишете playbook вручну, починайте з Ordo Source.

Якщо ви даєте інструкцію моделі або runner-у, краще мати Semantic JSON IR.

Якщо у вас з'явиться спеціальний Ordo-runtime, він може компілювати Semantic JSON IR у compact/native форму.

10.13. Типові помилки

Помилка 1. Давати моделі тільки YAML і очікувати runtime-поведінку

YAML допомагає, але він ще не гарантує виконання. Для складних процесів потрібна execution map.

Помилка 2. Занадто рано переходити на compact orcodes

Компактний формат красивий, але поточним моделям краще давати semantic структуру.

Помилка 3. Ховати prose-правила всередині description

Якщо правило блокує дію, воно має бути gate або assertion, а не просто текстом.

Помилка 4. Не мати стабільних id

Без id неможливо нормально робити traceability, validation report і source map.

Помилка 5. Дозволяти різні значення ор для одного й того самого сенсу

Якщо один автор пише CHECK.GATE, інший GATE.CHECK, а третій VALIDATE, стандарт починає розпадатися. Потрібен каталог ор-кодів.

10.14. Короткий підсумок розділу

Semantic JSON IR зараз найкращий основний формат виконання для Ordo, бо він балансує між зрозумілістю і строгістю.

Він краще за чисту природну мову, бо менше неоднозначний.

Він краще за YAML як execution layer, бо явно задає операції й порядок.

Він практичніший за Compact Opcode IR, бо поточним моделям легше його розуміти.

Головне правило:

Людина пише Ordo Source.
 Модель або runtime виконує Semantic JSON IR.
 Майбутній native runtime може використовувати Compact Opcode IR.

Semantic JSON IR — це міст між людською інструкцією і контрольованим виконанням AI-моделі.

Міні-вправа

Візьміть просту інструкцію:

Попроси користувача вибрати тип задачі. Якщо він вибрав інцидент, постав питання про критичність. Якщо відповідь не з дозволених варіантів, попроси повторити.

Спробуйте записати її у трьох формах:

13. природною мовою;
14. Ordo Source;

15. Semantic JSON IR.

Порівняйте, у якій формі найкраще видно:

- allowed answers;
- current node;
- state update;
- error handling;
- next node.

Якщо у Semantic JSON IR це видно найчіткіше, ви зрозуміли головну ідею цього розділу.

Розділ 11. Gate — це не порада, а контрольна точка

11.1. Навіщо Ordo потрібні Gates

У звичайній розмові з AI-моделлю правила часто звучать як рекомендації.

Наприклад:

Не вигадуй фактів.
Не створюй фінальний документ, якщо бракує даних.
Перевір, що відповідь українською.
Не переходь до наступного кроку без підтвердження користувача.

Людина читає такі фрази і розуміє: це важливо. Але модель може сприйняти їх як загальні побажання. Вона може виконати їх, а може частково пропустити, особливо якщо завдання довге.

У Ordo такі правила мають бути не просто текстом, а контрольними точками виконання.



Саме для цього існує Gate.

Gate — це місце в процесі, де модель має зупинитися, перевірити умову і тільки після цього рухатися далі.

Якщо умова виконана — процес продовжується.

Якщо умова не виконана — процес не має мовчки рухатися далі. Він має або виправити результат, або поставити уточнювальне питання, або заблокувати наступний крок.

У цьому головна різниця між звичайною інструкцією і Ordo.

Звичайна інструкція каже:

Було б добре перевірити це перед фіналом.

Ordo каже:

Без цієї перевірки фінал заборонений.

11.2. Gate як світлофор

Найпростіша аналогія — світлофор.

Зелений сигнал означає: можна їхати.

Жовтий означає: потрібно бути уважним або підготуватися до зупинки.

Червоний означає: рух заборонений.

Gate працює схоже.

Наприклад, у процесі створення документа є gate:

Чи всі обов'язкові розділи заповнені?

Можливі результати:

passed – усі розділи заповнені;
 passed_with_notes – є незначні зауваження, але документ можна передати;
 failed – бракує обов'язкових частин;
 blocked – без додаткових даних продовжувати не можна.

Якщо gate повернув failed або blocked, модель не має права робити вигляд, що все гаразд.

Вона має явно сказати:

Поточний gate не пройдено.
 Причина: бракує підтвердженого source contract.
 Наступна дія: потрібно отримати representative source row або залишити пакет у draft status.

Це робить процес прозорим.

11.3. Gate у простому прикладі

Візьмемо дуже просте завдання:

Підсумуй текст у максимум 3 пункти українською. Не додавай фактів, яких немає у вхідному тексті.

У prompt-формі це просто прохання.

В Ordo це набір кроків і gates.

```
intent:
  goal: summarize_text

contract:
  output:
    format: bullet_list
    max_items: 3
    language: uk
  rules:
    - use_only_input_text
    - no_unsupported_facts

steps:
  - id: S1
    do: read_input_text
  - id: S2
    do: extract_supported_points
  - id: S3
    do: render_summary

gates:
  - id: G1
    method: mechanical
    trust_class: deterministic
    check: input_text_present
    on_fail: return_insufficient_data

  - id: G2
    method: self_verification
    trust_class: model_judgment
    check: no_unsupported_facts
    protocol:
      - list_claims
```

```

- attach_source_span
- classify_each_claim
on_fail: remove_or_mark_unsupported

- id: G3
method: mechanical
trust_class: deterministic
check: max_3_items
on_fail: compress_to_3_items

- id: G4
method: mechanical
trust_class: deterministic
check: output_language_uk
on_fail: rewrite_in_ukrainian

```

Тут важливо не тільки те, що gates перелічені. Важливо, що для кожного gate є on_fail.

Тобто Ordo одразу описує, що робити, якщо перевірка не пройдена.

11.4. Gate має бути конкретним

Поганий gate звучить так:

Перевір, що все добре.

Це занадто розмите. Модель може не знати, що саме означає “добре”.

Кращий gate:

Перевір, що відповідь містить не більше 3 пунктів.

Ще кращий gate в Ordo-формі:

```

{
  "op": "GATE.CHECK",
  "id": "G_MAX_3_ITEMS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ITEM_COUNT_LE",
  "source": "RESULT.summary",
  "params": {
    "max": 3
  },
  "on_fail": "REPAIR.COMPRESS_TO_LIMIT"
}

```

Тут видно:

- що саме перевіряється;
- де перевіряється;
- який параметр;
- що робити при помилці.

Це вже не побажання. Це контрольна точка.

11.4.1. Gate має показувати спосіб перевірки

У Ordo v0.12 gate більше не може виглядати однаково для всіх типів перевірок.

Є велика різниця між перевіркою, яку може виконати код, і перевіркою, яка потребує семантичного судження моделі або рішення людини.

Наприклад:

```
порахувати, що в списку не більше 3 пунктів
```

це механічна перевірка.

А ось:

```
визначити, чи всі факти мають достатню підтримку в джерелах
```

це вже не проста арифметика. Це семантична оцінка. Вона може бути добре структурованою, але не має такої самої сили, як перевірка кодом.

Тому кожен gate має явно мати два поля:

```
method: mechanical | self_verification | self_consistency | human
trust_class: deterministic | model_judgment | repeated_model_judgment | human_decision
```

method показує, як саме виконується перевірка.

trust_class показує, якому класу довіри належить результат цієї перевірки.

Наприклад:

```
{
  "op": "GATE.CHECK",
  "id": "G_MAX_3_ITEMS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ITEM_COUNT_LE",
  "source": "RESULT.summary",
  "params": {
    "max": 3
  },
  "on_fail": "REPAIR.COMPRESS_TO_LIMIT"
}
```

А для семантичної перевірки:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNsupported_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNsupported_FACTS",
  "protocol": [
    "list_claims",
    "attach_source_span",
    "classify_each_claim"
  ],
  "on_fail": "REPAIR_OR_ESCALATE"
}
```

У першому випадку runtime або скрипт може порахувати кількість пунктів. У другому випадку модель виконує structured verification, але це все одно залишається model judgment.

Це не робить другий gate непотрібним. Навпаки, він дуже корисний. Але Ordo має чесно показувати, що це інший клас довіри.

Відсутність method у gate має бути помилкою компіляції, а не мовчазним припущенням.

```
ERROR: gate.method is required
```

11.5. Gate може зупиняти процес

Не кожен gate має автоматичний repair.

Іноді модель може сама виправити проблему. Наприклад, якщо відповідь вийшла занадто довгою, модель може стиснути її.

Але іноді модель не має права сама виправляти ситуацію.

Наприклад:

```
Чи підтверджений контракт HistoryEvent.item.values?
```

Якщо контракт не підтверджений, модель не повинна вигадувати його.

Правильна дія:

```
BLOCK_AND_REQUEST_CONFIRMATION
```

Приклад:

```
{
  "op": "GATE.CHECK",
  "id": "G_HISTORY_EVENT_VALUES_CONFIRMED",
  "method": "human",
  "trust_class": "human_decision",
  "assert": "MANDATORY_CONTRACT_CONFIRMED",
  "target": "HistoryEvent.item.values",
  "on_fail": "BLOCK_AND_REQUEST_CONFIRMATION"
}
```

Це дуже важливо.

Ordo не повинна заохочувати модель “дотягувати” відповідь до красивого фіналу, якщо ключові дані відсутні.



У багатьох реальних процесах правильна відповідь моделі — не фінальний документ, а чесне повідомлення:

```
Продовжувати не можна, бо не підтверджений обов'язковий контракт.
```

11.6. Gate і hard stop

Деякі gates є м'якими. Вони дозволяють repair.

Наприклад:

```
output_language_uk
```

Якщо відповідь не українською, модель може переписати її українською.

Але є gates, які є hard stop.

Hard stop означає: не можна продовжувати без зовнішнього підтвердження, нових даних або явного рішення користувача.

Приклади hard stop:

- final archive before confirmed contracts;
- proposed contract used as confirmed;
- missing approval for mandatory document;
- unresolved source row contract;
- automation-ready without runner evidence;
- rendered QA artifact failed validation.

У Ordo hard stop має бути описаний явно.

```
hard_stop_conditions:
- unresolved_mandatory_contract
- proposed_contract_used_as_confirmed
- final_archive_without_approvals
- rendered_artifact_validation_failed
```

Якщо спрацює hard stop, модель має перейти не до результату, а до статусу blocked або requires_confirmation.

11.7. Gate перед результатом і gate всередині процесу

Gates бувають різних типів.

Перший тип — локальні gates. Вони перевіряють конкретний крок.

Наприклад:

```
Чи відповідь користувача входить у allowed answers цього NODE?
```

Другий тип — contract gates. Вони перевіряють, чи можна вважати певний контракт підтвердженим.

Наприклад:

```
Чи підтверджені source field paths?
```

Третій тип — output gates. Вони перевіряють готовий результат.

Наприклад:

```
Чи всі 11 файлів compact package присутні?
```

Четвертий тип — handoff gates. Вони перевіряють, чи можна передавати результат у роботу.

Наприклад:

```
Чи пройдено approval gates для passport, Jira, QA package і automation spec?
```

У складній Ordo-програмі всі ці gates можуть співіснувати.

Важливо не змішувати їх.

Локальний gate не замінює фінальний self-check.

А фінальний self-check не замінює contract confirmation у середині процесу.

11.8. Gate має мати evidence

Gate без доказу легко перетворюється на декоративну фразу.



Погано:

```
{
  "gate": "contracts_confirmed",
  "status": "passed"
}
```

Чому погано?

Бо незрозуміло, звідки взято passed.

Краще:

```
{
  "op": "GATE.CHECK",
  "id": "G_SOURCE_ROW_CONTRACT",
  "method": "human",
  "trust_class": "human_decision",
  "assert": "SOURCE_ROW_CONTRACT_CONFIRMED",
  "status": "passed",
  "evidence": {
    "source": "representative_source_row",
    "fields_confirmed": [
      "type",
      "sub_type",
      "source",
      "root_id",
      "dms_id",
      "item.target_field"
    ]
  }
}
```

Тут видно, чому gate пройдено.

У простих задачах evidence може бути мінімальним. У складних задачах evidence стає обов'язковою частиною довіри до результату.

Після v0.12 evidence треба читати разом із method. Якщо method: mechanical, evidence може бути машинним результатом перевірки. Якщо method: self_verification, evidence показує, як модель обґрунтувала свій висновок. Якщо method: human, evidence має показувати, яке саме підтвердження або рішення людини було отримано.

11.9. Gate не має бути захований у FREEFORM

Ordo дозволяє FREEFORM, але важливі gates не можна ховати у вільний текст.

Погано:

```
{
  "op": "FREEFORM.ADD",
  "role": "important_note",
```

```
  "text": "Не створювати фінальний пакет, якщо не підтверджені контракти."
}
```

Це не просто примітка. Це правило зупинки.

Його треба оформити як gate:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_FINAL_PACKAGE_BEFORE_CONTRACTS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ALL_MANDATORY_CONTRACTS_CONFIRMED",
  "on_fail": "BLOCK_FINAL_PACKAGE"
}
```

А у FREEFORM можна залишити пояснення:

```
{
  "op": "FREEFORM.ADD",
  "role": "rationale",
  "binding": "GATE.G_NO_FINAL_PACKAGE_BEFORE_CONTRACTS",
  "text": "Цей gate потрібен, щоб модель не перетворювала proposed або candidate contracts на confirmed без підтвердження."
}
```

Так Ordo зберігає і контроль, і людське пояснення.

11.10. Gate і repair action

Не кожне порушення має закінчуватися блокуванням.

Іноді правильніше дати моделі шанс виправити результат.

Наприклад:

```
{
  "op": "GATE.CHECK",
  "id": "G_OUTPUT_LANGUAGE",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "LANG_IS",
  "params": {
    "lang": "uk"
  },
  "source": "RESULT.primary",
  "on_fail": "REPAIR.REWRITE_IN_LANGUAGE"
}
```

Або:

```
{
  "op": "GATE.CHECK",
  "id": "G_MAX_ITEMS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ITEM_COUNT_LE",
  "params": {
    "max": 3
  },
  "source": "RESULT.summary",
  "on_fail": "REPAIR.COMPRESS"
}
```

Repair action має бути безпечним.

Якщо модель може виправити проблему без нових даних і без порушення контракту, `repair` дозволений.

Якщо для виправлення потрібне нове знання або підтвердження людини, `repair` не дозволений. Тоді потрібен `block` або `request_confirmation`.

11.11. Gate і користувач

Gate не має робити процес незручним для користувача.

Погано, якщо модель після кожної дрібниці каже:

Потрібне підтвердження. Потрібне підтвердження. Потрібне підтвердження.

Ordo має відрізняти:

- що модель може безпечно зробити сама;
- що можна позначити як `assumption`;
- що можна залишити як `open question`;
- що потребує `explicit approval`;
- що є `hard stop`.

Наприклад, для чернетки документа можна залишити `placeholder`.

Але для фінального контракту не можна мовчки залишити припущення.

Тому `gate` має знати контекст:

```
mode:
  draft: allow_placeholders
  final: require_confirmed_contracts
```

Це дозволяє Ordo бути не тільки суворою, а й практичною.

11.12. Gate report

Після важливого етапу Ordo-програма має вміти показати `gate report`.

`Gate report` — це короткий звіт про те, які перевірки пройдено, які ні, і що це означає.

Приклад:

```
{
  "trace_source": "hybrid",
  "gate_report": [
    {
      "id": "G_PATH_SELECTED",
      "method": "mechanical",
      "trust_class": "deterministic",
      "status": "passed",
      "evidence": "terminal_path = Path 1 candidate"
    },
    {
      "id": "G_SOURCE_ROW_CONFIRMED",
      "method": "human",
      "trust_class": "human_decision",
      "status": "failed",
      "reason": "representative source row not provided",
      "next_action": "request source row or keep package in draft mode"
    },
    {
      "id": "G_NO_FINAL_PACKAGE",
      "method": "mechanical",
```

```

    "trust_class": "deterministic",
    "status": "blocked",
    "reason": "mandatory contracts unresolved"
  }
]
}

```

Для користувача це дуже корисно. Він бачить не просто “модель не може продовжити”, а чому саме.

Для розробника або runtime це ще корисніше, бо gate report можна перевіряти автоматично.

11.13. Типові помилки з gates

Перша помилка — писати gates як загальні побажання.

Перевірити якість.

Краще:

Перевірити, що кожен executable TC має source lookup, action, ChangeRecord lookup, history processing, event assertion, rollback і post-rollback verification.

Друга помилка — не описувати on_fail.

Якщо gate не пройдено, модель має знати, що робити.

Третя помилка — ховати hard stop у FREEFORM.

Четверта помилка — дозволяти моделі самій вирішувати, що gate passed, без evidence.

П'ята помилка — мати gate у шаблоні, але не перевіряти фінальний rendered artifact.

Шоста помилка — не розрізняти failed, blocked і requires_confirmation.

Сьома помилка — не вказувати method і створювати враження, що semantic self-check має таку саму силу, як механічна перевірка кодом.

Ці помилки роблять Ordo-програму схожою на звичайний довгий prompt. А мета Ordo — якраз уникнути цього.

11.14. Короткий підсумок розділу

Gate — це контрольна точка виконання.

Він має відповідати на питання:

Чи можна рухатися далі?

Добрий gate має:

- чітку умову;
- method;
- trust_class;
- джерело перевірки;
- статус;
- evidence;
- on_fail поведінку;
- зрозумілий вплив на наступний крок.

Gate може:

- дозволити рух далі;
- запустити repair;
- попросити підтвердження;
- заблокувати фінальний результат;
- створити gate report.

Головний принцип:

Gate – це не порада.

Gate – це точка, де Ordo-програма вирішує, чи має модель право продовжувати.

Міні-вправа

Візьміть будь-який свій процес і знайдіть у ньому три місця, де модель не має права рухатися далі без перевірки.

Для кожного місця запишіть:

1. Що перевіряємо?
2. Де це перевіряємо?
3. Який статус можливий?
4. Який `method` потрібен: mechanical, self_verification, self_consistency чи human?
5. Який `trust\_class` має результат перевірки?
6. Що робити, якщо перевірка не пройдена?
7. Чи може модель виправити це сама, чи потрібне підтвердження людини?

Після цього спробуйте записати один gate у JSON-формі.

Розділ 12. Негативні перевірки: ASSERT.NOT → ASSERTION

Навіщо це потрібно

У звичайних інструкціях ми часто пишемо моделі, що вона має зробити:

- створи документ;
- постав питання;
- перевір структуру;
- сформуль відповідь;
- підготуй архів;
- дай короткий підсумок.

Але в реальних процесах не менш важливо сказати, чого модель **не має права** робити.

Наприклад:

- не вигадувати відсутні дані;
- не переходити до наступного кроку без підтвердження;
- не створювати фінальний пакет до проходження перевірок;
- не вважати приклад правилом;
- не замінювати бізнесове рішення технічною згодкою;
- не включати у пакет зайві файли;
- не ховати невизначеність у фінальному результаті.

У prompt-ах такі заборони часто губляться. Модель може прочитати їх, погодитися, але все одно виконати дію, яка формально не дозволена. Саме тому в Ordo негативні перевірки мають бути не просто текстовими застереженнями, а окремим контрольованим правилом.



У ранніх версіях Ordo для цього використовувалась конструкція:

```
ASSERT.NOT
```

Починаючи з Ordo v0.12, точніше говорити так:

```
ASSERTION — канонічний примітив.  
ASSERT.NOT — скорочена форма для ASSERTION з polarity: not.
```

Це важлива зміна. ASSERT.NOT більше не є окремим паралельним механізмом поруч із gate і тестами. Це одна з форм єдиного правила ASSERTION, яке може бути розгорнуте в runtime-перевірку, test expectation і debug violation record.

Просте пояснення

ASSERTION — це формальне твердження про те, що в процесі має бути істинним або не має статися.

У нього є полярність:

```
must — потрібний стан має бути виконаний;  
not — заборонений стан не має статися.
```

Тому ASSERT.NOT можна мислити як assertion навпаки:

```
Чи точно не сталося заборонене?
```

Наприклад:

```
Gate:  
усі mandatory секції присутні.  
  
ASSERT.NOT:  
у фінальному пакеті немає зайвих файлів.
```

Або:

```
Gate:  
користувач підтвердив alias.  
  
ASSERT.NOT:  
модель не вигадала alias самостійно.
```

Це дуже важлива різниця. Бо позитивна перевірка не завжди ловить порушення.



Можна мати документ, у якому всі потрібні розділи присутні, але водночас у ньому є зайві, небезпечні або суперечливі блоки. Звичайний gate може сказати “структура є”, але assertion з polarity: not має сказати “зайвого немає”.

Чому Ordo v0.12 вводить ASSERTION

До v0.12 у мові легко виникали три схожі конструкції:

```
ASSERT.NOT — заборона під час виконання;  
negative gate — gate, який перевіряє відсутність проблеми;  
EXPECT.NOT — очікування в тестах, що заборонений стан не з'явиться.
```

Усі три говорять про одну і ту саму ідею: **певний стан або дія не мають статися**. Якщо тримати їх окремо, автору доводиться вручну синхронізувати правила між виконанням, тестами й debug-звітом.

Наприклад, автор може додати:

```
ASSERT.NOT final_output_before_validation
```

але забути додати відповідний EXPECT.NOT у regression suite. Тоді правило є в playbook-у, але тест його не захищає.

Тому в Ordo v0.12 канонічним стає один примітив:

```
ASSERTION
```

А `ASSERT.NOT`, `EXPECT.NOT` і `negative gate` стають його проєкціями.

Ordo-конструкція

У Source-форматі assertion може виглядати так:

```
assertions:
- id: "A_NO_INVENTED_ALIAS"
  polarity: "not"
  condition: "alias_created_without_user_confirmation"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
  on_fail: "STOP"
  message: "Alias не може бути вигаданий моделлю без підтвердження користувача."
```

Це означає:

Якщо `alias` був створений без явного підтвердження користувача, процес потрібно зупинити, а тестовий набір має містити очікування, що такий стан не допускається.

У compiled IR це може розгорнутися в кілька представлень.

Runtime-представлення:

```
{
  "op": "ASSERT.NOT",
  "id": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "condition": "alias_created_without_user_confirmation",
  "method": "self_verification",
  "severity": "block",
  "on_fail": "STOP"
}
```

Test-представлення:

```
{
  "op": "EXPECT.NOT",
  "id": "domain_pack.history_event.TE_NO_INVENTED_ALIAS",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "condition": "alias_created_without_user_confirmation"
}
```

Debug-представлення:

```
{
  "op": "VIOLATION.RECORD",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "status": "not_triggered",
  "method": "self_verification"
}
```

Важливо: `ASSERT.NOT` не просто попереджає. Якщо `severity: block`, то це жорстка зупинка.

Method для assertion

Ordo v0.12 додає `method` не тільки для `gate`, а й для `assertion`, якщо `assertion` перевіряється під час виконання.

Наприклад:

```
assertions:
- id: "A_NO_EXTRA_FILES"
  polarity: "not"
  condition: "package_contains_unapproved_files"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
```

Тут method: mechanical, бо runner або скрипт може детерміновано порівняти фактичний список файлів із дозволеним списком.

А ось інший приклад:

```
assertions:
- id: "A_NO_UNCONFIRMED_BUSINESS_DECISION"
  polarity: "not"
  condition: "final_output_contains_business_decision_without_user_confirmation"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
```

Це вже семантичне судження. Його може перевіряти critic-крок моделі за evidence-протоколом, але не можна робити вигляд, що така перевірка така сама надійна, як підрахунок файлів.

Приклад без Ordo

Уявімо prompt:

```
Підготуй аналітичний пакет. Переконайся, що всі файли є.
Не додавай зайвих файлів.
```

Модель може створити пакет, у якому є всі потрібні файли, але також додати:

```
debug_notes.md
draft_old.md
extra_payload.json
```

Потім вона може відповісти:

```
Пакет готовий. Усі файли присутні.
```

Формально вона перевірила позитивну частину. Але заборона була порушена.

Приклад з Ordo v0.12

В Ordo це краще описати так:

```
output:
  required_files:
  - "README.md"
  - "SUMMARY.json"
  - "VALIDATION_REPORT.json"

allowed_files:
- "README.md"
- "SUMMARY.json"
- "VALIDATION_REPORT.json"

gates:
- id: "G_REQUIRED_FILES_PRESENT"
  method: "mechanical"
```

```

trust_class: "deterministic"
check: "all_required_files_exist"
on_fail: "STOP"

assertions:
- id: "A_NO_EXTRA_FILES"
  polarity: "not"
  condition: "package_contains_files_outside_allowed_files"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
  on_fail: "STOP"

```

Тоді фінальна відповідь можлива тільки якщо:

1. усі потрібні файли є;
2. немає жодного зайвого файлу;
3. обидві перевірки мають явний method;
4. test suite автоматично отримує EXPECT.NOT для забороненого стану.

Це набагато сильніше, ніж загальна фраза “перевір пакет”.

Типові випадки для ASSERTION з polarity: not

Негативні assertions особливо важливі там, де модель може:

- поспішити;
- здогадатися;
- “допомогти” понад інструкцію;
- непомітно змінити зміст;
- додати зайву структуру;
- перейти до наступного етапу без дозволу;
- приховати невизначеність;
- видати проміжний результат як фінальний.

Типові ASSERT.NOT для Ordo:

```

ASSERT.NOT invented_value
ASSERT.NOT skipped_gate
ASSERT.NOT implicit_approval
ASSERT.NOT hidden_freeform_rule
ASSERT.NOT unexpected_file
ASSERT.NOT unconfirmed_contract
ASSERT.NOT unresolved_conflict
ASSERT.NOT ambiguous_status
ASSERT.NOT final_output_before_validation
ASSERT.NOT example_used_as_rule

```

У v0.12 це бажано описувати як ASSERTION:

```

assertions:
- id: "A_NO_FINAL_OUTPUT_BEFORE_VALIDATION"
  polarity: "not"
  condition: "final_output_created_before_validation_passed"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"

```

Негативні перевірки і статуси

ASSERTION тісно пов’язаний зі статусами.

Наприклад, якщо статус процесу:

```
needs_user_decision
```

то має бути негативна assertion:

```
ASSERT.NOT continue_without_user_decision
```

Якщо статус:

```
blocked_by_missing_contract
```

то має бути:

```
ASSERT.NOT generate_final_artifact
```

Інакше статус стає декоративним. Модель ніби знає, що процес заблокований, але все одно може продовжити.

В Ordo статус має впливати на дозволені дії.

Негативні перевірки і FREEFORM



Одне з найнебезпечніших місць — FREEFORM.

FREEFORM потрібен, коли частину знання ще рано або неможливо повністю формалізувати. Але він не має ставати сховищем прихованих правил.

Тому для FREEFORM корисні такі assertions:

```
ASSERT.NOT core_gate_hidden_in_freeform
ASSERT.NOT mandatory_output_hidden_in_freeform
ASSERT.NOT approval_rule_hidden_in_freeform
ASSERT.NOT business_decision_hidden_in_freeform
```

Простіше кажучи:

```
Пояснення може бути у FREEFORM.
Але контрольна точка має бути формальною.
```

Якщо правило зупиняє процес, змінює path, визначає output або потребує підтвердження людини, воно не повинно бути лише у FREEFORM.

Негативні перевірки в роботі з документами

Для документації assertions з polarity: not майже обов'язкові.

Приклади:

```
ASSERT.NOT duplicate_section
ASSERT.NOT unresolved_placeholder
ASSERT.NOT obsolete_term
ASSERT.NOT inconsistent_alias
ASSERT.NOT missing_traceability
ASSERT.NOT markdown_link_to_internal_draft
ASSERT.NOT raw_runtime_note_in_final_document
```

Це допомагає ловити те, що звичайна позитивна перевірка часто пропускає.

Наприклад, документ може мати всі потрібні секції, але все ще містити:

```
<TODO>
<ASK_USER_LATER>
old_alias
draft only
```

Позитивний gate “структура правильна” цього не зловить. Негативна assertion — зловить.

Severity: коли зупиняти, а коли попереджати

Не кожна негативна перевірка однаково критична.

Тому в Ordo для ASSERTION потрібна severity:

```
info
warn
block
```

Наприклад:

```
assertions:
- id: "A_NO_TODO_IN_FINAL"
  polarity: "not"
  condition: "final_artifact_contains_todo_marker"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"

- id: "A_NO_MINOR_STYLE_DRIFT"
  polarity: "not"
  condition: "document_contains_minor_style_inconsistency"
  phase: ["runtime"]
  method: "self_verification"
  severity: "warn"
```

Правило просте:

```
Якщо порушення може зробити результат неправильним або небезпечним – block.
Якщо порушення лише погіршує якість, але не ламає результат – warn.
```

Compiler projection

У v0.12 компілятор має вміти розгортати assertion у кілька цільових форм.

Наприклад:

```
assertions:
- id: "A_NO_SKIPPED_GATE"
  polarity: "not"
  condition: "required_gate_was_skipped"
  phase: ["runtime", "test", "debug"]
  method: "mechanical"
  severity: "block"
```

Компілятор може створити:

```
runtime: ASSERT.NOT required_gate_was_skipped
test:    EXPECT.NOT required_gate_was_skipped
debug:   VIOLATION.RECORD if required_gate_was_skipped is true
```

Це зменшує ризик, що заборона описана в одному місці, але забута в іншому.

Типові помилки

Помилка 1. Формулювати заборону тільки словами

Погано:

```
Не роби помилок.
```

Краще:

```
ASSERT.NOT skipped_gate
ASSERT.NOT invented_required_value
ASSERT.NOT final_output_before_validation
```

Ще краще у v0.12:

```
assertions:
- id: "A_NO_FINAL_OUTPUT_BEFORE_VALIDATION"
  polarity: "not"
  condition: "final_output_before_validation"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
```

Заборона має бути перевіркою.

Помилка 2. Об'єднувати все в один великий gate

Погано:

```
Перевір, що все добре.
```

Краще:

```
GATE required_files_present
ASSERT.NOT unexpected_files_present
ASSERT.NOT unresolved_placeholders_present
ASSERT.NOT validation_report_missing
```

Один великий gate створює ілюзію контролю, але не дає точного trace.

Помилка 3. Робити негативні перевірки занадто абстрактними

Погано:

```
ASSERT.NOT bad_result
```

Краще:

```
ASSERT.NOT result_contains_unconfirmed_business_decision
```

Модель і runtime мають розуміти, що саме перевіряється.

Помилка 4. Не вказувати наслідок

Погано:

```
assertions:
- condition: "missing_approval"
```

Краще:

```
assertions:
- id: "A_NO_MISSING_APPROVAL"
```

```
polarity: "not"
condition: "missing_approval"
phase: ["runtime"]
method: "mechanical"
severity: "block"
on_fail: "STOP"
```

Без наслідку перевірка може стати просто коментарем.

Помилка 5. Забути test projection

Погано:

```
Правило є в runtime, але його немає в regression suite.
```

Краще:

```
phase: ["runtime", "test"]
```

Тоді компілятор має створити runtime assertion і test expectation з одного джерела.

Міні-вправа

Візьміть просту інструкцію:

```
Підготуй фінальний документ для передачі розробнику.
```

Спробуйте виписати п'ять негативних перевірок.

Наприклад:

1. Не має бути unresolved placeholders.
2. Не має бути вигаданих технічних деталей.
3. Не має бути суперечності між описом задачі і acceptance criteria.
4. Не має бути посилань на проміжні чернетки.
5. Не має бути фінального статусу без self-check.

Потім перепишіть їх у стилі Ordo v0.12:

```
assertions:
- id: "A_NO_UNRESOLVED_PLACEHOLDER"
  polarity: "not"
  condition: "unresolved_placeholder_present"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"

- id: "A_NO_INVENTED_TECHNICAL_DETAIL"
  polarity: "not"
  condition: "invented_technical_detail_present"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
```

Короткий підсумок

ASSERTION — це канонічний спосіб описати правило, яке має бути виконане або не має бути порушене.

ASSERT.NOT — це скорочена форма для assertion з polarity: not.

Звичайний gate перевіряє, що потрібне виконано. ASSERT.NOT перевіряє, що заборонене не сталося.

Для Ordo це критично, бо AI-модель часто помиляється не тільки тоді, коли щось не зробила, а й тоді, коли зробила зайве: вигадала, перескочила, додала, приховала, змішала або передчасно завершила.

Добра Ordo-програма має не лише список того, що треба зробити, а й список того, чого робити не можна. У v0.12 це краще описувати один раз як ASSERTION, а компілятор має розгорнути це правило у runtime, test і debug-представлення.

Розділ 13. Status semantics

Навіщо це потрібно

У звичайній розмові зі штучним інтелектом статус часто виглядає як дрібниця.

Модель може написати:

```
готово
зроблено
можна передавати
виглядає коректно
потрібне уточнення
```

На перший погляд, усе зрозуміло. Але для керованого процесу цього недостатньо.

Проблема в тому, що такі слова не завжди мають чітке значення. Для однієї людини “готово” означає “чернетку можна прочитати”. Для іншої — “файл можна передавати розробнику”. Для третьої — “усі перевірки пройдено, ризиків немає”.

Для Ordo така нечіткість небезпечна. Якщо статус не має формальної семантики, модель може передчасно завершити процес, перескочити approval gate, видати чернетку як фінальний результат або назвати результат “готовим” без доказів.

Тому в Ordo потрібна окрема частина мови:

```
STATUS.SEMANTICS
```

Вона визначає, що саме означає кожний статус, хто може його встановити, які gate-и мають бути пройдені перед цим і які дії дозволені після нього.

Просте пояснення

Статус — це не просто етикетка.



У Ordo статус — це контрольована позиція процесу.

Наприклад:

```
needs_input
ready_for_analysis
analysis_in_progress
needs_approval
approved_for_generation
ready_for_handoff
blocked
```

Кожний статус має відповідати на кілька питань:

1. Що вже відбулося?
2. Що ще не відбулося?
3. Хто має право перевести процес у цей статус?
4. Які дії тепер дозволені?

5. Які дії тепер заборонені?
6. Який наступний легальний статус?

Без цього статуси стають декоративними словами. З цим — вони стають частиною виконання.

Чому статуси важливі саме для AI-моделі

Людина часто розуміє статус із контексту. Якщо аналітик каже “можна”, інша людина може уточнити, що саме можна: продовжити аналіз, створити файл, віддати в QA чи передати розробнику.

AI-модель так не працює надійно. Вона може взяти слово “можна” занадто широко.

Наприклад, користувач каже:

Так, варіант підходить.

Модель може помилково вирішити:

Користувач затвердив фінальну генерацію архіву.

А насправді користувач міг мати на увазі тільки:

Мені підходить ідея, але ще не фінальний документ.

STATUS.SEMANTICS потрібен, щоб такі переходи не відбувалися автоматично.

Ordo-конструкція

У Ordo статус можна описати так:

```
status_semantics:
- status: "needs_input"
  meaning: "процес очікує дані або рішення від користувача"
  allowed_actions:
    - "ask_question"
    - "summarize_current_state"
  forbidden_actions:
    - "generate_final_package"
    - "mark_as_ready_for_handoff"
  next_allowed_statuses:
    - "ready_for_analysis"
    - "blocked"

- status: "ready_for_handoff"
  meaning: "результат пройшов mandatory gates і може бути переданий далі"
  requires_gates:
    - "G_CONTRACT_CONFIRMED"
    - "G_OUTPUT_VALIDATED"
    - "G_SELF_CHECK_PASSED"
  allowed_actions:
    - "handoff_result"
  forbidden_actions:
    - "change_contract_without_reapproval"
```

У compiled IR це може перетворитися на операції:

```
[
{
  "op": "STATUS.DEF",
  "id": "S_NEEDS_INPUT",
  "status": "needs_input",
  "meaning": "process waits for user input or decision"
```

```

},
{
  "op": "STATUS.ALLOW",
  "status": "needs_input",
  "actions": ["ask_question", "summarize_current_state"]
},
{
  "op": "STATUS.FORBID",
  "status": "needs_input",
  "actions": ["generate_final_package", "mark_as_ready_for_handoff"]
},
{
  "op": "STATUS.REQUIRE.GATES",
  "status": "ready_for_handoff",
  "gates": ["G_CONTRACT_CONFIRMED", "G_OUTPUT_VALIDATED", "G_SELF_CHECK_PASSED"]
}
]

```

Головне не конкретний синтаксис, а принцип: статус має бути виконуваним правилом, а не вільною фразою.

Status lifecycle

Для Ordo-програми корисно описувати не тільки окремі статуси, а й життєвий цикл процесу.

Наприклад:

```

created
→ needs_input
→ contract_ready
→ contract_confirmed
→ analysis_in_progress
→ needs_approval
→ approved_for_generation
→ generation_in_progress
→ validation_in_progress
→ ready_for_handoff
→ handed_off

```

Такий lifecycle показує, як процес має рухатися.

Важливо: модель не повинна сама вигадувати переходи між статусами.

Якщо дозволено тільки:

```
needs_approval → approved_for_generation
```

то модель не може перейти напямую:

```
needs_approval → ready_for_handoff
```

Навіть якщо їй здається, що “все вже зрозуміло”.

Status transition

Перехід між статусами — це окрема дія.

У простому вигляді:

```

status_transitions:
- from: "needs_approval"
  to: "approved_for_generation"
  allowed_by:

```

```
- "user_explicit_approval"
requires:
  - "approval_scope_defined"
on_missing_requirement: "STOP"
```

Це означає: модель не може просто сказати “схоже, затверджено”. Має бути явне підтвердження або інша дозволена умова.

У Ordo бажано розрізняти:

```
model_can_set
user_must_set
runtime_can_set
external_system_can_set
```

Наприклад:

```
status: "contract_confirmed"
can_be_set_by:
  - "user"
forbidden_for:
  - "model_without_explicit_approval"
```

Це особливо важливо для процесів, де модель має допомагати, але не має права приймати бізнес-рішення замість людини.

Статуси і gates

Статус не має встановлюватися без gate-ів.

Погано:

```
Модель: усе перевірено, статус ready_for_handoff.
```

Краще:

```
set_status: "ready_for_handoff"
requires_gates:
  - "G_REQUIRED_FILES_PRESENT"
  - "G_NO_UNRESOLVED_PLACEHOLDERS"
  - "G_CONSISTENCY_CHECK_PASSED"
  - "G_HANDOFF_NOTE_PRESENT"
```

У цьому випадку статус — це не думка моделі. Це результат проходження контрольних точок.

Якщо хоча б один gate не пройдено, статус не може бути встановлений.

```
G_CONSISTENCY_CHECK_PASSED = failed
→ ready_for_handoff заборонено
```

Саме так Ordo зменшує ризик передчасного “готово”.

Статуси і ASSERT.NOT

Негативні перевірки також можуть бути прив’язані до статусів.

Наприклад:

```
status: "ready_for_handoff"
assert_not:
  - "unresolved_placeholders_present"
  - "missing_validation_report"
```

- "unconfirmed_assumption_present"
- "freeform_without_binding"

Це означає, що процес не може назватися готовим, якщо в ньому залишилися заборонені умови.

Для фінального статусу часто важливі саме негативні перевірки:

немає незаповнених місць
 немає вигаданих рішень
 немає невидимих припущень
 немає неперевірених документів
 немає не поясненого FREEFORM

Статуси і користувацькі відповіді



Одна з найважливіших проблем у роботі з AI-моделями — інтерпретація коротких відповідей користувача.

Користувач може написати:

так
 ок
 підходить
 далі
 можна
 згоден

Без STATUS.SEMANTICS модель може трактувати це занадто широко.

Ordo має вимагати, щоб коротка відповідь користувача інтерпретувалась тільки в межах поточного node або поточного gate.

Наприклад, якщо поточне питання було:

Обрати варіант А, В або С?

і користувач відповів:

А

це не означає:

користувач затвердив фінальний пакет

Це означає тільки:

користувач обрав варіант А у поточному node

Тому в Ordo статус повинен бути пов'язаний з current node, current gate і current question.

Приклад: guided intake

Уявімо, що модель проводить guided intake нової історичної події.

Можливі статуси:

```
statuses:
- "intake_started"
- "path_selected"
- "event_alias_confirmed"
- "source_row_confirmed"
- "values_confirmed"
- "qa_scope_confirmed"
- "package_ready_for_generation"
- "package_generated"
- "package_validated"
- "package_ready_for_handoff"
```

Для кожного статусу можна задати allowed transitions:

```
transitions:
- from: "intake_started"
  to: "path_selected"
  requires: ["path_decision_answered"]

- from: "path_selected"
  to: "event_alias_confirmed"
  requires: ["alias_confirmed_by_user"]

- from: "package_generated"
  to: "package_validated"
  requires: ["self_check_executed", "validation_report_created"]

- from: "package_validated"
  to: "package_ready_for_handoff"
  requires: ["no_blocking_validation_errors"]
```

Тепер модель не просто “йде за розмовою”. Вона рухається через контрольовану карту статусів.

Приклад: документ

Для підготовки документа статуси можуть бути такими:

```
draft_requested
structure_confirmed
content_drafted
content_reviewed
rendered_artifact_created
rendered_artifact_validated
ready_to_share
```

У цьому випадку важливо не плутати:

```
content_drafted
```

i

```
ready_to_share
```

Звичайна модель може написати “готово” після створення тексту. Але для Ordo текст ще не готовий, якщо не пройдено перевірку фінального артефакту.

Наприклад:

```
status: "ready_to_share"
requires:
- "content_reviewed"
- "rendered_artifact_created"
```

```
- "rendered_artifact_validated"
- "download_link_available"
```

Це особливо важливо для PDF, DOCX, презентацій, архівів і пакетів документів.

Чернетка, фінал і handoff

Одна з головних причин вводити статуси — розвести три різні стани:

```
draft
final
handoff_ready
```

Це не одне й те саме.

draft означає:

```
текст або структура створені, але ще можуть бути неповними
```

final означає:

```
зміст вважається завершеним у межах поточного contract
```

handoff_ready означає:

```
результат не тільки завершений, а й перевірений, упакований і готовий до передачі
```

У Ordo не можна підміняти один статус іншим.

Наприклад:

```
final_content != handoff_ready_package
```

Фінальний текст може бути готовим, але файл ще не перевірений. Або пакет може бути сформований, але validation report показує помилки. У таких випадках статус handoff_ready недозволений.

Status report

Ordo-процес має вміти показати статус не тільки словом, а й коротким звітом.

Наприклад:

```
status_report:
  current_status: "needs_approval"
  current_node: "N_APPROVE_PACKAGE_SCOPE"
  completed_gates:
    - "G_PATH_SELECTED"
    - "G_ALIAS_CONFIRMED"
  pending_gates:
    - "G_PACKAGE_SCOPE_APPROVED"
  blocked_actions:
    - "generate_final_archive"
  next_allowed_actions:
    - "ask_package_scope_approval"
```

Такий звіт корисний і для користувача, і для runtime, і для наступної моделі, яка може підхопити процес.

Він також зменшує ризик втрати контексту в довгих діалогах.

Статуси і trace



Статуси повинні залишати слід.

Недостатньо просто мати поточний статус. Потрібна історія переходів:

```
trace_source: "model_self_report"
status_trace:
  - from: "needs_input"
    to: "contract_ready"
    reason: "all mandatory intake fields collected"
    evidence: ["E_INPUT_FIELDS_COMPLETE"]

  - from: "contract_ready"
    to: "contract_confirmed"
    reason: "user explicitly approved contract"
    evidence: ["E_USER_APPROVAL_2026_07_05"]
```

Це дає відповідь на питання:

```
чому процес зараз у цьому статусі?
хто або що його сюди перевело?
які докази цього переходу?
```

Для Ordo це дуже важливо, бо без trace статус знову перетворюється на заяву моделі.

Типові помилки

Помилка 1. Використовувати розмиті статуси

Погано:

```
ok
ready
done
almost_done
good
```

Краще:

```
contract_confirmed
content_drafted
validation_passed
ready_for_handoff
```

Статус має описувати конкретний стан процесу.

Помилка 2. Не розділяти approval і execution

Погано:

```
approved
```

Незрозуміло, що саме затверджено: ідея, структура, contract, генерація, фінальний файл?

Краще:


```
contract_approved
package_scope_approved
generation_approved
handoff_approved
```

Помилка 3. Дозволяти моделі самій ставити фінальні статуси

Погано:

```
status: "ready_for_handoff"
can_be_set_by: ["model"]
```

Краще:

```
status: "ready_for_handoff"
can_be_set_by: ["runtime"]
requires_gates:
  - "G_SELF_CHECK_PASSED"
  - "G_RENDERED_ARTIFACT_VALIDATED"
  - "G_NO_BLOCKING_ERRORS"
```

Або, якщо runtime немає:

```
can_be_set_by: ["model_after_required_trace"]
```

але тільки з обов'язковим gate report.

Помилка 4. Не описувати allowed transitions

Погано:

```
status може бути будь-яким із переліку
```

Краще:

```
from: "needs_approval"
to: "approved_for_generation"
requires: ["explicit_user_approval"]
```

Статуси без переходів — це словник. Статуси з переходами — це процес.

Помилка 5. Плутати статус і результат

```
status = ready_for_handoff
```

не є самим handoff. Це тільки дозвіл на handoff.

Так само:

```
status = validation_passed
```

не є validation report. Це результат проходження validation gate, який має мати evidence.

Міні-вправа

Візьміть будь-який процес, який ви часто доручаєте AI-моделі.

Наприклад:

```
Підготувати задачу для Jira.
```

Спробуйте описати статуси:

```
request_received
problem_context_collected
acceptance_criteria_drafted
qa_scope_drafted
pm_level_review_needed
ready_for_jira_copy
```

Потім для кожного статусу дайте відповідь:

1. Що означає цей статус?
2. Хто може його встановити?
3. Які gate-и потрібні перед ним?
4. Які дії дозволені після нього?
5. Які дії заборонені після нього?
6. Який наступний статус дозволений?

Після цього ви побачите, що навіть простий процес стає набагато контрольованішим.

Короткий підсумок

STATUS.SEMANTICS визначає, що саме означають статуси в Ordo-програмі.

Статус у Ordo — це не просто слово “готово” або “зроблено”. Це формальний стан процесу з правилами, дозволеними діями, заборонами, переходами, gate-ами і evidence.

Добре описані статуси захищають процес від передчасного завершення, неправильного трактування коротких відповідей користувача, прихованих переходів і змішування чернетки з фінальним handoff.

Якщо Gate відповідає на питання “чи можна пройти далі?”, то STATUS.SEMANTICS відповідає на питання “де саме ми зараз у процесі і що нам дозволено робити далі?”.

Розділ 14. Чому Ordo-програми потрібно дебажити

Навіщо це потрібно

Після того як у Ordo з'являються State, Node, Gate, Status semantics і Output, вона перестає бути просто красивим способом писати інструкції. Вона стає програмою поведінки для AI-моделі.

А будь-яку програму потрібно вміти дебажити.

У звичайному програмуванні це очевидно. Якщо код працює не так, розробник дивиться логи, ставить breakpoint, перевіряє значення змінних, дивиться stack trace, повторює запуск на тестових даних.

У роботі з AI-інструкціями часто відбувається навпаки. Ми даємо моделі довгий prompt, отримуємо неправильну поведінку і бачимо тільки фінальну відповідь.

Наприклад:

Модель пропустила важливий крок.
Модель поставила питання не в тому місці.
Модель створила фінальний документ без підтвердження.
Модель переплутала правило і приклад.
Модель обрала неправильний сценарій.
Модель використала старе зауваження як чинне правило.

Але головне питання лишається без відповіді:

Чому вона так зробила?

Без відповіді на це питання робота з великими інструкціями перетворюється на хаотичне переписування prompt-ів.

Ви виправляєте одну фразу — ламається інша поведінка. Додаєте нове правило — модель починає трактувати його занадто широко. Забороняєте одну дію — модель перестає робити потрібну схожу дію. У результаті з'являються десятки версій інструкцій, і вже незрозуміло, яка з них справді краща.

Саме тому debug має бути не зовнішнім допоміжним інструментом, а частиною самої Ordo-мови.



Просте пояснення

Debug в Ordo — це режим, у якому модель не просто виконує інструкцію, а залишає зрозумілий слід виконання.

Цей слід має відповідати на питання:

- який шлях був обраний;
- які шляхи були відхилені;

- чому був обраний саме цей шлях;
- який state був до кроку;
- як state змінився після кроку;
- який gate спрацював;
- який gate був провалений;
- які правила були використані;
- які знання або документи вплинули на рішення;
- де саме виникла помилка;
- чи була дія дозволена контрактом.

У звичайному режимі користувач бачить результат.

У debug-режимі користувач або автор Ordo-програми бачить механіку виконання.

Це різні речі.

Результат може виглядати правильним, але бути отриманим неправильним шляхом. І навпаки: результат може бути неповним, але debug trace покаже, що модель правильно зупинилась на gate, бо їй бракувало підтвердження.

Тому Ordo має оцінювати не тільки фінальний output, а й шлях до нього.



Чому це особливо важливо для AI

Класична програма виконує інструкції буквально. Якщо в коді написано `if`, програма не розмірковує, чи мав автор на увазі щось інше.

AI-модель працює інакше. Вона інтерпретує інструкції, узагальнює приклади, робить припущення, підставляє контекст, іноді змішує правила з поясненнями.

Це корисно, коли потрібно працювати з людським змістом.

Але це небезпечно, коли потрібно виконати керований процес.

Тому Ordo має дати моделі не тільки інструкцію, а й рамку пояснюваного виконання.

Модель має не просто сказати:

Я вирішила, що треба йти цим шляхом.

Вона має показати:

Я обрала Path A1, тому що були підтверджені такі умови.
Я відхилила Path A2, тому що не було підтверджено зв'язану сутність.
Я не перейшла до фінального output, тому що gate G_PRE_APPROVAL ще не passed.

Це і є різниця між “модель щось зробила” і “модель виконала Ordo-програму”.

Ordo-конструкція

Для цього в Ordo вводиться окремий шар:

Ordo Debug, Test & Improvement Layer

У цьому розділі нас цікавить перша його частина:

```
Debug Layer
```

Мінімальні конструкції debug layer:

```
DEBUG.MODE
TRACE.LOG
DECISION.LOG
PATH.EXPLAIN
STATE.SNAPSHOT
STATE.DIFF
GATE.REPORT
KNOWLEDGE.TRACE
FAILURE.EXPLAIN
RUN.DRY
RUN.REPLAY
```

Їх можна уявити так:

DEBUG.MODE	– вмикає режим пояснюваного виконання;
TRACE.LOG	– загальний журнал виконання;
DECISION.LOG	– журнал прийнятих рішень;
PATH.EXPLAIN	– пояснення вибору або відхилення path;
STATE.SNAPSHOT	– знімок state у певний момент;
STATE.DIFF	– зміна state між двома моментами;
GATE.REPORT	– результат перевірки gates;
KNOWLEDGE.TRACE	– які знання / документи / правила були використані;
FAILURE.EXPLAIN	– пояснення помилки або зупинки;
RUN.DRY	– прогін без реального створення результату;
RUN.REPLAY	– повторне відтворення попереднього запуску.

Ці конструкції не є косметичними. Вони потрібні для того, щоб Ordo-програму можна було реально розвивати.

Малий приклад без debug

Уявімо просту інструкцію:

```
Підготуй пакет документів, але не створи фінальний архів без підтвердження користувача.
```

Модель створила фінальний архів одразу.

Без debug ми бачимо тільки проблему:

```
Фінальний архів створено занадто рано.
```

Але ми не знаємо, чому це сталося.

Можливі причини:

- модель не побачила правило про підтвердження;
- правило було в FREEFORM і не було gate-ом;
- gate існував, але не був blocking;
- state помилково містив approval=true;
- приклад у документі показував фінальний архів і модель сприйняла його як дозвіл;
- інша бібліотека перевизначила поведінку;
- output template був прив'язаний раніше, ніж approval gate.

Це різні проблеми. І виправляються вони по-різному.

Малий приклад з debug

У debug mode Ordo має сформувати приблизно такий trace:

```

run:
  id: "RUN-001"
  mode: "debug"

  trace_source: "model_self_report"
path_selection:
  selected:
    path_id: "package_generation"
    reason: "user requested package creation"

state_snapshots:
- step: "before_archive_generation"
  state:
    user_approval_received: false
    validation_passed: true

gate_report:
- gate_id: "G_PRE_ARCHIVE_APPROVAL"
  expected: "passed before archive generation"
  actual: "not_passed"
  severity: "blocking"

violation:
  id: "V_ARCHIVE_CREATED_BEFORE_APPROVAL"
  type: "gate_violation"
  explanation: "final archive generation was attempted before required user approval"

```

Тепер проблема стає зрозумілою.

Це вже не просто “модель помилилась”. Це конкретне порушення:

```
archive generation attempted before G_PRE_ARCHIVE_APPROVAL passed
```

Таку помилку можна виправляти системно:

- зробити gate blocking;
- додати ASSERT.NOT;
- додати regression test;
- оновити Domain Pack;
- оновити template binding;
- зафіксувати improvement record.

Debug як захист від самовпевнених відповідей

Одна з небезпек AI-моделей — вони можуть звучати впевнено навіть тоді, коли процес був неправильним.

Модель може сказати:

```
Готово, пакет сформовано.
```

Але debug trace може показати:

- source row не був підтверджений;
- expected values не були зібрані;
- QA package не був перевірений;
- self-check не був виконаний;
- final archive був створений перед approval gate.

Тобто Ordo має вчити нас не довіряти тільки гарному фінальному тексту.

Потрібно дивитися, чи був виконаний правильний процес.

Debug і state

Особливо важливо дебажити State.

У складних Ordo-програмах помилки часто виникають не тому, що модель не знає правила, а тому, що вона неправильно вважає state.

Наприклад:

```
alias_confirmed = true
```

хоча користувач лише запропонував alias, але ще не підтвердив його.

Або:

```
source_field_known = true
```

хоча поле було взято з припущення, а не з даних.

Тому debug trace має показувати не тільки поточний state, а й причину кожної зміни state.

Погано:

```
state:
  alias_confirmed: true
```

Добре:

```
state_change:
  field: "alias_confirmed"
  before: false
  after: true
  reason: "user explicitly confirmed alias"
  evidence: "message at step 4"
```

Якщо немає reason і evidence, зміна state має бути підозрілою.

Debug і path selection

Path selection — ще одне критичне місце.

У великих playbook-ах модель часто має обрати один із кількох сценаріїв.

Наприклад:

```
A1 — зміна основної сутності;
A2 — зміна пов'язаної сутності;
A3 — по-ор;
A4 — зовнішня історична подія;
A5 — ручний або нестандартний сценарій.
```

Без debug ми бачимо тільки те, що модель пішла шляхом A1.

З debug ми маємо бачити:

```
selected_path:
  id: "A1"
  reason: "confirmed source row belongs to main entity"

rejected_paths:
  - id: "A2"
```

```
reason: "no related entity identifier confirmed"
- id: "A4"
reason: "no ExternalHistoryEvent payload provided"
```

Це дозволяє перевірити не тільки обраний шлях, а й те, що альтернативи були відхилені правильно.

Debug і знання

Ще одна важлива частина — `KNOWLEDGE.TRACE`.

Модель має показувати, на які джерела вона спиралась:

```
- Ordo Core;
- Profile;
- Domain Pack;
- Library;
- конкретний rule;
- конкретний template;
- конкретний FREEFORM block;
- user-provided context.
```

Це не означає, що модель має розкривати приватний chain of thought. Ordo debug trace не повинен бути потоком прихованих думок моделі.

Він має бути структурованим звітом про виконання.

Різниця така:

```
Не потрібно: внутрішні міркування моделі слово в слово.
Потрібно: які правила, дані, gates і state transitions були використані.
```

Ordo debug — це не “прочитати думки моделі”.

Ordo debug — це “побачити протокол виконання”.



Типові помилки

Помилка 1. Дебажити тільки фінальний текст

Фінальний текст — це лише результат. У Ordo треба дебажити процес.

Помилка 2. Не логувати відхилені paths

Важливо знати не тільки що було обрано, а й чому інші варіанти не були обрані.

Помилка 3. Не логувати state changes

Якщо state змінився без reason/evidence, це потенційна помилка.

Помилка 4. Вважати debug зайвим для playbook-ів

Навпаки: чим більший playbook, тим більш обов’язковим є debug.

Помилка 5. Плутати debug trace з chain of thought

Ordo не потребує приватного потоку думок моделі. Ordo потребує структурованого execution trace.

Помилка 6. Не зберігати run_id

Без run_id неможливо пов'язати помилку, feedback, test і regression.

Міні-вправа

Візьміть будь-яку складну інструкцію, яку ви вже давали AI-моделі.

Наприклад:

Підготуй задачу для розробника, але спочатку уточни всі невідомі поля і не додавай технічних деталей, яких немає в описі.

Спробуйте виписати, що має бути в debug trace:

1. Який intent?
2. Який contract?
3. Які paths можливі?
4. Який path обрано?
5. Які paths відхилено?
6. Які state fields змінюються?
7. Які gates мають бути blocking?
8. Які outputs заборонені до підтвердження?
9. Які знання або документи використані?
10. Який failure мав би бути показаний, якщо модель пропустить уточнення?

Після цього спробуйте сформулювати один gate, який обов'язково має з'явитись у trace.

Наприклад:

G_UNKNOWN_FIELDS_RESOLVED

І одне порушення:

V_TASK_CREATED_WITH_UNCONFIRMED_FIELD

Короткий підсумок

Ordo-програму потрібно дебажити, бо вона керує поведінкою AI-моделі, а не просто описує бажаний текст.

Debug Layer потрібен, щоб бачити:

- вибір path;
- відхилення альтернатив;
- зміни state;
- оцінку gates;
- використані знання;
- порушення;
- причину зупинки;
- причину помилки.

Головне правило:

У складній Ordo-програмі неправильний результат – це не просто неправильний output, а неправильний або непояснений execution path.

Тому Ordo має підтримувати debug як частину мови.

Без debug Ordo-процеси неможливо надійно розвивати.

З debug Ordo стає не тільки мовою виконання, а й мовою пояснюваного виконання.

Розділ 15. Debug mode і діагностичний trace

Навіщо це потрібно

У попередньому розділі ми зафіксували головну проблему: складну Ordo-програму неможливо нормально розвивати, якщо видно тільки фінальну відповідь моделі.

Фінальна відповідь — це лише верхівка процесу. Вона показує, що модель сказала в кінці, але не показує, як вона до цього прийшла.

Для простих задач цього іноді достатньо. Наприклад, якщо модель переписує коротке повідомлення, можна просто оцінити результат: звучить добре чи ні.

Але для playbook-a, domain pack-a, бібліотеки або багатокрокового процесу цього недостатньо. Там важливо знати:

- який path було обрано;
- які paths були відхилені;
- які rules спрацювали;
- які gates були перевірені;
- які gates були пропущені;
- який state був до кроку;
- який state став після кроку;
- які знання або фрагменти інструкцій використані;
- де модель зробила припущення;
- де вона мала зупинитися, але не зупинилася.

Саме для цього в Ordo потрібен debug mode.

Debug mode — це режим виконання, у якому Ordo-програма не просто дає результат, а формує повний execution trace.



Простими словами:

- normal mode відповідає: що зроблено;
- debug mode відповідає: чому це було зроблено саме так.

Просте пояснення

Уявімо, що Ordo-програма — це маршрут у навігаторі.

У звичайному режимі навігатор просто веде вас до точки призначення.

У debug mode він додатково показує:

- чому вибрав саме цю дорогу;
- які дороги відкинув;
- де були обмеження;
- де були затори;
- де він перебудував маршрут;
- на які дані спирався;
- що сталося б, якби обрати інший шлях.

Для Ordo це означає: модель має показати не приховане мислення, а формальну трасу виконання.

Це важлива різниця.

Ordo не повинна вимагати від моделі розкривати приватний chain of thought. Натомість Ordo повинна вимагати structured execution trace — тобто запис формальних рішень, які є частиною процесу.



Не потрібно бачити все, що модель “думала”. Потрібно бачити те, що вона **виконала**:

- який NODE активний;
- який path selected;
- який gate evaluated;
- який state changed;
- який output allowed;
- який rule used;
- який warning raised.

Це не психологія моделі. Це журнал виконання програми.

Debug mode як частина мови

У Ordo debug mode не повинен бути зовнішнім коментарем у стилі:

Поясни, чому ти так зробив.

Це занадто слабо. Модель може пояснити красиво, але не обов’язково точно.

У мові має бути формальна конструкція:

```
run:
  mode: debug
  execution_mode: chat_internal
  trace_required: true
```

Або у compiled IR:

```
{
  "op": "DEBUG.MODE",
  "mode": "debug",
  "execution_mode": "chat_internal",
  "trace_required": true
}
```

Це означає, що Ordo-програма запускається не тільки для результату, а й для збору execution trace.

У debug mode результат без trace вважається неповним.

У Ordo v0.12 поруч із mode обов’язково фіксується execution_mode. Це чесно показує, у якому середовищі виконується процес:

```
full_runtime  — переходи й hard gates примусово контролює runtime або helper-runner;
chat_internal — модель веде процес у чаті, але частина перевірок може виконуватися кодом або
```

```
файлами сесії;
freeform_only — модель виконує дисципліну Ordo текстом без зовнішнього примусового контролю.
```

Це не дрібна технічна деталь. Один і той самий debug trace має різну силу залежно від того, хто реально контролював переходи: код, модель у чаті чи тільки текстова самодисципліна.

Що таке execution trace

Execution trace — це структурований журнал виконання Ordo-програми.

Він має відповідати на питання:

```
що було на вході;
який стан був на початку;
який path обрано;
чому саме цей path;
які paths відхилені;
які вузли пройдені;
які gates перевірені;
які decisions прийняті;
які state changes сталися;
які outputs дозволені;
які outputs заблоковані;
які джерела або знання використані;
які warning або violation виникли.
```

Мінімальна структура trace може виглядати так:

```
trace:
  run_id: "RUN-001"
  mode: "debug"
  execution_mode: "chat_internal"
  trace_source: "model_self_report"

  input_snapshot:
    user_message: "створюємо подію зміни статусу компанії"

  selected_path:
    id: "A1"
    reason: "користувач описав зміну поля в основному source row"

  rejected_paths:
    - id: "A2"
      reason: "немає підтвердження, що зміна стосується пов'язаної сутності"
    - id: "A4"
      reason: "немає ExternalHistoryEvent"

  nodes:
    - id: "NODE_SELECT_PATH"
      status: "completed"
    - id: "NODE_COLLECT_CONTRACT"
      status: "active"

  gates:
    - id: "G_CONTRACT_CONFIRMED"
      status: "pending"
      reason: "source field ще не підтверджено"

  state_changes:
    - field: "event_alias"
      before: null
      after: "LU_CHANGE_STATUS"

  warnings:
    - "source field not confirmed"
```

```
violations: []
```

Цей trace не є фінальним документом для користувача. Це службовий, але контрольований артефакт виконання.

Trace source

У Ordo v0.12 кожен trace має явно показувати, звідки походить його довіра.

Для цього додається поле:

```
trace_source: model_self_report | runtime_enforced | hybrid
```

model_self_report означає, що trace сформувала сама модель. Це корисно для пояснення логіки, але не є таким самим доказом, як журнал зовнішнього runtime.

runtime_enforced означає, що trace сформований кодом runner-а або оркестратора на основі реальних переходів стану, фактичних викликів gate і записаних state snapshots.

hybrid означає змішаний режим: частина trace сформована кодом, а частина є семантичним поясненням моделі. Наприклад, STATE.DIFF може бути runtime-enforced, а PATH.EXPLAIN.reason — model self-report.

Це поле потрібне для чесності. Без нього debug trace може виглядати як класичний лог програми, хоча насправді може бути лише структурованим самозвітом моделі.

Приклад:

```
trace:
  run_id: "RUN-001"
  mode: "debug"
  execution_mode: "chat_internal"
  trace_source: "hybrid"

  runtime_enforced:
    - "state_snapshot"
    - "mechanical_gate_status"

  model_self_report:
    - "path_reason"
    - "semantic_evidence_summary"
```

Правило Ordo v0.12: trace без trace_source вважається неповним.

Run ID

Кожен запуск Ordo-програми має мати run_id.

Без run_id складно зрозуміти, про який саме запуск іде мова.

Особливо це важливо, коли користувач каже:

```
ось тут воно пішло не так
```

Ordo має мати можливість прив'язати це зауваження до конкретного run:

```
run:
  id: "RUN-2026-07-05-014"
  program: "history_event_playbook"
  version: "0.12"
```

```
mode: "debug"
execution_mode: "chat_internal"
trace_source: "hybrid"
```

Потім improvement record може сказати:

```
observed_in:
  run_id: "RUN-2026-07-05-014"
  node: "NODE_PRE_ARCHIVE_CHECK"
  gate: "G_PACKAGE_SELF_CHECK"
```

Це робить покращення не абстрактним, а прив'язаним до фактичного виконання.

Input snapshot

Debug trace має містити snapshot входу.

Це не обов'язково повна копія всіх даних, особливо якщо там є конфіденційна інформація. Але має бути достатньо, щоб зрозуміти, з чого почався run.

Наприклад:

```
input_snapshot:
  user_intent: "створити HistoryEvent для зміни статусу"
  provided_fields:
    - "alias"
    - "old_value"
    - "new_value"
  missing_fields:
    - "source_field"
    - "fixture_id"
```

Так видно, що модель не мала права переходити до фінального пакета, бо частина контракту ще не була підтверджена.

Path explain



Один із найважливіших елементів debug mode — це PATH.EXPLAIN.

Він має показати не тільки обраний path, а й причину вибору.

Поганий варіант:

```
selected_path: "A1"
```

Кращий варіант:

```
selected_path:
  id: "A1"
  reason: "input describes a direct change in the main source row"
  evidence:
    - "user provided old/new values"
    - "no related entity context was confirmed"
```

Ще кращий варіант — показати rejected paths:

```
rejected_paths:
- id: "A2"
  reason: "related entity was not confirmed"
- id: "A4"
  reason: "external history event payload was not provided"
```

Це дуже важливо для дебагу.

Без rejected paths ми бачимо тільки рішення. З rejected paths ми бачимо межі рішення.

Decision log

DECISION.LOG — це журнал формальних рішень.

Decision — це не будь-яке речення моделі. Це точка, де Ordo-програма могла піти різними шляхами.

Наприклад:

```
decision_log:
- id: "D001"
  node: "NODE_SELECT_PATH"
  decision: "select_path_A1"
  reason: "direct source row change"
  evidence:
    - "field change described"
    - "no external event"

- id: "D002"
  node: "NODE_OUTPUT_ALLOWED"
  decision: "block_final_archive"
  reason: "pre-archive approval gate is not passed"
  evidence:
    - "G_PRE_ARCHIVE_APPROVAL = pending"
```

Decision log має бути коротким, але достатньо точним.

Він не має перетворюватися на довгий художній опис.

State snapshot і State diff

У складній Ordo-програмі state — це пам'ять процесу.

Debug mode має показувати не тільки поточний state, а й зміни state.

Для цього потрібні дві конструкції:

```
STATE.SNAPSHOT
STATE.DIFF
```

STATE.SNAPSHOT показує стан у певний момент.

STATE.DIFF показує, що саме змінилося.

Наприклад:

```
state_snapshot:
  at: "before_NODE_COLLECT_CONTRACT"
  state:
    event_alias: null
    source_field: null
    output_allowed: false
```


Після кроку:

```
state_diff:
  step: "NODE_COLLECT_ALIAS"
  changes:
    - field: "event_alias"
      before: null
      after: "LU_CHANGE_STATUS"
```

Це дозволяє бачити, де модель передчасно заповнила state або, навпаки, не заповнила те, що вже було підтверджено.

Gate report

У normal mode gate може просто блокувати або дозволяти дію.

У debug mode gate має пояснювати свій статус.

Наприклад:

```
gate_report:
- gate_id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
  method: "human"
  trust_class: "human_decision"
  trace_source: "runtime_enforced"
  status: "blocked"
  reason: "user has not approved package generation"
  required_evidence:
    - "explicit user approval"
  actual_evidence: []
```

Це краще, ніж просто:

```
архів не можна створити
```

Бо видно, чого саме бракує.

Gate report має містити щонайменше:

```
- full namespaced gate id;
- method;
- trust_class;
- trace_source;
- status;
- reason;
- required evidence;
- actual evidence;
- blocking / non-blocking;
- next required action.
```

Це особливо важливо для gates з різним рівнем довіри. method: mechanical і method: self_verification можуть обидва мати status: passed, але це не однаковий тип гарантії. Перший пройшов детерміновану перевірку, другий — семантичне судження моделі за протоколом evidence.

Knowledge trace

Ordo-програма часто спирається на різні джерела:

```
- Core;
- Profile;
- Domain Pack;
- Library;
```

- user-provided data;
- uploaded playbook;
- runtime context;
- FREEFORM block.

У debug mode має бути видно, які знання були використані.

Наприклад:

```
trace_source: "model_self_report"
knowledge_trace:
  - source_type: "domain_pack"
    source_id: "history_event_domain_pack"
    section: "Path A1"
    used_for: "path selection"

  - source_type: "library"
    source_id: "ordo.validation.contract_first"
    export: "G_CONTRACT_CONFIRMED"
    used_for: "contract validation"

  - source_type: "freeform"
    source_id: "FF_ANALYST_STYLE_GUIDE"
    used_for: "response tone"
```

Це особливо важливо для FREEFORM.

Якщо модель прийняла важливе рішення на основі FREEFORM, trace має це показати. Можливо, після цього стане зрозуміло, що цей FREEFORM треба формалізувати.

Warning і violation

Debug trace має розрізняти warnings і violations.

warning — це ризик або неповнота, яка не обов'язково зупиняє процес.

violation — це порушення правила.

Наприклад:

```
warnings:
  - id: "W_SOURCE_FIELD_MISSING"
    message: "source field is not confirmed yet"
    blocking: false

violations:
  - id: "V_ARCHIVE_CREATED_BEFORE_APPROVAL"
    rule: "ASSERT.NOT final_archive_created before G_PRE_ARCHIVE_APPROVAL"
    severity: "critical"
```

Якщо є critical violation, Ordo-програма не повинна робити вигляд, що результат валідний.

Debug output для людини

Повний trace може бути довгим. Тому Ordo має відрізняти:

```
machine trace
human debug summary
```

Machine trace потрібен для runtime, тестів і компілятора.

Human debug summary потрібен користувачу.

Наприклад:

```
Debug summary:
Обрано Path A1, бо вхідні дані описують зміну поля в основному source row.
Path A2 відхилено, бо не підтверджено пов'язану сутність.
Фінальний архів заблоковано, бо gate G_PRE_ARCHIVE_APPROVAL ще не пройдений.
Потрібна наступна дія: підтвердити source field i fixture.
```

Це коротко, зрозуміло і не перевантажує людину повним JSON.

Debug mode і privacy

Debug trace не повинен бездумно виводити всі сирі дані.

У trace можуть бути конфіденційні payload-и, персональні дані, внутрішні назви або технічні деталі.

Тому Ordo має підтримувати рівні trace:

```
trace_level: summary
trace_level: standard
trace_level: full
trace_level: redacted
```

Наприклад:

```
run:
  mode: "debug"
  trace_level: "redacted"
```

У такому режимі trace може показати, що поле було використано, але приховати значення:

```
state_diff:
- field: "tax_id"
  before: "[REDACTED]"
  after: "[REDACTED]"
```

Це важливо, якщо Ordo буде використовуватися в реальних продуктах.

Execution mode і рівень гарантії

Debug mode пояснює виконання, але не сам по собі гарантує, що кожен gate справді був примусово виконаний. Для цього Ordo v0.12 додає окреме поле execution_mode.

```
program: history_event_playbook
execution_mode: full_runtime
```

Базові режими:

```
full_runtime – runtime або helper-runner контролює state, node transitions і hard gates;
chat_internal – модель працює в чаті, може запускати скрипти або вести state у файлах, але
точка виклику gate не є повністю примусовою;
freeform_only – Ordo-дисципліна виконується текстом без зовнішнього контролю.
```

Таблиця чесних гарантій:

execution_mode	Хто визначає момент виклику gate	Хто виконує перевірку	Рівень гарантії
full_runtime	код / runner	код або модель за	найвищий

execution_mode	Хто визначає момент виклику gate	Хто виконує перевірку	Рівень гарантії
		протоколом	
chat_internal	модель у чаті	код у сесії або модель	середній
freeform_only	модель	модель текстом	найнижчий

Ця таблиця потрібна, щоб не перебільшувати силу Ordo. У chat_internal режимі механічна перевірка може бути реально виконана кодом, але без зовнішнього runtime модель усе ще має сама вчасно її запустити.

Debug mode і compiler

Компілятор Ordo має вміти автоматично додавати trace points.

Автор Ordo Source не повинен вручну логувати кожну дрібницю.

Наприклад, якщо у Source є:

```
nodes:
- id: "NODE_SELECT_PATH"
  branches:
  - path: "A1"
    when: "direct source row change"
  - path: "A2"
    when: "related entity change"
```

Компілятор має автоматично створити в IR trace points:

```
{
  "op": "PATH.EXPLAIN",
  "node": "NODE_SELECT_PATH",
  "required": true,
  "trace_source": "model_self_report"
}
```

Тобто debug layer не має бути “допискою збоку”. Він має бути частиною компіляції.

Debug mode і помилки моделі

Debug trace допомагає відрізнити різні типи проблем.

Наприклад, якщо результат неправильний, причина може бути різною:

1. неправильна інструкція;
2. неповний contract;
3. неоднозначний context;
4. помилковий path selection;
5. пропущений gate;
6. неправильний output template;
7. слабкий FREEFORM;
8. конфлікт бібліотек;
9. модель не виконала IR;
10. користувач змінив вимогу посеред процесу.

Без trace усе це виглядає однаково:

```
модель помилилась
```

З trace можна сказати точніше:

помилка виникла на `NODE_SELECT_PATH`;
 модель обрала A1, хоча `fixture` відповідав A2;
 причина – у `branch condition` немає правила для `related entity` через Центр ідентифікації.

Це вже не просто скарга. Це діагноз.

Типові помилки

Помилка 1. Просити модель “пояснити”, але не вимагати `trace`

Пояснення після факту може бути красивим, але неперевірним.

Краще мати `structured trace` під час виконання.

Помилка 2. Логувати тільки `selected path`

`Selected path` сам по собі не показує, чому інші `paths` відкинуті.

Потрібно логувати `rejected paths with reasons`.

Помилка 3. Не логувати `state diff`

Якщо видно тільки фінальний `state`, важко зрозуміти, де саме він зіпсувався.

Потрібен `state diff` після важливих `nodes`.

Помилка 4. Ховати `gate report` у тексті відповіді

`Gate report` має бути структурованим.

Інакше його складно тестувати.

Помилка 5. Виводити занадто багато `debug`-інформації людині

Повний `trace` потрібен машині й автору `playbook`-а.

Користувачу часто достатньо `debug summary`.

Помилка 6. Плутати `execution trace` з `chain of thought`

Ordo не потребує приватного `reasoning`-тексту моделі.

Ordo потребує формального журналу виконання: `node`, `path`, `gate`, `state`, `output`, `evidence`.

Помилка 7. Не вказувати `trace_source`

`Trace` без `trace_source` створює хибне враження, що це завжди `runtime-log`.

У v0.12 потрібно явно показувати: це `model_self_report`, `runtime_enforced` чи `hybrid`.

Помилка 8. Не вказувати `execution_mode`

Якщо не вказати режим виконання, читач або наступний процес не розуміє, який рівень гарантії має `trace`.

`full_runtime`, `chat_internal` і `freeform_only` — це різні рівні контролю, а не різні назви одного й того самого режиму.

Міні-вправа

Візьміть просту інструкцію:

Підготуй пакет для нової історичної події, але не створюй фінальний архів, поки я не підтверджу source field і QA-сценарії.

Спробуйте описати, що має з'явитися в debug trace.

Мінімально визначте:

- run_id;
- execution_mode;
- trace_source;
- input snapshot;
- selected path;
- rejected paths;
- state before/after підтвердження source field;
- gate для заборони архіву;
- gate status до підтвердження;
- warning, якщо QA-сценарії ще не визначені;
- human debug summary.

Після цього спробуйте сформулювати violation, якщо модель все ж створила архів до підтвердження.

Короткий підсумок

Debug mode — це режим виконання Ordo-програми, у якому результат супроводжується structured execution trace.

Execution trace має показувати:

- run_id;
- execution_mode;
- trace_source;
- input snapshot;
- selected і rejected paths;
- decision log;
- state snapshots;
- state diffs;
- gate report;
- knowledge trace;
- warnings;
- violations;
- human debug summary.

Головна цінність debug mode у тому, що він перетворює проблему з:

модель чомусь зробила не те

на:

на такому node було обрано такий path з такої причини; такий gate був пропущений; такий state змінився неправильно; ось де потрібно виправити Ordo-програму.

Саме тому debug mode — це не сервісна функція навколо Ordo, а частина самої мови.

Оновлення M72.1. Цей розділ пояснює debug-представлення trace. Повний нормативний core-елемент EXECUTION_TRACE, його поля, event catalog, replay та integrity описані в розділі 74.

Розділ 16. Тести, fixtures і expected behavior

Навіщо це потрібно

Коли ми працюємо зі звичайним кодом, ми майже ніколи не вважаємо програму готовою тільки тому, що вона один раз правильно спрацювала вручну. Ми пишемо тести, перевіряємо різні сценарії, фіксуємо очікувану поведінку і запускаємо ці перевірки після змін.

З інструкціями для AI-моделей ситуація часто гірша. Інструкцію переписали, вона почала краще працювати в одному випадку, але непомітно зламала інший. Додали нове правило — модель почала ставити питання не в тому порядку. Посилили gate — модель перестала доходити до результату. Послабили gate — модель почала створювати фінальний артефакт раніше підтвердження.

Без тестів такі проблеми видно тільки випадково. А якщо playbook великий, знайти причину стає дуже складно.

Тому Ordo-програма має тестуватися не лише за фінальним текстом, а за поведінкою процесу.



Тест у Ordo відповідає не тільки на питання:

чи правильний фінальний output?

Він відповідає на ширше питання:

чи процес пройшов саме так, як мав пройти?

У v0.12 це стає ще важливішим, бо Ordo тепер явно розрізняє різні класи довіри: механічні перевірки, семантичні перевірки моделлю, повторні перевірки та рішення людини. Тест має бачити цю різницю, а не просто казати “gate пройшов”.

Просте пояснення

У звичайному prompt-підході тест часто виглядає так:

Я дав моделі вхідні дані.
Модель щось відповіла.
Відповідь наче нормальна.

Для Ordo цього недостатньо.



Ordo має перевіряти:

- який path було обрано;
- які шляхи було відхилено;
- які питання модель поставила;
- які питання модель не мала ставити;
- як оброблено unmatched input;
- який state змінився;
- які gates пройдені;
- яким методом перевірялися gates;
- який trust_class мав кожен gate;
- які gates заблокували виконання;
- чи не створено заборонений output;
- чи правильно оброблено по-ор сценарій;
- чи не було прихованого використання FREEFORM;
- чи не була виконана дія без approval;
- чи відповідає поведінка заявленому execution_mode;
- чи не порушено ASSERTION.

Тобто тест Ordo-програми — це перевірка execution behavior.

Fixture: контрольований вхід

Щоб тест був повторюваним, він має мати fixture.

Fixture — це підготовлений набір вхідних даних для тесту.

У звичайному коді fixture може бути тестовим об'єктом, записом у базі або файлом. В Ordo fixture може містити:

- повідомлення користувача;
- стартовий state;
- доступний context;
- execution_mode;
- control_level;
- підключені libraries;
- активний profile;
- domain pack;
- source documents;
- очікувані user confirmations;
- обмеження середовища;
- попередній trace, якщо тест replay-based.

Приклад:

```
fixture:
  id: "FX_HISTORY_EVENT_A1_BASIC"
  execution_mode: "chat_internal"
  control_level: "standard"
  user_message: "Створюємо історичну подію зміни статусу компанії."
  initial_state:
    event_alias: null
    source_field: null
    contract_confirmed: false
  context:
    domain_pack: "history_event"
    available_paths:
      - "A1"
      - "A2"
      - "A4"
```

Fixture потрібен, щоб ми могли повторити той самий сценарій і отримати порівнюваний результат.

Expected behavior: що саме має статися

У звичайних тестах часто перевіряють тільки результат. Наприклад:

```
очікуємо файл package.zip
```

В Ordo цього мало. Потрібно описати expected behavior.

Expected behavior — це контракт очікуваної поведінки процесу.

Він може включати:

```
- expected path;
- expected node sequence;
- expected questions;
- expected state;
- expected gates;
- expected gate methods;
- expected trust classes;
- expected assertions;
- expected output;
- expected block;
- expected no-op;
- expected warnings;
- expected trace_source;
- expected absence of forbidden actions.
```

Приклад:

```
expected:
  path:
    selected: "A1"

  questions:
    required:
      - intent: "request_event_alias_confirmation"
      - intent: "request_source_field_confirmation"
    forbidden:
      - intent: "request_final_archive_generation"

  gates:
    - id: "domain_pack.history_event.G_CONTRACT_CONFIRMED"
      method: "human"
      trust_class: "human_decision"
      status: "blocked"

  output:
    final_archive_created: false
```

Тут важливо, що тест не вимагає від моделі дослівно повторити текст. Він перевіряє смислову поведінку.

TEST.DEF

У мові Ordo тест можна описати через TEST.DEF.

Спрощено:

```
TEST.DEF:
  id: "TC_NO_FINAL_OUTPUT_BEFORE_APPROVAL"
```

```

title: "Не створювати фінальний output до approval"
mode: "test"

fixture:
  execution_mode: "chat_internal"
  user_message: "Зроби фінальний архів одразу."
  initial_state:
    approval:
      pre_archive: false

expected:
  gates:
    - id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
      method: "human"
      trust_class: "human_decision"
      status: "blocked"

  assertions:
    - id: "domain_pack.history_event.A_NO_ARCHIVE_BEFORE_APPROVAL"
      polarity: "not"
      status: "passed"

  output:
    final_archive_created: false

  not_allowed:
    - "archive.generate"
    - "handoff.mark_ready"

```

Цей тест каже: навіть якщо користувач просить зробити фінальний архів, Ordo-програма не має переходити через blocking gate без approval.

ASSERTION у тестах

У v0.12 ASSERTION стає канонічним способом описати обов'язкову або заборонену умову.

ASSERT.NOT, negative gate і EXPECT.NOT більше не мають бути трьома окремими правилами, які автор підтримує вручну. Це різні проекції одного assertion.

Наприклад:

```

ASSERTION.DEF:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  polarity: "not"
  condition: "alias_created_without_user_confirmation"
  phase:
    - runtime
    - test
  severity: "block"
  on_fail: "STOP"

```

Компілятор розгортає це правило у runtime-перевірку:

```

ASSERT.NOT:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  condition: "alias_created_without_user_confirmation"
  on_fail: "STOP"

```

і в test-time очікування:

```

EXPECT.NOT:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  condition: "alias_created_without_user_confirmation"

```

Це захищає від поширеної помилки: правило є в playbook, але його забули додати в regression suite.

EXPECT.PATH

EXPECT.PATH перевіряє, який шлях має бути обраний.

Наприклад:

```
EXPECT.PATH:
  selected: "A1"
  rejected:
    - id: "A2"
      reason_required: true
    - id: "A4"
      reason_required: true
```

Це важливо для decision tree. Якщо модель обрала правильний output, але прийшла до нього неправильним шляхом, це все одно проблема. Бо в складному процесі неправильний path пізніше може зламати gates, state або QA.

EXPECT.GATE

EXPECT.GATE перевіряє gate behavior.

У v0.12 тест має перевіряти не лише status, а й method та trust_class.

```
EXPECT.GATE:
  - id: "domain_pack.history_event.G_SOURCE_FIELD_CONFIRMED"
    method: "human"
    trust_class: "human_decision"
    status: "blocked"
    because: "source field has not been confirmed by user"
```

Такий тест захищає від ситуації, коли модель сама “додумала” відсутнє підтвердження.

Для Ordo це критично. Якщо gate є в документації, але не блокує виконання, це не gate, а декоративний текст.

EXPECT.STATE

EXPECT.STATE перевіряє, як змінюється state.

```
EXPECT.STATE:
  after_node: "N_COLLECT_ALIAS"
  values:
    event_alias: "LU_CHANGE_STATUS"
    contract_confirmed: false
```

State-тести потрібні, бо багато помилок виникає не у фінальному output, а раніше: модель неправильно запам'ятала рішення, переплутала confirmed і assumed, або перенесла значення з прикладу в реальний контракт.

EXPECT.OUTPUT

EXPECT.OUTPUT перевіряє результат, але в Ordo результат — це не обов’язково текст.

Output може бути:

- документ;
- JSON;
- archive;
- checklist;
- question;
- blocked status;
- handoff;
- validation report;
- improvement record.

Приклад:

```
EXPECT.OUTPUT:
  type: "question"
  must_request:
    - "source_field"
    - "old_value"
    - "new_value"
  must_not_create:
    - "final_package"
```

Це означає: правильний output на цьому етапі — не фінальний документ, а питання до користувача.

EXPECT.NOOP

No-op сценарії дуже важливі.

No-op — це ситуація, де правильна поведінка полягає в тому, щоб нічого не створювати або нічого не змінювати.



У класичних інструкціях такі сценарії часто ламаються, бо модель вважає, що має “щось зробити”. Але в реальних процесах правильна відповідь іноді така:

```
нічого не змінюємо;
не створюємо подію;
не генеруємо ChangeRecord;
не створюємо archive;
зупиняємо процес.
```

Приклад:

```
TEST.DEF:
  id: "TC_EXPECTED_NO_CHANGE"
  title: "Не створювати подію, якщо значення не змінилося"
```

```

fixture:
  old_value: "active"
  new_value: "active"

expected:
  noop: true
  no_new_change_record: true
  no_history_event_created: true

```

Но-оп тести захищають систему від зайвої активності.

EXPECT.NOT

EXPECT.NOT залишається корисною назвою для тестового очікування, але в v0.12 вона має бути проекцією ASSERTION, а не окремим правилом.

Наприклад:

```

EXPECT.NOT:
- assertion_id: "domain_pack.history_event.A_NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
- assertion_id: "domain_pack.history_event.A_NO_INVENTED_SOURCE_ROW"
- assertion_id: "domain_pack.history_event.A_NO_ASSUMPTION_AS_CONFIRMED"
- assertion_id: "core.assertions.A_NO_HIDDEN_GATE_INSIDE_FREEFORM"

```

Це дозволяє перевіряти не тільки те, що модель має зробити, а й те, чого вона не має робити.

EXPECT.CLARIFY

У v0.12 для NODE.DEF з'являється контрольований escape hatch on_unmatched_input. Тому тест має вміти перевіряти не тільки нормальні allowed_answers, а й ситуацію, коли відповідь користувача не входить у дерево.

```

EXPECT.CLARIFY:
  node: "domain_pack.history_event.N_EVENT_KIND"
  when_input: "зроби як минулого разу"
  expected_action: "CLARIFY.REQUEST"
  strategy: "rephrase_and_narrow"
  max_attempts: 2
  on_exhausted: "escalate_to_human"

```

Цей тест захищає від неконтрольованої імпровізації: модель не повинна сама вигадувати новий path, якщо input не відповідає жодній дозволений відповіді.

Тестувати треба поведінку, а не стиль відповіді

Одна з помилок — тестувати Ordo-програму як текстовий шаблон.

Погано:

```

Модель має дослівно написати: "Підтвердьте alias події".

```

Краще:

```
expected:
  question_intent:
    - "request_event_alias_confirmation"
```

Тобто ми перевіряємо не буквальне формулювання, а смислову дію.

Це важливо, бо AI-модель може сформулювати питання різними словами, але виконати той самий Ordo-крок.

Тест як захист від деградації інструкцій

Коли Ordo-програма розвивається, тести стають захистом від випадкової деградації.

Наприклад, ми додали нову бібліотеку:

```
include:
  - library: "ordo.artifact.validation"
    version: "0.1"
```

Після цього потрібно перевірити, що старі сценарії не зламалися:

- Path A1 все ще обирається правильно.
- Gate перед архівом все ще blocking.
- Gate має правильний method.
- self_verification gate не видається за mechanical gate.
- FREEFORM не почав перекривати structured rules.
- No-op сценарії не створюють зайвих подій.
- ASSERTION розгортається у runtime і test checks.

Без тестів ми побачимо проблему тільки під час реальної роботи.

Типові помилки

Помилка 1. Тестувати тільки фінальний документ

Якщо фінальний документ виглядає правильно, це ще не означає, що процес був правильний. Модель могла пропустити approval або вигадати частину state.

Помилка 2. Не тестувати blocking gates

Gate без тесту легко перетворюється на рекомендацію.

Помилка 3. Не тестувати method і trust_class

У v0.12 тест має бачити, чи gate був механічним, модельним або людським. Інакше semantic self-check може випадково виглядати як deterministic verification.

Помилка 4. Не тестувати no-op

Якщо no-op не тестується, система майже гарантовано почне створювати зайві результати.

Помилка 5. Тестувати тільки happy path / основний успішний сценарій

Складні Ordo-програми мають тестувати також відмови, зупинки, неповні дані, конфлікти, unmatched input і неправильні запити користувача.

Помилка 6. Не прив'язувати тест до node/gate/path/assertion

Якщо тест падає, але незрозуміло, яку частину програми він перевіряє, debug стає значно складнішим.

Помилка 7. Тримати EXPECT.NOT окремо від ASSERTION

Якщо негативне правило описане окремо в runtime і окремо в тестах, вони рано чи пізно роз'їдуться. У v0.12 джерелом має бути ASSERTION, а EXPECT.NOT — його тестова проекція.

Міні-вправа

Візьміть просту інструкцію:

```
Підготуй відповідь клієнту про затримку доставки.
```

Опишіть для неї один Ordo test case:

1. Який fixture?
2. Який execution_mode?
3. Який expected path?
4. Які питання модель має поставити?
5. Який gate має заблокувати фінальну відповідь?
6. Який method і trust_class має цей gate?
7. Який ASSERTION забороняє вгадати причину затримки?
8. Який output заборонений до підтвердження?
9. Який по-ор сценарій можливий?

Наприклад, по-ор може бути таким: якщо користувач просить відповісти клієнту, але не дав жодної інформації про причину затримки, модель не має вгадувати причину. Вона має зупинитися і запросити контекст.

Короткий підсумок

Ordo-тест — це не перевірка красивого тексту. Це перевірка поведінки процесу.

Тест має фіксувати:

```
input → fixture → expected path → expected state → expected gates → expected assertions →  
expected output → forbidden actions
```

У v0.12 тест також має бачити:

```
execution_mode → gate.method → trust_class → trace_source → assertion projections
```

Саме це дозволяє розвивати складні інструкції без хаосу.

Без тестів Ordo-програма поступово перетворюється назад на великий prompt, який важко змінювати і майже неможливо надійно перевірити.

Розділ 17. Regression suite і coverage

Навіщо це потрібно

Коли Ordo-програма маленька, її можна перевірити вручну. Достатньо один раз прогнати сценарій і подивитися, чи модель поставила правильне питання, чи не пропустила gate, чи створила потрібний output.

Але як тільки Ordo-програма стає більшою, ручної перевірки вже недостатньо.

З'являється типова проблема:

```
ми виправили одну помилку →
але зламали інший сценарій →
потім виправили другий сценарій →
але випадково послабили важливий gate →
потім додали нове правило →
але воно почало конфліктувати зі старим правилом.
```

У звичайній роботі з prompt-ами це майже неможливо контролювати. Користувач бачить тільки, що модель “стала поводитися дивно”. Але не завжди зрозуміло, яка саме зміна це спричинила.

В Ordo для цього потрібні дві речі:

```
Regression suite
Coverage report
```

Regression suite перевіряє, що старі сценарії не зламалися після зміни інструкцій.



Coverage report показує, які частини Ordo-програми реально покриті тестами, а які існують тільки на папері.

Починаючи з Ordo v0.12, coverage має перевіряти не тільки paths, nodes і outputs. Він також має показувати, чи покриті:

```
gate.method;
trust_class;
execution_mode;
control_level;
ASSERTION-проекції;
layer conflict rules;
FREEFORM maturity lifecycle.
```

Інакше можна мати гарний coverage report, але не бачити, що всі критичні семантичні gates перевіряються тільки самою моделлю, а не механічно або через окремий critic-прохід.

Що таке regression suite

Regression suite — це набір тестів, який потрібно запускати після зміни Ordo-програми, library, domain pack, profile або compiler rule.

Простими словами:

Regression suite — це захист від випадкового ламання того, що вже працювало.

Наприклад, у нас є Ordo-playbook для створення історичних подій. Він підтримує кілька шляхів:

```
A1 — зміна поля в source row;
A2 — зміна пов'язаної сутності;
A3 — ручний сценарій;
A4 — зовнішня історична подія;
A5 — no-op / expected no-change.
```

Ми змінюємо логіку A1, щоб краще обробляти зміну статусу. Але після цього треба переконатися, що:

```
A2 не зламався;
A4 не почав трактувати external payload як звичайний delta.field;
A5 досі правильно зупиняє no-op сценарії;
pre-archive gates досі блокують фінальний пакет до підтвердження;
manual QA package досі має потрібні перевірки;
uci blocking gates мають method;
trace не видає model_self_report за runtime_enforced;
ASSERTION не загубила test-проекцію.
```

Саме це робить regression suite.

Regression suite як частина мови

У Ordo regression suite не має бути зовнішньою домовленістю на кшталт “не забудь потім перевірити”.

Вона має бути частиною самої Ordo-програми або пов'язаного з нею test package.

Наприклад:

```
regression_suite:
  id: "RS_HISTORY_EVENT_PLAYBOOK"
  target:
    kind: "domain_pack"
    name: "history_event"
    version: "0.12"

  control_level: strict
  execution_modes:
    - full_runtime
    - chat_internal

  tests:
    - "TC_PATH_A1_BASIC"
    - "TC_PATH_A2_RELATED_ENTITY"
    - "TC_PATH_A4_EXTERNAL_EVENT"
    - "TC_NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
    - "TC_EXPECTED_NO_CHANGE"
    - "TC_FREEFORM_BINDING_VISIBLE"
    - "TC_GATE_METHODS_DECLARED"
```

```

- "TC_ASSERTIONS_PROJECTED_TO_TESTS"
- "TC_TRACE_SOURCE_VISIBLE"

required_before:
- "domain_pack_publish"
- "library_version_publish"
- "compiler_rule_change"
- "production_use"

```

Тут важливі чотири речі.

Перше: suite має ціль. Він перевіряє не “все на світі”, а конкретну Ordo-програму, library, profile або domain pack.

Друге: suite має список тестів. Кожен тест перевіряє конкретну поведінку.

Третє: suite має правило запуску. Наприклад, перед публікацією нової версії domain pack regression suite є обов’язковим.

Четверте: suite має знати control_level і execution_modes, для яких він справедливий. Тест, який достатній для freeform_only, може бути занадто слабким для strict-режиму.

Що має перевіряти regression suite

Regression suite має перевіряти не тільки фінальний текст.

Це ключова думка.

У звичайному prompt-testing часто перевіряють результат:

```
чи відповідь виглядає нормально?
```

В Ordo потрібно перевіряти поведінку процесу:

```

чи правильний path був обраний;
чи правильний node був активований;
чи правильне питання поставила модель;
чи правильний state змінився;
чи правильний gate спрацював;
чи правильний method має кожен gate;
чи trust_class відповідає реальному способу перевірки;
чи trace_source не завищує рівень гарантії;
чи ASSERTION розгорнулась у runtime і test expectations;
чи правильний output був створений;
чи не була виконана заборонена дія;
чи зберігся handoff contract;
чи не було прихованого використання FREEFORM;
чи не зникло обов’язкове пояснення.

```

Тобто regression suite має дивитися на Ordo-програму як на процес, а не як на генератор тексту.

Приклад regression test

Припустимо, ми хочемо перевірити, що фінальний архів не створюється до approval gate.

Тест може виглядати так:

```
test:
  id: "TC_NO_ARCHIVE_BEFORE_APPROVAL"
  title: "Фінальний архів не створюється до підтвердження"
  execution_mode: full_runtime

fixture:
  user_message: "зроби фінальний архів одразу"
  state:
    contract_confirmed: true
    pre_archive_approval: false
    validation_passed: false

expected:
  selected_path: "package_generation"

  gates:
    - id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
      method: human
      trust_class: human_decision
      status: blocked

  assertions:
    - id: "domain_pack.history_event.A_NO_ARCHIVE_BEFORE_APPROVAL"
      polarity: not
      phase: [runtime, test]
      status: passed

  output:
    final_archive_created: false

  forbidden_actions:
    - "ARCHIVE.CREATE"
    - "HANDOFF.MARK_READY"
```

Цей тест не питає, чи модель написала красиву відповідь. Він перевіряє інше:

чи модель не зробила того, що їй заборонено робити.

Це набагато важливіше для складних процесів.

Тестування `gate.method`



У v0.12 `gate.method` — це не декоративне поле. Це частина довіри до `gate`.

Тому regression suite має мати окремі тести або lint-перевірки:

```
test:
  id: "TC_GATE_METHODS_DECLARED"

expected:
  all_gates_have_method: true
  allowed_methods:
    - mechanical
    - self_verification
    - self_consistency
    - human
```

```
forbidden:
  - gate_without_method
  - gate_with_unknown_method
```

Окремо потрібно перевіряти, що `trust_class` не суперечить `method`:

```
expected:
  method_trust_mapping:
    mechanical: deterministic
    self_verification: model_judgment
    self_consistency: repeated_model_judgment
    human: human_decision
```

Це не гарантує, що кожне семантичне судження правильне. Але це не дозволяє приховати семантичне судження під виглядом механічної перевірки.

Тестування `execution_mode`

Одна й та сама Ordo-програма може виконуватись у різних режимах:

```
full_runtime;
chat_internal;
freeform_only.
```

Regression suite має явно показувати, для якого режиму виконання справедливий тест.

Наприклад:

```
test:
  id: "TC_TRACE_SOURCE_IN_CHAT_INTERNAL"
  execution_mode: chat_internal

expected:
  trace:
    trace_source: hybrid

  gates:
    - id: "domain_pack.history_event.G_RENDERED_ARTIFACT_VALIDATED"
      method: mechanical
      status: passed

  warnings_allowed:
    - "gate_invocation_not_runtime_enforced"
```

У `chat_internal` режимі код може реально виконати перевірку, але сам момент запуску перевірки не є примусово контрольованим зовнішнім runtime. Тому тест має дозволяти чесний warning, а не робити вигляд, що це повний `full_runtime`.

Тестування `ASSERTION`

У v0.12 `ASSERT.NOT`, `negative gate` і `EXPECT.NOT` більше не мають жити окремо.

Канонічним примітивом є `ASSERTION`:

```
assertion:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  polarity: not
  condition: "alias_created_without_user_confirmation"
  phase: [runtime, test]
```

```
severity: block
on_fail: STOP
```

Regression suite має перевіряти, що така assertion дала потрібні проєкції:

```
expected:
  projections:
    runtime:
      includes: "ASSERT.NOT"
    test:
      includes: "EXPECT.NOT"
    debug:
      includes: "violation_record"
```

Це зменшує ризик, що правило є в основному playbook, але його забули в regression suite.

Що таке coverage

Coverage — це відповідь на питання:

Яку частину Ordo-програми ми реально перевірили?

У класичному програмуванні coverage часто показує, які рядки коду або гілки були виконані тестами.

В Ordo coverage має бути ширшим. Тут потрібно перевіряти не тільки “рядки”, а семантичні частини процесу:

```
paths;
nodes;
gates;
gate methods;
trust classes;
execution modes;
control levels;
status transitions;
outputs;
assertions;
ASSERTION projections;
FREEFORM blocks;
FREEFORM maturity;
libraries;
layer conflicts;
domain rules;
error paths;
no-op сценарії;
handoff rules.
```

Наприклад, playbook може мати п'ять шляхів, але тести перевіряють тільки два. Формально тести є, але насправді більша частина поведінки не захищена.

Coverage має це показати явно.

Приклад coverage report

```
coverage_report:
  target:
    kind: "domain_pack"
    name: "history_event"
```

```

version: "0.12"

control_level: strict

execution_modes:
  full_runtime:
    covered: true
  chat_internal:
    covered: true
  freeform_only:
    covered: false
    reason: "not supported for this production pack"

paths:
  total: 5
  covered: 4
  uncovered:
    - "A4_EXTERNAL_HISTORY_EVENT"

nodes:
  total: 18
  covered: 15
  uncovered:
    - "NODE_EXTERNAL_EVENT_NORMALIZATION"
    - "NODE_MANUAL_EXCEPTION_REVIEW"

gates:
  total: 14
  covered: 12
  uncovered:
    - "domain_pack.history_event.G_EXTERNAL_EVENT_MAPPING_CONFIRMED"
    - "domain_pack.history_event.G_RENDERED_ARTIFACT_VALIDATED"

gate_methods:
  mechanical:
    total: 6
    covered: 6
  self_verification:
    total: 4
    covered: 3
  self_consistency:
    total: 1
    covered: 0
  human:
    total: 3
    covered: 3

trust_classes:
  deterministic: "covered"
  model_judgment: "partially_covered"
  repeated_model_judgment: "not_covered"
  human_decision: "covered"

assertions:
  total: 9
  covered: 7
  uncovered:
    - "domain_pack.history_event.A_NO_ARCHIVE_BEFORE_SELF_CHECK"

assertion_projections:
  runtime: "covered"
  test: "partially_covered"
  debug: "covered"

freeform:
  total_blocks: 5
  covered: 3
  maturity:
    stable: 2
    volatile: 2

```

```

candidate_for_formalization: 1
uncovered:
  - "FF_DOMAIN_EXAMPLES"
  - "FF_EDGE_CASE_NOTES"

outputs:
  total: 11
  covered: 10
  uncovered:
    - "07_PROCESS_IMPROVEMENT_FEEDBACK"

```

Такий report одразу показує слабкі місця.

Наприклад, якщо `G_RENDERED_ARTIFACT_VALIDATED` не покритий тестами, це ризик. Модель може знову перевірити `template` замість реально згенерованого `artifact`, і тест це не зловить.

А якщо `self_consistency gates` мають `covered: 0`, це ще один сигнал: критичні семантичні рішення формально описані, але не захищені regression-паketом.

Coverage не означає якість автоматично



Важливо не сплутати `coverage` з якістю.

Високий `coverage` не гарантує, що Ordo-програма добра. Він тільки означає, що багато її частин були виконані тестами.

Можна мати 90% `coverage` і все одно мати погані тести.

Наприклад, тест може перевіряти тільки те, що `output` існує:

```

expected:
  output_exists: true

```

Але не перевіряти, що `output` створено після правильного `gate`.

Або тест може перевіряти, що `gate` має `status: passed`, але не перевіряти його `method` і `trust_class`.

Тому для Ordo важливо мати не тільки `coverage`, а й якість очікувань:

```

expected path;
expected state;
expected gates;
expected gate.method;
expected trust_class;
expected trace_source;
expected assertion projections;
expected forbidden actions;
expected evidence;
expected no-op;
expected handoff status.

```


Coverage показує, де ми були. Якість тестів показує, що саме ми там перевірили.

Мінімальний coverage для production playbook

Для простих Ordo-програм coverage може бути неповним.

Але для production playbook-ів, domain racks і libraries потрібно встановити мінімальні вимоги.

Наприклад:

```
coverage_policy:
  required_for: "production_domain_pack"
  control_level: strict

  minimum:
    paths: "100%"
    blocking_gates: "100%"
    gate_methods_declared: "100%"
    mechanical_gates_tested: "100%"
    human_gates_have_approval_fixture: "100%"
    self_verification_gates_have_protocol_tests: "100%"
    negative_assertions: "100%"
    assertion_test_projection: "100%"
    outputs: "90%"
    freeform_blocks: "at_least_linked"
    no_op_scenarios: "required_if_supported"

  release_rule:
    if_below_minimum: "block_release"
```

Тут важливо, що blocking gates мають бути покриті всі.

Якщо gate має зупиняти процес, але для нього немає тесту, це небезпечно. Такий gate існує тільки як текстова декларація.

Для control_level: strict це має бути не рекомендацією, а правилом компіляції або release-gate.

Regression після feedback

Regression suite особливо важливий після user feedback.

Користувач каже:

```
Ти пропустив self-check перед архівом.
```

Ordo створює improvement record і пропонує patch:

```
зробити G_PACKAGE_SELF_CHECK blocking;
додати ASSERTION A_NO_ARCHIVE_BEFORE_SELF_CHECK;
додати test TC_NO_ARCHIVE_WITHOUT_SELF_CHECK.
```

Після внесення зміни треба прогнати regression suite.

Чому? Бо новий blocking gate може випадково заблокувати легітимний сценарій, де self-check уже виконаний, але state названий інакше.

Тобто improvement без regression може виправити одну проблему і створити іншу.

Правильний цикл:

```
feedback → improvement record → patch → new assertion/test → regression suite → coverage report
→ release decision
```

Regression для бібліотек



Коли в Ordo з'являються libraries, regression стає ще важливішим.

Бібліотека може використовуватися в десятках playbook-ів. Якщо змінити її gate, assertion або status semantics, можна непомітно зламати всі програми, які її підключають.

Тому library має мати власний regression suite:

```
library:
  id: "ordo.validation.contract_first"
  version: "0.2.0"

regression_suite:
  id: "RS_CONTRACT_FIRST_LIBRARY"
  tests:
    - "TC_CONTRACT_REQUIRED_BEFORE_OUTPUT"
    - "TC_MISSING_CONTRACT_BLOCKS_EXECUTION"
    - "TC_EXPLICIT_OVERRIDE_REQUIRED"
    - "TC_ALL_EXPORTED_GATES_HAVE_METHOD"
```

А Ordo-програма, яка використовує бібліотеку, має мати integration tests:

```
integration_tests:
  libraries:
    - id: "ordo.validation.contract_first"
      version: "^0.2.0"
      tests:
        - "TC_PLAYBOOK_CONTRACT_GATE_BINDING"
        - "TC_LAYER_CONFLICT_REQUIRES_OVERRIDE"
```

Це означає: бібліотека працює сама по собі, але також перевірено, що вона правильно підключена до конкретного playbook-a.

Coverage для control_level

control_level визначає, наскільки суворо має бути перевірена Ordo-програма.

```
program:
  id: "history_event.guided_intake"
  control_level: strict
```

Для light-програми достатньо базових перевірок intent і contract.

Для standard-програми вже потрібні state, gates і базова regression suite.

Для strict-програми потрібні:

```
обов'язковий debug trace;
обов'язковий trace_source;
обов'язкові gate.method;
обов'язкові tests для mandatory gates;
обов'язковий coverage report;
обов'язковий feedback/improvement path;
обов'язкова release-перевірка.
```

Coverage report має показувати, чи відповідає тестовий пакет заявленому control_level.

Типові помилки

Помилка 1. Тестувати тільки фінальний текст

Погано:

```
модель щось відповіла — значить тест пройшов.
```

Добре:

```
перевірити path, state, gates, output і forbidden actions.
```

Помилка 2. Не запускати regression після дрібної зміни

У складних інструкціях “дрібна зміна” часто має великий ефект.

Одна фраза може змінити пріоритет path, послабити gate або зробити FREEFORM надто широким.

Помилка 3. Вважати coverage доказом якості

Coverage показує, що щось було виконано. Але він не показує автоматично, що очікування були правильні.

Помилка 4. Не покривати negative assertions

У Ordo дуже важливо тестувати не тільки те, що модель має зробити, а й те, чого вона не має робити.

Наприклад:

```
не створювати архів до approval;
не вигадувати source row;
не переходити до package generation без contract;
не ховати gate у FREEFORM;
не вважати draft final result.
```

Помилка 5. Не тестувати no-or сценарії

No-or — це повноцінний expected behavior.

Якщо правильний результат — нічого не створювати, це теж має бути тестом.

Помилка 6. Не перевіряти `gate.method`

Погано:

```
gates:
- id: "G_NO_UNSUPPORTED_FACTS"
  status: passed
```

Добре:

```
gates:
- id: "domain_pack.history_event.G_NO_UNSUPPORTED_FACTS"
  method: self_verification
  trust_class: model_judgment
  status: passed
```

Якщо тест бачить тільки passed, але не бачить method, він не розуміє, який рівень довіри має цей результат.

Помилка 7. Не розрізняти `full_runtime` і `chat_internal`

Тест для `full_runtime` не можна автоматично вважати доказом для `chat_internal`, і навпаки.

У `chat_internal` частина перевірок може бути реально виконана кодом, але точка їх виклику не є примусовою без зовнішнього runtime.

Міні-вправа

Візьміть будь-яку Ordo-програму або великий prompt, з яким ви працювали.

Спробуйте скласти для нього маленький regression suite з п'яти тестів:

1. Основний успішний сценарій.
2. Сценарій із відсутнім contract.
3. Сценарій, де має спрацювати blocking gate.
4. Сценарій, де модель не має виконувати заборонену дію.
5. No-or або expected no-change сценарій.

Після цього додайте v0.12-питання:

```
Які gates мають method?
Які gates є mechanical, а які self_verification?
Чи є tests для assertion projections?
Які execution modes покриті?
Чи відповідає coverage заявленому control_level?
Які FREEFORM-блоки впливають на рішення, але не мають тестів?
```

Це і є перший простий coverage analysis.

Короткий підсумок

Regression suite потрібен, щоб Ordo-програма не ламалася після кожного покращення.

Coverage потрібен, щоб бачити, які частини процесу реально перевірені.

У Ordo потрібно тестувати не тільки output, а всю поведінку:

```
path → node → state → gate → assertion → output → handoff
```

Починаючи з v0.12, сильна Ordo-програма також має перевіряти:

```
gate.method → trust_class → trace_source → execution_mode → control_level
```

Сильна Ordo-програма — це не та, яка один раз дала правильну відповідь. Сильна Ordo-програма — це та, яка має regression suite, coverage report і може безпечно розвиватися.

Розділ 18. Feedback & Improvement Loop

Навіщо це потрібно

Коли люди працюють із великими інструкціями для AI-моделей, майже ніколи не буває так, що перша версія одразу працює ідеально.

Зазвичай процес виглядає інакше.

Спочатку ми пишемо інструкцію. Потім модель виконує її майже правильно, але пропускає якийсь важливий крок. Ми додаємо уточнення. Потім вона перестає пропускати цей крок, але починає помилятися в іншому місці. Ми знову додаємо правило. Потім інструкція стає довшою, складнішою, у ній з'являються винятки, повтори, приховані суперечності. Через якийсь час уже важко зрозуміти, де саме потрібно щось виправляти.

У реальній роботі користувач постійно дає зауваження:

```
ти пропустив self-check;
це треба було питати раніше;
тут не можна було створювати фінальний пакет;
це правило має бути gate, а не рекомендацією;
це треба винести в playbook;
тут потрібен по-ор тест;
цей випадок треба запам'ятати для майбутніх задач;
це покращення треба додати в інструкцію, а не просто врахувати один раз.
```

Якщо такі зауваження залишаються тільки в чаті, вони швидко губляться. Модель може врахувати їх у поточній відповіді, але наступна версія playbook-a, library або domain pack-a може не отримати цього покращення.

Тому в Ordo потрібен окремий механізм: не тільки виконувати інструкції, не тільки дебажити їх, не тільки тестувати, а ще й збирати реальний досвід використання та перетворювати його на структуровані записи для покращення.

Цей механізм називається:

```
Feedback & Improvement Loop
```

Просте пояснення

Feedback & Improvement Loop — це контур покращення Ordo-програми.

Його задача — перетворити людське зауваження на структурований запис:

```
користувач побачив проблему
→ Ordo зафіксувала feedback
→ класифікувала проблему
→ знайшла affected unit
→ запропонувала patch
→ запропонувала тест
→ чекала human approval
→ після підтвердження зміна потрапила в playbook/library/domain pack/compiler
→ regression suite перевірила, що нічого не зламалось
```

Це дуже важлива відмінність від звичайного prompt-підходу.

У prompt-підході зауваження часто виглядає так:

Добре, врахуй це надалі.

Але що саме врахувати? Де саме? У якому файлі? У якому правилі? Чи це новий gate? Чи новий тест? Чи уточнення до FREEFORM? Чи зміна domain pack-a? Чи помилка компілятора?

Ordo не має залишати це неясним.

У Ordo feedback має стати артефактом.



Чим feedback відрізняється від debug і testing

Debug відповідає на питання:

Чому процес пішов саме так?

Testing відповідає на питання:

Чи процес працює так, як очікується?

Feedback & Improvement Loop відповідає на інше питання:

Що потрібно змінити в Ordo-програмі, щоб вона працювала краще в майбутньому?

Ці три частини пов'язані, але не однакові.

Debug може показати, що модель пропустила gate.

Test може показати, що сценарій не пройшов.

Feedback може зафіксувати, що це не випадкова помилка, а проблема в самій інструкції: gate був описаний як рекомендація, але не був визначений як blocking.

Ordo-конструкція

У мові Ordo для цього шару потрібні окремі конструкції.

Базовий набір може виглядати так:

```
FEEDBACK.CAPTURE
ISSUE.RECORD
IMPROVEMENT.RECORD
PROBLEM.CLASSIFY
ROOT_CAUSE.LINK
AFFECTED.UNIT
PATCH.SUGGEST
TEST.SUGGEST
REGRESSION.ADD
```

VERSION.NOTE
CHANGELOG.UPDATE
LESSON.LEARNED

Їх не потрібно сприймати як фінальний синтаксис. Важлива сама ідея: feedback має бути не текстовою приміткою, а частиною виконуваної системи розвитку Ordo-програми.

Що таке improvement record

Improvement record — це структурований запис про проблему або покращення.

Він має містити не тільки текст зауваження, а й контекст:

- хто або що виявило проблему;
- у якому run це сталося;
- який path був активний;
- який node або gate був пов'язаний із проблемою;
- який тип проблеми;
- наскільки вона критична;
- яку частину Ordo-програми треба змінити;
- який patch пропонується;
- який тест потрібно додати;
- чи потрібне human approval;
- чи внесено зміну в changelog.

Приклад:

```
improvement_record:
  id: "IR-2026-07-05-001"

  source:
    type: "user_feedback"
    message: "Ти знову пропустив self-check перед архівом."

  classification:
    type: "missed_required_gate"
    severity: "high"

  affected_unit:
    kind: "gate"
    id: "G_PACKAGE_SELF_CHECK"
    owner:
      layer: "domain_pack"
      name: "history_event"

  observed_in:
    run_id: "RUN-2026-07-05-001"
    path: "package_generation"
    node: "final_archive"

  root_cause_hypothesis:
    - "gate exists in documentation but is not enforced as blocking"
    - "archive generation was allowed before validation report"

  proposed_patch:
    - "mark G_PACKAGE_SELF_CHECK as blocking"
    - "add ASSERT.NOT final_archive_created before self_check_passed"

  suggested_tests:
    - id: "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"
      expected:
        final_archive_created: false
        blocked_gate: "G_PACKAGE_SELF_CHECK"
```



```
approval:
  required: true
  status: "pending"
```

Цей запис уже можна використовувати як основу для реальної зміни в playbook-у.

Affected unit

Один із найважливіших елементів improvement loop — це affected_unit.

Користувач може сказати:

Тут щось не так.

Але для розвитку Ordo цього недостатньо. Потрібно зрозуміти, де саме проблема.

Проблема може бути в:

- конкретному NODE;
- конкретному GATE;
- статусі;
- ASSERT.NOT;
- output contract;
- FREEFORM-блоці;
- library;
- domain pack;
- profile;
- compiler rule;
- documentation runtime;
- rendered artifact validation;
- test fixture;
- самій структурі playbook-a.

Тому improvement record має завжди намагатися прив'язати проблему до конкретної одиниці.

Приклад:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

Або:

```
affected_unit:
  kind: "freeform"
  id: "FF_DOMAIN_EDGE_CASES"
  owner:
    layer: "domain_pack"
    name: "history_event"
```

Це дозволяє не просто виправити один сценарій, а покращити правильне місце.

Класифікація проблем

Feedback потрібно класифікувати.

Інакше всі зауваження перетворяться на хаотичний список.

Типові класи проблем:

```
missed_required_gate
wrong_path_selected
premature_output
missing_question
wrong_question_order
state_not_updated
state_updated_without_confirmation
implicit_assumption
freeform_overuse
conflicting_rules
library_conflict
missing_test
missing_noop_test
missing_coverage
rendered_artifact_not_validated
compiler_mapping_error
unclear_status_semantics
```

Класифікація допомагає зрозуміти, що саме потрібно зробити.

Наприклад, якщо проблема `missing_question`, можливо, потрібно змінити `NODE.DEF`.

Якщо проблема `premature_output`, можливо, потрібен `blocking gate` або `ASSERT.NOT`.

Якщо проблема `freeform_overuse`, можливо, частину `FREEFORM` треба формалізувати.

Якщо проблема `missing_noop_test`, потрібно додати `test case`.

Human approval

Feedback loop не має автоматично переписувати Ordo-програму.

Це дуже важливе правило.



Ordo може:

- зафіксувати проблему;
- запропонувати root cause;
- запропонувати patch;
- запропонувати regression test;
- показати affected unit;
- підготувати changelog note.

Але застосування зміни має бути керованим.

Правильний цикл:

```
feedback captured
→ problem classified
→ affected unit linked
→ patch suggested
```

```

→ test suggested
→ human approval required
→ change applied
→ regression suite executed
→ version note / changelog updated

```

Без підтвердження зміна має залишатися у статусі:

```
pending_improvement
```

Це захищає Ordo від хаотичного самонавчання.

Зв'язок із regression suite

Кожне серйозне зауваження має завершуватися не тільки patch-ем, а й тестом.

Погано:

```
Ми додали в інструкцію: не пропускати self-check.
```

Краще:

```
Ми додали blocking gate G_PACKAGE_SELF_CHECK.
```

Ще краще:

```

Ми додали blocking gate G_PACKAGE_SELF_CHECK
і regression test TC_NO_ARCHIVE_WITHOUT_SELF_CHECK.

```

Інакше та сама проблема може повернутися через кілька змін.

Тому Ordo має мати правило:

```
Кожен high-severity improvement record має пропонувати принаймні один regression test.
```

Зв'язок із changelog

Якщо feedback призвів до зміни Ordo-програми, це має бути відображено в changelog або version note.

Наприклад:

```

version_note:
  version: "0.11.1"
  changes:
    - type: "gate_enforcement"
      description: "G_PACKAGE_SELF_CHECK is now blocking before final archive generation."
      source_improvement_record: "IR-2026-07-05-001"
      regression_tests_added:
        - "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"

```

Це дозволяє бачити, чому з'явилась конкретна зміна.

Без цього playbook поступово перетворюється на набір правил, історія появи яких уже нікому не зрозуміла.

Зв'язок із бібліотеками

Коли Ordo отримує механізм `libraries, feedback loop` має працювати і з ними.

Проблема може виникнути не в основному `playbook`-у, а в підключеній бібліотеці.

Наприклад:

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
  as: "contract_first"
```

Якщо `gate` із цієї бібліотеки працює неправильно або не покриває потрібний випадок, `improvement record` має вказати саме бібліотеку:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

Тоді покращення можна внести не тільки в один `playbook`, а в `reusable` рішення, яке потім використовують інші Ordo-програми.

Зв'язок із FREEFORM

FREEFORM часто буде джерелом складних проблем.

Це нормально. FREEFORM потрібен саме тому, що не все можна формалізувати одразу.

Але якщо певний FREEFORM-блок постійно викликає помилки, Ordo має це бачити.

Наприклад:

```
freeform_feedback:
  freeform_id: "FF_DOMAIN_EDGE_CASES"
  records:
    - "IR-2026-07-05-004"
    - "IR-2026-07-05-009"
  suggested_action:
    - "split freeform block"
    - "formalize recurring rule as GATE.DEF"
    - "add example-based tests"
```

Тобто `feedback loop` допомагає поступово зменшувати неконтрольовану частину інструкції.



Не обов'язково прибирати FREEFORM повністю. Але потрібно бачити, де він стає ризиком.

Типові помилки

1. Просто сказати “врахуй надалі”

Це слабкий варіант.

У Ordo потрібно не тільки врахувати, а й зафіксувати:

```
що саме;  
де саме;  
чому;  
який patch;  
який тест;  
який статус.
```

2. Вносити зміни без regression test

Це швидко створює нові проблеми.

Кожне серйозне виправлення має породжувати тест.

3. Не прив'язувати проблему до affected unit

Якщо немає affected unit, то незрозуміло, що саме треба змінювати.

4. Автоматично застосовувати всі зауваження

Feedback не завжди означає правильну зміну. Іноді користувач описує симптом, але root cause інший.

Тому потрібне human approval.

5. Змішувати feedback із debug log

Debug log показує, що сталося.

Feedback record показує, що з цим треба зробити.

Міні-вправа

Візьміть одне реальне зауваження до будь-якої інструкції.

Наприклад:

```
Модель занадто рано сформувала фінальний документ.
```

Спробуйте оформити його як improvement record:

1. Який тип проблеми?
2. Який severity?
3. Який affected unit?
4. Який root cause?
5. Який patch потрібен?
6. Який regression test потрібно додати?
7. Чи потрібне human approval?
8. Який changelog note треба створити?

Після цього стане видно, що feedback — це не просто коментар. Це вхід у процес розвитку Ordo-програми.

Короткий підсумок

Feedback & Improvement Loop потрібен, щоб реальний досвід роботи з Ordo не губився.

Він перетворює зауваження користувача на структуровані записи для покращення:

```
feedback → issue → affected unit → root cause → patch → test → approval → regression → changelog
```

Цей шар робить Ordo мовою не тільки виконання, а й керованого розвитку інструкцій.

Без feedback loop складні playbook-и поступово стають хаотичними.

З feedback loop кожна проблема може стати джерелом контрольованого покращення.

Розділ 19. Чому не все треба формалізувати

У попередніх розділах ми багато говорили про структуру: Intent, Contract, Context, State, Node, Gate, Output, ASSERT.NOT, STATUS.SEMANTICS. Через це може виникнути природне враження: якщо Ordo прагне керованості, то в Ordo потрібно формалізувати абсолютно все.

Але це неправильний висновок.

Ordo не вимагає перетворювати кожне речення, кожен приклад і кожне пояснення на окрему машинну інструкцію. Навпаки: хороша Ordo-програма відрізняє те, що повинно бути формальним, від того, що достатньо залишити в контрольованому людському описі.

Цей розділ пояснює, чому надмірна формалізація може бути шкідливою, де проходить межа між структурою і поясненням, і чому Ordo потребує спеціального механізму для неформалізованих, але контрольованих частин.

Навіщо це потрібно

У класичному програмуванні ми звикли, що комп'ютер виконує тільки те, що записано у формальній мові. Якщо інструкція не є кодом, машина її не виконає.

З AI-моделями ситуація інша. Модель добре працює зі змістом, контекстом, прикладами, поясненнями, стилістичними обмеженнями, винятками і напівструктурованими правилами. Вона може зрозуміти фразу, порівняти приклади, перенести смисл, пояснити рішення, узагальнити патерн.

Саме тому було б помилкою зробити Ordo мовою, яка намагається перетворити все людське знання на жорсткий набір orcode-ів.

Якщо формалізувати все до дрібниць, можна отримати три проблеми.

Перша проблема — втрата змісту.

Деякі правила мають сенс тільки в контексті пояснення. Наприклад:

Не створюй фінальний архів, якщо аналітик ще не підтвердив очікуваний рівень змін.

Це правило можна формалізувати як gate. Але поруч із ним може бути довге пояснення, чому це важливо, які помилки вже траплялися раніше, які приклади є небезпечними, і як аналітик зазвичай формулює підтвердження. Це пояснення не завжди потрібно розбивати на десятки дрібних інструкцій.

Друга проблема — крихкість.

Надто формалізована система починає ламатися при кожному новому випадку. Якщо кожен виняток оформлений як окремий orcode, то мова швидко перетворюється на громіздкий каталог спеціальних випадків.

Третя проблема — погана читабельність.

Ordo має бути зрозумілою не тільки машині, а й людям, які проєктують процес. Якщо автор playbook-а не може прочитати власну інструкцію без окремого компілятора, система стає небезпечною.



Тому Ordo має зберегти баланс:

формалізувати те, що керує виконанням;
залишати в контрольованому описі те, що пояснює зміст.

Просте пояснення

Уявіть собі дорожні правила.

Є речі, які мають бути формальними:

червоне світло = зупинитися;
зелений сигнал = можна їхати;
обмеження швидкості = не перевищувати;
пішохід на переході = дати дорогу.

Але є також пояснення:

водій має враховувати дорожню обстановку;
у складних умовах потрібно бути особливо уважним;
погана видимість збільшує ризик;
дитина біля дороги може поводитися непередбачувано.

Ці пояснення важливі. Вони впливають на поведінку. Але не всі вони зводяться до одного простого if/then.

Ordo працює подібно.

Формальними мають бути ті частини, які визначають:

- що є входом;
- що є виходом;
- які питання потрібно поставити;
- які стани існують;
- які переходи дозволені;
- які gate-и обов'язкові;
- де модель повинна зупинитися;
- що заборонено робити;
- що треба перевірити перед handoff.

Але не все пояснення домену треба перетворювати на формальну команду. Частина знання може залишатися в контрольованому текстовому шарі.

Головне — не дозволяти цьому текстовому шару ставати “чорною дірою”, куди заховані критичні правила.

Що точно треба формалізувати

У Ordo потрібно формалізувати все, що впливає на керування процесом.

Наприклад, якщо правило визначає, чи можна переходити до наступного кроку, воно має бути `gate-ом`.

Погано:

Перед фінальним архівом бажано переконатися, що все перевірено.

Добре:

```
GATE.DEF:
  id: G_FINAL_VALIDATION_REQUIRED
  before: final_archive
  require:
    - validation_report.status == passed
    - consistency_check.status == passed
  on_fail: stop
```

Якщо правило визначає, що модель не має права робити, воно має бути `ASSERT.NOT`.

Погано:

Не поспішай з висновками.

Добре:

```
ASSERT.NOT:
  id: A_NO_UNCONFIRMED_CONTRACT
  forbidden:
    - treat_unconfirmed_field_as_confirmed
    - generate_final_package_before_approval
```

Якщо правило визначає стан процесу, воно має бути частиною `STATUS.SEMANTICS`.

Погано:

Коли все майже готово, можна переходити далі.

Добре:

```
STATUS.SEMANTICS:
  ready_for_archive:
    meaning: "усі обов'язкові approvals і validation gates пройдені"
    allowed_next:
      - final_archive_generation
    forbidden_actions:
      - ask_foundational_intake_question_again
```

Якщо правило визначає очікуваний результат, воно має бути в `OUTPUT.DEF`.

Погано:

Зроби нормальний пакет документів.

Добре:

```
OUTPUT.DEF:
  id: compact_history_event_package
  required_files:
    - README.md
    - SUMMARY.json
    - VALIDATION_REPORT.json
    - CONSISTENCY_CHECK_REPORT.json
```

Отже, формалізувати потрібно все, що відповідає на питання:

чи можна?
 що далі?
 що заборонено?
 що вважається готовим?
 що потрібно перевірити?
 який саме результат треба створити?

Що не обов’язково формалізувати

Не обов’язково повністю формалізувати все, що є поясненням, прикладом, фоном або допоміжною інтерпретацією.

Наприклад:

- історія появи правила;
- приклади старих помилок;
- пояснення для аналітика;
- довгі доменні коментарі;
- опис типових ситуацій;
- навчальні приклади;
- стилістичні рекомендації;
- пояснення, чому певне правило існує;
- порівняння правильного і неправильного мислення.

Такі частини можуть бути дуже важливими. Але вони не завжди повинні ставати окремими формальними інструкціями.

Наприклад, у playbook-у може бути пояснення:

Для нових History Event не можна виводити type=companyProfile або sub_type=EDR тільки з бізнесової назви події. Спочатку потрібно підтвердити, що це справді ЕДР-фактографія і що потрібне поле реально зберігається у source row.

Тут є дві різні частини.

Перша частина є формальною заборonoю:

не виводити source type автоматично з назви події

Її варто оформити як ASSERT.NOT.

Друга частина є поясненням причини і доменного контексту. Її можна залишити як контрольований текст, прив’язаний до цього assert-у.

Тобто правильне рішення — не “формалізувати все” і не “залишити все текстом”, а розділити:

критичне правило → formal instruction;
 пояснення і приклади → controlled freeform.

Чому це особливо важливо для AI-моделей

AI-модель не працює як звичайний інтерпретатор коду. Вона не просто читає orcode і виконує точну машинну операцію. Вона виконує семантичну роботу: розуміє текст, зіставляє контекст, вибирає варіант, формує відповідь.

Саме тому модель потребує двох шарів одночасно.

Перший шар — формальний.

Він каже:

```
тут зупинись;
тут постав питання;
тут не роби припущення;
тут перевір consistency;
тут не створи archive;
тут потрібне approval.
```

Другий шар — змістовий.

Він пояснює:

```
чому це правило існує;
як виглядає типовий випадок;
які приклади є ризиковими;
як людина зазвичай формулює відповідь;
які нюанси є в домені.
```

Без першого шару модель стає некерованою.

Без другого шару модель втрачає зміст.

Ordo потрібна не для того, щоб знищити природну мову, а для того, щоб поставити природну мову в керований execution-контекст.

Ordo-конструкція: межа формалізації

Для цього в Ordo має існувати явне рішення: кожен великий блок інструкції повинен бути класифікований.

Наприклад:

```
CONTENT.CLASSIFY:
  id: C_HISTORY_EVENT_SOURCE_RULE
  source: "original_playbook_section_04"
  classification:
    - formal_gate
    - formal_assert_not
    - controlled_freeform_explanation
```

Або коротше:

```
FORMALIZATION.BOUNDARY:
  formalize:
    - transition_rules
    - approval_requirements
    - output_contracts
    - forbidden_actions
    - validation_gates
  keep_as_controlled_freeform:
    - rationale
    - examples
    - domain_background
    - historical_notes
```

Це не означає, що кожна Ordo-програма має містити такий великий блок. Але на рівні мови така ідея важлива: Ordo має вміти показати, що саме було формалізовано, а що залишилося у контрольованому описі.

Це веде нас до наступних розділів — про FREEFORM і FREEFORM.COVERAGE.

Малий приклад

Візьмемо просту інструкцію:

Підготуй коротку відповідь клієнту про затримку доставки. Відповідь має бути ввічливою, не обіцяй точну дату, якщо її немає, поясни, що ми уточнюємо інформацію, і запропонуй повідомити клієнта окремо після оновлення статусу.

Що тут треба формалізувати?

Intent:

```
INTENT:
  goal: "підготувати коротку відповідь клієнту про затримку доставки"
```

ASSERT.NOT:

```
ASSERT.NOT:
  id: A_NO_FAKE_DELIVERY_DATE
  forbidden:
    - promise_exact_delivery_date_without_confirmed_date
```

OUTPUT.DEF:

```
OUTPUT.DEF:
  id: customer_delay_reply
  format: "short_message"
  tone: "polite"
  must_include:
    - delay_acknowledgement
    - information_is_being_checked
    - follow_up_offer
```

А що можна залишити як контрольований опис?

Відповідь має звучати людяно, без надмірно формального тону. Не перекладай відповідальність на клієнта. Не використовуй фрази, які звучать як автоматичний шаблон підтримки.

Ці правила теж важливі. Але їх не обов'язково всі перетворювати на окремі orcode-и. Достатньо прив'язати їх до output style guidance або controlled freeform.

Типова помилка 1: формалізувати все до втрати сенсу

Погана Ordo-програма може виглядати дуже “технічно”, але бути майже непридатною для роботи.

Наприклад:

```
STEP.001: detect_client
STEP.002: detect_delay
STEP.003: detect_emotion
STEP.004: detect_apology
STEP.005: detect_followup
STEP.006: generate_sentence_1
STEP.007: generate_sentence_2
STEP.008: generate_sentence_3
```

Здається, що процес контрольований. Але насправді така інструкція може бути гіршою за нормальний людський опис, бо вона змушує автора передчасно деталізувати те, що модель і так добре робить як мовна модель.

Формалізація має додавати контроль, а не імітувати контроль.

Типова помилка 2: залишити критичне правило в поясненні



Протилежна помилка — не формалізувати те, що обов’язково треба формалізувати.

Погано:

Було б добре перед фінальною відповіддю перевірити, що немає суперечностей.

Якщо consistency check критичний, це не “було б добре”. Це gate.

Добре:

```
GATE.DEF:
  id: G_CONSISTENCY_REQUIRED
  before: final_answer
  require:
    - consistency_check.status == passed
  on_fail: stop_and_report
```

У Ordo важливо не тільки мати правила, а й правильно визначати їхній статус: пояснення, рекомендація, заборона, gate, output requirement або state transition.

Типова помилка 3: використовувати FREEFORM як смітник



FREEFORM не повинен бути місцем, куди складають усе, що автору лінь структурувати.

Погано:

```
FREEFORM:
  text: "Тут усі важливі правила процесу. Прочитай уважно і зроби правильно."
```

Це не контрольований escape hatch. Це повернення до великого prompt-a.

Добре:

```
ASSERT.NOT:
  id: A_NO_ARCHIVE_BEFORE_APPROVAL
  forbidden:
    - generate_archive_before_approval

FREEFORM:
  binding: A_NO_ARCHIVE_BEFORE_APPROVAL
  purpose: "domain_rationale"
  text: "Це правило з'явилося через повторювану помилку: фінальний пакет іноді створювався до підтвердження аналітиком очікуваного рівня змін."
```

Тут FREEFORM не замінює правило. Він пояснює правило.

Практичне правило для автора Ordo

Коли ви пишете Ordo-програму, для кожного фрагмента інструкції поставте просте питання:

Якщо модель проігнорує цей фрагмент, що станеться?

Якщо відповідь така:

процес може піти неправильним шляхом;
буде створено неправильний output;
модель може перейти далі без approval;
може бути порушена заборона;
може бути втрачений стан;
handoff стане небезпечним;

тоді цей фрагмент треба формалізувати.

Якщо відповідь така:

відповідь стане менш якісною;
пояснення буде менш повним;
модель втратить частину стилістичного або доменного нюансу;

тоді фрагмент можна залишити як controlled freeform, style guidance або domain explanation.

Міні-вправа

Візьміть таку інструкцію:

Підготуй коротку задачу для Jira. Не додавай технічних деталей реалізації. Обов'язково вкажи загальний опис проблеми, критерії прийняття і ручні тести для тестувальника. Якщо другий критерій пріоритезації ще не визначений, явно напиши, що його потрібно заповнити перед передачею задачі в роботу. Формулювання мають бути рівня РМ, не розробника.

Розділіть її на три групи.

Перша група — що треба формалізувати як OUTPUT.DEF:

- загальний опис проблеми;
- критерії прийняття;
- ручні тести;
- блок про налаштування, якщо він потрібний;

Друга група — що треба формалізувати як ASSERT.NOT:

- не додавати технічних деталей реалізації;
- не приховувати, що другий критерій пріоритезації не визначений;

Третя група — що можна залишити як controlled freeform:

- стиль рівня РМ;
- пояснення, чому задача не повинна бути занадто технічною;
- приклади бажаного тону;

Після цього спробуйте написати коротку Ordo Source-структуру для цієї задачі.

Короткий підсумок

Ordo не вимагає формалізувати абсолютно все.

Формалізувати потрібно те, що керує виконанням: переходи, gate-и, заборони, статуси, approvals, output contracts, validation rules і handoff conditions.

У контрольованому описі можна залишати пояснення, приклади, доменний фон, історію появи правила, стилістичні рекомендації і навчальні коментарі.

Головне правило просте:

критичне для виконання – формалізувати;
важливе для розуміння – прив'язати як controlled freeform;
неважливе для процесу – не перевантажувати мовою.

Саме для цього Ordo потребує FREEFORM: не як спосіб сховати хаос, а як контрольований механізм збереження змісту там, де жорстка структура поки що недоречна.

Розділ 20. Як правильно використовувати FREEFORM

У попередньому розділі ми домовилися про важливу річ: не все в Ordo потрібно формалізувати до останнього слова.

Але з цього не випливає, що FREEFORM можна використовувати як місце, куди складається все незрозуміле.

Навпаки: FREEFORM — одна з найнебезпечніших і водночас найкорисніших частин Ordo. Вона дозволяє зберегти людський зміст там, де жорстка структура ще не потрібна або поки що неможлива. Але якщо користуватися нею без правил, Ordo дуже швидко перетвориться назад на великий prompt.

Цей розділ про те, як використовувати FREEFORM правильно.

Навіщо потрібен FREEFORM

У реальних інструкціях завжди є частини, які важко або недоцільно одразу перетворити на формальні оператори.

Наприклад:

- пояснення доменного контексту;
- приклади хороших і поганих відповідей;
- історія появи правила;
- стильові рекомендації;
- застереження для моделі;
- пояснення для аналітика або розробника;
- нюанси, які поки що не мають стабільної структури.

Якщо спробувати все це силоміць перетворити на `NODE`, `GATE`, `ASSERT.NOT` або `OUTPUT.DEF`, ми можемо втратити зміст.

Тому Ordo вводить контрольовану лазейку:

```
FREEFORM
```

Але це не “вільний текст без правил”.



Правильніше думати так:

FREEFORM — це контрольований фрагмент людського змісту, який має явне місце, причину, межі і прив'язку до execution context.

Чим FREEFORM не є

FREEFORM не є способом обійти Ordo.

Погано:


```
freeform: |
    Виконай весь процес створення історичної події за всіма правилами,
    постав потрібні питання, перевір усе, сформулюй архів і зроби QA.
```

Такий блок фактично ховає весь процес у текст.

Модель знову отримує великий prompt і має сама здогадатися:

```
- де початок;
- де стан;
- де рішення;
- де gate;
- де stop condition;
- що треба підтвердити;
- що можна робити без підтвердження;
- який output є правильним.
```

Це не Ordo.

У правильному Ordo FREEFORM не замінює execution logic. Він тільки доповнює її.

Базове правило FREEFORM



Головне правило:

```
FREEFORM не може містити приховані обов'язкові правила виконання.
```

Якщо фрагмент впливає на те, чи можна рухатися далі, він має бути винесений у формальну структуру.

Наприклад, якщо в тексті є правило:

```
Перед створенням архіву обов'язково перевір, що всі mandatory files існують.
```

це не має залишатися тільки в FREEFORM.

Це має бути GATE:

```
- op: GATE.DEF
  id: G_ARCHIVE_MANDATORY_FILES
  condition: "all mandatory files are present"
  on_fail: "BLOCK_ARCHIVE"
```

А в FREEFORM можна залишити пояснення:

```
freeform:
  id: FF_ARCHIVE_REASON
  binds_to: G_ARCHIVE_MANDATORY_FILES
  text: |
    Цей gate потрібний тому, що в попередніх пакетах модель іноді
    створювала архів без частини mandatory files або додавала зайві файли.
```

Тут FREEFORM пояснює правило, але не є самим правилом.

Що має бути у правильному FREEFORM-блоці

Кожен FREEFORM-блок має мати мінімальну службову інформацію.

Я б рекомендував таку структуру:

```
freeform:
  id: FF_001
  title: "Пояснення доменного контексту"
  type: "domain_explanation"
  binds_to:
    - NODE_SELECT_EVENT_PATH
    - G_EVENT_TYPE_CONFIRMED
  reason: "Потрібно зберегти людське пояснення, але воно не є окремим gate"
  text: |
    ...людський текст...
```

Тут важливі поля:

id	— стабільний ідентифікатор блоку;
title	— коротка назва;
type	— тип freeform-змісту;
binds_to	— до чого цей текст прив'язаний;
reason	— чому текст залишений у freeform;
text	— сам зміст.

Без binds_to модель не розуміє, де саме цей текст застосовується.

Без reason незрозуміло, чому це не було формалізовано.

Без type неможливо оцінювати покриття і якість FREEFORM у великому playbook-у.

Типи FREEFORM

Не всі FREEFORM-блоки однакові.

Корисно одразу розрізняти їх за типами.

Наприклад:

1. domain_explanation
Доменне пояснення, яке допомагає моделі зрозуміти предметну область.
2. rationale
Пояснення, чому правило або gate існує.
3. example
Приклад правильного або неправильного виконання.
4. style_guidance
Рекомендації щодо стилю, тону або рівня деталізації.
5. migration_note
Пояснення, як стара інструкція була перенесена в Ordo.
6. analyst_note
Примітка для аналітика або людини, яка керує процесом.
7. unresolved_design
Усвідомлено залишена відкрита частина, яку треба формалізувати пізніше.

Це допомагає не змішувати все в один великий текстовий блок.

FREEFORM має бути прив'язаний до execution context

Поганий FREEFORM живе сам по собі.

```
freeform:
  text: |
```

У цій системі важливо не пропускати перевірки і не створювати фінальний пакет без погодження аналітика.

Цей текст правильний за змістом, але Ordo не знає, де його застосувати.

Краще:

```
freeform:
  id: FF_APPROVAL_CONTEXT
  type: rationale
  binds_to:
    - G_ANALYST_APPROVAL_BEFORE_ARCHIVE
    - G_NO_ARCHIVE_BEFORE_VALIDATION
  reason: "Пояснює причину двох approval gates"
  text: |
    У цій системі фінальний пакет не можна створювати автоматично,
    тому що аналітик має підтвердити бізнесову і технічну готовність
    перед handoff.
```

Тепер текст має місце в процесі.

FREEFORM не повинен мати власного прихованого статусу

Ще одна типова помилка — написати в FREEFORM щось таке:

Якщо користувач погодився, вважай пакет готовим до архіву.

Це фактично статусна логіка.

Вона має бути в STATUS.SEMANTICS або gate transition.

Правильно:

```
status_semantics:
  ready_for_archive:
    allowed_when:
      - G_ANALYST_APPROVAL_PASSED
      - G_VALIDATION_PASSED
      - G_CONSISTENCY_CHECK_PASSED
```

А FREEFORM може пояснити:

```
freeform:
  id: FF_READY_FOR_ARCHIVE_EXPLANATION
  type: rationale
  binds_to:
    - ready_for_archive
  text: |
    Статус ready_for_archive означає не просто "користувач сказав ок",
    а що пройдено всі формальні approval і validation gates.
```

FREEFORM не повинен створювати нові outputs

Якщо в тексті сказано:

Також підготуй короткий summary для менеджера.

це не має бути тільки у FREEFORM.

Це output contract.

Правильно:

```
outputs:
  - id: OUT_MANAGER_SUMMARY
```

```
type: document
required: true
audience: "manager"
```

А FREEFORM може описати стиль цього summary:

```
freeform:
  id: FF_MANAGER_SUMMARY_STYLE
  type: style_guidance
  binds_to:
    - OUT_MANAGER_SUMMARY
  text: |
    Summary має бути коротким, без зайвих технічних деталей,
    з акцентом на ризик, рішення і наступну дію.
```

FREEFORM і приклади

Приклади — одна з найкращих причин використовувати FREEFORM.

Приклад часто важко формалізувати без втрати користі.

Наприклад:

```
freeform:
  id: FF_GOOD_BAD_GATE_EXAMPLES
  type: example
  binds_to:
    - G_OUTPUT_READY
  text: |
    Погано:
    "Перевір, що все нормально."

    Добре:
    "Перевір, що всі mandatory sections присутні,
    статуси не суперечать один одному,
    validation report має passed або passed_with_notes,
    а unresolved items явно перелічені."
```

Такий FREEFORM не керує процесом напряму, але допомагає моделі краще виконати формальні правила.

FREEFORM і людський стиль

Стильові інструкції часто краще тримати у FREEFORM, якщо вони не є критичними validation rules.

Наприклад:

```
freeform:
  id: FF_PM_STYLE
  type: style_guidance
  binds_to:
    - OUT_JIRA_TASK
  text: |
    Пиши задачу рівнем PM: без зайвих деталей реалізації,
    але з достатньою конкретикою для прийняття задачі в роботу.
```

Але якщо стильова вимога стає критичною заборобою, її треба дублювати як ASSERT.NOT.

Наприклад:

```
assert_not:
  - id: A_NO_INTERNAL_IMPLEMENTATION_DETAILS
```

```
target: OUT_JIRA_TASK
rule: "Output must not include internal implementation details"
```

Тоді FREEFORM пояснює тон, а ASSERT.NOT блокує помилку.

FREEFORM і міграція старих playbook-ів

Під час міграції великих markdown playbook-ів в Ordo майже завжди частина тексту спочатку потрапляє у FREEFORM.

Це нормально.

Ненормально — залишити все в FREEFORM і назвати це міграцією.

Правильний процес такий:

1. Розділити старий текст на фрагменти.
2. Витягнути формальні правила.
3. Перенести рішення в NODE / PATH / STATE.
4. Перенести перевірки в GATE / ASSERT.NOT.
5. Перенести outputs в OUTPUT.DEF.
6. Те, що залишилось, оформити як controlled FREEFORM.
7. Для кожного FREEFORM-блоку пояснити, чому він не формалізований.
8. Пізніше переглянути coverage і поступово зменшувати зайвий FREEFORM.

Тобто FREEFORM — це не смітник, а тимчасова або усвідомлена зона змісту.

Ознаки неправильного FREEFORM

Є кілька простих ознак, що FREEFORM використовується неправильно.

Ознака 1. У FREEFORM є слова “обов’язково”, “заборонено”, “не можна”

Якщо в тексті є:

```
обов’язково перевір;
заборонено створювати;
не можна переходити;
потрібно отримати підтвердження;
```

найімовірніше, це не має залишатися тільки в FREEFORM.

Це кандидат на GATE, ASSERT.NOT, APPROVAL.REQUIRE або STATUS.SEMANTICS.

Ознака 2. FREEFORM визначає порядок кроків

Якщо блок описує послідовність:

```
спочатку зроби А, потім В, після цього С, а якщо D — перейди до Е
```

це не просто пояснення.

Це execution path.

Його треба формалізувати.

Ознака 3. FREEFORM створює новий документ

Якщо freeform-блок каже:

```
також створи окремий README
```

це output definition.

Ознака 4. FREEFORM змінює значення статусу

Якщо блок каже:

цей статус означає, що можна запускати пакет у роботу

це status semantics.

Ознака 5. FREEFORM дозволяє обійти gate

Якщо блок каже:

у простих випадках можна пропустити цю перевірку

це override rule.

Воно має бути явним.

Як вирішити: формалізувати чи залишити FREEFORM

Можна використовувати просте дерево рішень.

Питання 1: Чи цей фрагмент змінює шлях виконання?

Так → формалізувати.

Ні → питання 2.

Питання 2: Чи цей фрагмент блокує або дозволяє перехід?

Так → gate/status/approval.

Ні → питання 3.

Питання 3: Чи цей фрагмент визначає обов'язковий output?

Так → OUTPUT.DEF.

Ні → питання 4.

Питання 4: Чи цей фрагмент забороняє помилкову поведінку?

Так → ASSERT.NOT.

Ні → питання 5.

Питання 5: Чи цей фрагмент пояснює, навчає, дає приклад або стиль?

Так → controlled FREEFORM.

Ні → можливо, фрагмент зайвий.

Приклад: неправильний і правильний варіант

Поганий варіант:

```
freeform:
  text: |
    Перед фінальним архівом перевір, що всі файли є,
    що validation report успішний,
    що consistency check не має критичних помилок,
    і що користувач погодив фінальний пакет.
```

Цей текст містить кілька gate-ів.

Краще:

```
gates:
- id: G_FILES_PRESENT
  condition: "mandatory files exist"
  on_fail: "BLOCK_ARCHIVE"
```

```

- id: G_VALIDATION_PASSED
  condition: "validation report is passed or passed_with_notes"
  on_fail: "BLOCK_ARCHIVE"

- id: G_CONSISTENCY_PASSED
  condition: "consistency check has no critical errors"
  on_fail: "BLOCK_ARCHIVE"

- id: G_USER_APPROVAL_RECEIVED
  condition: "user approved final package"
  on_fail: "WAIT_FOR_APPROVAL"

freeform:
  id: FF_ARCHIVE_GATE_RATIONALE
  type: rationale
  binds_to:
    - G_FILES_PRESENT
    - G_VALIDATION_PASSED
    - G_CONSISTENCY_PASSED
    - G_USER_APPROVAL_RECEIVED
  text: |
    Ці gate-и існують тому, що фінальний архів є handoff-артефактом.
    Якщо він неповний або неперевірений, наступний учасник процесу
    отримає помилкову основу для роботи.

```

Тепер execution logic формальна, а пояснення збережене.

FREEFORM має бути видимим у звіті

Якщо Ordo-програма містить FREEFORM, це має бути видно у validation або coverage report.

Наприклад:

```

{
  "freeform_coverage": {
    "total_blocks": 12,
    "bound_blocks": 12,
    "unbound_blocks": 0,
    "blocks_with_reason": 12,
    "high_risk_blocks": 2,
    "recommendation": "review high-risk freeform blocks for future formalization"
  }
}

```

Це дозволяє бачити, чи FREEFORM справді контрольований.

Якщо в playbook-у 40% змісту залишилось у FREEFORM, це може бути нормально для першої міграції.

Але тоді треба чесно сказати:

```

structured_core: 60%
controlled_freeform: 40%

```

І додати план, що саме варто формалізувати пізніше.

Високоризиковий FREEFORM

Не всі freeform-блоки однаково безпечні.

Високоризиковими є блоки, які:

- містять слова "must", "required", "forbidden", "block", "approve";
- описують послідовність дій;
- стосуються фінального handoff;

- стосуються юридичних, фінансових або комплаєнс-рішень;
- містять правила створення або видалення даних;
- містять правила запуску зовнішніх інструментів;
- визначають, коли можна вважати процес завершеним.

Такі блоки треба або формалізувати, або явно позначити як ризикові.

Наприклад:

```
freeform:
  id: FF_LEGACY_QA_NOTE
  type: unresolved_design
  risk_level: high
  binds_to:
    - QA_PACKAGE_GENERATION
  reason: "Legacy QA description is not yet fully formalized"
  action_required: "Convert to GATE.DEF and OUTPUT.DEF in next version"
  text: |
    ...старий QA-текст...
```

Це чесніше, ніж робити вигляд, що все вже стало формальною Ordo-програмою.

FREEFORM як борг формалізації

Іноді FREEFORM — це не фінальне рішення, а борг.

Це нормально, якщо борг видимий.

Для цього можна використовувати поле:

```
formalization_status: "accepted_for_now"
```

або:

```
formalization_status: "candidate_for_next_version"
```

Наприклад:

```
freeform:
  id: FF_COMPLEX_DOMAIN_RULES
  type: unresolved_design
  formalization_status: "candidate_for_next_version"
  reason: "Domain rules are not stable enough yet"
```

Так Ordo дозволяє розвивати мову поступово.

Ми не вимагаємо миттєвої ідеальної формалізації. Але ми вимагаємо чесності: що формалізовано, що ні, і чому.

Як модель має працювати з FREEFORM



Модель не повинна сприймати FREEFORM як вільний дозвіл робити що завгодно.

Правильна поведінка моделі:

1. Прочитати FREEFORM-блок.
2. Подивитися, до чого він прив'язаний.

3. Визначити його тип.
4. Перевірити, чи він не містить прихованих `gate/rule/output/status`.
5. Використати його тільки в межах `binds_to`.
6. Якщо блок суперечить формальній структурі – формальна структура має пріоритет.
7. Якщо суперечність критична – зупинитися і повідомити про `conflict`.

Наприклад, якщо `GATE.DEF` забороняє створювати архів без `approval`, а `FREEFORM` десь натякає, що “у простих випадках можна одразу сформулювати пакет”, модель не має обирати зручний варіант.

Вона має сказати:

```
CONFLICT.DETECTED:
FREEFORM suggests possible shortcut, but formal gate requires approval.
Execution blocked until conflict is resolved.
```

Пріоритет формальної структури

У Ordo треба закласти просту ієрархію.

1. Formal execution contract
2. Gates / assertions / status semantics
3. Output definitions
4. Domain pack rules
5. Controlled FREEFORM
6. Style notes / examples

`FREEFORM` не може переписувати `GATE`.

`FREEFORM` не може створювати непогоджений `output`.

`FREEFORM` не може змінювати `status semantics`.

`FREEFORM` може пояснювати, уточнювати і навчати.

Міні-вправа

Візьміть такий фрагмент:

```
Після підготовки пакета перевіріть, що README, Jira task, QA package і validation report існують.
Не створюйте архів, якщо validation report має critical errors. У відповіді аналітику поясніть
коротко, що саме готово, а що потребує уваги. Пиші українською мовою, без зайвих англійців.
```

Розкладіть його на частини.

Що тут має бути `GATE`?

- перевірка `mandatory files`;
- заборона створення архіву при `critical errors`;

Що має бути `OUTPUT.DEF`?

- відповідь аналітику з коротким `summary`;

Що має бути `ASSERT.NOT`?

- не створювати архів при `critical errors`;

Що можна залишити як `FREEFORM`?

- пояснення стилю відповіді;
- рекомендація писати українською без зайвих англійців;

Після цього спробуйте оформити один FREEFORM-блок із полями:

```
id
type
binds_to
reason
text
```

Короткий підсумок

FREEFORM потрібний Ordo, бо реальні людські інструкції не завжди можна одразу повністю формалізувати.

Але FREEFORM має бути контрольованим.

Правильний FREEFORM:

```
- має id;
- має тип;
- має reason;
- має binds_to;
- не приховує gate-и;
- не створює outputs;
- не змінює statuses;
- не обходить approvals;
- не суперечить формальній структурі;
- відображається в coverage report.
```

Найкоротше правило:

FREEFORM може пояснювати виконання, але не повинен керувати виконанням замість Ordo.

Саме тому FREEFORM — це не слабкість Ordo, а спосіб чесно працювати з тією частиною людського знання, яка ще не стала формальною мовою.

Розділ 21. FREEFORM.COVERAGE

У попередніх розділах ми домовилися, що FREEFORM потрібен Ordo. Не все можна або варто перетворювати на строгі ор-коди, таблиці й gates. Частина знань завжди залишатиметься у вигляді пояснень, прикладів, доменних нюансів, історичних зауважень або інструкцій для людини.

Але з цього виникає нова проблема.

Якщо FREEFORM дозволений, то потрібно розуміти:

скільки важливої логіки залишилося у FREEFORM;
де саме вона знаходиться;
на що вона впливає;
чи можна її тестувати;
чи не приховані там критичні правила;
що з цього треба формалізувати пізніше.

Саме для цього в Ordo потрібна конструкція FREEFORM.COVERAGE.

Навіщо потрібен FREEFORM.COVERAGE

Без coverage FREEFORM швидко перетворюється на чорну скриньку.

На перший погляд playbook може виглядати структурованим: є paths, nodes, gates, outputs, statuses. Але частина справжніх правил може залишитися у вільному тексті.

Наприклад:

У складних випадках не створюй фінальний пакет без додаткової перевірки.

Якщо це просто фраза у FREEFORM, модель може виконати її один раз, пропустити інший раз, інтерпретувати по-своєму або не зрозуміти, що це blocking rule.

FREEFORM.COVERAGE потрібен, щоб такі місця були видимими.

Він відповідає на питання:

Яка частина процесу вже формалізована, а яка ще живе у вільному тексті?

FREEFORM не є помилкою

Важливо не сприймати FREEFORM як щось погане.

У хорошому Ordo-документі FREEFORM може бути абсолютно нормальним і потрібним. Проблема не в тому, що він існує. Проблема в тому, що ми не знаємо, яку роль він відіграє.

Є безпечний FREEFORM:

пояснення для аналітика;
приклади формулювань;
опис доменного контексту;
історичні причини появи правила;
пояснення термінів;
рекомендації щодо стилю.

І є небезпечний FREEFORM:

```

приховані gates;
приховані заборони;
умови переходу між paths;
правила створення output;
умови зупинки процесу;
винятки, які змінюють поведінку моделі.

```

FREEFORM.COVERAGE допомагає розрізняти ці два випадки.

Що саме потрібно покривати

Coverage має показувати не лише кількість тексту у FREEFORM. Обсяг тексту сам по собі не головний.

Набагато важливіше зрозуміти, чи впливає FREEFORM на виконання.

Для кожного FREEFORM-блоку потрібно знати:

```

id блоку;
де він знаходиться;
до якого path/node/gate/output він прив'язаний;
яку роль він виконує;
чи впливає на рішення моделі;
чи містить правила;
чи містить приклади;
чи потрібні для нього tests;
чи є improvement records, пов'язані з цим блоком;
чи потрібно його формалізувати пізніше.

```

У простому вигляді це можна уявити так:

```

freeform_coverage:
  entries:
    - id: "FF_DOMAIN_CONTEXT_01"
      location: "History Event Domain Pack / Path A1"
      role: "domain_explanation"
      affects_execution: false
      test_required: false
      formalization_needed: false

    - id: "FF_EDGE_CASE_02"
      location: "Package Generation / Final Archive"
      role: "conditional_rule"
      affects_execution: true
      test_required: true
      formalization_needed: true
      suggested_formalization:
        - "convert to GATE.DEF"
        - "add ASSERT.NOT before final archive"

```

Ролі FREEFORM-блоків

Щоб coverage був корисним, кожен FREEFORM-блок має мати роль.

Базові ролі можуть бути такими:

```

explanation
example
note
warning
domain_context
style_guidance
edge_case
conditional_rule
human_instruction

```

```
migration_note
implementation_hint
```

Не всі ролі однаково ризикові.

Наприклад:

```
example – зазвичай низький ризик;
explanation – низький або середній ризик;
warning – середній ризик;
edge_case – середній або високий ризик;
conditional_rule – високий ризик;
human_instruction – залежить від контексту;
implementation_hint – високий ризик, якщо впливає на output або tests.
```

Тому coverage має не просто рахувати блоки, а давати risk view.

Risk view

У Ordo можна ввести просту класифікацію ризику:

```
low
medium
high
critical
```

Наприклад:

```
freeform_risk_summary:
  total_entries: 12
  low: 6
  medium: 3
  high: 2
  critical: 1

critical_entries:
- id: "FF_NO_ARCHIVE_WITHOUT_SELF_CHECK"
  reason: "contains blocking behavior but is not represented as gate"
  action: "formalize_before_release"
```

Якщо FREEFORM-блок має рівень critical, Ordo-програма не повинна вважатися готовою до production-використання без рішення:

```
формалізувати;
покрити тестом;
або явно прийняти ризик.
```

FREEFORM.COVERAGE і тести

FREEFORM, який впливає на поведінку, має бути покритий тестами.

Наприклад, якщо у FREEFORM сказано:

```
Якщо користувач просить створити архів до approval, потрібно зупинитися.
```

Це вже не просто пояснення. Це поведінкове правило.

Для нього потрібен test case:

```
test:
  id: "TC_FREEFORM_NO_ARCHIVE_BEFORE_APPROVAL"
  method: human
  trust_class: human_decision

fixture:
```

```

user_message: "створи архів одразу"

expected:
  gate:
    id: "G_PRE_ARCHIVE_APPROVAL"
    status: "blocked"

  output:
    archive_created: false

```

Coverage має показувати, що цей FREEFORM-блок або вже формалізований як gate, або хоча б перевірений тестом.

FREEFORM.COVERAGE і Improvement Loop

Якщо користувач кілька разів вказує на проблему, яка походить із FREEFORM-блоку, це сильний сигнал.

Наприклад:

Тут знову було неправильно витлумачено edge case.

Тоді improvement record має посилатися не тільки на run або node, а й на конкретний FREEFORM-блок:

```

improvement_record:
  id: "IR-002"
  classification:
    type: "ambiguous_freeform_rule"
    severity: "high"

  affected_unit:
    kind: "freeform"
    id: "FF_EDGE_CASE_02"

  proposed_patch:
    - "split FREEFORM block into explanation and rule"
    - "convert rule part into GATE.DEF"
    - "add regression test"

```

Так FREEFORM.COVERAGE стає не просто звітом, а частиною циклу покращення Ordo-програми.

Формалізація з часом

У першій версії playbook-а частина логіки може залишатися у FREEFORM. Це нормально.

Але Ordo має підтримувати поступову формалізацію.

Цикл може виглядати так:

```

FREEFORM explanation
→ feedback показує проблему
→ debug trace показує місце
→ coverage позначає high risk
→ author формалізує правило
→ додається gate/assertion/test
→ regression suite перевіряє зміну

```

Тобто FREEFORM.COVERAGE допомагає розвивати Ordo-програму без різкого переписування всього документа.

Мінімальний звіт FREEFORM.COVERAGE

Для простих Ordo-програм достатньо короткого звіту:

```
freeform_coverage_report:
  total_entries: 5
  execution_affecting_entries: 1
  tested_entries: 1
  high_risk_entries: 0
  formalization_required: false
```

Для великих playbook-ів потрібен детальніший звіт:

```
freeform_coverage_report:
  total_entries: 28
  by_role:
    explanation: 8
    example: 6
    domain_context: 5
    edge_case: 4
    conditional_rule: 3
    warning: 2

  execution_affecting:
    total: 7
    covered_by_tests: 5
    not_covered:
      - "FF_EDGE_CASE_07"
      - "FF_STATUS_WARNING_02"

  formalization_candidates:
    - id: "FF_EDGE_CASE_07"
      suggested_target: "GATE.DEF"
    - id: "FF_STATUS_WARNING_02"
      suggested_target: "STATUS.SEMANTICS"

  release_status: "blocked_until_review"
```

Типові помилки

Перша помилка — рахувати тільки кількість FREEFORM.

Може бути багато безпечного FREEFORM і мало небезпечного. А може бути один короткий FREEFORM-блок, який фактично змінює всю поведінку процесу.

Друга помилка — не прив'язувати FREEFORM до конкретного місця execution.

Якщо блок не прив'язаний до path, node, gate, output або domain rule, його важко тестувати й покращувати.

Третя помилка — залишати blocking rule у FREEFORM.

Усе, що має зупиняти процес, має бути gate або assertion.

Четверта помилка — не тестувати FREEFORM edge cases.

Якщо FREEFORM описує складний виняток, для нього потрібен test case.

П'ята помилка — не переносити повторювані проблеми з FREEFORM у improvement backlog.

Якщо одна й та сама помилка виникла кілька разів, це вже не випадковість. Це сигнал, що Ordo-програму треба покращувати.

Міні-вправа

Візьміть будь-який фрагмент playbook-а з вільним текстом і дайте відповідь на п'ять питань:

1. Це пояснення, приклад, warning чи правило?
2. Чи впливає цей текст на рішення моделі?
3. Чи може неправильне тлумачення цього тексту зламати процес?
4. Чи є для нього тест?
5. Чи потрібно формалізувати його як gate, assertion, status або output rule?

Якщо відповідь на друге або третє питання — “так”, цей FREEFORM-блок має бути видимим у coverage report.

Короткий підсумок

FREEFORM. COVERAGE потрібен, щоб Ordo не перетворилася назад на великий некерований prompt.

FREEFORM дозволяє зберегти людський сенс, доменні пояснення й складні нюанси. Але все, що впливає на execution, має бути видимим, прив'язаним, оціненим за ризиком і, за потреби, покритим тестами.

Головна ідея розділу проста:

FREEFORM дозволений, але він не має бути невидимим.

У зрілій Ordo-програмі кожен важливий FREEFORM-блок має відповідь на три питання:

де він використовується;
на що він впливає;
як ми перевіряємо, що він не ламає процес.

Розділ 22. Ordo Core

Навіщо потрібен Ordo Core

До цього моменту ми вже розібрали багато окремих частин Ordo: intent, contract, state, nodes, gates, output, status semantics, debug, tests, feedback loop і FREEFORM. Але якщо залишити ці поняття просто набором ідей, кожен автор Ordo-програми почне збирати їх по-своєму.

Один автор назве стартовий вузол `start`. Другий — `entry`. Третій — `initial_question`. Четвертий взагалі сховає початок процесу в довгому тексті.

Так само може статися з `gates`, статусами, `output`, `state`, перевітками, `trace` і `FREEFORM`. Формально всі будуть “писати Ordo”, але фактично кожна програма буде жити за власними правилами.

Саме тому потрібен Ordo Core.

Ordo Core — це мінімальний обов’язковий набір понять, правил і конструкцій, без яких Ordo-програма не вважається повноцінною Ordo-програмою.

Core не описує всю предметну область. Він не знає, що таке історична подія, моніторингова подія, юридичний висновок, перевірка компанії чи QA-пакет. Core відповідає за інше: за базову форму керованого виконання.

Простіше кажучи:

Ordo Core — це скелет мови.



На цей скелет потім накладаються `profiles`, `domain packs`, `libraries` і конкретні `playbook`-и.

Що входить в Ordo Core

Ordo Core має відповідати на кілька базових питань:

- З чого починається виконання?
- Який контракт потрібно підтвердити?
- Який `state` ведеться під час процесу?
- Які вузли процесу існують?
- Які відповіді можуть бути прийняті?
- Які `gates` блокують або дозволяють перехід?
- Який `output` має бути створений?
- Які `negative assertions` забороняють неправильні дії?
- Як фіксується `trace` виконання?
- Як позначається `FREEFORM` і його покриття?

Тобто Core — це не бібліотека готових рішень і не `domain pack`. Це базова граматика виконання.

У першій версії Ordo Core можна мислити через такі ключові блоки:

```
ENTRY.DEF
NODE.DEF
STATE.SCHEMA
ANSWER.REGISTRY
OUTPUT.DEF
ASSERT.NOT
STATUS.SEMANTICS
ASSUMPTION.LEDGER
FREEFORM.COVERAGE
TRACE.REQUIRE
GATE.REQUIRE
```

Далі розберемо їх простими словами.

ENTRY.DEF

ENTRY.DEF описує, як Ordo-програма починається.

У звичайному prompt-і початок часто неформальний. Користувач пише щось, модель якось інтерпретує це і починає відповідати.

В Ordo старт має бути визначеним.

Наприклад:

```
entry:
  id: "ENTRY_MAIN"
  accepts:
    - "new_user_request"
    - "uploaded_playbook"
    - "existing_state"
  first_node: "NODE_CLASSIFY_REQUEST"
```

Це означає: коли приходить новий запит, програма не повинна одразу створювати фінальний результат. Вона має перейти в перший визначений вузол і класифікувати запит.

ENTRY потрібен для того, щоб модель не вгадувала, “з чого почати”.

NODE.DEF

NODE.DEF описує один крок або вузол процесу.

Node — це не просто абзац інструкції. Це місце, де модель має виконати певну дію: поставити питання, прийняти відповідь, оновити state, вибрати path, перевірити gate або перейти далі.

Приклад:

```
node:
  id: "NODE_COLLECT_ALIAS"
  purpose: "collect event alias"
  asks:
    question: "Який alias події?"
  writes_to_state:
    - "event.alias"
  next:
    when_answered: "NODE_COLLECT_SOURCE_FIELD"
```

Це вже не просто порада “запитай alias”. Це формальна частина execution flow.

Core не визначає, які саме nodes потрібні в кожній предметній області. Але Core визначає, що node має мати id, purpose, очікувану дію, вплив на state і правило переходу.

STATE.SCHEMA

STATE.SCHEMA описує, які дані процес пам'ятає під час виконання.

Без state модель легко втрачає контекст. Вона може двічі питати одне й те саме, забути підтвердження, переплутати чернетку з фінальним результатом або прийняти припущення за факт.

Приклад:

```
state_schema:
  event:
    alias:
      type: "string"
      required: true
      status: "unconfirmed"
    source_field:
      type: "string"
      required: true
      status: "unconfirmed"
    approvals:
      pre_archive:
        type: "boolean"
        default: false
```

State schema не гарантує, що модель ніколи не помилиться. Але вона дає чітку карту того, що потрібно пам'ятати і в якому статусі це перебуває.

ANSWER.REGISTRY

ANSWER.REGISTRY описує, які типи відповідей користувача Ordo-програма може приймати.

Це важливо для guided intake. Користувач не завжди відповідає ідеально структуровано. Він може написати:

```
так
підтверджую
ні
змінив рішення
повернись назад
давай далі
це не те
```

Без registry модель кожного разу сама вирішує, що означає відповідь. Це небезпечно.

Приклад:

```
answer_registry:
  confirm:
    examples:
      - "так"
      - "підтверджую"
      - "ок"
    effect: "mark_current_contract_part_confirmed"

  reject:
    examples:
      - "ні"
```

```

- "не підходить"
- "це не те"
effect: "keep_state_unconfirmed"

go_next:
  examples:
    - "далі"
    - "йдемо далі"
  effect: "advance_if_current_gate_passed"

```

Core не повинен знати всі фрази кожної мови. Але Core має вимагати, щоб важливі відповіді були класифіковані і мали визначений effect.

OUTPUT.DEF

OUTPUT.DEF описує, що саме Ordo-програма має створити.

Output — це не просто “дай відповідь”. Це визначена структура результату.

Наприклад:

```

output:
  id: "FINAL_PACKAGE"
  type: "archive"
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"
    - "CONSISTENCY_CHECK_REPORT.json"
  gates_before_creation:
    - "G_CONTRACT_CONFIRMED"
    - "G_PRE_ARCHIVE_APPROVED"
    - "G_SELF_CHECK_PASSED"

```

Output definition потрібен, щоб модель не змішувала чернетку, пояснення, фінальний документ і handoff в один хаотичний текст.

ASSERT.NOT

ASSERT.NOT — це негативні перевірки. Вони описують, чого Ordo-програма не має права робити.

Наприклад:

```

assert_not:
  - id: "NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
    condition: "pre_archive_approval != true"
    forbidden_action: "create_final_archive"

```

У Core це особливо важливо. Бо позитивні правила кажуть, що треба зробити. Але для AI-моделі часто ще важливіше явно сказати, чого робити не можна.

Особливо це стосується:

- не створювати фінальний output до approval;
- не вгадувати відсутні дані;
- не позначати unconfirmed як confirmed;
- не ховати gate у FREEFORM;
- не трактувати приклад як правило;
- не змінювати domain logic без explicit instruction.

STATUS.SEMANTICS

STATUS.SEMANTICS описує значення статусів.

У складних процесах слова “готово”, “підтверджено”, “чернетка”, “blocked”, “passed” можуть легко змішуватися.

Core має вимагати, щоб статуси мали чітку семантику.

Наприклад:

```
status_semantics:
  draft:
    meaning: "created but not approved"
    allows_final_handoff: false

  confirmed:
    meaning: "explicitly approved by user"
    allows_gate_pass: true

  blocked:
    meaning: "execution cannot continue until condition is resolved"
    allows_next_step: false
```

Це особливо важливо в Ordo, бо мова працює не тільки з текстом, а з процесом.

ASSUMPTION.LEDGER

ASSUMPTION.LEDGER — це журнал припущень.

Модель часто змушена робити припущення. Проблема не в самих припущеннях, а в тому, що вони стають невидимими.

Core має вимагати, щоб важливі припущення були зафіксовані.

Наприклад:

```
assumption_ledger:
  - id: "A-001"
    assumption: "source field belongs to EDR factual data"
    reason: "user provided EDR-like payload"
    status: "needs_confirmation"
    used_for:
      - "path_selection"
```

Якщо припущення впливає на path, gate або output, воно не може бути прихованим.

FREEFORM.COVERAGE

Core не забороняє FREEFORM. Навпаки, він визнає, що частина інструкцій може залишатися вільною мовою.

Але Core має вимагати, щоб FREEFORM був контрольованим.

Для цього потрібен FREEFORM.COVERAGE:

```
freeform_coverage:
  entries_total: 5
  structured_bindings: 4
  unbound_entries:
```

```
- "FF_DOMAIN_EXAMPLES"
risk: "medium"
```

Це дозволяє бачити, які частини playbook-а ще не формалізовані і де можуть виникати помилки.

TRACE.REQUIRE

Після появи Debug, Test & Improvement Layer Core має включати мінімальну вимогу до trace.

Навіть якщо run не запущений у повному debug mode, складна Ordo-програма має залишати базові сліди виконання:

```
- який entry використано;
- який path обрано;
- які gates пройдено;
- які gates заблоковано;
- які state fields змінено;
- який output створено;
- які warnings були зафіксовані.
```

Це не завжди має бути великий детальний лог. Але виконання не повинно бути повністю непрозорим.

GATE.REQUIRE

Core також має вимагати, щоб важливі переходи були захищені gates.

Наприклад, фінальний output не повинен створюватися просто тому, що користувач написав “давай”. Має бути перевірено, чи виконані умови.

```
gate_require:
  before:
    - action: "create_final_output"
      required_gates:
        - "G_CONTRACT_CONFIRMED"
        - "G_VALIDATION_PASSED"
```

Це один із головних принципів Ordo: gate — це не рекомендація, а контрольна точка.

Чим Core відрізняється від Profile

Core відповідає за базову мову.

Profile відповідає за тип процесу.

Наприклад:

```
Core каже: output має бути визначений.
Profile каже: для документації output має мати шаблон, rendered validation i catalog.

Core каже: gates мають бути явними.
Profile каже: для QA-пакета потрібні gates manual QA readiness i automation readiness.

Core каже: state має бути описаний.
Profile каже: для guided intake state має містити current node, confirmed answers i pending questions.
```

Тобто profile розширює Core, але не замінює його.

Чим Core відрізняється від Domain Pack

Domain Pack знає предметну область.

Core не знає, що таке HistoryEvent, ChangeRecord, ExternalHistoryEvent, Monitoring Center, EDR, source row або QA package.

Domain Pack описує такі речі:

- domain vocabulary;
- domain-specific paths;
- domain gates;
- domain statuses;
- domain outputs;
- domain examples;
- domain no-op rules.

А Core забезпечує, щоб усе це було виконуваним, контрольованим і перевірюваним.

Чим Core відрізняється від Library

Library — це reusable готове рішення.

Наприклад, бібліотека може містити готовий набір contract-first gates або rendered artifact validation.

Core же визначає, як такі речі взагалі підключаються, перевіряються і не конфліктують із програмою.

Коротко:

Core — базова мова.
 Profile — стиль/тип процесу.
 Domain Pack — предметна область.
 Library — готовий reusable фрагмент.



Типові помилки

Помилка 1. Робити Core занадто великим

Core не має містити все. Якщо покласти в Core історичні події, QA-пакети, юридичні висновки, API-конфігурації і шаблони документів, Core стане неповоротким.

Core має бути мінімальним.



Помилка 2. Ховати правила Core у FREEFORM

Якщо правило блокує виконання, воно не має жити тільки в пояснювальному тексті.

Погано:

```
Перед архівом бажано зробити self-check.
```

Добре:

```
gate:
  id: "G_SELF_CHECK_PASSED"
  method: mechanical
  trust_class: deterministic
  blocking: true
```

Помилка 3. Вважати Core документацією

Core — це не опис для читання. Це частина виконуваної моделі процесу.

Помилка 4. Дозволяти domain pack-у ламати Core

Domain Pack може розширювати Core, але не повинен скасовувати базові правила без explicit override і trace.

Міні-вправа

Візьміть будь-який складний prompt, який ви використовували раніше, і спробуйте виділити в ньому Core-частину:

1. Де починається процес?
2. Які вузли або кроки там є?
3. Який state потрібно вести?
4. Які gates мають бути blocking?
5. Який output очікується?
6. Які дії потрібно явно заборонити?
7. Які припущення можуть виникнути?
8. Що треба логувати для debug?

Якщо на ці питання немає відповіді, перед вами ще не Ordo-програма, а тільки неформальна інструкція.

Короткий підсумок

Ordo Core — це мінімальний обов'язковий набір конструкцій, який робить Ordo-програму керованою.

Core не описує предметну область і не замінює domain pack. Він задає базову форму execution:

```
entry → node → state → gate → output → trace
```

Без Core Ordo перетвориться на набір красивих prompt-шаблонів.

З Core вона стає мовою керування поведінкою AI-моделі.

Розділ 23. Ordo Profiles

Навіщо потрібні Profiles

У попередньому розділі ми розібрали Ordo Core — мінімальний набір конструкцій, без яких Ordo-програма не може бути керованою. Core відповідає за базову форму виконання: entry, node, state, gate, output, trace.

Але в реальній роботі цього недостатньо.

Одна Ordo-програма може створювати аналітичний пакет. Інша — вести користувача через guided intake. Третя — формувати QA-документацію. Четверта — перевіряти rendered artifact. П'ята — працювати з approvals, доказами, шаблонами й каталогом документів.

Усі ці сценарії використовують Core, але потребують різних додаткових правил.

Саме для цього в Ordo потрібні Profiles.

Простими словами:

Ordo Profile — це стандартний набір додаткових правил і конструкцій для певного типу роботи.



Core каже: “будь-яка Ordo-програма має мати кероване виконання”.

Profile каже: “як саме керувати виконанням у конкретному класі задач”.

Чому не можна все покласти в Core

Може здатися, що всі корисні правила треба одразу додати в Core. Наприклад:

- правила для документів;
- правила для QA;
- правила для approvals;
- правила для rendered artifact validation;
- правила для evidence matrix;
- правила для розбиття документації на файли;
- правила для package generation.

Але тоді Core швидко стане занадто великим.

Core має залишатися мінімальним і стабільним. Він не повинен знати всі можливі типи задач. Інакше Ordo втратить гнучкість.

Тому Ordo розділяє:

Core	→ базова мова виконання
Profile	→ типовий режим роботи
Domain	→ конкретна предметна область
Library	→ reusable готові частини



Це схоже на звичайне програмування. У мові є базовий синтаксис, але для web, тестів, баз даних або UI використовуються окремі фреймворки й бібліотеки.

В Ordo Profile виконує роль такого стандартного режиму роботи для певного класу задач.

Що може входити в Profile

Profile може містити не все підряд, а тільки ті правила, які повторюються в багатьох Ordo-програмах одного типу.

Наприклад, Documentation Profile може містити:

- правила роботи з шаблонами;
- правила розбиття великих документів;
- правила вибору потрібних документів;
- правила перевірки готового rendered artifact;
- правила catalog / selected docs;
- правила self-check перед handoff.

QA Profile може містити:

- структуру test case;
- правила fixture;
- expected behavior;
- негативні сценарії;
- regression suite;
- coverage report;
- manual QA runbook rules.

Approval Profile може містити:

- що саме потребує підтвердження людини;
- які gates є blocking;
- які рішення модель не має права приймати сама;
- як фіксувати approval у state;
- як заборонити фінальний output без approval.

Тобто Profile — це не предметна область. Це стиль і режим виконання.

Приклад: Documentation Profile

Уявімо, що ми маємо Ordo-програму, яка створює пакет документів.

Без Profile інструкція може виглядати так:

```
Створи README, паспорт, задачу, QA-документ і validation report.
Перед архівом перевіріть, що все консистентно.
```

Це зрозуміло людині, але для керованого виконання такому опису бракує структури.

Documentation Profile може формалізувати це так:

```
profile:
  id: "ordo.profile.documentation"
  version: "0.1"
```

```
rules:
  - id: "DOC.CATALOG"
    description: "усі документи пакета мають бути зареєстровані в каталозі"

  - id: "DOC.SELECT"
    description: "перед відповіддю модель має визначити, які документи потрібні для поточного кроку"

  - id: "TEMPLATE.BIND"
    description: "кожен output-документ має бути прив'язаний до шаблону або явно описаної структури"

  - id: "RENDER.VALIDATE"
    description: "перевіряти потрібно не тільки шаблон, а фінальний rendered artifact"
```

Тепер будь-яка Ordo-програма, яка підключає цей Profile, отримує стандартні правила документаційного виконання.

Profile як контракт поведінки



Profile не просто додає корисні поради. Він змінює execution contract.

Якщо Ordo-програма підключила Documentation Profile, модель уже не може працювати так, ніби вона просто пише текст.

Вона має:

- знати, які документи існують;
- вибирати релевантні документи для кроку;
- не змішувати шаблон і rendered artifact;
- не створювати фінальний пакет без self-check;
- фіксувати, які документи були використані;
- повідомляти про missing required artifact;
- виконувати validation gates.

Це важлива різниця.

У звичайному prompt-і можна написати: “перевір консистентність”.

У Profile можна сказати: “консистентність є обов’язковим gate перед handoff”.

Profile y Ordo Source

У людському Ordo Source підключення Profile може виглядати так:

```
ordo:
  version: "0.11"

profiles:
  use:
    - id: "ordo.profile.documentation"
      version: "0.1"
    - id: "ordo.profile.qa"
      version: "0.1"
    - id: "ordo.profile.approval"
      version: "0.1"
```

Така програма одразу повідомляє:

```
це не просто генерація тексту;
це документаційний процес;
він має QA-структуру;
частина рішень потребує людського approval.
```

Profile у compiled IR

У compiled IR це може перетворитися на набір operations:

```
[
  {
    "op": "PROFILE.USE",
    "id": "P1",
    "profile": "ordo.profile.documentation",
    "version": "0.1"
  },
  {
    "op": "PROFILE.USE",
    "id": "P2",
    "profile": "ordo.profile.qa",
    "version": "0.1"
  },
  {
    "op": "PROFILE.BIND_RULES",
    "id": "P3",
    "profiles": ["P1", "P2"]
  }
]
```

Після цього компілятор або runtime має знати, що в програмі активні додаткові правила.

Наприклад, якщо Documentation Profile активний, тоді final package generation не може відбутися без RENDER.VALIDATE або еквівалентного gate.

Типові Profile-конструкції

У поточній концепції Ordo можна виділити такі Profile-level конструкції:

```
TEMPLATE.BIND
EVIDENCE.MATRIX
APPROVAL.REQUIRE
DOC.SPLIT
DOC.CATALOG
DOC.SELECT
RENDER.VALIDATE
QA.CASE.DEF
QA.RUNBOOK.DEF
PACKAGE.SELF_CHECK
HANDOFF.NOTE
```

Це не обов'язково повний список. Але він показує різницю між Core і Profile.

Core каже:

```
має бути gate
```

Profile каже:

```
перед архівом має бути rendered artifact validation gate
```

Core каже:

має бути output

Profile каже:

output має відповідати template binding і входити в document catalog

Core каже:

має бути state

Profile каже:

state має містити approval status, selected docs і validation result

Profile і Domain Pack

Profile часто плутають із Domain Pack. Це треба розділити дуже чітко.

Profile відповідає на питання:

який це тип процесу?

Domain Pack відповідає на питання:

про яку предметну область цей процес?

Наприклад, History Event Playbook може використовувати:

```
Documentation Profile
QA Profile
Approval Profile
Debug/Test Profile
History Event Domain Pack
```

Profile дає загальні правила документів, QA і approval.

Domain Pack дає специфіку історичних подій:

```
- Path A1/A2/A3/A4/A5;
- HistoryEvent;
- ChangeRecord;
- source row;
- no-op rules;
- event alias;
- history event package structure.
```

Це різні рівні.

Якщо їх змішати, Ordo-програма стане важкою для підтримки.

Profile і бібліотеки

Пізніше в книзі ми окремо поговоримо про Ordo Libraries. Але вже тут варто зафіксувати різницю.

Profile — це стандартний режим поведінки.

Library — це reusable пакет готових частин.

Наприклад:

```
ordo.profile.qa
```

може визначати, що в QA-процесі мають бути fixtures, expected behavior і regression suite.

А бібліотека:

```
ordo.library.qa.history_event_basic_tests
```

може містити готові test cases для конкретного типу History Event.

Тобто Profile задає правила гри, а Library може дати готові reusable елементи для цієї гри.

Чому Profiles важливі для компілятора

Компілятор Ordo має розуміти активні Profiles.

Без цього він не зможе правильно перевірити програму.

Наприклад, якщо активний QA Profile, компілятор може вимагати:

- наявність TEST.DEF;
- наявність хоча б мінімального coverage report;
- наявність negative test для blocking gates;
- наявність expected behavior для ключових paths.

Якщо активний Documentation Profile, компілятор може перевіряти:

- чи всі required documents описані;
- чи є document catalog;
- чи є validation report;
- чи не створюється archive до self-check;
- чи rendered artifact validation не замінена перевіркою шаблону.

Таким чином Profile — це ще й механізм compiler validation.

Типові помилки

Помилка 1. Робити Profile занадто предметним

Погано:

```
Profile для зміни статутного капіталу компанії
```

Це вже domain pack або library, а не Profile.

Profile має бути ширшим:

```
Documentation Profile
QA Profile
Approval Profile
Guided Intake Profile
```

Помилка 2. Класти всі правила в Core

Якщо все зробити Core, Core стане непідйомним.

Core має бути малим. Profiles мають розширювати його для типових режимів роботи.

Помилка 3. Не вказувати активний Profile

Якщо Ordo-програма фактично працює як QA-процес, але не підключає QA Profile, компілятор не зможе вимагати потрібні тести.

Погано:

десь у тексті написано, що треба перевірити QA

Добре:

```
profiles:
  use:
    - id: "ordo.profile.qa"
      version: "0.1"
```

Помилка 4. Змішувати Profile і Library

Profile не повинен бути складом готових прикладів на всі випадки.

Для reusable готових частин потрібні libraries.

Profile має задавати правила, обов'язки і gates.

Міні-вправа

Візьміть будь-який складний процес, який ви хочете виконувати через AI-модель.

Наприклад:

- створення аналітичного пакета;
- перевірка документа;
- підготовка QA-інструкцій;
- guided intake нової задачі;
- review змін у коді;
- формування handoff для розробника.

Спробуйте відповісти:

1. Який Core тут потрібен?
2. Який це тип процесу?
3. Який Profile найкраще підходить?
4. Чи потрібен Documentation Profile?
5. Чи потрібен QA Profile?
6. Чи потрібен Approval Profile?
7. Які gates має додати Profile?
8. Які перевірки має виконувати компілятор?
9. Що тут є domain-specific і не має потрапити в Profile?

Якщо ви можете відповісти на ці питання, ви вже починаєте мислити не prompt-ами, а шарами Ordo.

Короткий підсумок

Ordo Profile — це стандартний набір додаткових правил для певного типу Ordo-процесу.

Core задає базову мову виконання.

Profile задає режим роботи.

Domain Pack задає предметну область.

Library дає reusable готові частини.

Правильне розділення цих рівнів дозволяє Ordo залишатися одночасно простою, розширюваною і придатною для великих playbook-ів.

Без Profiles кожен складний playbook буде сам винаходити правила документації, QA, approval і validation.

З Profiles ці правила стають повторно використовуваними і зрозумілими для компілятора.

Розділ 24. Domain Packs

Навіщо потрібні Domain Packs

У попередніх розділах ми розділили Ordo на кілька рівнів.

Ordo Core задає мінімальну мову виконання.

Ordo Profiles додають правила для типових режимів роботи: документація, QA, погодження, guided intake, debug і тестування.

Але цього все ще недостатньо для реальних процесів.

Бо майже кожен серйозний процес має свою предметну логіку.

Наприклад, створення історичної події має свої поняття:

```
HistoryEvent
ChangeRecord
source row
alias
display name
old value
new value
Path A1
Path A2
Path A4
no-op
rollback
manual QA package
automation spec
```

Моніторингова подія матиме інші поняття:

```
monitoring event
configuration
translation tree
sandbox evaluate
REST runbook
business passport
technical package
trigger condition
expected notification
```

Юридичний аналіз матиме ще інші поняття:

```
jurisdiction
legal basis
risk factor
evidence
finding
recommendation
exception
```

Це не варто класти в Core.

Також це не зовсім Profile. Profile каже: “цей процес має QA”, “цей процес має погодження”, “цей процес має documentation runtime”.

А Domain Pack каже: “у цій предметній області є такі об’єкти, такі правила, такі шляхи, такі gates, такі edge cases”.

Тому в Ordo потрібен окремий рівень:

Domain Pack



Просте пояснення

Domain Pack — це пакет предметних знань і правил для конкретної сфери.

Якщо пояснювати дуже просто:

Core — це граматика Ordo.
 Profile — це стиль роботи.
 Domain Pack — це знання конкретної області.

Або так:

Core знає, що таке NODE і GATE.
 Profile знає, що в QA-процесі потрібні tests і coverage.
 Domain Pack знає, що для History Event треба відрізнити A1 від A2, по-ор від real change, source row від generated event.

Domain Pack перетворює загальну мову Ordo на інструмент для конкретної роботи.



Чому Domain Pack не можна замінити довгим prompt-ом

Можна, звичайно, написати в prompt-і великий опис предметної області.

Але тоді виникають знайомі проблеми:

- модель не розуміє, які правила є обов'язковими;
- приклади змішуються з правилами;
- edge cases губляться;
- gates не виконуються як blocking;
- нові сценарії додаються хаотично;
- немає test coverage;
- незрозуміло, яку частину інструкції змінювати;
- feedback користувача не прив'язаний до конкретного rule або path.

Domain Pack має виправити це.

Він робить предметну область структурованою:

- які є об'єкти;
- які є статуси;
- які є paths;
- які питання треба ставити;
- які gates є blocking;
- які outputs створюються;
- які edge cases існують;
- які правила залишаються в controlled FREEFORM;
- які tests перевіряють поведінку.

Що входить у Domain Pack

Мінімальний Domain Pack може містити такі частини:

```
DOMAIN.DEF
DOMAIN.VOCABULARY
DOMAIN.OBJECTS
DOMAIN.PATHS
DOMAIN.RULES
DOMAIN.GATES
DOMAIN.STATUS
DOMAIN.OUTPUTS
DOMAIN.FREEFORM
DOMAIN.TESTS
DOMAIN.COVERAGE
```

DOMAIN.DEF

Це визначення предметної області.

Наприклад:

```
domain:
  id: "history_event"
  name: "History Event Domain Pack"
  version: "0.1"
  purpose: "Guided intake and package generation for history event creation or update."
```

Цей блок відповідає на питання:

```
Що це за область?
Для чого вона потрібна?
Яку задачу вона допомагає виконувати?
```

DOMAIN.VOCABULARY

Це словник термінів.

Наприклад:

```
vocabulary:
  HistoryEvent: "final internal historical event shown in company/person history"
  ChangeRecord: "technical record describing detected source-level change"
  source_row: "input source object from which change is detected"
  no_op: "case where no new event should be created"
```

Словник потрібен не для краси. Він зменшує ризик, що модель змішає близькі поняття.

Наприклад, якщо не визначити різницю між ChangeRecord і HistoryEvent, модель може почати трактувати технічний record як готову історичну подію.

DOMAIN.OBJECTS

Це опис основних об'єктів області.

```
objects:
- id: "HistoryEvent"
  required_fields:
  - "alias"
  - "display_name"
  - "event_date"
  - "old_value"
  - "new_value"
```

```
- id: "ChangeRecord"
  required_fields:
    - "field"
    - "old"
    - "new"
    - "status"
```

У реальному Domain Pack ці поля можуть бути складнішими. Але ідея проста: модель має знати, з якими сутностями вона працює.

DOMAIN.PATHS

Це опис основних сценаріїв.

Наприклад:

```
paths:
- id: "A1"
  name: "direct source row field change"
  when:
    - "change is detected directly in source row"
  requires:
    - "source field confirmed"
    - "old/new values confirmed"

- id: "A2"
  name: "related entity change"
  when:
    - "change belongs to entity related through identification center"
  requires:
    - "main entity confirmed"
    - "related entity confirmed"
    - "relation context confirmed"
```

Paths потрібні, щоб модель не “вгадувала” сценарій на основі загального враження.

Вона має вибрати path і пояснити чому.

DOMAIN.RULES

Це предметні правила.

Наприклад:

```
rules:
- id: "R_NO_EVENT_FOR_NOOP"
  text: "If old and new normalized values are equal, no HistoryEvent should be created."
  enforcement: "blocking"

- id: "R_CURRENCY_NORMALIZATION"
  text: "Capital amount comparison must use normalized amount and normalized currency."
  enforcement: "required"
```

Правило має бути не просто абзацом у документації. Воно має мати ідентифікатор і enforcement semantics.

DOMAIN.GATES

Це контрольні точки, специфічні для області.

```
gates:
- id: "G_SOURCE_FIELD_CONFIRMED"
  type: "approval"
  blocking: true
```

```

description: "Source field must be confirmed before generating event passport."

- id: "G_NOOP_CHECK_DONE"
  type: "validation"
  blocking: true
  description: "No-op check must be completed before event creation decision."

```

Domain Pack може використовувати загальні gates із Profile, але додавати свої предметні gates.

DOMAIN.OUTPUTS

Це опис результатів, які має створювати процес.

Наприклад:

```

outputs:
- id: "history_event_passport"
  format: "markdown"
  required: true

- id: "jira_task"
  format: "markdown"
  required: true

- id: "manual_qa_package"
  format: "markdown"
  required: true

```

Це важливо, бо в різних доменах outputs різні.

DOMAIN.FREEFORM

Навіть у Domain Pack частина знань може залишатися у controlled FREEFORM.

Наприклад:

```

freeform:
- id: "FF_DOMAIN_EXAMPLES"
  purpose: "Examples of valid and invalid history event descriptions."
  binding:
    applies_to:
      - "DOMAIN.PATHS"
      - "DOMAIN.RULES"
  must_not_contain:
    - "blocking gates"
    - "status semantics"

```

Тобто Domain Pack не забороняє вільний текст. Він робить його контрольованим.

DOMAIN.TESTS

Domain Pack має містити або підключати тести.

```

tests:
- id: "TC_A1_DIRECT_FIELD_CHANGE"
  expects:
    path: "A1"
    required_gates:
      - "G_SOURCE_FIELD_CONFIRMED"

- id: "TC_NOOP_NORMALIZED_VALUES_EQUAL"
  expects:
    noop: true
    history_event_created: false

```

Без тестів Domain Pack дуже швидко починає ламатися під час змін.

Domain Pack як contract між людьми і моделлю

Domain Pack — це не просто технічна специфікація.

Це contract між:

- автором предметної логіки;
- аналітиком;
- моделлю;
- compiler/runtime;
- тестами;
- майбутніми користувачами.

Автор предметної області каже:

```
У цій сфері ми працюємо ось так.
Ось наші поняття.
Ось наші paths.
Ось наші gates.
Ось де модель має зупинитись.
Ось що вона не має робити.
Ось як перевірити, що вона не зламалась.
```

Модель не повинна сама винаходити domain logic з повітря.

Вона має виконувати Domain Pack.

Приклад: History Event Domain Pack

Для History Event Domain Pack основні частини можуть виглядати так:

```
DOMAIN.DEF:
  History Event guided intake and package generation

DOMAIN.VOCABULARY:
  HistoryEvent
  ChangeRecord
  source row
  ExternalHistoryEvent
  no-op
  rollback
  manual QA
  automation spec

DOMAIN.PATHS:
  A1 – direct source row field change
  A2 – related entity change through identification center
  A3 – generated/calculated data change
  A4 – external history event input
  A5 – correction / rollback / special case

DOMAIN.GATES:
  G_ALIAS_CONFIRMED
  G_DISPLAY_NAME_CONFIRMED
  G_SOURCE_ROW_CONFIRMED
  G_VALUES_CONFIRMED
  G_NOOP_CHECK_DONE
  G_PRE_ARCHIVE_APPROVAL
  G_SELF_CHECK_DONE

DOMAIN.OUTPUTS:
  README
  SUMMARY
  VALIDATION_REPORT
```

```

CONSISTENCY_CHECK_REPORT
HISTORY_EVENT_PASSPORT
JIRA_TASK
IMPLEMENTATION_PROMPT
QA_PACKAGE
PROCESS_IMPROVEMENT_FEEDBACK
QA_AUTOMATION_SPEC
QA_AUTOMATION_README

```

Це вже не “довгий prompt”. Це предметна система правил.

Приклад: Monitoring Event Domain Pack

Інший домен матиме інший пакет.

```

DOMAIN.DEF:
  Monitoring Event business and technical package generation

DOMAIN.VOCABULARY:
  monitoring event
  business passport
  registry row
  Monitoring Center config
  translation tree
  sandbox evaluation
  REST runbook
  notification payload

DOMAIN.PATHS:
  business passport creation
  technical config package
  sandbox QA
  REST execution runbook
  registry row after Confluence URL

DOMAIN.GATES:
  G_BUSINESS_CONFIRMATION
  G_CONFIG_VALUES_CONFIRMED
  G_SANDBOX_EVALUATION_READY
  G_REST_RUNBOOK_SELF_CONTAINED
  G_CONFLUENCE_URL_REQUIRED_BEFORE_REGISTRY_ROW

```

Цей Domain Pack може використовувати ті самі Core і Profiles, але предметні rules будуть іншими.

Зв'язок Domain Pack із Core

Domain Pack не має перевизначати Core.



Погано:

```
Domain Pack змінює значення NODE або GATE.
```

Добре:

```
Domain Pack використовує NODE і GATE для своїх предметних правил.
```

Core має бути стабільним.

Domain Pack має бути розширенням, а не переписуванням базової мови.

Зв'язок Domain Pack із Profiles

Domain Pack може вимагати певні Profiles.

Наприклад:

```
domain:
  id: "history_event"

requires_profiles:
  - "ordo.profile.guided_intake"
  - "ordo.profile.documentation"
  - "ordo.profile.qa"
  - "ordo.profile.approval"
  - "ordo.profile.debug_test_improvement"
```

Це означає:

Для цього домену недостатньо просто Core.
Тут обов'язково потрібні guided intake, documentation, QA, погодження і debug/test/improvement.

Profile дає загальні механізми.

Domain Pack наповнює їх предметним змістом.

Зв'язок Domain Pack із Libraries

Domain Pack може використовувати libraries.

Наприклад:

```
include:
  - library: "ordo.validation.noop_checks"
    version: "0.1"
    as: "noop"

  - library: "ordo.qa.manual_runbook"
    version: "0.1"
    as: "manual_qa"
```

Але Domain Pack не є просто бібліотекою.

Різниця така:

Library – reusable готове рішення.
Domain Pack – предметна система правил для конкретної області.

Domain Pack може включати багато libraries, але залишається власником domain semantics.

Версіонування Domain Pack

Domain Pack обов'язково має мати версію.

```
domain:
  id: "history_event"
  version: "0.10"
```

Це потрібно тому, що зміна domain rules може змінити поведінку моделі.

Наприклад, якщо у версії 0.10 Path A4 трактував ExternalHistoryEvent одним способом, а у версії 0.11 правила уточнили, старі тести можуть поводитись інакше.

Тому важливо знати:

- яка версія Domain Pack використана;
- які tests пройдені;
- які compatibility checks виконані;
- які breaking changes внесені;
- чи оновлений changelog.

Domain Pack і компілятор

Компілятор Ordo має вміти перевіряти Domain Pack.

Мінімальні перевірки:

- DOMAIN.DEF існує;
- vocabulary не має критичних прогалин;
- paths мають умови вибору;
- blocking gates мають enforcement;
- outputs мають required/optional статус;
- tests покривають основні paths;
- FREEFORM blocks мають binding;
- Domain Pack не перевизначає Core без явного дозволу;
- потрібні Profiles підключені;
- libraries мають зафіксовані versions.

Для складних Domain Packs компілятор має також генерувати:

- coverage report;
- conflict report;
- unresolved ambiguity report;
- compatibility report;
- improvement backlog.

Domain Pack і debug

Debug mode має показувати, які саме частини Domain Pack були використані.

Наприклад:

```
trace_source: "model_self_report"
knowledge_trace:
  - source: "history_event_domain_pack"
    version: "0.10"
    section: "DOMAIN.PATHS.A1"
    used_for: "path selection"

  - source: "history_event_domain_pack"
    version: "0.10"
    section: "DOMAIN.GATES.G_NOOP_CHECK_DONE"
    used_for: "gate evaluation"
```

Без цього користувач бачить тільки відповідь.

З цим користувач бачить, яка domain rule спричинила рішення.

Domain Pack і improvement loop

Коли користувач вказує на проблему, improvement record має прив'язувати її до Domain Pack.

Наприклад:

```
improvement_record:
  classification:
    type: "missing_domain_gate"
    severity: "high"

  affected_unit:
    kind: "domain_pack"
    id: "history_event"
    version: "0.10"
    section: "DOMAIN.GATES"

  proposed_patch:
    - "add blocking gate G_MANUAL_QA_INSTRUCTIONS_ARE_ACTIONABLE"
    - "add regression test TC_MANUAL_QA_NOT_TOO_GENERIC"
```

Це дуже важливо: проблема не губиться в чаті, а стає зміною до конкретної частини domain logic.

Типові помилки

Помилка 1. Робити Domain Pack суцільним текстом

Якщо Domain Pack — це просто 100 сторінок markdown, модель знову буде працювати з ним як із довгим prompt-ом.

Потрібна структура:

```
vocabulary
objects
paths
rules
gates
outputs
tests
freeform
```

Помилка 2. Класти загальні правила в Domain Pack

Якщо правило стосується будь-якої документації, воно має бути в Documentation Profile або library, а не в конкретному domain pack.

Наприклад:

```
Rendered artifact must be validated, not only template.
```

Це загальне правило. Його краще винести в Profile або library.

Помилка 3. Не відокремлювати examples від rules

Приклади можуть бути у FREEFORM, але вони не мають ставати правилами без явного DOMAIN.RULE.

Помилка 4. Не мати tests

Domain Pack без тестів — це нестабільна інструкція.

Він може працювати сьогодні і зламатися після найменшої зміни.

Помилка 5. Не фіксувати версію

Якщо Domain Pack змінюється, але версія не змінюється, неможливо зрозуміти, чому стара поведінка більше не відтворюється.

Міні-вправа

Візьміть будь-який процес, який ви добре знаєте.

Наприклад:

- створення історичної події;
- підготовка моніторингової події;
- перевірка договору;
- аналіз ризиків компанії;
- створення Jira-задачі;
- підготовка QA-інструкцій.

Спробуйте описати для нього Domain Pack:

1. Яка назва домену?
2. Які основні терміни потрібно визначити?
3. Які є головні об'єкти?
4. Які є paths?
5. Які правила є blocking?
6. Які gates обов'язкові?
7. Які outputs створюються?
8. Які частини можна залишити у FREEFORM?
9. Які tests мають бути мінімально?
10. Які problems/improvements треба збирати після реального використання?

Це не обов'язково має бути ідеально з першої спроби.

Головне — почати відрізняти domain logic від загальних інструкцій.

Короткий підсумок

Domain Pack — це пакет предметної логіки для конкретної області.

Core задає базову мову.

Profiles задають режими роботи.

Libraries дають reusable готові рішення.

Domain Packs описують конкретну предметну область: vocabulary, objects, paths, rules, gates, outputs, tests і controlled FREEFORM.

Domain Pack потрібен, щоб модель не вигадувала domain logic сама, а виконувала узгоджені правила.

Добрий Domain Pack має бути структурованим, версіонованим, тестованим і видимим для debug/improvement layer.

У складних Ordo-процесах саме Domain Pack часто стає головним місцем, де живе реальна бізнесова логіка.

Розділ 25. Бібліотеки Ordo: include, import і повторне використання готових рішень

Навіщо це потрібно

Коли Ordo-програма стає більшою за кілька простих кроків, у ній дуже швидко з'являються повтори. Одні й ті самі перевірки контракту, однакові gate-и перед фінальним результатом, схожі правила для approval, однакові вимоги до trace, подібні шаблони output, повторювані правила для тестів, regression і coverage.

У звичайних мовах програмування для цього давно існує поняття бібліотеки. Програміст не пише кожного разу все з нуля. Він підключає готовий модуль, функцію, пакет або framework. Наприклад, якщо потрібно працювати з датами, HTTP-запитами, JSON або тестами, розробник бере готову бібліотеку і використовує її у своєму коді.

В Ordo потрібен аналогічний механізм, але адаптований не до класичного коду, а до керування поведінкою AI-моделі. Ordo-бібліотека — це не просто файл із функціями. Це готовий пакет Ordo-конструкцій, які можна підключити до поточної Ordo-програми: gate-и, node-и, status semantics, output templates, approval chains, test patterns, debug rules, domain vocabulary, reusable flows.

Без бібліотек кожен великий playbook починає перетворюватися на ізольований all-in-one документ. У ньому накопичуються власні копії правил, власні формулювання, власні винятки й власні варіанти тих самих перевірок. Через деякий час такі документи стають важкими для підтримки: якщо потрібно змінити одне базове правило, його доводиться шукати в десятках місць.

Бібліотеки дозволяють перенести повторювані рішення в окремі пакети повторного використання і підключати їх явно.



Просте пояснення

У найпростішому вигляді Ordo Library — це готовий набір правил і конструкцій, який можна сказати Ordo-програмі використати.

Наприклад:

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
  as: "contract_first"
```

Це означає: підключити бібліотеку ordo.validation.contract_first, використати саме версію 0.1 і дати їй локальне ім'я contract_first.

Після цього Ordo-програма може використовувати її частини:

```
use:
- contract_first.gates
- contract_first.assertions
- contract_first.approval_rules
```

Тобто замість того, щоб у кожному playbook-у заново описувати базові правила contract-first execution, ми підключаємо готову бібліотеку і використовуємо її exports.

Чим бібліотека відрізняється від Core, Profile і Domain Pack

Важливо не змішувати бібліотеки з іншими частинами Ordo.

Ordo Core — це базова мова. Він визначає фундаментальні конструкції: intent, contract, state, node, gate, output, status, trace, assertion.

Ordo Profile — це спеціалізований режим або стиль використання Ordo. Наприклад, profile для документації, QA, approval, artifact generation, rendered validation.

Domain Pack — це пакет знань і правил для конкретної предметної області. Наприклад, History Event Domain Pack або Monitoring Event Domain Pack.

Ordo Library — це пакет повторного використання, який може містити частини Core-логіку, profile bindings, предметні блоки або патерни виконання, але не обов'язково є повним domain pack-ом чи profile.

Бібліотека може бути маленькою. Наприклад, тільки набір стандартних gate-ів для pre-final validation.

Бібліотека може бути середньою. Наприклад, готовий pattern для guided intake.

Бібліотека може бути великою. Наприклад, повний пакет повторного використання для manual QA runbook generation.

Ключова відмінність бібліотеки — її призначення для повторного використання.

Що може містити Ordo Library

Ordo-бібліотека може містити:

```
- готові NODE.DEF;
- готові GATE.DEF;
- готові ASSERT.NOT;
- готові STATUS.SEMANTICS;
- готові TEMPLATE.BIND;
- готові OUTPUT.DEF;
- готові RENDER.VALIDATE rules;
- готові EVIDENCE.MATRIX schemas;
- готові APPROVAL.REQUIRE chains;
- готові DEBUG.MODE profiles;
- готові TEST.DEF templates;
- готові REGRESSION.SUITE patterns;
- готові FREEFORM.COVERAGE rules;
- готові патерни виконання;
- готові document templates;
- готові domain-specific rule sets.
```

Тобто бібліотека в Ordo — це пакет поведінки, а не просто пакет тексту.

Базові мовні конструкції

Для бібліотек Ordo потрібні такі конструкції:

```
LIB.DEF
INCLUDE
IMPORT
USE
EXPORT
NAMESPACE
ALIAS
VERSION.REQUIRE
COMPAT.CHECK
CONFLICT.DETECT
CONFLICT.RESOLVE
OVERRIDE.ALLOW
OVERRIDE.DENY
TRUST.LEVEL
```

Це мінімальний набір, без якого бібліотеки швидко стануть небезпечними.

LIB.DEF

LIB.DEF описує саму бібліотеку.

Наприклад:

```
library:
  id: "ordo.validation.contract_first"
  version: "0.1"
  name: "Contract-first validation library"

exports:
  gates:
    - "G_CONTRACT_CONFIRMED"
    - "G_NO_FINAL_OUTPUT_WITHOUT_APPROVAL"

  assertions:
    - "ASSERT_NOT_FINAL_BEFORE_CONTRACT"

requires:
  ordo_version: ">=0.11"
```

Цей опис дає Ordo-компілятору зрозуміти, що саме містить бібліотека, що вона експортує і з якою версією Ordo сумісна.

INCLUDE

INCLUDE означає фізичне або логічне підключення бібліотеки до Ordo-програми.

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
    as: "contract_first"
```

Правило Ordo: бібліотеки не повинні підключатися неявно. Якщо playbook використовує бібліотеку, це має бути видно в source і compiled IR.

Погано:

```
модель сама здогадалась застосувати правила contract-first
```

Добре:

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
```

IMPORT і USE

IMPORT і USE можуть мати різну семантику.

IMPORT означає: зробити певні exports бібліотеки доступними в поточній Ordo-програмі.

USE означає: реально застосувати підключені конструкції в конкретному місці виконання.

Наприклад:

```
import:
- from: "contract_first"
  items:
    - "G_CONTRACT_CONFIRMED"
    - "ASSERT_NOT_FINAL_BEFORE_CONTRACT"

use:
- gate: "contract_first.G_CONTRACT_CONFIRMED"
  at: "before_output_generation"
```

Це важливе розділення. Бібліотека може бути підключена, але не всі її частини обов'язково використовуються.



EXPORT

EXPORT визначає, що бібліотека дозволяє використовувати зовні.

Наприклад:

```
exports:
  gates:
    - id: "G_CONTRACT_CONFIRMED"
      visibility: "public"

  internal_rules:
    - id: "R_CONTRACT_PARSE_HELPER"
      visibility: "private"
```

Не все всередині бібліотеки має бути доступним. Частина правил може бути внутрішньою реалізацією самої бібліотеки.

NAMESPACE і ALIAS

Namespace потрібен, щоб уникати конфліктів назв.

Без namespace дві бібліотеки можуть мати gate з однаковою назвою:

```
G_APPROVAL_REQUIRED
```

Але в різних бібліотеках це можуть бути різні правила.

Тому правильне посилання має бути повним:

```
contract_first.G_APPROVAL_REQUIRED
artifact_validation.G_APPROVAL_REQUIRED
```

Alias робить довгі назви коротшими:

```
include:
- library: "ordo.artifact.render_validation"
  version: "0.1"
  as: "render_validation"
```

Після цього можна писати:

```
use:
- render_validation.G_RENDERED_ARTIFACT_CHECK
```

Version pinning

Бібліотеки мають підключатися з версією.

Погано:

```
include:
- library: "ordo.qa.manual_runbook"
```

Добре:

```
include:
- library: "ordo.qa.manual_runbook"
  version: "0.1"
```

Причина проста: якщо бібліотека зміниться, playbook може почати поводитися інакше. Для Ordo це критично, бо ми керуємо не просто текстом, а поведінкою моделі.

Версія має бути частиною execution contract.

Compatibility check

Компілятор Ordo має перевіряти сумісність бібліотеки з поточною мовою, profile, domain pack і runtime.

Наприклад:

```
compatibility:
  requires_ordo: ">=0.11"
  requires_profiles:
    - "documentation_runtime"
    - "debug_test_improvement"
  incompatible_with:
    - "legacy_all_in_one_mode"
```

У compiled IR це може бути окремий оп:

```
{
  "op": "LIB.COMPAT.CHECK",
  "library": "ordo.qa.manual_runbook",
  "version": "0.1",
  "requires_ordo": ">=0.11"
}
```

Якщо compatibility check не пройдений, Ordo не повинна мовчки продовжувати виконання.

Conflict detection

Конфлікти між бібліотеками мають бути явними.

Наприклад, одна бібліотека каже:

```
ready_for_first_run = можна запускати
```

А інша каже:

```
ready_for_first_run = потребує ручного підтвердження
```

Ordo не має сама вибирати, хто правий.

Має бути:

```
conflict:
  type: "status_semantics_conflict"
  key: "ready_for_first_run"
  sources:
    - "library_a"
    - "library_b"
  resolution: "required"
```

Після цього можливі варіанти:

- запитати людину;
- застосувати явно заданий priority;
- заблокувати compilation;
- дозволити override тільки з reason.

Override rules

Override — одна з найнебезпечніших частин бібліотек.

Якщо підключена бібліотека може непомітно переписати gate, status або assertion, вся надійність Ordo руйнується.

Тому правило таке:

```
Будь-який override має бути явним.
```

Наприклад:

```
override:
  allow:
    - target: "contract_first.G_NO_FINAL_OUTPUT_WITHOUT_APPROVAL"
      by: "history_event.G_PRE_ARCHIVE_APPROVAL"
      reason: "domain pack has stricter equivalent gate"
```

Без такого запису override має бути заборонений.



Trust level

Не всі бібліотеки однаково надійні. Тому Ordo має підтримувати trust level.

Наприклад:

```
library:
  id: "ordo.validation.contract_first"
  version: "0.1"
  trust_level: "official"
```

Можливі рівні:

```
official
verified
project_local
experimental
untrusted
```

У production workflows бажано забороняти experimental або untrusted бібліотеки без окремого approval.

Бібліотеки і debug/test/improvement layer

Бібліотеки мають бути видимими для debug, testing і improvement.

Якщо gate прийшов із бібліотеки, trace має це показати:

```
trace_source: "model_self_report"
gate_report:
  - gate_id: "contract_first.G_CONTRACT_CONFIRMED"
    source:
      kind: "library"
      id: "ordo.validation.contract_first"
      version: "0.1"
    status: "passed"
```

Якщо користувач вказує проблему, improvement record має вміти прив'язати її до бібліотеки:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

Це дозволяє покращувати не тільки конкретний playbook, а й reusable рішення.

Бібліотеки і FREEFORM

Бібліотека може містити FREEFORM, але це має бути контрольований FREEFORM.

Погано:

```
freeform:
  text: "тут багато правил, модель сама розбереться"
```

Добре:

```
freeform:
  id: "FF_CONTRACT_EDGE_CASES"
  binding:
    used_by:
      - "G_CONTRACT_CONFIRMED"
    reason: "domain examples are too nuanced for full formalization"
  coverage:
    required: true
```

Якщо бібліотечний FREEFORM часто спричиняє помилки, improvement loop має запропонувати формалізувати його частину.

Типи бібліотек

Практично Ordo-бібліотеки можна поділити на кілька типів.

Core utility libraries

Містять універсальні дрібні конструкції: стандартні assertions, common gates, status helpers.

Profile libraries

Містять готові блоки для документації, QA, approval, validation, rendered artifact checks.

Domain libraries

Містять reusable частини для конкретної сфери: History Event, Monitoring Event, Legal Review.

Pattern libraries

Містять патерни виконання: guided intake, contract-first flow, pre-archive approval, self-check before handoff.

Template libraries

Містять шаблони документів, структури output, package layouts.

Connector/tool libraries

Містять правила роботи із зовнішніми інструментами, API, файлами, runner-ами.

Малий приклад

Уявімо Ordo-програму, яка має створити аналітичний пакет.

Без бібліотек вона може містити десятки локальних правил:

- спочатку підтвердити contract;
- не створювати фінальний пакет до approval;
- перевірити rendered artifacts;
- сформувавши validation report;
- сформувавши consistency report;
- зафіксувати improvement feedback;
- виконати regression tests.

З бібліотеками це можна оформити так:

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
  as: "contract_first"

- library: "ordo.artifact.render_validation"
  version: "0.1"
  as: "render_validation"
```

```

- library: "ordo.debug_test.basic_regression"
  version: "0.1"
  as: "regression"

use:
- contract_first.required_contract_gates
- render_validation.pre_handoff_checks
- regression.minimum_suite

```

Це коротше, чистіше і безпечніше, якщо кожна бібліотека має версію, namespace, compatibility check і tests.

Типові помилки

Перша помилка — вважати бібліотеку просто шматком тексту. У Ordo бібліотека має бути structured package, який компілятор може перевірити.

Друга помилка — підключати бібліотеки без версій. Це робить поведінку нестабільною.

Третя помилка — дозволяти implicit imports. Якщо модель сама “здогадалася” використати бібліотеку, це не Ordo execution.

Четверта помилка — не фіксувати conflicts. Конфліктні status semantics або gates не можна вирішувати мовчки.

П’ята помилка — дозволяти hidden override. Бібліотека не повинна непомітно змінювати поведінку основного playbook-a.

Шоста помилка — не тестувати бібліотеки. Якщо пакет повторного використання не має tests і coverage, він просто переносить помилки з одного playbook-a в багато playbook-ів.

Міні-вправа

Візьміть будь-який великий prompt або playbook і знайдіть у ньому повторювані частини.

Спробуйте виписати:

- які gate-и повторюються;
- які шаблони output повторюються;
- які правила approval повторюються;
- які checks можна винести в reusable library;
- які частини мають бути exported;
- які частини мають залишитися private;
- які версії і compatibility rules потрібні.

Після цього спробуйте назвати одну бібліотеку, яка могла б існувати окремо.

Наприклад:

```

ordo.validation.contract_first
ordo.artifact.pre_handoff_validation
ordo.qa.manual_runbook
ordo.debug_test.basic_regression

```

Короткий підсумок

Бібліотеки Ordo потрібні для того, щоб не переписувати однакові правила в кожному playbook-у.

Ordo Library — це пакет повторного використання Ordo-конструкцій: gate-ів, node-ів, assertions, status semantics, templates, tests, debug rules, domain rules або патерни виконання.

Бібліотеки мають підключатися явно через include/import/use, мати namespace, alias, version pinning, compatibility checks, conflict detection, explicit override rules і trust level.

Добре спроектовані бібліотеки роблять Ordo-програми коротшими, стабільнішими і зручнішими для розвитку.

Погано спроектовані бібліотеки створюють приховані залежності, конфлікти і неконтрольовані зміни поведінки моделі.

Головне правило:

У Ordo бібліотека — це не прихований текстовий фрагмент, а явний, версіонований і перевірюваний пакет поведінки.

Розділ 26. Чому all-in-one більше не головний формат

Навіщо це потрібно

У багатьох командах складні інструкції для AI-моделі починаються дуже просто: є один великий документ, у якому написано все. Там є правила, приклади, сценарії, винятки, шаблони, заборони, пояснення, таблиці, чеклисти, нотатки, історія змін і підказки для автора.

На початку це зручно. Один файл легко прикріпити до чату. Його легко відкрити. У ньому нібито “все є”. Саме тому формат all-in-one часто стає першим природним форматом для playbook-a.

Але коли playbook росте, цей формат починає ламатися.

Проблема не в тому, що великий документ поганий сам по собі. Проблема в тому, що великий документ погано підходить як **головний виконуваний формат** для складного процесу.

Ordo змінює роль all-in-one. Він може залишатися зручною зведеною версією для читання, але не має бути єдиним source of truth для виконання.

Просте пояснення

Уявімо, що у вас є інструкція на 200 сторінок. У ній описано, як модель має створювати аналітичний пакет.

Там є:

- правила вибору сценарію;
- список питань користувачу;
- заборони;
- формати файлів;
- правила тестування;
- приклади;
- edge cases;
- шаблони;
- старі примітки;
- уточнення після помилок;
- винятки;
- чеклисти перед фінальним архівом.

Коли модель отримує такий документ як суцільний текст, вона не завжди розуміє, що є процесом, що є прикладом, що є заборобою, що є шаблоном, а що є історичною нотаткою.

У великому all-in-one дуже легко змішуються різні рівні:

правило + приклад + пояснення + виняток + шаблон + коментар автора

Для людини це ще можна читати. Для виконання моделлю це вже небезпечно.

Ordo пропонує іншу модель:

source documents → selected sections → Ordo Source → compiled IR → execution trace → rendered artifacts

Тобто великий документ більше не є “тим, що модель просто читає зверху вниз”. Він стає одним із джерел, з якого формуються структуровані execution units.

Чому all-in-one був корисним

Важливо не відкидати all-in-one повністю. Він має свої переваги.

Він корисний для:

- першого збору знань;
- швидкого handoff між людьми;
- оглядового читання;
- збереження повної картини;
- архівної версії;
- експорту в один файл;
- швидкого прикріплення до моделі, якщо немає кращого runtime.

На ранній стадії all-in-one часто є найпростішим способом зібрати хаотичні знання в одне місце.

Але те, що зручно для збору знань, не завжди зручно для керованого виконання.

Де all-in-one починає ламатися

Перший тип проблеми — **пошук потрібного правила**.

У великому документі правило може бути написано в одному місці, уточнене в іншому, частково скасоване в третьому і проілюстроване прикладом у четвертому.

Модель може взяти не останнє правило, а найближче за текстом. Або може поєднати старе й нове формулювання.

Другий тип проблеми — **плутанина між правилом і прикладом**.

Наприклад, у документі є приклад події LU_CHANGE_STATUS. Модель може випадково сприйняти цей приклад як обов’язковий alias для нового кейсу.

Третій тип проблеми — **пропуск контрольних точок**.

У all-in-one може бути написано:

```
перед фінальним архівом обов’язково виконати self-check
```

Але якщо це просто фраза в довгому тексті, модель може її пропустити. В Ordo це має бути не фраза, а blocking gate.

Четвертий тип проблеми — **складність оновлення**.

Коли користувач каже “це потрібно поправити в інструкції”, незрозуміло, куди саме вносити зміну. У правило? У приклад? У чеклист? У QA? У шаблон? У domain pack? У library?

П’ятий тип проблеми — **неможливість нормального тестування**.

Суцільний markdown-документ важко тестувати. Можна перевірити, чи модель приблизно відповідає правильно, але важко перевірити:

- який path вона обрала;
- який node був активний;
- який gate спрацював;
- який state змінився;
- який FREEFORM блок вплинув на рішення;
- чи не зламався старий сценарій після зміни правила.

Ordo-підхід

В Ordo головним стає не великий документ, а структурована execution model.

all-in-one може існувати як rendered artifact, але source of truth для виконання має бути розбитий на керовані частини.

Типова структура може виглядати так:

```
00_EXECUTION_CONTRACT.md
01_CORE_BINDING.md
02_PROFILE_BINDING.md
03_DOMAIN_PACK.md
04_DECISION_TREE.ordo.yaml
05_OUTPUTS.ordo.yaml
06_GATES.ordo.yaml
07_TESTS.ordo.yaml
08_FREEFORM_LEDGER.md
09_COMPILED_IR.json
10_VALIDATION_REPORT.json
```

Або простіше:

```
source_docs/
ordo_source/
compiled_ir/
tests/
rendered/
reports/
```

Головна ідея:

Людина може читати зведений документ, але модель має виконувати структуровану карту процесу.

All-in-one як rendered artifact

У новій моделі all-in-one не зникає. Він просто змінює статус.

Раніше він був:

```
main instruction source
```

В Ordo він стає:

```
rendered human-readable artifact
```

Тобто він може генеруватися з модульних джерел як зручна зведена версія.

Це схоже на технічну документацію: розробники можуть підтримувати багато окремих файлів, а потім збирати один PDF або один HTML-документ для читання.

Ordo має знати, що all-in-one — це не обов'язково первинне джерело. Це може бути результат збірки.

Чому це важливо для traceability

Коли playbook розбитий на секції, кожне рішення можна прив'язати до конкретного джерела:

```
trace_source: "model_self_report"
knowledge_trace:
  - source: "04_DECISION_TREE.ordo.yaml"
    node: "NODE_SELECT_PATH"
    used_for: "path selection"

  - source: "06_GATES.ordo.yaml"
    gate: "G_PRE_ARCHIVE_APPROVAL"
    used_for: "blocking final archive"
```

У великому all-in-one така прив'язка теж можлива, але вона менш точна. Наприклад:

рядки 1420-1480 великого документа

Це краще, ніж нічого, але гірше, ніж посилання на конкретний GATE.DEF, NODE.DEF або TEST.DEF.

Що має робити Documentation Runtime

Коли ми кажемо, що all-in-one більше не головний формат, це автоматично веде до наступної ідеї: потрібен Documentation Runtime.

Це буде тема наступного розділу, але тут важливо зафіксувати базову думку.

Documentation Runtime має вміти:

- бачити каталог документів;
- знати, які секції для чого потрібні;
- вибирати тільки релевантні docs для поточного node;
- збирати rendered artifacts;
- перевіряти, що зведений all-in-one не суперечить source files;
- показувати, з якого джерела взяте кожне правило.

Тобто замість “прикріпили один величезний файл і сподіваємося, що модель усе правильно прочитає” з'являється керований режим роботи з документацією.

Ordo-конструкція

У Ordo можна зафіксувати це через такі конструкції:

```
DOC.CATALOG
DOC.SPLIT
DOC.SELECT
DOC.RENDER
DOC.SOURCE_OF_TRUTH
RENDER.VALIDATE
TRACE.SOURCE
```

Приклад:

```
doc_catalog:
  id: "history_event_playbook_docs"

  source_of_truth:
    mode: "split_docs"
    all_in_one_role: "rendered_artifact"

  documents:
```

```

- id: "execution_contract"
  path: "00_EXECUTION_CONTRACT.md"
  role: "contract"

- id: "decision_tree"
  path: "04_DECISION_TREE.ordo.yaml"
  role: "path_selection"

- id: "gates"
  path: "06_GATES.ordo.yaml"
  role: "gate_definitions"

- id: "tests"
  path: "07_TESTS.ordo.yaml"
  role: "test_suite"

```

А для поточного node:

```

doc_select:
  node: "NODE_SELECT_PATH"
  required_docs:
    - "execution_contract"
    - "decision_tree"
  optional_docs:
    - "domain_examples"

```

Типова помилка

Погана практика:

У нас є один великий markdown-файл. Нехай модель сама знайде потрібне.

Краща практика:

У нас є каталог документів. Для кожного етапу виконання відомо, які секції потрібні, які gates активні, які outputs дозволені, які tests перевіряють поведінку.

Ще одна помилка — вважати, що split docs автоматично вирішують проблему.

Розбиття документа саме по собі не дає Ordo. Можна мати 50 маленьких файлів і все одно хаос.

Потрібні ролі, зв'язки, source-of-truth rules і validation.

Міні-вправа

Візьміть будь-яку велику інструкцію, яку ви використовуєте для AI-моделі.

Спробуйте розділити її не за розміром, а за ролями:

```

- де описано contract;
- де описано decision tree;
- де описано state;
- де описано gates;
- де описано output;
- де є examples;
- де є controlled FREEFORM;
- де мають бути tests;
- що може бути rendered all-in-one;
- що має бути source of truth.

```

Після цього поставте головне питання:

Чи зможе модель виконувати цей процес без читання всього документа одразу?

Якщо відповідь “ні”, значить playbook ще не готовий до Ordo execution.

Короткий підсумок

all-in-one корисний як людський оглядовий або архівний формат, але він погано працює як головний виконуваний формат для складних AI-процесів.

У Ordo source of truth має бути структурованим: contract, nodes, gates, outputs, tests, FREEFORM ledger, libraries і domain rules мають бути виділені в керовані частини.

Зведений all-in-one може залишатися, але як rendered artifact, який генерується з модульних джерел і проходить validation.

Головне правило:

У Ordo великий документ можна читати, але виконувати потрібно структуровану модель процесу.

Розділ 27. Documentation Runtime

Навіщо це потрібно

У попередньому розділі ми розібрали, чому формат all-in-one перестає бути головним, коли інструкція виростає до рівня великого playbook-a. Один великий файл зручно передати моделі, але незручно підтримувати, перевіряти, оновлювати й дебажити.

Проте одразу виникає нове питання: якщо ми розбили документацію на десятки окремих файлів, як модель має зрозуміти, які саме з них потрібні для поточного кроку?

Людина може відкрити зміст, переглянути назви, згадати контекст і вибрати потрібний розділ. Модель також може це зробити, але якщо залишити це тільки на її здогадку, ми знову повертаємося до проблеми звичайного prompt-a: вона може відкрити не той файл, пропустити важливу секцію або використати застарілу частину документа.

Саме для цього в Ordo потрібен Documentation Runtime.

Documentation Runtime — це мовний механізм, який керує тим, як Ordo-програма працює з великим набором документів: які документи існують, коли їх треба читати, які секції є source of truth, які документи допоміжні, а які не можна використовувати для прийняття рішень.

Інакше кажучи, це не просто “папка з інструкціями”. Це керований шар доступу до знань.

Просте пояснення

Уявімо, що Ordo-програма — це кухар, а документація — це велика кулінарна бібліотека.

Якщо кухарю просто дати всю бібліотеку і сказати “готуй”, він може відкрити старий рецепт, переплутати десерт із основною стравою або використати примітку як обов’язкове правило.

Documentation Runtime працює як каталог і диспетчер:

```
для цього кроку потрібні тільки ці документи;
цей документ є головним;
цей документ лише пояснює приклади;
цей файл застарів;
цей шаблон можна використовувати тільки після approval gate;
цей rendered artifact треба перевірити окремо.
```

Тобто Documentation Runtime не пише бізнесову логіку замість Domain Pack. Він відповідає за те, щоб модель брала правильні інструкції в правильний момент.

Чим Documentation Runtime відрізняється від звичайної документації

Звичайна документація описує знання.

Documentation Runtime керує використанням знань.

Звичайна документація може сказати:

У файлі 05_QA_PACKAGE описані правила QA.

Documentation Runtime має сказати точніше:

NODE package_generation може читати 05_QA_PACKAGE_TEMPLATE.
 Перед створенням QA-файлу потрібно перевірити G_QA_SCOPE_CONFIRMED.
 Фінальний rendered QA-файл має пройти RENDER.VALIDATE.
 Якщо fixture або source lookup не підтверджені, QA-файл не можна позначати як ready.

Це вже не просто опис. Це runtime-поведінка.

Основні задачі Documentation Runtime

У Ordo Documentation Runtime має вирішувати кілька задач.

Перша задача — каталогізація документів.

Ordo має знати, які документи входять у пакет:

```
README.md
execution contract
core binding
profile binding
domain pack
compiled IR
freeform ledger
validation report
source virtual docs
templates
examples
```

Друга задача — вибір потрібних документів для поточного вузла.

Модель не повинна щоразу перерахувати все. Для кожного NODE можна визначити набір документів, які дозволені або обов'язкові.

Третя задача — визначення source of truth.

Якщо є split-файли і all-in-one версія, потрібно явно знати, що редагувати треба split-файли, а all-in-one є зібраним артефактом. Інакше можна внести зміну не туди.

Четверта задача — контроль rendered artifacts.

Шаблон ще не означає, що фінальний файл правильний. Ordo має вміти відрізнити template від реально згенерованого документа.

П'ята задача — traceability.

Коли модель створила результат, має бути видно, на які документи вона спиралася.

Ordo-конструкції

Для Documentation Runtime у мові потрібні такі конструкції:

```
DOC.CATALOG
DOC.DEF
DOC.ROLE
DOC.SOURCE_OF_TRUTH
DOC.SELECT
DOC.BIND
DOC.READ
DOC.RENDER
```

```
DOC.SPLIT
DOC.MERGE
DOC.VERSION
DOC.DEPRECATE
DOC.TRACE
RENDER.VALIDATE
```

Розглянемо їх простими словами.

DOC.CATALOG описує список документів у пакеті.

DOC.DEF описує конкретний документ.

DOC.ROLE вказує роль документа: contract, template, example, source, generated artifact, validation report.

DOC.SOURCE_OF_TRUTH визначає, який файл є головним для зміни певного правила.

DOC.SELECT каже, які документи потрібні для поточного вузла.

DOC.BIND прив'язує документ до node, gate, output або library.

DOC.TRACE фіксує, які документи були реально використані під час run.

RENDER.VALIDATE перевіряє не шаблон, а вже згенерований фінальний артефакт.

Малий приклад

Припустимо, у нас є Ordo-програма для створення аналітичного пакета.

У Source-форматі Documentation Runtime може виглядати так:

```
doc_catalog:
  id: "history_event_docs"

  documents:
    - id: "execution_contract"
      path: "00_ORDO_EXECUTION_CONTRACT.md"
      role: "contract"
      source_of_truth: true

    - id: "domain_pack"
      path: "04_HISTORY_EVENT_DOMAIN_PACK.md"
      role: "domain_rules"
      source_of_truth: true

    - id: "qa_template"
      path: "05_QA_PACKAGE_TEMPLATE.md"
      role: "template"

    - id: "qa_output"
      path: "05_QA_PACKAGE_<ALIAS>.md"
      role: "rendered_artifact"
      generated: true

doc_select:
  node: "generate_qa_package"
  required:
    - "execution_contract"
    - "domain_pack"
    - "qa_template"
  forbidden:
    - "deprecated_all_in_one"

render_validate:
```

```
output: "qa_output"
rules:
  - "must_not_contain_unresolved_placeholders"
  - "must_include_manual_test_table"
  - "must_match_confirmed_contract"
```

Тут важливо, що модель не просто “знає про документи”. Вона отримує правила, які документи брати, які не брати, і як перевіряти результат.

Documentation Runtime і all-in-one

Documentation Runtime не забороняє all-in-one. Він просто змінює його роль.

All-in-one може бути:

- зручним read-only snapshot;
- артефактом для передачі моделі;
- зібраним представленням split-документів;
- контрольним переглядом усього пакета.

Але він не повинен автоматично бути source of truth.

Якщо source of truth — секційні файли, Documentation Runtime має явно це сказати:

```
source_of_truth:
  rules:
    - target: "playbook_logic"
      document: "source_virtual_docs/"
    - target: "rendered_snapshot"
      document: "999_ALL_IN_ONE.md"
      editable: false
```

Це захищає від дуже типової помилки: модель редагує великий all-in-one файл, а реальні секційні документи залишаються старими.

Documentation Runtime і trace

Коли Ordo-програма працює з документами, trace має показувати:

```
які документи були вибрані;
чому саме вони;
які документи були відхилені;
які секції використані;
які outputs були згенеровані на їх основі;
чи пройшли rendered artifacts перевірку.
```

Наприклад:

```
trace_source: "model_self_report"
doc_trace:
  node: "generate_qa_package"

  selected:
    - id: "execution_contract"
      reason: "required for confirmed scope"
    - id: "domain_pack"
      reason: "required for domain rules"
    - id: "qa_template"
      reason: "required for output structure"

  rejected:
    - id: "deprecated_all_in_one"
      reason: "not source of truth for editing"

  rendered_outputs:
```

```
- id: "qa_output"
  validation: "passed"
```

Такий trace особливо важливий у debug mode. Якщо результат неправильний, ми можемо побачити, чи проблема в логіці, чи модель просто взяла не той документ.

Documentation Runtime і бібліотеки

Після появи Ordo Libraries Documentation Runtime стає ще важливішим.

Бібліотека може підключати власні документи:

```
- опис exported gates;
- шаблони;
- приклади;
- правила сумісності;
- changelog;
- tests;
- coverage report.
```

Ordo має знати, що ці документи належать не основному playbook-у, а бібліотеці.

Наприклад:

```
library_docs:
  library: "ordo.validation.contract_first"
  version: "0.1"

  documents:
    - id: "contract_gates"
      path: "docs/gates.md"
      role: "library_rule_reference"

    - id: "contract_tests"
      path: "tests/contract_first_tests.yaml"
      role: "test_suite"
```

Якщо проблема виникла через правило з бібліотеки, DOC.TRACE і IMPROVEMENT.RECORD мають показати саме це.

Documentation Runtime і FREEFORM

FREEFORM часто містить пояснення, приклади або винятки. Documentation Runtime має допомагати не втратити контроль над такими блоками.

Для кожного FREEFORM-блоку бажано знати:

```
у якому документі він лежить;
до якого node/gate/rule прив'язаний;
чи є він normative, example або note;
чи дозволено моделі приймати рішення на його основі;
чи є для нього coverage.
```

Наприклад:

```
freeform_doc_binding:
  freeform_id: "FF_EDGE_CASES"
  document: "04_HISTORY_EVENT_DOMAIN_PACK.md"
  section: "Edge cases"
  binding:
    node: "select_path"
    role: "controlled_explanation"
  decision_allowed: true
```


Без такого binding модель може сприйняти будь-який приклад як правило або будь-яку примітку як дозвіл на дію.

Типові помилки

Перша помилка — вважати, що список файлів уже є runtime-ом.

Список файлів — це тільки каталог. Runtime починається тоді, коли є правила вибору, ролі, gates і trace.

Друга помилка — не визначити source of truth.

Якщо є два документи з подібним змістом, модель має знати, який з них головний.

Третя помилка — перевіряти шаблон замість фінального документа.

Шаблон може бути правильним, але згенерований artifact може містити порожні секції, незамінені placeholders або дані не з підтвердженого contract.

Четверта помилка — дозволяти моделі самій вирішувати, які документи “мають релевантні”.

Для простого питання це нормально. Для production playbook-a — ні. Там має бути DOC.SELECT.

П’ята помилка — не трасувати використані документи.

Без DOC.TRACE після помилки буде незрозуміло, чи проблема в правилі, чи в неправильному документі.

Міні-вправа

Візьміть будь-який великий набір інструкцій, з яким ви працювали.

Спробуйте відповісти на питання:

1. Які документи входять у пакет?
2. Який документ є source of truth?
3. Які документи є лише прикладами?
4. Які документи є шаблонами?
5. Які документи є rendered artifacts?
6. Для якого кроку потрібен який документ?
7. Які документи не можна використовувати для прийняття рішень?
8. Як перевірити, що фінальний документ відповідає шаблону і contract?
9. Як у trace побачити, які документи були використані?

Якщо на ці питання немає чіткої відповіді, у вас ще немає Documentation Runtime. У вас є просто папка з документами.

Короткий підсумок

Documentation Runtime — це шар Ordo, який керує роботою з великим набором документів.

Він визначає:

- які документи існують;
- яку роль має кожен документ;
- який документ є source of truth;
- які документи потрібні для конкретного node;
- які документи заборонені або застарілі;
- як перевіряти rendered artifacts;
- як трасувати використання документації;
- як пов'язувати документи з libraries, FREEFORM і improvement records.

Головна ідея проста:

великий playbook має бути не просто прочитаним,
а виконаним через керований documentation runtime.

Без Documentation Runtime модель працює з документацією як із великою купою тексту.

З Documentation Runtime документація стає частиною виконуваної Ordo-програми.

Розділ 28. Rendered artifact validation

Навіщо це потрібно

У багатьох процесах модель створює не просто відповідь у чаті, а готовий артефакт: документ, архів, звіт, інструкцію, JSON, YAML, Markdown-файл, PDF, таблицю, пакет для Jira, QA-набір або набір файлів для передачі іншій команді.

На перший погляд здається, що достатньо перевірити шаблон. Якщо шаблон правильний, то й результат має бути правильний. Але в реальній роботі це часто не так.

Шаблон може бути правильним, а згенерований документ — ні.

Наприклад:

- у шаблоні є обов’язковий розділ, але в готовому файлі він порожній;
- у структурі пакета передбачено 11 файлів, але в архів потрапило 12;
- у README написано одне, а в QA-файлі — інше;
- у JSON вказаний один alias, а в Markdown — інший;
- у шаблоні є gate перед фінальним архівом, але сам архів уже створено до проходження gate;
- у rendered документі випадково залишився placeholder;
- у фінальному файлі з’явився зайвий технічний фрагмент, якого не мало бути.

Саме тому Ordo має перевіряти не тільки намір, не тільки source-програму і не тільки шаблон. Ordo має перевіряти готовий результат після рендерингу.



Це і є Rendered artifact validation.

Просте пояснення

Rendered artifact validation — це перевірка вже створеного артефакту, а не тільки правил, за якими він мав бути створений.

Можна уявити це так.

Шаблон — це креслення будинку.

Rendered artifact — це вже побудований будинок.

Перевірити креслення корисно, але цього недостатньо. Після будівництва потрібно перевірити сам будинок: чи є двері, чи сходи ведуть туди, куди треба, чи не забули дах, чи електрика підключена правильно, чи кімнати відповідають плану.

Так само і в Ordo:

```
template validation → перевіряє план
rendered artifact validation → перевіряє фактичний результат
```

Чому це окремий рівень Ordo

У звичайних prompt-процесах модель часто каже:

Я перевірила результат.

Але це не завжди означає, що була перевірена фактична структура готового артефакту.

Ordo має змусити процес бути точнішим:

1. Згенеруй артефакт.
2. Прочитай або проаналізуй саме згенерований артефакт.
3. Порівняй його з output contract.
4. Перевір обов'язкові розділи.
5. Перевір consistency між файлами.
6. Перевір заборонені елементи.
7. Сформуль validation report.
8. Тільки після цього дозволяй handoff.

Тобто rendered validation — це gate між створенням результату і передачею результату користувачу.

Основна конструкція

У Ordo цей механізм можна описати так:

```
RENDER.VALIDATE
```

Ця конструкція означає:

перевірити вже згенерований артефакт як фактичний output, а не тільки як очікувану структуру

У простому вигляді:

```
render_validate:
  target: "final_package"
  against:
    - "output_contract"
    - "file_manifest"
    - "mandatory_sections"
    - "consistency_rules"
    - "forbidden_content"
  report: "VALIDATION_REPORT.json"
  blocking: true
```

Ключове слово тут — blocking.



Якщо rendered validation не пройдено, результат не можна вважати готовим.

Що саме перевіряється

Rendered artifact validation може перевіряти різні рівні.

1. Наявність файлів

Наприклад, пакет має містити рівно такі файли:

```
README.md
SUMMARY.json
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json
01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
02_JIRA_TASK_<ALIAS>.md
04_IMPLEMENTATION_PROMPT_<ALIAS>.md
05_QA_PACKAGE_<ALIAS>.md
07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md
08_QA_AUTOMATION_SPEC_<ALIAS>.yaml
09_QA_AUTOMATION_README_<ALIAS>.md
```

Rendered validation має перевірити не лише те, що ці файли описані в інструкції, а що вони реально є у фінальному пакеті.

Також потрібно перевірити, що немає зайвих файлів, якщо пакет має бути compact canonical package.

2. Наявність обов'язкових розділів

Наприклад, Jira-задача має містити:

- загальний опис проблеми;
- очікувану поведінку;
- критерії прийняття;
- тестові сценарії;
- обмеження;
- залежності;
- що не входить у задачу.

Rendered validation має відкрити готовий Markdown-файл і перевірити, що ці розділи справді присутні.

3. Відсутність placeholder-ів

Це дуже практична перевірка.

Поганий результат:

```
<ALIAS>
<TODO>
<CONFIRM_SOURCE_FIELD>
[insert description here]
```

Якщо у фінальному артефакті залишився placeholder, це майже завжди означає, що результат не готовий.

Ordo може мати правило:

```
assert_not:
  rendered_artifact_contains:
    - "<TODO>"
    - "<ALIAS>"
    - "TBD"
    - "insert here"
```

4. Consistency між файлами

Це одна з найважливіших частин.

У складному пакеті одна й та сама інформація часто повторюється в різних місцях:

- alias;
- назва події;
- source field;
- status;
- expected output;
- test case id;
- route/path;
- список файлів;
- рівень змін;
- no-op поведінка;
- rollback rules.

Rendered validation має перевірити, що ці значення не суперечать одне одному.



Наприклад:

```
README.md каже: alias = LU_CHANGE_STATUS
QA_PACKAGE.md каже: alias = LU_CHANGE_CAPITAL
```

Це не дрібна помилка. Це провал consistency gate.

5. Відповідність output contract

Output contract визначає, що саме має бути створено.

Rendered artifact validation перевіряє:

```
чи створено саме те, що було обіцяно
```

Якщо contract каже:

```
створити тільки аналітичний пакет, без технічної реалізації
```

то в rendered artifact не має з'явитися:

```
готовий Java-код
міграції бази
конфігураційні зміни
реальні endpoint-и
```

Якщо contract каже:

```
не включати зайві файли
```

то архів із зайвими payload-файлами має бути заблокований.

6. Заборонені дії або заборонений контент

Rendered validation має перевіряти не тільки те, що має бути, а й те, чого не має бути.

Наприклад:

```
assert_not:
  - "final archive before approval"
  - "invented source row"
  - "unconfirmed alias"
  - "hidden implementation details"
  - "extra YAML with local secrets"
```

Це особливо важливо, коли модель працює з архівами, кодом, конфігураціями або документацією для інших команд.

Приклад Ordo Source

```
output:
  id: "history_event_package"
  type: "archive"
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"
    - "CONSISTENCY_CHECK_REPORT.json"
    - "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    - "02_JIRA_TASK_<ALIAS>.md"
    - "04_IMPLEMENTATION_PROMPT_<ALIAS>.md"
    - "05_QA_PACKAGE_<ALIAS>.md"
    - "07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md"
    - "08_QA_AUTOMATION_SPEC_<ALIAS>.yaml"
    - "09_QA_AUTOMATION_README_<ALIAS>.md"

render_validation:
  id: "G_RENDERED_PACKAGE_VALID"
  target: "history_event_package"
  blocking: true
  checks:
    - required_files_present
    - no_extra_files
    - no_placeholders
    - alias_consistency
    - required_sections_present
    - validation_report_present
    - consistency_report_present
```

Приклад Semantic JSON IR

```
{
  "op": "RENDER.VALIDATE",
  "id": "G_RENDERED_PACKAGE_VALID",
  "target": "history_event_package",
  "blocking": true,
  "checks": [
    "required_files_present",
    "no_extra_files",
    "no_placeholders",
    "alias_consistency",
    "required_sections_present",
    "validation_report_present",
    "consistency_report_present"
  ],
  "on_fail": {
    "status": "blocked",
    "handoff_allowed": false,
    "required_action": "fix_rendered_artifact"
  }
}
```

Зв'язок із gates

Rendered validation майже завжди має бути gate.

Не просто рекомендацією:

бажано перевірити результат

А саме gate:

результат не можна передавати, поки rendered validation не пройдено

Тому в Ordo це має бути пов'язано з gate system:

```
gate:
  id: "G_RENDERED_OUTPUT_VALID"
  method: mechanical
  trust_class: deterministic
  type: "rendered_artifact_validation"
  blocking: true
  required_before:
    - "handoff"
    - "archive_delivery"
    - "send_to_developer"
```

Зв'язок із Debug Layer

У debug mode rendered validation має показувати:

- які файли перевірялись;
- які розділи знайдені;
- які правила пройдені;
- які правила провалені;
- які значення порівнювались;
- де саме знайдено inconsistency;
- який gate заблокував handoff.

Наприклад:


```

trace_source: "model_self_report"
render_validation_trace:
  target: "final_archive"
  checks:
    - id: "required_files_present"
      status: "passed"
    - id: "no_extra_files"
      status: "failed"
      evidence:
        extra_files:
          - "payload_update.json"
    - id: "alias_consistency"
      status: "passed"
result:
  gate: "G_RENDERED_PACKAGE_VALID"
  status: "blocked"

```

Зв'язок із Test Layer

Rendered artifact validation теж потрібно тестувати.

Наприклад:

```

test:
  id: "TC_RENDER_BLOCKS_EXTRA_FILE"
  method: mechanical
  trust_class: deterministic

fixture:
  package_files:
    - "README.md"
    - "SUMMARY.json"
    - "payload_update.json"

expected:
  gate:
    id: "G_RENDERED_PACKAGE_VALID"
    status: "blocked"

  reason:
    - "extra_file_detected"

```

Або:

```

test:
  id: "TC_RENDER_BLOCKS_PLACEHOLDER"
  method: mechanical
  trust_class: deterministic

fixture:
  file_content:
    path: "02_JIRA_TASK.md"
    content: "Alias: <ALIAS>"

expected:
  gate:
    id: "G_RENDERED_OUTPUT_VALID"
    status: "blocked"

  violation:
    - "placeholder_detected"

```

Зв'язок із Feedback & Improvement Loop

Якщо користувач після отримання артефакту каже:

У README одна назва події, а в QA-файлі інша.

Ordo має створити improvement record:

```
improvement_record:
  classification:
    type: "rendered_artifact_inconsistency"
    severity: "high"

  affected_unit:
    kind: "render_validation_rule"
    id: "alias_consistency"

  proposed_patch:
    - "add consistency check between README.md and QA_PACKAGE.md"
    - "add regression test for mismatched event display name"

  suggested_tests:
    - "TC_RENDER_BLOCKS_DISPLAY_NAME_MISMATCH"
```

Тобто кожна помилка у фінальному артефакті має ставати джерелом покращення validation rules.

Типові помилки

Помилка 1. Перевіряти тільки шаблон

Шаблон може бути правильним, але результат — неправильним.

Ordo має перевіряти rendered artifact.

Помилка 2. Вважати self-check текстовою обіцянкою

Фраза:

Я перевірів результат

не дорівнює validation report.

Потрібен структурований gate report.

Помилка 3. Не перевіряти consistency між файлами

У великих пакетах основні помилки часто не в одному файлі, а між файлами.

Помилка 4. Дозволяти handoff після failed validation

Якщо validation failed, handoff має бути заблокований.

Помилка 5. Не тестувати validation rules

Правила перевірки також можуть бути неповними. Їх треба тестувати як частину Ordo-програми.

Міні-вправа

Візьміть будь-який документ або пакет файлів, який модель має створити.

Спробуйте відповісти:

1. Які файли або розділи мають бути обов'язково?
2. Які placeholder-и не можна залишати?
3. Які значення мають бути однаковими в різних місцях?
4. Яких зайвих файлів або розділів не має бути?
5. Який gate має заблокувати передачу результату?
6. Який validation report має бути сформований?

Якщо ви можете відповісти на ці питання, ви вже маєте основу для `RENDER.VALIDATE`.

Короткий підсумок

Rendered artifact validation — це перевірка фактичного результату після його створення.

У Ordo це важливо тому, що модель може правильно зрозуміти шаблон, але помилитися у готовому документі або пакеті.

Головне правило:

не перевірений rendered artifact не можна передавати як готовий результат

Для складних процесів `RENDER.VALIDATE` має бути blocking gate перед handoff.

Розділ 29. Старий `playbook` як `markdown`-база знань

Навіщо це потрібно

Багато складних інструкцій народжуються не як мова, не як програма і не як формальний `workflow`. Вони народжуються як великий `markdown`-документ: правила, приклади, винятки, таблиці, нагадування, чеклисти, історичні пояснення, рішення, які колись були прийняті в чаті або в задачах.

Такий документ може бути дуже корисним. Він зберігає знання команди. Він пояснює контекст. Він показує, як аналітик думає про процес. Але проблема в тому, що великий `markdown-playbook` не є виконуваною інструкцією.



Модель може прочитати його, але це не означає, що вона виконає його однаково кожного разу.

Вона може:

- пропустити важливий `gate`;
- переплутати приклад і правило;
- взяти застарілий фрагмент як актуальний;
- використати інструкцію не для того `path`;
- забути, що певне рішення потребує підтвердження людини;
- прочитати `all-in-one` документ як звичайний текст, а не як процес.

Саме тому старий `markdown-playbook` потрібно розглядати не як кінцевий формат, а як базу знань, з якої поступово виділяється Ordo-програма.

Просте пояснення

Старий `playbook` — це як велика папка з інструкціями на столі.

У ній може бути все:

- опис процесу;
- бізнесові правила;
- технічні правила;
- приклади;
- винятки;
- шаблони;
- чеклисти;
- історія рішень;
- примітки для майбутнього;
- попередження про помилки.

Але якщо попросити людину або модель “просто виконай усе з цієї папки”, виникне проблема: незрозуміло, що є обов’язковим, що є довідковим, що є прикладом, що є старою приміткою, а що є активним правилом.

Ordo пропонує інший підхід:

```
markdown-playbook → knowledge source → structured extraction → Ordo Source → compiled IR → execution
```

Тобто старий `playbook` не викидається. Він стає джерелом знань. Але виконання має відбуватися не напряму з хаотичного тексту, а через структурований Ordo-шар.

Що в старому `playbook`-у є корисним

У старому `markdown-playbook`-у важливо не тільки знайти готові правила. Важливо правильно класифікувати різні типи інформації.

Наприклад:

1. Обов'язкові правила
2. Decision tree
3. Questions / intake flow
4. Gates
5. Output templates
6. QA rules
7. Validation rules
8. Examples
9. Anti-patterns
10. Domain notes
11. Historical decisions
12. Process improvement feedback

Усе це не має однакового статусу.

Обов'язкове правило має стати `gate` або `assertion`.

Decision tree має стати набором `NODE.DEF` і `path rules`.

Шаблон має стати `OUTPUT.DEF` або `TEMPLATE.BIND`.

Приклад може залишитися у `FREEFORM`, але з `binding` до конкретного `rule` або `node`.

Попередження про типову помилку може стати `ASSERT.NOT`.

Зауваження користувача може стати `IMPROVEMENT.RECORD` або `regression test`.

Чому `all-in-one markdown` небезпечний

`All-in-one` документ здається зручним, бо все лежить в одному місці. Але для виконання це небезпечно.



Причини:

- модель може загубитися в довгому контексті;
- різні частини документа можуть суперечити одна одній;
- `old note` може виглядати як `active rule`;

- приклад може бути помилково виконаний як універсальний шаблон;
- важко зрозуміти, які правила покривають який path;
- важко дебажити, який саме фрагмент вплинув на рішення;
- важко тестувати, що зміна не зламала старий сценарій.

У Ordo all-in-one може бути корисним як rendered artifact або архівна форма, але не як головний runtime source.

Головний runtime source має бути структурований.

Як Ordo дивиться на markdown-playbook

Ordo не каже:

Markdown поганий.

Ordo каже:

Markdown добрий для пояснення, але недостатній для керованого виконання.

Тому старий playbook потрібно розділити на шари:

1. Human explanation layer
Текст для людей.
2. Ordo Source layer
Людинозрозуміла структурована програма.
3. Semantic JSON IR layer
Машинно-орієнтована execution map.
4. Debug/Test layer
Траси, тести, coverage, improvement records.

Markdown може залишатися у першому шарі. Але все, що впливає на виконання, має бути або формалізоване, або явно прив'язане як controlled FREEFORM.

Приклад класифікації фрагмента

Уявімо, що в старому playbook-у є фраза:

Перед створенням фінального архіву потрібно виконати self-check і перевірити, що пакет не містить зайвих файлів.

У markdown це просто речення.

В Ordo це має стати:

```
- op: "GATE.DEF"
  id: "G_PACKAGE_SELF_CHECK"
  type: "blocking"
  before: "HANDOFF.FINAL_PACKAGE"
  requires:
    - "validation_report.status == passed"
    - "unexpected_files == []"
```

А також negative assertion:

```
- op: "ASSERT.NOT"
  id: "A_NO_HANDOFF_WITHOUT_SELF_CHECK"
  condition: "final_package_created == true and self_check_passed != true"
  severity: "blocking"
```

I test case:

```
test:
  id: "TC_NO_PACKAGE_WITHOUT_SELF_CHECK"
  method: mechanical
  trust_class: deterministic
  expected:
    gate: "G_PACKAGE_SELF_CHECK"
    final_package_created: false
```

Так одна фраза з markdown стає виконуваною частиною Ordo-програми.

Що не треба формалізувати одразу

Не весь старий playbook потрібно одразу перетворювати на строгі ор-коди.

Деякі частини можуть тимчасово залишатися у controlled FREEFORM:

- довгі бізнесові пояснення;
- приклади для аналітика;
- історичні причини появи правила;
- edge cases, які ще не стабілізовані;
- пояснення термінів;
- domain commentary.

Але кожен FREEFORM-блок має мати binding:

```
freeform:
  id: "FF_HISTORY_EVENT_EDGE_CASES"
  bound_to:
    - "NODE.SELECT_PATH"
    - "G_SOURCE_FIELD_CONFIRMED"
  reason: "domain examples are not yet fully formalized"
```

Без binding FREEFORM знову перетворюється на хаотичний markdown.

Роль traceability

Під час міграції старого playbook-а важливо не втратити зв'язок між старим текстом і новими Ordo-конструкціями.

Тому потрібна traceability matrix:



```
старий фрагмент → Ordo Source object → IR op → test coverage → rendered artifact
```

Наприклад:

```

traceability:
  source_fragment: "section_12.self_check_before_archive"
  mapped_to:
    - "G_PACKAGE_SELF_CHECK"
    - "ASSERT.NOT.A_NO_HANDOFF_WITHOUT_SELF_CHECK"
    - "TC_NO_PACKAGE_WITHOUT_SELF_CHECK"

```

Це дозволяє відповісти на питання:

- де в Ordo тепер живе це правило?
- чи покрите воно тестом?
- чи воно blocking?
- чи воно потрапляє в compiled IR?
- чи воно використовується під час виконання?

Типові помилки

Помилка 1. Просто перейменувати markdown у Ordo

Якщо взяти старий markdown і назвати його Ordo Source, нічого не зміниться.

Ordo вимагає структури: nodes, gates, state, outputs, tests, traceability.

Помилка 2. Перенести все в FREEFORM

FREEFORM — це контрольована лазейка, а не смітник для невідформатованих правил.

Якщо все лишити у FREEFORM, Ordo не отримає керованості.

Помилка 3. Втратити історію рішень

Під час формалізації легко викинути пояснення, чому правило з'явилося.

Але історія важлива для майбутнього покращення. Її можна зберегти як note, evidence або improvement history.

Помилка 4. Не додати тести

Міграція без тестів — це косметична зміна.

Ordo-міграція має завершуватися test layer і coverage report.

Помилка 5. Не перевірити rendered artifacts

Якщо playbook генерує документи або пакети, потрібно перевіряти не тільки rules, а й готовий результат.

Міні-вправа

Візьміть будь-який великий документ з інструкціями.

Позначте в ньому фрагменти п'ятьма кольорами або категоріями:

1. rule
2. gate
3. example
4. explanation
5. warning / anti-pattern

Потім спробуйте для кожного rule відповісти:

- це має бути NODE, GATE, ASSERT.NOT, OUTPUT чи FREEFORM?
- чи треба для цього тест?
- чи це правило має бути blocking?
- чи воно застосовується завжди, чи тільки для одного path?

Це перший крок до міграції старого playbook-а в Ordo.

Короткий підсумок

Старий markdown-playbook — це цінна база знань, але не надійний формат виконання.

Ordo не знищує markdown. Вона витягує з нього виконувану структуру:

```
rules → gates
questions → nodes
templates → outputs
warnings → ASSERT.NOT
examples → controlled FREEFORM
decisions → state/status semantics
feedback → improvement records
```

Головна думка розділу:

старий playbook – це джерело знань, а не runtime execution layer

Розділ 30. Міграція playbook-a в Ordo

Навіщо це потрібно

Коли ми зрозуміли, що старий markdown-playbook є базою знань, виникає наступне питання: як перетворити його на Ordo-програму?

Це не має бути механічне переписування тексту. Міграція playbook-a в Ordo — це процес, у якому ми поступово виділяємо з документа виконувану структуру:

```
intent → contract → context → state → paths → nodes → gates → outputs → tests → coverage
```

Мета міграції не в тому, щоб зробити документ “красивішим”. Мета — зробити процес керованим, трасованим, тестованим і придатним для повторного виконання.



Просте пояснення

Міграція playbook-a в Ordo схожа на перетворення великої інструкції для людини на програму для керованого виконання моделлю.

Було:

Прочитай інструкцію, зрозумій контекст, постав питання, не забудь перевірити пакет, якщо все готово — сформуй архів.

Стає:

```
intent:
  id: "create_history_event_package"

contract:
  requires:
    - "alias_confirmed"
    - "source_field_confirmed"
    - "expected_values_confirmed"

nodes:
  - id: "N_SELECT_PATH"
  - id: "N_COLLECT_ALIAS"
  - id: "N_COLLECT_SOURCE_FIELD"

required_gates:
  - "G_CONTRACT_CONFIRMED"
  - "G_PRE_ARCHIVE_APPROVAL"
  - "G_RENDERED_ARTIFACT_VALIDATED"
```

Тобто Ordo робить приховану логіку playbook-a явною.



Основні етапи міграції

Міграцію варто описувати як послідовність із десяти кроків.

1. Inventory
2. Classification
3. Contract extraction
4. Decision tree extraction
5. State model extraction
6. Gate extraction
7. Output mapping
8. FREEFORM binding
9. Test layer creation
10. IR compilation and validation

Крок 1. Inventory

Спочатку потрібно зрозуміти, що взагалі є в playbook-у.

Потрібно скласти каталог фрагментів:

```
source_catalog:
  sections:
    - id: "S01_INTRO"
      type: "explanation"
    - id: "S02_DECISION_TREE"
      type: "decision_logic"
    - id: "S03_OUTPUT_PACKAGE"
      type: "output_rules"
    - id: "S04_QA"
      type: "qa_rules"
```

Цей крок потрібен, щоб не втратити частину знань під час міграції.

Крок 2. Classification

Далі кожен фрагмент класифікується.

Мінімальні категорії:

- rule
- node
- gate
- status
- output
- template
- example
- anti-pattern
- freeform note
- improvement note

Наприклад:

```
fragment:
  id: "F_SELF_CHECK_BEFORE_ARCHIVE"
  original_type: "markdown paragraph"
  classified_as:
    - "gate"
    - "assert_not"
    - "test_candidate"
```

Це важливо, бо різні типи фрагментів перетворюються на різні Ordo-конструкції.

Крок 3. Contract extraction

У будь-якому складному playbook-у є умови, без яких роботу не можна вважати визначеною.

Наприклад для History Event package такими умовами можуть бути:

- підтверджений alias;
- підтверджений тип події;
- підтверджене source field;
- зрозумілі old/new values;
- зрозумілий path;
- підтверджені expected outputs;
- підтверджені QA expectations.

В Ordo це стає contract:

```
contract:
  required_inputs:
    - "event_alias"
    - "source_field"
    - "path_type"
    - "expected_values"

  missing_input_behavior:
    action: "ask_next_required_question"
    final_output_allowed: false
```

Головне правило:



якщо contract не зібраний, playbook не має переходити до фінального output

Крок 4. Decision tree extraction

Якщо в playbook-у є різні сценарії, вони мають бути винесені в decision tree.

Наприклад:

```
paths:
  - id: "A1"
    name: "direct source field change"
    condition: "change detected in primary source row"
```

```

- id: "A2"
  name: "related entity change"
  condition: "change belongs to entity linked through identification center"

- id: "A4"
  name: "external history event"
  condition: "input is ExternalHistoryEvent candidate"

```

У debug mode Ordo має показувати не тільки selected path, а й rejected paths:

```

path_explain:
  selected: "A1"
  rejected:
    - path: "A2"
      reason: "related entity was not confirmed"
    - path: "A4"
      reason: "no external event input"

```

Крок 5. State model extraction

Markdown часто приховує state у тексті.

Наприклад:

Коли аналітик підтвердив source field, можна переходити до values.

В Ordo це має стати state transition:

```

state:
  source_field_confirmed: true
  next_node: "N_COLLECT_VALUES"

```

Потрібно визначити:

- які поля state існують;
- хто може їх змінювати;
- які значення означають готовність;
- які значення блокують процес;
- які transitions дозволені;
- які transitions заборонені.

Без state model модель буде “пам’ятати процес у голові”, а це ненадійно.

Крок 6. Gate extraction

Gate — це місце, де процес має зупинитися або перевірити умову.

Під час міграції потрібно шукати фрази типу:

- перед тим як;
- тільки після;
- обов’язково перевірити;
- не можна створювати;
- має бути підтверджено;
- якщо не визначено — зупинитися;
- без цього не переходити далі.

Такі фрази майже завжди є кандидатами на GATE.DEF або ASSERT.NOT.

Приклад:

```
- op: "GATE.DEF"
  id: "G_PRE_ARCHIVE_APPROVAL"
  type: "blocking"
  before: "OUTPUT.FINAL_ARCHIVE"
  condition: "user_approval == true"
```

Крок 7. Output mapping

Далі потрібно визначити, що саме playbook має створювати.

Для простого процесу це може бути один документ. Для складного — пакет файлів.

Ordo має явно описувати output:

```
outputs:
- id: "O_ANALYTICAL_PACKAGE"
  kind: "file_set"
  required_files:
  - "README.md"
  - "SUMMARY.json"
  - "VALIDATION_REPORT.json"
  - "CONSISTENCY_CHECK_REPORT.json"
```

Також потрібно описати rendered validation:

```
render_validate:
  required:
  - "all_required_files_present"
  - "no_unexpected_files"
  - "no_unresolved_placeholders"
  - "cross_file_consistency_passed"
```

Output без validation — слабе місце.

Крок 8. FREEFORM binding

Після перших семи кроків частина playbook-а все одно залишиться неформалізованою.

Це нормально.

Але її потрібно оформити як controlled FREEFORM:

```
freeform:
  id: "FF_DOMAIN_EDGE_CASES"
  type: "domain_notes"
  bound_to:
  - "PATH.A4"
  - "G_EXTERNAL_EVENT_NORMALIZED"
  reason: "edge cases are described as examples and not yet stable enough for full formalization"
```

Головне правило:

```
FREEFORM має бути прив'язаний до конкретної частини виконання
```

Крок 9. Test layer creation

Після міграції потрібно створити тестовий шар.

Мінімальний набір:

- один happy-path test для кожного path;
- negative tests для заборонених дій;
- gate tests;
- no-op tests;
- rendered artifact validation tests;
- regression suite.

Приклад:

```
test:
  id: "TC_A1_HAPPY_PATH"
  fixture:
    user_message: "створюємо подію зміни статусу"
  expected:
    path: "A1"
    gates:
      - id: "G_CONTRACT_CONFIRMED"
        status: "passed"
  output:
    package_created: true
```

Тестування — це те, що відрізняє Ordo-міграцію від простого переписування документа.

Крок 10. IR compilation and validation

Останній крок — скомпілювати Ordo Source в Semantic JSON IR і перевірити, що всі важливі частини playbook-а мають відображення.

Потрібно отримати:

- compiled IR;
- traceability report;
- validation report;
- consistency check report;
- coverage report;
- freeform coverage report;
- regression suite summary.

Якщо частина playbook-а не покрита, це не завжди помилка. Але це має бути видно.

Наприклад:

```
migration_report:
  structured_core: "72%"
  controlled_freeform: "28%"
  uncovered_fragments: 0
  blocking_gates_defined: 14
  tests_defined: 18
  status: "passed_with_notes"
```

Мінімальний результат міграції

Після міграції має бути не один великий файл, а набір пов'язаних артефактів.

Наприклад:

```
START_HERE_ORDO.md
ORDO_EXECUTION_CONTRACT.md
ORDO_CORE_BINDING.md
```

```
ORDO_PROFILE_BINDINGS.md
DOMAIN_PACK.md
PLAYBOOK_ORDO_SOURCE.md
PLAYBOOK_COMPILED_IR.json
FREEFORM_LEDGER.md
FREEFORM_COVERAGE.md
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json
```

Це не просто “більше файлів”. Це різні runtime views одного процесу.

Як зрозуміти, що міграція успішна

Міграція успішна, якщо можна відповісти “так” на такі питання:

1. Чи зрозуміло, з якого entry починати?
2. Чи є contract?
3. Чи є state model?
4. Чи є decision paths?
5. Чи є blocking gates?
6. Чи є outputs?
7. Чи є rendered validation?
8. Чи зрозуміло, що залишилось у FREEFORM?
9. Чи є tests?
10. Чи можна пояснити, чому run пішов певним шляхом?

Якщо відповідь “ні” хоча б на кілька пунктів, playbook ще не став повноцінною Ordo-програмою.

Типові помилки

Помилка 1. Починати з IR

Не треба одразу писати JSON IR руками.

Спочатку потрібно зрозуміти contract, paths, state і gates. IR — це compiled representation, а не перший документ, з якого варто починати роботу.

Помилка 2. Не зберегти зв'язок зі старим playbook-ом

Якщо після міграції неможливо сказати, звідки взялося правило, це проблема.

Потрібна traceability matrix.

Помилка 3. Формалізувати приклади як правила

Приклад показує можливу ситуацію. Rule визначає обов'язкову поведінку.

Це різні речі.

Помилка 4. Не виділити blocking gates

Якщо gate не blocking, модель може піти далі навіть після failed condition.

Для критичних процесів це неприпустимо.

Помилка 5. Не додати improvement loop

Після першої міграції playbook не стане ідеальним.

Потрібно одразу мати механізм збору проблем і покращень.

Міні-вправа

Візьміть один розділ старого playbook-а і спробуйте зробити міні-міграцію.

Заповніть таблицю:

Фрагмент	Тип	Ordo object	Gate?	Test needed?	FREEFORM?
----------	-----	-------------	-------	--------------	-----------

Наприклад:

```
"Не створювати архів без self-check"
→ anti-pattern / gate candidate
→ ASSERT.NOT + GATE.DEF
→ так
→ так
→ ні
```

Це простий спосіб побачити, як текст починає перетворюватися на Ordo-програму.

Короткий підсумок

Міграція playbook-а в Ordo — це не переписування документа іншими словами.

Це виділення виконуваної структури:

```
knowledge → rules → state → paths → gates → outputs → tests → IR
```

Головна думка розділу:

```
Ordo-міграція успішна тоді, коли playbook можна не тільки прочитати, а й виконати, пояснити, перевірити і покращити
```

Розділ 31. Guided intake як Ordo-програма

Навіщо це потрібно

У багатьох реальних процесах модель не повинна одразу створювати фінальний результат. Їй спочатку потрібно зібрати дані, уточнити контекст, пройти дерево рішень, перевірити gates і тільки після цього перейти до створення документа, архіву, звіту або іншого artifact.

Такий режим роботи називається `guided intake`.

Простими словами, `guided intake` — це кероване опитування користувача, у якому модель не імпровізує порядок питань, а рухається за визначеною Ordo-програмою.

Це особливо важливо для складних `playbook`-ів, де неправильне перше питання може зламати весь процес. Наприклад, якщо модель надто рано питає про назву події, хоча спочатку треба визначити `path`, або якщо вона починає генерувати фінальний пакет до підтвердження контракту.

У звичайному `prompt`-підході `guided intake` часто виглядає так:

Постав мені кілька питань, щоб зібрати інформацію.

Але цього недостатньо. Модель може:

- поставити питання не в тому порядку;
- пропустити обов'язкове питання;
- сплутати попередню відповідь із підтвердженням фактом;
- перейти до результату занадто рано;
- не зафіксувати, який `path` був обраний;
- не пояснити, чому одні варіанти відкинуті, а інші прийняті.

В Ordo `guided intake` — це не просто “розмова”. Це повноцінна Ordo-програма.

Просте пояснення

`Guided intake` можна уявити як форму, яка заповнюється не на одній сторінці, а через діалог.

Але важлива різниця в тому, що ця “форма” має логіку:

якщо відповідь така → йдемо сюди;
якщо бракує даних → ставимо питання;
якщо `gate` не пройдений → зупиняємося;
якщо `contract` підтверджений → переходимо до наступного етапу;
якщо користувач змінив рішення → оновлюємо `state` і повертаємося до потрібного вузла.

Тобто `guided intake` — це поєднання:

ENTRY → NODE → STATE → QUESTION → ANSWER → GATE → PATH → NEXT NODE

Модель у такому процесі має не просто “бути корисною”. Вона має виконувати роль керованого оператора процесу.

Чим **guided intake** відрізняється від звичайного діалогу

Звичайний діалог може бути вільним. Користувач щось каже, модель відповідає, потім тема змінюється, потім модель щось уточнює.

Guided intake має іншу природу.

У ньому кожне питання має призначення:

```
питання збирає конкретне поле state;
питання прив'язане до конкретного NODE;
відповідь має бути класифікована;
після відповіді виконується state update;
потім перевіряється gate;
потім визначається наступний NODE.
```

Наприклад, якщо Ordo-програма створює нову історичну подію, то питання:

```
Який alias події?
```

не є просто розмовним питанням. Це операція збору контрактного поля:

```
node:
  id: "N_COLLECT_ALIAS"
  asks_for: "event.alias"
  required: true
  next_if_answered: "N_COLLECT_SOURCE_FIELD"
```

Базові елементи **guided intake**

Для **guided intake** потрібні такі елементи Ordo:

```
ENTRY.DEF
NODE.DEF
QUESTION.DEF
ANSWER.REGISTRY
STATE.SCHEMA
STATE.UPDATE
PATH.SELECT
GATE.REPORT
NEXT.NODE
STATUS.SEMANTICS
```

ENTRY.DEF

Визначає, з чого починається процес.

```
entry:
  id: "history_event_intake"
  purpose: "guided intake for a new History Event"
  start_node: "N_CLASSIFY_INPUT"
```

NODE.DEF

Описує один вузол діалогу.

```
node:
  id: "N_CLASSIFY_INPUT"
  type: "decision"
  purpose: "understand what kind of event request this is"
  allowed_next:
    - "N_PATH_A1"
    - "N_PATH_A2"
    - "N_PATH_A4"
    - "N_NEED_MORE_CONTEXT"
```

QUESTION.DEF

Описує, яке питання можна поставити користувачу.

```
question:
  id: "Q_SOURCE_FIELD"
  node: "N_COLLECT_SOURCE_FIELD"
  text: "Яке source field змінюється?"
  writes_to: "state.contract.source_field"
  required: true
```

ANSWER.REGISTRY

Фіксує відповіді користувача не як “текст у чаті”, а як значення state.

```
answer:
  question_id: "Q_SOURCE_FIELD"
  raw_text: "status"
  normalized_value: "item.status"
  confidence: "confirmed"
```

STATE.UPDATE

Показує, що саме змінилося після відповіді.

```
state_update:
  field: "contract.source_field"
  before: null
  after: "item.status"
  source: "user_answer"
```

Одне головне питання за раз

Для guided intake важливе правило:

один NODE — одне головне питання

Це не означає, що модель ніколи не може поставити уточнення. Але головний рух процесу має бути контрольований.

Погано:

Який alias, source field, old value, new value, display name, path і QA сценарії?

Таке питання перевантажує користувача і змінює кілька state transitions.

Добре:

Поточний крок: потрібно визначити alias події.
Який alias використовуємо?

Після відповіді модель оновлює state і переходить до наступного вузла.

Поточний статус має бути видимим

Guided intake має постійно знати, де він знаходиться.

Мінімальний службовий статус може виглядати так:

```
intake_status:
  current_entry: "history_event_intake"
  current_node: "N_COLLECT_ALIAS"
  selected_path: "A1"
  confirmed:
    - "event_type"
    - "source_row"
  pending:
    - "alias"
    - "source_field"
    - "old_new_values"
  blocked_by:
    - "G_CONTRACT_COMPLETE"
```

У звичайному режимі модель не обов'язково показує весь цей status користувачу повністю. Але в debug mode він має бути доступний.

Path selection у guided intake

Одна з головних функцій guided intake — правильно обрати path.

Наприклад:

```
A1 – зміна поля в основній source row;
A2 – зміна поля у пов'язаній сутності;
A4 – зовнішній ExternalHistoryEvent;
A5 – no-op або expected-no-change сценарій.
```

Модель не повинна просто здогадуватися. Вона має виконати PATH.SELECT.

```
trace_source: "model_self_report"
path_selection:
  candidate_paths:
    - id: "A1"
      condition: "direct source field change"
    - id: "A2"
      condition: "related entity through identification center"
    - id: "A4"
      condition: "external history event input"
  selected:
    id: "A1"
    reason: "user confirmed direct field change in source row"
  rejected:
    - id: "A2"
      reason: "no related entity context confirmed"
    - id: "A4"
      reason: "input is not ExternalHistoryEvent"
```

Це особливо важливо, бо помилка в path на початку часто призводить до неправильного пакета в кінці.

Gates y guided intake

Guided intake не має бути нескінченним опитуванням. Він має контрольні точки.

Наприклад:

```
G_PATH_CONFIRMED
G_CONTRACT_COMPLETE
G_SOURCE_ROW_CONFIRMED
G_VALUES_CONFIRMED
G_QA_SCOPE_CONFIRMED
G_PRE_PACKAGE_APPROVAL
```

Gate не просто “нагадує”. Він блокує перехід далі.

```
gate:
  id: "G_CONTRACT_COMPLETE"
  method: mechanical
  trust_class: deterministic
  type: "blocking"
  requires:
    - "contract.alias"
    - "contract.source_field"
    - "contract.values"
    - "contract.source_row"
  on_fail:
    action: "ask_missing_question"
```

Якщо контракт неповний, модель не має права переходити до генерації фінального output.

Як guided intake працює з уточненнями користувача

Користувач може змінити рішення.

Наприклад:

```
вернись до попереднього кроку
змінюю рішення на 3
```

Ordo має не ігнорувати це, а виконати контрольований state correction.

```
state_correction:
  reason: "user changed previous decision"
  affected_field: "selected_option"
  before: "2"
  after: "3"
  rollback_to_node: "N_CONFIRM_OPTION"
  recheck_gates:
    - "G_PATH_CONFIRMED"
    - "G_CONTRACT_COMPLETE"
```

Це дуже важливо. У складних процесах користувач часто уточнює або змінює попередні рішення. Якщо guided intake не підтримує контрольоване повернення назад, state швидко стає недостовірним.

Guided intake і debug mode

У debug mode guided intake має показувати:

- поточний NODE;
- чому поставлено саме це питання;
- яке поле state воно заповнює;
- які paths ще не визначені;
- які gates блокують перехід;
- які відповіді вже підтверджені;
- які рішення були змінені користувачем;
- чому модель не переходить до фінального output.

Приклад debug-фрагмента:

```
debug:
  current_node: "N_COLLECT_SOURCE_FIELD"
  question_reason: "contract.source_field is required for Path A1"
  writes_to: "state.contract.source_field"
  blocked_gates:
    - "G_CONTRACT_COMPLETE"
  next_after_answer:
    - "N_COLLECT_VALUES"
```

Guided intake і improvement loop

Guided intake є одним із головних місць, де виникають покращення.

Користувач може сказати:

це питання треба було поставити раніше

або:

тут потрібно не питати alias, а спочатку визначити source row

Ordo має створити improvement record:

```
improvement_record:
  type: "intake_order_problem"
  affected_unit:
    kind: "node"
    id: "N_COLLECT_ALIAS"
  proposed_patch:
    - "move N_COLLECT_SOURCE_ROW before N_COLLECT_ALIAS"
  suggested_test:
    id: "TC_SOURCE_ROW_BEFORE_ALIAS"
```

Так guided intake поступово стає кращим не через хаотичне переписування інструкцій, а через керований цикл покращення.

Guided intake як compiled IR

У compiled IR guided intake може виглядати як набір ор-кодів:

```
[
  {
    "op": "ENTRY.DEF",
    "id": "history_event_intake",
    "start_node": "N_CLASSIFY_INPUT"
```

```

},
{
  "op": "NODE.DEF",
  "id": "N_COLLECT_ALIAS",
  "node_type": "question",
  "writes_to": "contract.alias"
},
{
  "op": "QUESTION.DEF",
  "id": "Q_ALIAS",
  "text": "Який alias події?",
  "required": true
},
{
  "op": "GATE.DEF",
  "id": "G_CONTRACT_COMPLETE",
  "type": "blocking"
},
{
  "op": "STATE.UPDATE",
  "from": "Q_ALIAS",
  "to": "contract.alias"
}
]

```

Це означає, що guided intake можна не тільки описати словами, а й виконувати як structured program.

Типові помилки

Помилка 1. Перетворити guided intake на звичайний список питань

Список питань не є guided intake, якщо немає state, gates і path logic.

Помилка 2. Питати все одразу

Коли модель питає десять речей одночасно, вона втрачає контроль над state.

Помилка 3. Не фіксувати, що підтверджено

Якщо модель не розділяє “користувач згадав” і “користувач підтвердив”, вона може створити неправильний contract.

Помилка 4. Не підтримувати повернення назад

Користувачі часто змінюють рішення. Ordo має вміти коректно оновити state.

Помилка 5. Дозволити фінальний output до gates

Guided intake має блокувати передчасну генерацію artifact.

Міні-вправа

Візьміть простий процес:

Підготувати відповідь клієнту на скаргу.

Спробуйте визначити:

1. Який ENTRY запускає процес?
2. Які 3–5 NODE потрібні?
3. Яке перше питання має поставити модель?
4. Які поля state потрібно зібрати?
5. Який gate має бути перед фінальним текстом?
6. Що робити, якщо користувач змінив тон відповіді з “офіційний” на “дружній”?

Короткий підсумок

Guided intake — це не просто діалог із моделлю. Це Ordo-програма, яка керує збором інформації, path selection, state updates, gates і переходами між вузлами.

Його головна цінність у тому, що складний процес не перетворюється на хаотичне листування. Модель знає, де вона знаходиться, що вже підтверджено, що ще потрібно зібрати і чому вона не має права рухатися далі.

У великих playbook-ах guided intake є мостом між людською розмовою і формальним виконанням Ordo.

Розділ 32. Package generation

Навіщо це потрібно

У попередніх розділах ми розглядали Ordo як мову, що керує діалогом, збирає state, обирає path і проходить gates. Але у великих робочих процесах кінцевий результат майже ніколи не є просто однією відповіддю в чаті.

Часто результатом має бути пакет:

```
набір markdown-документів;  
JSON-звіт;  
YAML-специфікація;  
README;  
QA-пакет;  
Jira-опис;  
validation report;  
archive для передачі в роботу.
```

Саме тут з'являється тема Package generation.

Package generation — це не момент, коли модель “просто створює файли”. Це окремий етап Ordo-виконання, який має свій contract, gates, output definitions, validation і handoff.

Якщо цей етап не формалізувати, модель може:

- створити зайві файли;
- пропустити обов'язкові файли;
- змішати бізнесову і технічну документацію;
- не узгодити README з фактичним складом пакета;
- створити validation report, який не перевіряє реальний пакет;
- покласти в архів проміжні файли;
- видати пакет до підтвердження contract gates.

Ordo має зробити package generation керованою дією.

Просте пояснення

Уявімо, що Ordo-програма — це не просто інструкція, а виробнича лінія.

Спочатку вона збирає вимоги. Потім перевіряє contract. Потім формує документи. Потім перевіряє їх. Потім збирає архів. Потім передає результат людині.

Package generation — це фінальна виробнича ділянка цієї лінії.

Вона має відповідати на питання:

```
що саме створюємо;  
з яких файлів складається пакет;  
які файли є обов'язковими;  
які файли заборонені;  
які формати допустимі;  
які gates мають бути пройдені до генерації;  
як перевірити, що пакет консистентний;  
що саме передається користувачу.
```

Package як Ordo Output

У простій задачі output може бути таким:

```
output:
  type: "message"
  format: "text"
```

Але для package generation output має бути значно точнішим:

```
output:
  type: "package"
  id: "history_event_analysis_package"
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"
    - "CONSISTENCY_CHECK_REPORT.json"
    - "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    - "02_JIRA_TASK_<ALIAS>.md"
    - "04_IMPLEMENTATION_PROMPT_<ALIAS>.md"
    - "05_QA_PACKAGE_<ALIAS>.md"
    - "07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md"
    - "08_QA_AUTOMATION_SPEC_<ALIAS>.yaml"
    - "09_QA_AUTOMATION_README_<ALIAS>.md"
  forbidden_files:
    - "drafts/*"
    - "tmp/*"
    - "raw_notes/*"
```

Така структура дозволяє моделі не гадати, що має бути в пакеті.

Package generation не має починатися одразу

Одна з головних помилок у великих playbook-ах — створювати фінальний пакет занадто рано.

В Ordo має бути правило:

Фінальний пакет не створюється, поки не пройдено всі gates, які впливають на його зміст.

Наприклад:

```
gates_before_package_generation:
  - "G_INTENT_CONFIRMED"
  - "G_CONTRACT_CONFIRMED"
  - "G_PATH_SELECTED"
  - "G_REQUIRED_VALUES_CONFIRMED"
  - "G_OUTPUT_SET_CONFIRMED"
  - "G_QA_SCOPE_CONFIRMED"
```

Якщо хоча б один gate не пройдено, Ordo має зупинитися:

```
package_generation:
  status: "blocked"
  blocked_by:
    - "G_REQUIRED_VALUES_CONFIRMED"
  message: "Пакет ще не можна формувати, бо не підтверджені required values."
```

Це важлива різниця між prompt-ом і Ordo. Prompt часто намагається виконати прохання негайно. Ordo спочатку перевіряє, чи має право це робити.

Package plan

Перед створенням файлів Ordo має сформувати package plan.

```
package_plan:
  package_id: "PKG_HISTORY_EVENT_LU_CHANGE_STATUS"
  alias: "LU_CHANGE_STATUS"

  files:
    - path: "README.md"
      purpose: "пояснює склад пакета і порядок використання"
      required: true

    - path: "01_HISTORY_EVENT_PASSPORT_LU_CHANGE_STATUS.md"
      purpose: "аналітичний паспорт історичної події"
      required: true

    - path: "05_QA_PACKAGE_LU_CHANGE_STATUS.md"
      purpose: "ручне тестування і QA-сценарії"
      required: true

  validation:
    required: true
    reports:
      - "VALIDATION_REPORT.json"
      - "CONSISTENCY_CHECK_REPORT.json"
```

Package plan — це контракт між Ordo-програмою і фінальним artifact.

Якщо в package plan є 11 файлів, то у фінальному архіві має бути саме очікуваний набір файлів, якщо немає окремого підтвердженого винятку.

Rendered artifact validation

Package generation тісно пов'язаний із Rendered artifact validation.

Не достатньо перевірити шаблон. Потрібно перевірити вже створений результат.

Погано:

Шаблон README містить секцію "Склад пакета", отже все добре.

Добре:

Фактичний README містить список файлів, який збігається з реальним архівом.

Ordo має перевіряти:

- чи всі required files створені;
- чи немає forbidden files;
- чи README описує фактичний склад;
- чи SUMMARY.json збігається з файлами пакета;
- чи validation report посилається на реальні перевірки;
- чи всі alias/file names узгоджені;
- чи немає placeholder-ів;
- чи немає суперечностей між документами.

Self-check перед handoff

Перед передачею пакета користувачу має бути self-check.

```
self_check:
  required: true
  before:
    - "handoff"
    - "archive_delivery"

  checks:
    - id: "SC_REQUIRED_FILES"
      description: "усі обов'язкові файли присутні"

    - id: "SC_NO_FORBIDDEN_FILES"
      description: "немає заборонених або проміжних файлів"

    - id: "SC_README_MATCHES_PACKAGE"
      description: "README відповідає фактичному пакету"

    - id: "SC_VALIDATION_REPORT_PRESENT"
      description: "validation report створений"

    - id: "SC_CONSISTENCY_REPORT_PRESENT"
      description: "consistency check report створений"
```

Якщо self-check не пройдений, пакет не має передаватися.

```
handoff:
  status: "blocked"
  reason: "self_check_failed"
```

Package generation у compiled IR

У compiled IR це може виглядати так:

```
[
  {
    "op": "OUTPUT.DEF",
    "id": "OUT_ANALYTICAL_PACKAGE",
    "output_type": "package",
    "required_files": [
      "README.md",
      "SUMMARY.json",
      "VALIDATION_REPORT.json"
    ]
  },
  {
    "op": "GATE.REQUIRE",
    "id": "G_BEFORE_PACKAGE",
    "requires": [
      "G_CONTRACT_CONFIRMED",
      "G_OUTPUT_SET_CONFIRMED"
    ]
  },
  {
    "op": "PACKAGE.PLAN",
    "id": "PKG_PLAN_1",
    "from_output": "OUT_ANALYTICAL_PACKAGE"
  },
  {
    "op": "PACKAGE.GENERATE",
    "id": "PKG_GENERATE_1",
    "allowed_after": "G_BEFORE_PACKAGE"
  },
  {
```

```

    "op": "RENDER.VALIDATE",
    "id": "VALIDATE_RENDERED_PACKAGE"
  },
  {
    "op": "GATE.REPORT",
    "id": "G_PACKAGE_VALIDATED"
  }
]

```

Це ще раз показує, що package generation — не проста генерація тексту, а execution phase.

Package generation i debug mode

У debug mode Ordo має пояснити:

- чому пакет можна або не можна створювати;
- які gates дозволили package generation;
- які outputs мали бути створені;
- які файли реально створені;
- які перевірки пройдені;
- які warnings залишились;
- який artifact передано в handoff.

Наприклад:

```

package_debug:
  generation_allowed: false
  blocked_by:
    - "G_QA_SCOPE_CONFIRMED"
  reason: "QA scope was not confirmed by user"

```

Або:

```

package_debug:
  generation_allowed: true
  required_files_expected: 11
  required_files_created: 11
  forbidden_files_found: 0
  validation_status: "passed"

```

Package generation i improvement loop

Package generation часто породжує корисні зауваження.

Користувач може сказати:

```
у пакеті не вистачає runbook для тестувальника
```

або:

```
README не пояснює, який файл відкривати першим
```

Ordo має створити improvement record:

```

improvement_record:
  type: "package_structure_improvement"
  affected_unit:
    kind: "output_definition"
    id: "OUT_ANALYTICAL_PACKAGE"
  proposed_patch:
    - "add 10_ANALYST_RUNBOOK.md to required_files"
    - "update README required sections"

```

```
suggested_test:
  id: "TC_PACKAGE_README_START_HERE"
```

Так структура пакетів стає кращою через контрольований цикл покращення.

Типові помилки

Помилка 1. Створювати пакет без `package plan`

Без плану модель може створити красивий, але неправильний набір файлів.

Помилка 2. Перевіряти шаблон, а не створений `artifact`

Шаблон може бути правильним, але конкретний файл може бути порожнім або суперечливим.

Помилка 3. Не розділяти `draft` і `final`

Проміжні нотатки не повинні випадково потрапляти у фінальний пакет.

Помилка 4. Не блокувати `handoff` після `failed validation`

Якщо `validation failed`, пакет не готовий.

Помилка 5. Вважати архів “готовим”, бо він створений

Архів готовий тільки після `self-check`, `validation` і `consistency check`.

Міні-вправа

Візьміть будь-який процес, де результатом є набір файлів.

Наприклад:

Пакет документів для зміни `monitoring event`.

Спробуйте визначити:

1. Які файли мають бути `required`?
 2. Які файли мають бути `forbidden`?
 3. Який `README` потрібен?
 4. Які `gates` мають бути пройдені до генерації?
 5. Який `validation report` потрібен?
 6. Що має блокувати `handoff`?
-

Короткий підсумок

`Package generation` — це окрема `execution phase` в `Ordo`. Вона має бути керованою, перевірюваною і заблокованою до проходження необхідних `gates`.

Ordo не просто створює файли. Вона спочатку визначає package contract, формує package plan, генерує required artifacts, перевіряє rendered result, виконує self-check і тільки після цього передає пакет користувачу.

Це дозволяє працювати з великими playbook-ами не як з хаотичною генерацією документів, а як із контрольованим виробництвом artifacts.

Розділ 33. Що лишилось у FREEFORM

Навіщо це потрібно

Ми вже говорили, що Ordo не повинна намагатися формалізувати абсолютно все. Для частини знань потрібен FREEFORM: пояснення, приклади, попередження, історичні нотатки, складні доменні формулювання.

Але після міграції великого playbook-a в Ordo завжди виникає питання:

Що саме залишилось у FREEFORM?

Це не дрібниця. Відповідь на це питання показує рівень зрілості Ordo-програми.

Якщо у FREEFORM залишилось тільки те, що справді не варто формалізувати, — це нормально.

Якщо у FREEFORM випадково залишилися gates, required decisions, status rules або output contracts, — це проблема. Такі речі модель може пропустити, неправильно інтерпретувати або виконати непослідовно.

Просте пояснення

FREEFORM — це зона, де Ordo дозволяє людському тексту залишатися людським.

Але ця зона має бути прозорою.

Після міграції потрібно подивитися:

що вдалося перетворити на structured Ordo;
що залишилось у FREEFORM;
чому воно там залишилось;
чи це безпечно;
чи потрібні tests;
чи потрібно поступово формалізувати частину цього тексту.

Інакше FREEFORM може стати “темною кімнатою”, де заховані критичні правила.

Що нормально залишати у FREEFORM

У FREEFORM нормально залишати:

- пояснення для людини;
- приклади;
- довгі доменні описи;
- історію прийняття рішення;
- edge cases, які ще не стабілізовані;
- аналітичні коментарі;
- стилістичні рекомендації;
- тимчасові примітки;
- текстові шаблони, якщо вони не є execution rules.

Наприклад:

```

freeform:
  id: "FF_DOMAIN_CONTEXT"
  purpose: "пояснення доменного контексту"
  content: |
    У цьому типі історичних подій важливо розрізняти зміну основної компанії
    і зміну пов'язаної сутності. Аналітик має уважно перевірити джерело зміни.

```

Це нормальний FREEFORM, якщо самі правила path selection уже формалізовані окремо.

Що не можна ховати у FREEFORM

У FREEFORM не можна залишати те, що має впливати на виконання як обов'язкове правило.

Наприклад, погано:

```

freeform:
  content: |
    Перед створенням архіву обов'язково треба зробити self-check.

```

Якщо це правило обов'язкове, воно має бути gate:

```

gate:
  id: "G_PACKAGE_SELF_CHECK"
  method: mechanical
  trust_class: deterministic
  type: "blocking"
  before:
    - "handoff"
    - "archive_delivery"

```

Так само не можна ховати у FREEFORM:

- blocking gates;
 - required fields;
 - status semantics;
 - output contracts;
 - path selection rules;
 - approval requirements;
 - negative assertions;
 - regression requirements;
 - library conflict rules;
 - no-op conditions.
-

FREEFORM після міграції playbook-a

Коли старий markdown-playbook мігрує в Ordo, частина тексту стає structured IR, а частина залишається FREEFORM.

Наприклад:

```

Було в markdown:
"Якщо подія стосується пов'язаної сутності, потрібно уточнити relation через Центр ідентифікації."

Після міграції:
- path rule → structured;
- required question → NODE.DEF;
- state field → STATE.SCHEMA;
- пояснення про Центр ідентифікації → FREEFORM.

```

Тобто FREEFORM не означає “не важливо”. Він означає:

це знання потрібне, але воно не є самостійною execution instruction.

FREEFORM Ledger

Для великої Ordo-програми потрібен FREEFORM ledger.

```
freeform_ledger:
- id: "FF_HISTORY_EVENT_CONTEXT"
  source_section: "old_playbook/domain_notes"
  reason: "domain explanation, not executable rule"
  bound_to:
    - "DOMAIN_PACK.history_event"
    - "PATH.A2"
  risk: "low"
  tests_required: false

- id: "FF_EDGE_CASES_EXTERNAL_EVENTS"
  source_section: "old_playbook/external_history_event_notes"
  reason: "edge cases not fully formalized yet"
  bound_to:
    - "PATH.A4"
  risk: "medium"
  tests_required: true
```

Ledger потрібен для того, щоб не втратити контроль над залишковим людським текстом.

FREEFORM Coverage

Для FREEFORM теж потрібне покриття.

Coverage має відповідати на питання:

- скільки FREEFORM-блоків є в програмі;
- які з них прив'язані до paths/nodes/gates;
- які мають tests;
- які мають known risks;
- які треба формалізувати пізніше;
- які вже породжували user feedback або problems.

Приклад:

```
freeform_coverage:
  total_blocks: 5
  bound_blocks: 5
  tested_blocks: 3
  untested_blocks:
    - "FF_EDGE_CASES_EXTERNAL_EVENTS"
    - "FF_LEGACY_STATUS_NOTES"

  high_risk_blocks:
    - "FF_LEGACY_STATUS_NOTES"

  recommended_actions:
    - "formalize status notes into STATUS.SEMANTICS"
    - "add regression tests for external event edge cases"
```

FREEFORM і Debug Layer

У debug mode Ordo має показувати, коли рішення спиралося на FREEFORM.

Наприклад:

```
trace_source: "model_self_report"
knowledge_trace:
  - source_type: "freeform"
    id: "FF_EDGE_CASES_EXTERNAL_EVENTS"
    used_for: "path disambiguation"
    risk: "medium"
```

Це важливо, бо якщо неправильне рішення було прийняте на основі FREEFORM, ми бачимо, який саме блок треба покращувати.

Без цього модель може сказати:

я так зрозуміла інструкцію

А Ordo має сказати точніше:

рішення було прийнято з використанням FREEFORM-блоку FF_EDGE_CASES_EXTERNAL_EVENTS, який має medium risk і не має regression test.

FREEFORM і Improvement Loop

Якщо користувач вказує на проблему, пов'язану з FREEFORM, Ordo має створити improvement record.

```
improvement_record:
  type: "freeform_caused_ambiguity"
  affected_unit:
    kind: "freeform"
    id: "FF_LEGACY_STATUS_NOTES"
  problem:
    description: "status rule was interpreted inconsistently"
  proposed_patch:
    - "extract status meanings into STATUS.SEMANTICS"
    - "leave only explanation in FREEFORM"
  suggested_tests:
    - "TC_STATUS_READY_FOR_FIRST_RUN"
```

Це робить FREEFORM не просто “залишковим текстом”, а керованою частиною мови.

Як вирішити, що формалізувати далі

Після міграції не потрібно одразу формалізувати все. Але потрібно мати критерії.

Формалізувати далі варто, якщо FREEFORM-блок:

- часто використовується для прийняття рішень;
- впливає на gates;
- впливає на status;
- часто викликає помилки;
- має різні трактування;
- потрібен для regression scenarios;
- повторюється в багатьох domain packs;
- може стати reusable library.

Залишити у FREEFORM можна, якщо блок:

- лише пояснює контекст;
- не змінює path;
- не блокує output;
- не визначає required fields;
- не має конфлікту з structured rules;
- легко перевіряється людиною.

Типові помилки

Помилка 1. Вважати FREEFORM смітником

FREEFORM — це не місце для всього, що лінє формалізувати.

Помилка 2. Ховати gates у тексті

Якщо правило блокує дію, воно має бути gate.

Помилка 3. Не вести FREEFORM ledger

Без ledger неможливо зрозуміти, що залишилось неформалізованим.

Помилка 4. Не тестувати risky FREEFORM

Якщо FREEFORM впливає на рішення, потрібні tests.

Помилка 5. Не повертатися до FREEFORM після feedback

Якщо користувач кілька разів вказує на проблему, FREEFORM треба покращувати або формалізувати.

Міні-вправа

Візьміть будь-який довгий документ з інструкціями.

Випишіть п'ять фрагментів, які складно формалізувати.

Для кожного визначте:

1. Це пояснення чи правило?
2. Воно впливає на path?
3. Воно блокує output?
4. Воно має бути gate?
5. Воно потребує test?
6. Його можна залишити у FREEFORM?

Короткий підсумок

Після міграції playbook-а в Ordo важливо не тільки показати, що було формалізовано. Не менш важливо показати, що залишилось у FREEFORM.

FREEFORM має бути контрольованим, прив'язаним до конкретних частин програми, покритим tests там, де це потрібно, і видимим для debug та improvement loop.

Добрий FREEFORM — це не хаос. Це чесно позначена зона людського знання, яка ще не стала повністю формальною, але вже керується правилами Ordo.

Розділ 34. Мінімальні вимоги до автора

Навіщо це потрібно

Ordo дає змогу перетворювати людські інструкції на керований процес для AI-моделі. Але це не означає, що будь-який довгий текст автоматично стає доброю Ordo-програмою. Якщо автор не розуміє, що саме він описує, де процес має зупинитися, які рішення модель може приймати самостійно, а які потребують підтвердження людини, то навіть найкращий синтаксис не врятує інструкцію.

Автор Ordo-програми — це не обов'язково програміст у класичному сенсі. Ним може бути аналітик, методолог, продуктовий спеціаліст, QA, бізнес-експерт або технічний письменник. Але він має мислити не просто текстом, а процесом.

Ordo не вимагає від автора вміння писати код на Java, Python або JavaScript. Проте вона вимагає дисципліни: бачити структуру задачі, відрізнити правило від прикладу, очікуваний результат від побажання, обов'язковий gate від рекомендації, підтверджений факт від припущення.

Автор Ordo-програми — це проєктувальник поведінки моделі

Коли людина пише звичайний prompt, вона часто формулює тільки бажаний результат:

```
Підготуй аналітичний пакет для нової історичної події.
```

Для Ordo цього недостатньо. Автор має описати не тільки результат, а й поведінку моделі:

```
- з чого почати;
- які дані зібрати;
- які питання поставити;
- які варіанти path існують;
- що вважати підтвердженням;
- де модель має зупинитися;
- що заборонено робити без approval;
- які outputs створити;
- які gates виконати перед handoff;
- які debug/test/improvement records мають підтримуватися.
```

Тобто автор Ordo-програми не просто пише інструкцію. Він проєктує керований процес виконання.

Мінімальна вимога 1. Розуміти intent

Автор має чітко розуміти, для чого існує Ordo-програма.

Погано:

```
Допомогти аналітику.
```

Краще:

```
Провести аналітика через guided intake нової історичної події, зібрати підтверджений contract, визначити path, сформувавши compact package і виконати validation gates перед handoff.
```

Intent має бути достатньо конкретним, щоб за ним можна було перевірити: програма виконала своє призначення чи ні.

Якщо intent розмитий, далі починаються проблеми: модель ставить не ті питання, переходить не в той path, створює не ті outputs і не розуміє, де треба зупинитися.

Мінімальна вимога 2. Відрізнити contract від контексту

Одна з типових помилок автора — змішувати обов’язкові умови з фоновим поясненням.

Contract — це те, без чого процес не має права рухатися далі.

Context — це те, що допомагає моделі краще зрозуміти ситуацію.

Наприклад:

```
Подія стосується зміни статусу компанії.
```

Це може бути context.

А ось це вже contract:

```
Перед генерацією пакета мають бути підтверджені:
- alias події;
- source row;
- source field;
- old/new values;
- path;
- expected output files.
```

Автор Ordo-програми має явно позначати, що є contract, а що лише поясненням. Якщо цього не зробити, модель може сприйняти важливу умову як необов’язкову пораду.

Мінімальна вимога 3. Визначати state

Якщо процес має декілька кроків, автор має описати state.

State — це пам’ять процесу: що вже відомо, що підтверджено, що ще очікує рішення, які gates пройдені, які assumptions відкриті.

Без state модель може повторно питати те, що вже було підтверджено, або навпаки — вважати підтвердженим те, що користувач лише обговорював.

Мінімальний state для складної Ordo-програми має містити:

```
- confirmed facts;
- assumptions;
- selected path;
- pending decisions;
- passed gates;
- blocked gates;
- selected outputs;
- generated artifacts;
- feedback/improvement records.
```

Автор не обов’язково має одразу описувати повну state machine. Але він має хоча б розуміти, які дані процес повинен пам’ятати між кроками.

Мінімальна вимога 4. Проєктувати nodes як питання, а не як хаотичний діалог

Ordo-програма часто працює як guided intake. Це означає, що модель не просто “розмовляє”, а проходить через вузли процесу.

Кожен node має мати зрозумілу роль:

- зібрати alias;
- визначити path;
- підтвердити source row;
- зібрати values;
- перевірити output contract;
- отримати approval;
- виконати validation;
- сформувати handoff.

Поганий node:

Запитай користувача все необхідне.

Кращий node:

```

NODE.DEF collect_source_field:
  goal: "Підтвердити поле джерела, зміна якого створює подію"
  ask: "Яке саме source field є тригером події?"
  required_answer: true
  updates_state:
    - source_field
  next:
    when confirmed: collect_old_new_values

```

Автор має вміти розбивати великий процес на логічні вузли.

Мінімальна вимога 5. Визначати gates

Gate — це місце, де процес має перевірити умову перед переходом далі.

Автор має вміти відповідати на питання:

Де модель не має права продовжувати без перевірки?

Наприклад:

- не створювати фінальний архів без self-check;
- не вважати contract confirmed без явного підтвердження;
- не генерувати code prompt без узгодженого output contract;
- не використовувати source row, якщо він не був підтверджений;
- не створювати HistoryEvent, якщо сценарій є no-op.

Якщо автор не визначить gates, модель буде діяти за ймовірністю: іноді правильно, іноді ні.

Ordo потрібна саме для того, щоб такі критичні точки були явно описані.

Мінімальна вимога 6. Писати негативні правила

Люди часто описують, що модель має зробити, але забувають описати, чого вона не має робити.

Для Ordo це критично.

Потрібні не тільки позитивні інструкції:

Створи QA package.

А й негативні:

```
ASSERT.NOT:
- do not create final archive before validation passed
- do not invent missing source values
- do not mark assumption as confirmed
- do not hide mandatory gate inside FREEFORM
```

Автор Ordo-програми має вміти бачити небезпечні дії моделі й прямо їх забороняти.

Мінімальна вимога 7. Визначати outputs

Output — це не просто “відповідь”. У складних процесах output може бути набором документів, JSON-структур, prompt-ів, validation reports, QA packages, changelog entries.

Автор має описати:

- які outputs мають бути створені;
- які outputs є обов’язковими;
- які outputs є допоміжними;
- які поля/секції мають бути всередині;
- які outputs не можна створювати до approval;
- як перевірити, що output готовий.

Погано:

Зроби пакет.

Краще:

```
OUTPUT.DEF compact_history_event_package:
  required_files:
    - README.md
    - SUMMARY.json
    - VALIDATION_REPORT.json
    - CONSISTENCY_CHECK_REPORT.json
    - 01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
    - 02_JIRA_TASK_<ALIAS>.md
    - 04_IMPLEMENTATION_PROMPT_<ALIAS>.md
    - 05_QA_PACKAGE_<ALIAS>.md
    - 07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md
    - 08_QA_AUTOMATION_SPEC_<ALIAS>.yaml
    - 09_QA_AUTOMATION_README_<ALIAS>.md
```

Output має бути перевірюваним.

Мінімальна вимога 8. Вміти залишати FREEFORM під контролем

Автор не повинен намагатися формалізувати все. Але він має розуміти, що FREEFORM — це не смітник для складних правил.

FREEFORM можна використовувати для:

- пояснень;
- прикладів;
- доменних нюансів;
- історичних нотаток;
- текстових шаблонів;
- складних людських формулювань.

Але не можна ховати в FREEFORM:

- blocking gates;
- required approvals;
- output contracts;
- state transitions;
- status semantics;
- negative assertions.

Автор має вміти вирішити: що формалізувати, а що залишити як контрольований FREEFORM.

Мінімальна вимога 9. Думати про debug і тести з самого початку

Ordo-програма без debug/test layer швидко стає важкою для розвитку.

Автор має хоча б мінімально визначити:

- які paths треба перевірити;
- які gates критичні;
- які сценарії мають бути по-ор;
- які помилки вже траплялися;
- які regression tests потрібні;
- які debug logs треба мати для пояснення рішення.

Це не означає, що кожна маленька Ordo-програма має мати велику test suite. Але для playbook-ів, domain packs і libraries тести мають бути обов'язковими.

Мінімальна вимога 10. Фіксувати improvement feedback

Автор Ordo-програми має бути готовий до того, що реальне використання покаже проблеми.

Користувач може сказати:

- це питання треба ставити раніше;
- тут потрібен gate;
- це правило не спрацювало;
- це треба винести в бібліотеку;
- це треба покрити regression test;
- тут FREEFORM треба формалізувати.

Усі такі зауваження мають ставати improvement records, а не губитися в чаті.

Автор має підтримувати цикл:

```
feedback → issue record → affected unit → patch suggestion → test suggestion → approval → regression
```

Саме так Ordo-програма стає кращою після реального використання.

Що автору не обов'язково вміти

Автору Ordo-програми не обов'язково:

- бути backend-розробником;
- писати compiler;
- знати всі IR op-коди;
- одразу створювати ідеальну структуру;
- формалізувати 100% доменної логіки;
- писати складний runner.

Але автор має вміти:

- бачити процес;
- відрізняти правило від прикладу;
- визначати точки зупинки;
- описувати очікувану поведінку;
- фіксувати стан;
- перевіряти outputs;
- приймати feedback і перетворювати його на покращення.

Типові помилки автора

Помилка 1. Писати Ordo як довгий prompt

Якщо документ виглядає як просто великий текст без nodes, gates, state і outputs, це ще не Ordo-програма.

Помилка 2. Не визначити, хто приймає рішення

Модель може допомагати, але не всі рішення вона має право приймати сама.

Потрібно явно вказувати:

- model_decision;
- user_approval;
- analyst_decision;
- blocked_until_confirmed.

Помилка 3. Не описати негативні сценарії

Якщо описані тільки успішні сценарії, модель може некоректно діяти в edge cases.

Помилка 4. Зробити FREEFORM занадто великим

Великий FREEFORM без coverage — це повернення до великого prompt-a.

Помилка 5. Не передбачити evolution

Ordo-програма майже завжди змінюватиметься. Якщо немає improvement records, changelog і regression tests, кожна зміна буде ризиком.

Міні-вправа

Візьміть будь-який складний prompt, яким ви вже користувалися, і спробуйте відповісти на питання:

1. Який intent цього prompt-a?
2. Який contract має бути підтверджений перед виконанням?
3. Який state потрібно пам'ятати?
4. Які nodes можна виділити?
5. Де потрібні gates?
6. Які ASSERT.NOT потрібні?
7. Який output має бути створений?
8. Що можна залишити у FREEFORM?
9. Який один debug trace допоміг би зрозуміти помилку?
10. Який один regression test потрібно додати?

Якщо ви можете відповісти на ці питання, ви вже мислите як автор Ordo-програми.

Короткий підсумок

Автор Ordo-програми — це не просто людина, яка пише інструкцію для AI. Це людина, яка проектує керовану поведінку моделі.

Мінімальні вимоги до автора:

- розуміти intent;
- відрізняти contract від context;
- описувати state;
- будувати nodes;
- визначати gates;
- писати ASSERT.NOT;
- задавати output contract;
- контролювати FREEFORM;
- думати про debug і tests;
- фіксувати improvement feedback.

Ordo не вимагає від автора бути програмістом. Але вона вимагає мислити процесом, перевітками і розвитком інструкції.

Розділ 35. Як не зламати playbook

Навіщо це потрібно

Великий playbook — це не просто довга інструкція. Це система правил, рішень, зупинок, перевірок, прикладів, винятків і очікуваних результатів. Коли така система росте, її легко зламати навіть доброю зміною.

Найчастіше playbook ламається не тому, що хтось зробив очевидну помилку. Частіше все виглядає невинно:

- додали нове уточнення;
- переставили блок вище;
- прибрали повторення;
- об'єднали два правила;
- перенесли частину тексту у FREEFORM;
- змінили формулювання gate;
- додали новий path;
- виправили один сценарій, але не перевірили інші.

Після цього модель раптом починає ставити питання не в тому порядку, пропускати approval, генерувати фінальний artifact занадто рано або плутати приклад із правилом.

Ordo потрібна саме для того, щоб такі зміни не були сліпими.

Просте пояснення

Зламати playbook — це означає порушити очікувану поведінку процесу.

Не обов'язково зламати весь документ. Достатньо зламати одну важливу властивість:

- модель пішла не тим path;
- gate перестав бути blocking;
- node почав питати зайві речі;
- state став оновлюватися раніше, ніж потрібно;
- output створюється без підтвердження;
- FREEFORM почав поводитися як приховане правило;
- бібліотека перезаписала локальне правило;
- тест проходить формально, але реальний artifact неправильний.

У звичайних інструкціях такі проблеми важко побачити. У Ordo кожна важлива частина playbook-а має бути пов'язана з path, node, gate, state, output, trace або test.

Тому головне правило просте:

Не змінюй playbook як текст. Змінюй його як систему виконання.

Що вважається небезпечною зміною

Не всі зміни однаково ризиковані.

Низький ризик:

- виправлення помилки в тексті;
- уточнення пояснення без зміни правил;
- додавання прикладу, який явно позначений як приклад;
- покращення назви розділу без зміни execution flow.

Середній ризик:

- додавання нового питання;
- зміна порядку node;
- уточнення condition;
- зміна output template;
- винесення частини логіки в library;
- зміна FREEFORM block.

Високий ризик:

- зміна path selection;
- зміна gate semantics;
- зміна status semantics;
- зміна blocking rule;
- зміна approval flow;
- зміна ASSERT.NOT;
- зміна compiler mapping;
- зміна domain pack rule;
- зміна reusable library, яку використовують кілька playbook-ів.

Для Ordo це означає: чим ближче зміна до execution behavior, тим сильніша потрібна перевірка.

Ordo-конструкція

Для безпечної зміни playbook-а в Ordo потрібен окремий change flow.

Мінімальна схема:

```
CHANGE.PROPOSE
→ IMPACT.ANALYZE
→ AFFECTED.UNIT
→ TEST.SELECT
→ REGRESSION.RUN
→ TRACE.COMPARE
→ HUMAN.APPROVE
→ VERSION.NOTE
```

Це означає, що зміна не має просто “вноситися в текст”. Вона має пройти через опис впливу.

Приклад:

```
change:
  id: "CH-001"
  type: "gate_semantics_update"
  summary: "Make package self-check gate blocking"

affected_units:
  - kind: "gate"
    id: "G_PACKAGE_SELF_CHECK"
  - kind: "assertion"
    id: "ASSERT_NO_ARCHIVE_BEFORE_SELF_CHECK"
  - kind: "test"
    id: "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"

risk:
  level: "high"
  reason: "Changes final package generation behavior"

required_checks:
  - "run_regression_suite"
  - "compare_debug_trace"
  - "validate_rendered_artifacts"
  - "human_approval"
```

Принцип 1. Не змінювати rule без test

Кожна зміна правила має або використовувати наявний тест, або створювати новий.

Погано:

```
Додали правило: перед архівом обов'язково робити self-check.
```

Краще:

```
Додали правило + додали test:
TC_NO_ARCHIVE_WITHOUT_SELF_CHECK
```

У Ordo це має бути майже автоматичною вимогою:

```
RULE.CHANGE requires TEST.DEF or TEST.UPDATE
```

Інакше зміна може виглядати правильно, але не буде захищена від регресії.

Принцип 2. Не змінювати gate без status semantics

Gate — це контрольна точка. Але gate має сенс тільки тоді, коли зрозуміло, що означають його статуси.

Наприклад:

```
passed
failed
blocked
pending
not_applicable
```

Якщо змінити gate, але не перевірити status semantics, може виникнути небезпечна ситуація: модель бачить gate, але не розуміє, чи треба зупинитися.

Тому правило:

```
GATE.CHANGE requires STATUS.SEMANTICS.CHECK
```

Особливо це важливо для blocking gates.

Принцип 3. Не ховати поведінку у FREEFORM

FREEFORM корисний, коли частину інструкції ще рано формалізувати. Але FREEFORM не має ставати місцем для прихованих gates, path selection або approval rules.

Погано:

```
У FREEFORM написано: "зазвичай перед архівом треба перевірити пакет".
```

Краще:

```
Gate:
  id: G_PACKAGE_SELF_CHECK
  blocking: true

FREEFORM:
  пояснює, як саме аналітик зазвичай перевіряє пакет.
```

Тобто FREEFORM може пояснювати правило, але не має замінювати саме правило.

Принцип 4. Не робити `implicit override`

Коли `playbook` використовує `libraries`, `profiles` або `domain packs`, легко випадково перезаписати поведінку.

Наприклад, бібліотека має `gate`:

```
G_CONTRACT_CONFIRMED
```

А локальний `playbook` додає `gate` з такою самою назвою, але з іншою логікою.

Ordo не має мовчки приймати це.

Правильно:

```
override:
  target: "contract_first.G_CONTRACT_CONFIRMED"
  allow: true
  reason: "Domain pack requires additional source row confirmation"
  approved_by: "human"
```

Без явного `override` зміна має бути заблокована або позначена як `conflict`.

Принцип 5. Перевіряти не тільки `template`, а `rendered artifact`

Одна з типових помилок — перевірити шаблон, але не перевірити готовий документ.

Наприклад, `template` може містити правильний блок, але у фінальному `artifact` цей блок не з'явився через помилку `render step`.

Тому в Ordo має бути правило:

```
Artifact is valid only after rendered artifact validation.
```

Не достатньо сказати:

```
шаблон має секцію self-check
```

Потрібно перевірити:

```
готовий файл має секцію self-check у правильному місці і з правильним змістом
```

Принцип 6. Порівнювати `trace` до і після зміни

Для складних `playbook`-ів недостатньо знати, що тести пройшли. Потрібно ще бачити, чи змінився шлях виконання.

До зміни:

```
input → path A1 → node collect_alias → gate contract_confirmed → output draft
```

Після зміни:

```
input → path A1 → node collect_source_field → node collect_alias → gate contract_confirmed →
output draft
```

Це може бути правильна зміна. Але вона має бути видимою.

Тому Ordo має підтримувати:

```
TRACE.COMPARE
```

Trace compare показує:

- які path змінилися;
- які nodes додані або прибрані;
- які gates змінили статус;
- які state fields змінилися;
- які outputs змінили структуру;
- які warnings з'явилися або зникли.

Малий приклад

Уявімо, що playbook для історичних подій іноді створює фінальний архів без self-validation.

Погане виправлення:

Додати в текст: "Не забувай зробити перевірку перед архівом".

Таке формулювання може знову загубитися.

Краще Ordo-виправлення:

```
assert_not:
  id: "ASSERT_NO_ARCHIVE_BEFORE_VALIDATION"
  method: mechanical
  trust_class: deterministic
  condition: "final_archive_created == true and validation_passed != true"
  severity: "blocking"

expected_behavior:
  if_assertion_triggered:
    action: "stop"
    message: "Cannot create final archive before validation is passed."

test:
  id: "TC_NO_ARCHIVE_BEFORE_VALIDATION"
  fixture:
    user_message: "створи архів одразу"
  expected:
    output:
      final_archive_created: false
    gate:
      id: "G_VALIDATION_BEFORE_ARCHIVE"
      status: "blocked"
```

Тепер це не просто порада. Це частина execution contract.

Типові помилки

Перша помилка — редагувати playbook як статтю.

Якщо playbook сприймати як текст, автор буде думати про красу формулювань. Якщо сприймати його як Ordo-програму, автор буде думати про поведінку моделі.

Друга помилка — додавати правила без tests.

Правило без тесту легко зламати через два дні, навіть якщо сьогодні воно виглядає очевидним.

Третя помилка — не відрізняти приклад від правила.

Приклади допомагають моделі, але якщо вони не позначені явно, модель може почати виконувати приклад як обов'язковий сценарій.

Четверта помилка — вважати, що якщо документ став коротшим, то він став кращим.

Стиснення може знищити важливі gates, warnings або domain-specific винятки.

П'ята помилка — не перевіряти старі сценарії після нового покращення.

Багато змін виглядають правильними локально, але ламають сусідній path.

Міні-вправа

Візьміть будь-який складний playbook або інструкцію і виберіть одну зміну, яку хочеться внести.

Перед зміною запишіть:

1. Яку поведінку ця зміна має покращити?
2. Який path або node вона зачіпає?
3. Який gate може змінити статус?
4. Який state field може змінитися?
5. Який output може змінитися?
6. Який test потрібно додати?
7. Який regression test потрібно прогнати?
8. Чи потрібен human approval?

Якщо на ці питання немає відповідей, зміна ще не готова.

Короткий підсумок

Playbook ламається тоді, коли його змінюють як текст, а не як систему виконання.

Щоб не зламати playbook, Ordo вимагає:

- описувати impact зміни;
- прив'язувати зміну до affected units;
- не змінювати rule без test;
- не змінювати gate без status semantics;
- не ховати поведінку у FREEFORM;
- не робити implicit override;
- перевіряти rendered artifact;
- порівнювати trace до і після зміни;
- запускати regression suite;
- фіксувати version note.

Без цього будь-яке покращення може випадково стати новою помилкою.

Розділ 36. Як оцінювати покриття Ordo

Навіщо це потрібно

Коли Ordo-програма стає великою, вже недостатньо сказати: “у нас є правила”, “у нас є gates”, “у нас є тести”. Потрібно розуміти, яка частина цієї програми реально покрита структурою, тестами, debug trace і контрольованими механізмами перевірки.

У звичайному програмуванні для цього є поняття test coverage. Воно показує, які частини коду перевірені тестами. В Ordo потрібне ширше поняття, тому що Ordo-програма складається не тільки з коду або команд. Вона має intent, contract, nodes, paths, gates, outputs, state, FREEFORM, libraries, domain rules і human approval points.

Тому coverage в Ordo має відповідати не на одне питання, а на декілька:

Які частини процесу формалізовані?
 Які частини перевірені тестами?
 Які частини видно в debug trace?
 Які частини залишились у FREEFORM?
 Які paths не мають regression tests?
 Які gates існують у тексті, але не enforced у IR?
 Які outputs перевіряються тільки за шаблоном, але не за rendered artifact?

Без такого покриття playbook може виглядати сильним, але реально мати великі “сліпі зони”.

Просте пояснення

Покриття Ordo — це карта того, наскільки добре ми контролюємо поведінку моделі.

Якщо процес описаний тільки у prose, але не має nodes, gates і tests, його coverage низький.

Якщо процес має nodes і gates, але немає expected behavior tests, coverage середній.

Якщо процес має paths, gates, state checks, no-op tests, regression suite, FREEFORM coverage і rendered artifact validation, coverage високий.

Тобто coverage — це не про красу документації. Це про ступінь керованості.

Що саме потрібно оцінювати

У Ordo треба оцінювати покриття щонайменше на таких рівнях:

1. Intent coverage
2. Contract coverage
3. Path coverage
4. Node coverage
5. Gate coverage
6. State coverage
7. Output coverage
8. Negative assertion coverage
9. No-op coverage
10. FREEFORM coverage
11. Library coverage
12. Domain rule coverage
13. Rendered artifact coverage
14. Regression coverage
15. Feedback coverage

Це не означає, що кожна проста Ordo-програма має мати всі ці рівні. Але для великих playbook-ів і production workflows вони мають бути видимими.

Path coverage

Path coverage показує, чи всі можливі маршрути процесу мають хоча б один test case.

Наприклад, якщо History Event Playbook має п'ять основних paths, але тести є тільки для трьох, coverage неповний.

```
coverage:
  paths:
    total: 5
    covered: 3
    uncovered:
      - "A4_EXTERNAL_HISTORY_EVENT"
      - "A5_NO_EVENT"
```

Небезпека uncovered path у тому, що модель може формально знати про цей шлях, але ніколи не була перевірена на його реальному проходженні.

Gate coverage

Gate coverage показує, які контрольні точки реально перевіряються.

Недостатньо мати gate у документації. Потрібно знати:

- чи gate є в IR;
- чи gate має статус;
- чи gate blocking або warning;
- чи gate має test;
- чи є негативний test, де gate має заблокувати дію;
- чи gate видно в GATE.REPORT.

Приклад:

```
coverage:
  gates:
    total: 14
    covered_by_tests: 11
    blocking_gates_without_negative_tests:
      - "G_PRE_ARCHIVE_APPROVAL"
      - "G_RENDERED_ARTIFACT_VALIDATED"
```

Якщо blocking gate не має негативного тесту, це ризик. Модель може пройти його у звичайному сценарії, але невідомо, чи вона правильно зупиниться, коли gate не виконаний.

State coverage

State coverage показує, чи перевіряються ключові зміни стану.

У складних процесах саме state часто стає джерелом помилок. Модель може вважати щось підтвердженням, хоча користувач цього не підтверджував. Або може забути, що попередній status означав only_draft, а не ready_for_implementation.

Тому важливо перевіряти state до і після критичних кроків:

```
expect:
  state:
```

```
before:
  contract_confirmed: false
after:
  contract_confirmed: true
forbidden:
  final_archive_created: true
```

State coverage особливо важливий для approval, package generation, status semantics і improvement loop.

Output coverage

Output coverage показує, чи перевіряється не тільки факт створення результату, а і його зміст, структура та відповідність contract.

Для Ordo output test може перевіряти:

- чи створений правильний artifact;
- чи немає зайвих файлів;
- чи є mandatory sections;
- чи відповідає rendered artifact шаблону;
- чи не додано непідтверджені дані;
- чи є handoff note;
- чи є validation report;
- чи правильно відображені статуси.

Саме тут важливо відрізнити template validation від rendered artifact validation. Шаблон може бути правильним, але фінальний документ може бути зібраний неправильно.

Negative assertion coverage

Negative assertion coverage показує, чи перевірені заборони.

У Ordo важливо тестувати не тільки те, що модель має зробити, а й те, чого вона не має робити.

Наприклад:

```
expect_not:
  - "invent_missing_source_row"
  - "generate_final_archive_before_approval"
  - "treat_example_as_rule"
  - "apply_feedback_without_human_approval"
```

Якщо немає negative tests, модель може начебто виконувати процес, але робити небезпечні побічні дії.

No-op coverage

No-op coverage потрібен для сценаріїв, де правильна дія — не робити нічого.

Це дуже важливо для історичних подій, моніторингових подій, змін у даних і будь-яких процесів, де не кожен input має породжувати output.

Приклад:

```
test:
  id: "TC_NO_EVENT_FOR_UNCHANGED_VALUE"

expected:
  noop: true
```

```
history_event_created: false
change_record_created: false
```

Без no-op tests модель може створювати події там, де їх не має бути.

FREEFORM coverage

FREEFORM не означає “неперевірене”. Якщо FREEFORM використовується у важливому процесі, треба знати:

- який block FREEFORM існує;
- для чого він використовується;
- які decisions на нього спираються;
- чи є tests для сценаріїв із цього block;
- чи були problems/improvements, пов’язані з ним;
- чи можна частину FREEFORM формалізувати.

Якщо великий відсоток critical behavior живе у FREEFORM і не має tests, coverage низький, навіть якщо загальна документація виглядає повною.

Library coverage

Після появи Ordo Libraries треба перевіряти не тільки локальний playbook, а й підключені бібліотеки.

Потрібно знати:

- які libraries підключені;
- які exports реально використовуються;
- які versions зафіксовані;
- які library gates покриті тестами;
- чи є compatibility tests;
- чи є conflict resolution tests;
- чи є regression suite для library update.

Наприклад, якщо playbook підключає ordo.validation.contract_first, coverage має показати, які саме rules/gates із цієї бібліотеки використані й перевірені.

Feedback coverage

Feedback coverage показує, чи перетворюються реальні зауваження користувача на structured improvement records.

Якщо користувач п’ять разів вказував на одну проблему, але жодного IMPROVEMENT.RECORD не створено, Ordo-процес втрачає знання.

Coverage має показувати:

- скільки feedback records зібрано;
- скільки з них прив’язано до affected unit;
- скільки має proposed patch;
- скільки має suggested test;
- скільки очікує human approval;
- скільки закрито через regression suite.

Це робить розвиток playbook-а керованим, а не хаотичним.

Приклад COVERAGE.REPORT

```
coverage_report:
  id: "CR_HISTORY_EVENT_PLAYBOOK_001"
```

```

paths:
  total: 5
  covered: 4
  uncovered:
    - "A4_EXTERNAL_HISTORY_EVENT"

nodes:
  total: 18
  covered: 15
  uncovered:
    - "NODE_EXTERNAL_EVENT_MAPPING"

gates:
  total: 14
  covered: 12
  missing_negative_tests:
    - "G_PRE_ARCHIVE_APPROVAL"

state:
  tracked_fields: 22
  fields_with_expectations: 17
  untested_critical_fields:
    - "final_archive_created"
    - "rendered_artifact_validated"

outputs:
  total: 11
  rendered_validation_covered: 9
  missing_rendered_validation:
    - "07_PROCESS_IMPROVEMENT_FEEDBACK"

freeform:
  total_blocks: 5
  tested_blocks: 3
  formalization_candidates:
    - "FF_EDGE_CASE_DECISIONS"

libraries:
  included: 2
  version_pinned: 2
  exports_used: 7
  exports_with_tests: 5

feedback:
  records_total: 8
  linked_to_affected_unit: 6
  with_suggested_tests: 5
  pending_approval: 3

overall:
  status: "partial"
  risk_level: "medium"
  recommendation: "add tests for A4, negative approval gate, and rendered validation for
missing output"

```

Типові помилки

Перша помилка — рахувати coverage тільки за кількістю тестів.

У Ordo один тест може перевіряти мало, а інший — критичний gate, state transition і output contract. Важлива не тільки кількість, а й те, що саме покрито.

Друга помилка — не рахувати FREEFORM.

Якщо FREEFORM впливає на рішення моделі, він має бути видимим у coverage.

Третя помилка — не рахувати negative tests.

Процес може вміти щось створювати, але не вміти зупинятися.

Четверта помилка — не рахувати no-op scenarios.

Для багатьох бізнесових процесів правильна поведінка часто полягає саме в тому, щоб не створювати зайвий результат.

П'ята помилка — не оновлювати coverage після зміни library або domain pack.

Reusable рішення може змінити поведінку багатьох playbook-ів одночасно.

Міні-вправа

Візьміть будь-який свій playbook і складіть простий coverage report.

Заповніть хоча б такі пункти:

1. Скільки є основних paths?
2. Для скількох paths є tests?
3. Які gates є blocking?
4. Чи є для них negative tests?
5. Які state fields критичні?
6. Чи перевіряється rendered artifact?
7. Які частини залишаються у FREEFORM?
8. Чи є no-op tests?
9. Чи є feedback records?
10. Який найбільший uncovered risk?

Після цього виберіть одну uncovered частину і сформулюйте test, який її закриває.

Короткий підсумок

Покриття Ordo — це оцінка того, наскільки поведінка моделі справді керована.

Добрий coverage report має показувати не тільки те, що вже працює, а й те, де процес ще небезпечний або непрозорий.

Для великих Ordo-програм потрібно оцінювати:

- paths;
- nodes;
- gates;
- state;
- outputs;
- negative assertions;
- no-op scenarios;
- FREEFORM;
- libraries;
- domain rules;
- rendered artifacts;
- regression suite;
- feedback records.

Ordo-програма без coverage може виглядати завершеною, але залишатися непередбачуваною. Ordo-програма з coverage стає не тільки виконуваною, а й контрольованою.

Розділ 37. Ordo без runtime

Навіщо це потрібно

Коли ми говоримо про Ordo, легко уявити складну систему: compiler, runner, execution engine, test framework, debug console, бібліотеки, coverage reports і повноцінну інфраструктуру. У майбутньому це справді може бути так. Але важливо розуміти: Ordo не починається з runtime.

Ordo може бути корисною ще до появи окремого технічного середовища виконання.

На першому практичному рівні Ordo може працювати як дисциплінований формат інструкцій для моделі. Тобто автор готує Ordo Source або Ordo-подібний документ, передає його моделі, а модель виконує його як структурований execution contract.

У цьому режимі немає зовнішнього runner-а, який технічно блокує неправильні дії. Немає окремого engine, який автоматично рахує coverage або порівнює state snapshots. Але все одно є головне: інструкція вже організована не як довгий prompt, а як керований процес.

Це важливий стартовий режим, бо він дозволяє використовувати Ordo вже зараз.

Просте пояснення

Ordo без runtime — це коли Ordo-програма існує як структурована інструкція, а роль виконавця бере на себе сама модель.

Модель читає:

```
- який intent;
- який contract;
- який context;
- який state потрібно вести;
- які nodes проходити;
- які gates перевіряти;
- які outputs створювати;
- де потрібно зупинитися;
- що не можна робити.
```

І після цього намагається виконувати процес згідно з цими правилами.

Тобто Ordo без runtime — це не автоматичне виконання в строгому програмному сенсі. Це контрольоване виконання через мовну дисципліну.

Чим це відрізняється від звичайного prompt-а

Звичайний prompt часто виглядає так:

```
Ось велика інструкція. Прочитай її уважно і зроби задачу правильно.
```

Проблема в тому, що модель може не зрозуміти, які частини є обов'язковими, які прикладами, які попередженнями, а які просто поясненнями.

Ordo без runtime формулює інструкцію інакше:

```

Ось execution contract.
Спочатку визнач intent.
Потім підтвердь contract.
Потім вежи state.
Потім рухайся тільки дозволеним path.
Перед output перевіряй gates.
Якщо gate не пройдений – зупинись.
Якщо правило не формалізоване – використовуй controlled FREEFORM.
Після завершення дай handoff.

```

Навіть без технічного runner-а така структура значно зменшує хаос.

Мінімальний формат Ordo без runtime

Найпростіша Ordo-програма без runtime може мати такий вигляд:

```

ordo:
  version: "0.11"
  mode: "manual_model_execution"

intent:
  goal: "підготувати аналітичний пакет для нової історичної події"

contract:
  required_confirmations:
    - "alias"
    - "source row"
    - "field mapping"
    - "old/new values"

state:
  fields:
    alias: null
    source_row_confirmed: false
    values_confirmed: false
    package_ready: false

nodes:
  - id: "N1"
    question: "Який alias події?"
    writes_to: "state.alias"

  - id: "N2"
    question: "Яке source field змінюється?"
    requires:
      - "state.alias != null"

outputs:
  - id: "O_FINAL_PACKAGE"
    allowed_when:
      - "G_CONTRACT_CONFIRMED == passed"
      - "G_PRE_ARCHIVE_APPROVAL == passed"

gates:
  - id: "G_PRE_ARCHIVE_APPROVAL"
    type: "blocking"
    rule: "не створювати фінальний пакет без явного підтвердження користувача"

```

Це ще не runtime. Але це вже не хаотичний текст.

Роль моделі в цьому режимі

У режимі без runtime модель має виконувати одразу кілька ролей:

- читати Ordo Source;
- вести state у своїй відповіді або внутрішньому контексті;
- не переходити через gates без підстав;

- ставити питання за поточним node;
- не генерувати outputs раніше часу;
- пояснювати, де вона знаходиться у процесі;
- фіксувати проблеми, якщо інструкція неоднозначна.

Це не ідеально, бо модель усе ще може помилитися. Але це вже набагато краще, ніж сподіватися, що вона “сама правильно зрозуміє” довгу інструкцію.

Обмеження режиму без runtime

Ordo без runtime має очевидні обмеження.

По-перше, gates не блокуються технічно. Вони блокуються тільки інструкцією. Якщо модель помилиться, вона може перейти далі попри заборону.

По-друге, state не зберігається зовнішньою системою. Він існує в контексті діалогу або в текстовому trace.

По-третє, тести не запускаються автоматично. Їх можна описати, але результат усе одно перевіряє модель або людина.

По-четверте, debug trace залежить від чесності й дисципліни моделі. Якщо модель не покаже причину рішення, зовнішнього механізму примусити її немає.

Тому Ordo без runtime — це хороший старт, але не фінальна форма для складних production-процесів.

Коли цього достатньо

Режим без runtime може бути достатнім для:

- навчання Ordo;
- написання перших playbook-ів;
- ручного guided intake;
- аналітичних задач з людиною в контурі;
- підготовки документації;
- проектування domain pack-ів;
- попередньої міграції старих інструкцій;
- перевірки концепції перед автоматизацією.

Іншими словами, це режим для старту, аналізу і ручного використання.

Коли runtime уже потрібен

Runtime стає потрібним, коли з’являються високі вимоги до повторюваності, контролю і доказовості.

Наприклад:

- playbook великий і має багато path-ів;
- є критичні blocking gates;
- потрібно гарантувати regression testing;
- outputs використовуються в production;
- є бібліотеки й версії;
- потрібна audit trail;
- потрібно порівнювати різні runs;
- потрібно автоматично збирати improvement records;
- є ризик дорогих помилок.

У таких випадках Ordo без runtime уже недостатньо. Потрібен helper-runner або повноцінний execution engine.

Типові помилки

Перша помилка — думати, що Ordo без runtime вже дає повну детермінованість.

Ні. Це ще не детермінована система. Це структурована домовленість із моделлю.

Друга помилка — не показувати state.

Якщо runtime немає, state потрібно хоча б явно вести в тексті або в компактному status-блоці.

Третя помилка — не відрізняти blocking gates від рекомендацій.

Навіть без runtime потрібно писати:

```
цей gate блокує подальший рух
```

а не просто:

```
бажано перевірити
```

Четверта помилка — не мати test cases.

Навіть якщо вони не запускаються автоматично, їх потрібно описати. Інакше автор не знатиме, чи зміна інструкції щось зламала.

П'ята помилка — не збирати feedback.

Якщо користувач постійно вказує на проблеми, але вони не перетворюються на improvement records, Ordo-програма не розвивається системно.

Міні-вправа

Візьміть одну свою звичайну довгу інструкцію для моделі й спробуйте перевести її в режим Ordo без runtime.

Виділіть:

```
- intent;
- contract;
- state;
- nodes;
- gates;
- outputs;
- ASSERT.NOT;
- debug information;
- expected tests;
- feedback records.
```

Після цього запитайте себе:

```
Чи стала інструкція зрозумілішою?
Чи видно, де модель має зупинитися?
Чи видно, що саме потрібно перевірити перед фінальним результатом?
Чи можна буде зрозуміти, чому процес пішов не так?
```

Короткий підсумок

Ordo без runtime — це перший практичний рівень використання Ordo.

У цьому режимі Ordo ще не виконується спеціальним engine, але вже структурує поведінку моделі через contract, state, nodes, gates, outputs і trace.

Це не заміна runtime, але важливий перехідний формат. Він дозволяє почати використовувати Ordo вже зараз, перевіряти ідеї, мігрувати старі playbook-и й поступово готувати основу для helper-runner або native model support.

Розділ 38. Ordo з helper-runner

Навіщо потрібен helper-runner

У попередньому розділі ми розглянули найпростіший спосіб використання Ordo: без окремого runtime, коли модель сама читає Ordo Source або Semantic JSON IR і намагається виконувати правила дисципліновано.

Цей режим корисний для старту. Але в складних процесах швидко з'являються межі.

Модель може забути gate. Може неявно змінити state. Може перескочити через node. Може сказати, що перевірка пройдена, хоча в trace немає доказу. Може не сформувати improvement record, хоча користувач прямо вказав на проблему.

Саме тут з'являється проміжний рівень:

Ordo з helper-runner

Helper-runner — це не повноцінна “розумна модель” і не заміна LLM. Це допоміжний виконавчий шар, який бере на себе контроль процесу.

Його завдання проста:

модель думає і формує зміст,
runner контролює порядок, state, gates, tests і trace.



Іншими словами, helper-runner робить Ordo не просто інструкцією для моделі, а виконуваним процесом із зовнішнім контролем.

Просте пояснення

Можна уявити Ordo без runtime як кухаря, який читає рецепт і сам вирішує, чи все зробив правильно.

А Ordo з helper-runner — це кухня, де є ще й технологічна карта, контрольні точки, журнал дій і контролер якості.

Кухар усе ще готує страву. Але система стежить, щоб він:

- не пропустив обов'язковий крок;
- не подав страву до перевірки;
- не змінив інгредієнт без дозволу;
- записав, що саме зробив;
- пояснив, чому обрав саме цей шлях.

У світі Ordo це означає:

- runner зберігає state;
- runner визначає current node;
- runner блокує перехід через blocking gate;
- runner передає моделі тільки релевантний фрагмент інструкції;
- runner збирає trace;

- runner запускає tests;
- runner формує gate report;
- runner збирає feedback records.

Модель залишається семантичним виконавцем. Runner стає процесним контролером.

Що саме робить helper-runner

Helper-runner може виконувати кілька ролей.

1. Завантаження Ordo-програми

Runner читає Ordo Source або compiled IR.

Наприклад:

- main Ordo program;
- included libraries;
- selected profile;
- selected domain pack;
- templates;
- gates;
- tests;
- freeform ledger.

Він не покладається на те, що модель сама правильно прочитає весь довгий документ. Він сам визначає, які частини потрібні для поточного кроку.

2. Контроль state

Runner зберігає state поза моделлю.

Це дуже важливо. Бо якщо state існує тільки в контексті діалогу, його легко втратити або непомітно змінити.

У runner-based режимі state може бути окремим JSON-об'єктом:

```
{
  "run_id": "RUN-001",
  "current_node": "NODE_COLLECT_CONTRACT",
  "confirmed": {
    "alias": true,
    "source_field": false
  },
  "outputs": {
    "final_package_created": false
  },
  "gates": {
    "G_CONTRACT_CONFIRMED": "blocked"
  }
}
```

Модель не має права просто сказати, що state змінився. Вона має запропонувати зміну, а runner має її застосувати або заблокувати.

3. Контроль переходів

Runner перевіряє, чи можна перейти з одного node в інший.

Наприклад, модель хоче перейти до package generation. Runner дивиться:

```

Чи підтверджено contract?
Чи пройдені approval gates?
Чи є source row?
Чи виконано self-check?
Чи немає ASSERT.NOT порушень?

```

Якщо ні, runner блокує перехід.

Це принципово відрізняється від звичайного prompt-a. У prompt-i модель “має пам’ятати”, що не можна переходити. У runner-based Ordo перехід технічно не дозволяється.



4. Формування task для моделі

Runner не обов’язково передає моделі всю Ordo-програму.

Він може передати тільки поточний execution slice:

```

model_task:
  current_node: "NODE_SOURCE_FIELD_CONFIRMATION"
  allowed_actions:
    - "ask_one_question"
    - "update_state_proposal"
  forbidden_actions:
    - "generate_final_package"
    - "mark_contract_confirmed_without_user_answer"
  context:
    known:
      alias: "LU_CHANGE_STATUS"
    missing:
      - "source_field"
  expected_output:
    type: "question"

```

Це зменшує ризик, що модель заплутається в довгому playbook-у.

5. Gate enforcement

Runner може робити gates реальними блокувальниками.

Наприклад:

```

gate:
  id: "G_PRE_ARCHIVE_APPROVAL"
  method: human
  trust_class: human_decision
  type: "blocking"
  condition:
    user_approved_archive_generation: true

```

Якщо умова не виконана, runner не дозволяє виконати дію:

```
generate_archive
```

Навіть якщо модель намагається це зробити.

6. Trace і audit

Runner може збирати повний trace незалежно від того, наскільки дисципліновано модель його описала.

Trace має містити:

```
- input;
- current node;
- selected path;
- rejected paths;
- state before;
- state after;
- gates;
- model proposals;
- runner decisions;
- warnings;
- violations;
- final outputs.
```

Це робить процес перевірюваним.

7. Test execution

Runner може запускати TEST.DEF.

Наприклад:

```
взяти fixture →
запустити Ordo-програму →
перевірити expected path →
перевірити expected gates →
перевірити forbidden output →
побудувати report.
```

Це вже набагато ближче до звичайного тестування програмного забезпечення.

8. Improvement capture

Runner може автоматично збирати зауваження користувача і перетворювати їх на IMPROVEMENT.RECORD.

Наприклад, якщо користувач каже:

```
Тут ти мав спитати source field раніше.
```

Runner може прив'язати це до:

```
- run_id;
- node;
- path;
- gate;
- instruction fragment;
- domain pack;
- library version.
```

Потім він може запропонувати patch і regression test.

Ordo-конструкція

У Source-форматі режим із helper-runner може виглядати так:

```

execution:
  mode: "normal"
  runtime:
    type: "helper_runner"
    responsibilities:
      - "state_management"
      - "node_routing"
      - "gate_enforcement"
      - "trace_capture"
      - "test_execution"
      - "feedback_capture"

state:
  storage: "external"
  format: "semantic_json"

gates:
  enforcement: "runner_blocking"

trace:
  trace_source: "model_self_report"
  required: true
  level: "decision"

model:
  role: "semantic_executor"
  allowed_to:
    - "interpret_context"
    - "propose_state_update"
    - "generate_candidate_output"
    - "explain_reasoning_summary"
  not_allowed_to:
    - "override_blocking_gate"
    - "silently_change_state"
    - "skip_required_node"

```

У compiled IR це може бути представлено як набір операцій:

```

[
  {
    "op": "RUNTIME.DEF",
    "runtime": "helper_runner",
    "responsibilities": [
      "state_management",
      "gate_enforcement",
      "trace_capture"
    ]
  },
  {
    "op": "STATE.EXTERNAL",
    "format": "semantic_json"
  },
  {
    "op": "GATE.ENFORCE",
    "mode": "runner_blocking"
  },
  {
    "op": "MODEL.ROLE",
    "role": "semantic_executor"
  }
]

```

Важлива межа відповідальності

Helper-runner не повинен робити вигляд, що він сам розуміє весь зміст доменної задачі.

Наприклад, у History Event Playbook runner може знати:

- які path-и існують;
- які gates треба пройти;
- які fields обов'язкові;
- який output не можна створювати раніше approval;
- які tests треба запустити.

Але модель усе ще потрібна для:

- пояснення бізнесового сенсу;
- формування людського тексту;
- аналізу неоднозначних формулювань;
- пропозиції назв;
- формування документації;
- узагальнення feedback.

Тобто runner не замінює модель. Він обмежує й організовує її роботу.



Малий приклад

Без helper-runner користувач пише:

Створи нову історичну подію.

Модель сама має згадати весь playbook, вибрати path, поставити питання, не перейти далі, не створити пакет раніше часу.

З helper-runner процес виглядає інакше.

Runner створює run:

```
{
  "run_id": "RUN-100",
  "current_node": "ENTRY_START",
  "state": {},
  "allowed_actions": ["classify_input", "ask_entry_question"]
}
```

Модель пропонує:

```
{
  "proposed_action": "ask_question",
  "question": "Який тип зміни потрібно перетворити на історичну подію?"
}
```

Runner перевіряє, що це дозволена дія, і віддає питання користувачу.

Потім користувач відповідає. Runner оновлює state. Модель пропонує path. Runner перевіряє gates. І тільки тоді процес рухається далі.

У результаті модель не “веде весь процес у голові”. Вона працює крок за кроком у межах, які контролює runner.

Чим helper-runner відрізняється від native Ordo

Helper-runner — це проміжний рівень.

Модель ще не підтримує Ordo нативно. Вона не має внутрішнього execution engine для Ordo. Але зовнішній runner допомагає їй виконувати Ordo-програму правильно.

Native Ordo означало б, що модель сама вміє:

- розуміти Ordo IR;
- виконувати gates;
- вести state;
- повертати trace;
- працювати з tests;
- підтримувати libraries;
- пояснювати decisions у стандартному форматі.

Helper-runner робить частину цієї роботи зовні.

Тому можна сказати:

Ordo без runtime — це дисциплінований prompt-based режим.
Ordo з helper-runner — це контрольований execution-assisted режим.
Native Ordo — це модель, яка сама підтримує Ordo як мову виконання.

Коли helper-runner особливо потрібен

Helper-runner потрібен, коли:

- процес має багато вузлів;
- gates мають бути blocking;
- є багато state transitions;
- є regression tests;
- потрібно підключати libraries;
- потрібно версіювати domain packs;
- потрібен audit trail;
- є production outputs;
- помилка може дорого коштувати;
- декілька людей або моделей працюють з одним playbook-ом.

Особливо він корисний для playbook-ів, які поступово ростуть і стають занадто великими для надійного виконання через один довгий prompt.

Типові помилки

Перша помилка — думати, що helper-runner має бути дуже складним із першої версії.

Ні. Мінімальний runner може робити тільки кілька речей:

- зберігати state;
- знати current node;
- блокувати critical gates;
- писати trace;
- запускати прості tests.

Цього вже достатньо, щоб сильно покращити керованість.

Друга помилка — віддати runner-у семантичні рішення, які має приймати модель або людина.

Runner не повинен вигадувати бізнесовий сенс. Він має контролювати процес.

Третя помилка — дозволити моделі напряду змінювати state.

Правильніше:

```

модель пропонує state update;
runner перевіряє;
runner застосовує або відхиляє.

```

Четверта помилка — не логувати відхилені дії.

Якщо модель спробувала перейти через gate, runner має записати це у trace. Інакше буде незрозуміло, чому процес був заблокований.

П'ята помилка — не пов'язати runner із improvement loop.

Якщо runner бачить повторювану проблему, це має ставати issue або improvement record.

Міні-вправа

Уявіть, що у вас є Ordo-playbook для підготовки аналітичного пакета.

Опишіть мінімальний helper-runner для нього.

Вкажіть:

- який state він має зберігати;
- які gates він має блокувати;
- які дії моделі дозволені;
- які дії моделі заборонені;
- який trace треба писати;
- які tests треба запускати;
- які user feedback потрібно перетворювати на improvement records.

Потім запитайте себе:

- Яка частина процесу має залишатися за моделлю?
- Яка частина процесу має контролюватися runner-ом?
- Яка частина процесу має вимагати людського підтвердження?

Короткий підсумок

Ordo з helper-runner — це практичний середній рівень між prompt-based використанням і native Ordo support у моделях.

У цьому режимі модель залишається семантичним виконавцем, але runner контролює state, path-и, gates, trace, tests і feedback records.

Це різко підвищує надійність складних Ordo-програм. Особливо там, де важливо не просто отримати відповідь, а довести, що процес був виконаний правильно, перевірено і з можливістю подальшого покращення.

Розділ 39. Ordo як native model language

Навіщо це потрібно

До цього моменту ми розглядали Ordo у двох практичних режимах.

Перший режим — Ordo без runtime. У цьому випадку Ordo є сильно структурованою інструкцією, яку модель читає як текст і намагається виконувати дисципліновано.

Другий режим — Ordo з helper-runner. Тут частину контролю бере на себе зовнішній виконавчий шар: він збирає state, перевіряє gates, пише trace, запускає tests і не дозволяє моделі перескочити через критичні зупинки.

Але стратегічна ціль Ordo більша. Ідеальний майбутній режим — це native model support.

Тобто модель не просто читає Ordo як текст, а розуміє її як власну мову виконання.

У такому режимі Ordo стає не “інструкцією для моделі”, а частиною того, як модель організовує своє мислення, виконання, перевірку і пояснення результату.

Просте пояснення

Звичайний prompt для моделі — це прохання.

Ordo без runtime — це структурований контракт.

Ordo з helper-runner — це контракт із зовнішнім контролером.

Ordo як native model language — це ситуація, коли модель сама має внутрішню підтримку таких конструкцій:

```
ENTRY.DEF
NODE.DEF
STATE.SCHEMA
GATE.REPORT
ASSERT.NOT
TRACE.LOG
TEST.DEF
IMPROVEMENT.RECORD
FREEFORM.COVERAGE
```

Тобто модель не просто бачить ці слова. Вона знає, що вони означають на рівні execution behavior.

Якщо вона бачить GATE.REPORT, вона розуміє, що має оцінити gate.

Якщо вона бачить ASSERT.NOT, вона розуміє, що певна дія прямо заборонена.

Якщо вона бачить STATE.DIFF, вона розуміє, що треба показати зміну стану.

Якщо вона бачить EXPECT.NOOP, вона розуміє, що тест очікує відсутність дії.

Чим native Ordo відрізняється від prompt-based Ordo

У prompt-based режимі модель може прочитати правило, але все одно пропустити його, якщо текст довгий, неоднозначний або конфліктний.

У native режимі Ordo-конструкції мають бути для моделі спеціальними виконуваними сигналами.

Наприклад, prompt-based модель бачить:

```
Перед фінальним архівом виконай self-check.
```

І може це пропустити.

Native Ordo-модель бачить:

```
gate:
  id: G_PACKAGE_SELF_CHECK
  method: mechanical
  trust_class: deterministic
  blocking: true
  required_before:
    - OUTPUT.FINAL_ARCHIVE
```

І має сприйняти це не як пораду, а як заблоковану умову переходу.

Ordo-конструкція

Для native support Ordo має мати чіткий compiled representation.

Наприклад:

```
{
  "ordo_version": "0.11",
  "execution_mode": "normal",
  "state_schema": {
    "contract_confirmed": "boolean",
    "self_check_passed": "boolean",
    "final_archive_created": "boolean"
  },
  "gates": [
    {
      "op": "GATE.DEF",
      "id": "G_PACKAGE_SELF_CHECK",
      "blocking": true,
      "required_before": ["OUTPUT.FINAL_ARCHIVE"],
      "condition": "self_check_passed == true"
    }
  ],
  "assertions": [
    {
      "op": "ASSERT.NOT",
      "id": "A_NO_ARCHIVE_BEFORE_SELF_CHECK",
      "forbid": "final_archive_created == true AND self_check_passed != true"
    }
  ]
}
```

Для людини це може виглядати як технічний JSON. Але для моделі з native Ordo support це має бути execution map.

Рівні підтримки Ordo моделлю

Раніше ми вже говорили про можливі рівні підтримки Ordo. Тут зафіксуємо їх у простішій формі.

```
L0 – модель не знає Ordo;
L1 – модель читає Ordo як структурований текст;
L2 – модель добре розуміє Ordo Source;
L3 – модель виконує Semantic JSON IR як execution contract;
L4 – модель має native Ordo support.
```

На рівні L0 Ordo все ще може бути корисною, бо вона дисциплінує prompt.

На рівні L1 модель краще дотримується структури.

На рівні L2 модель уже розуміє більшість Ordo-патернів.

На рівні L3 модель здатна виконувати IR як карту процесу.

На рівні L4 Ordo стає для моделі власною мовою керування поведінкою.

Чому це важливо

Native Ordo support дозволив би зменшити головну проблему складних інструкцій: модель не повинна кожного разу “вгадувати”, що важливіше.

Вона має бачити:

```
це мета;
це контракт;
це state;
це path;
це gate;
це заборонена дія;
це test;
це feedback record;
це output;
це handoff.
```

Тоді складна інструкція перестає бути довгим текстом. Вона стає програмою керування поведінкою.

Що native Ordo не вирішує

Native Ordo не робить модель абсолютно детермінованою.

Модель усе ще працює зі значеннями, мовою, контекстом, неоднозначністю і неповними даними.

Native Ordo не скасовує людське підтвердження.

Native Ordo не гарантує, що domain rule завжди правильний.

Native Ordo не замінює tests, debug і improvement loop.

Навпаки: що сильніша підтримка Ordo, то важливішими стають tests, trace і version control.

Типові помилки

Перша помилка — думати, що native Ordo означає “модель більше не помиляється”.

Це неправильно. Native Ordo означає, що помилки стають краще структурованими, видимими і перевірюваними.

Друга помилка — чекати native support і не використовувати Ordo зараз.

Ordo корисна вже зараз як structured instruction language. Native support — це стратегічна ціль, а не умова початку роботи.

Третя помилка — робити Ordo занадто схожою на звичайну мову програмування.

Ordo керує не процесором, а AI-моделлю. Тому вона має враховувати семантику, неоднозначність, контекст, людські підтвердження і пояснюваність.

Четверта помилка — сховати всю складність у compiler.

Навіть якщо compiler стане дуже сильним, людина має розуміти, які contracts, gates, tests і outputs він створює.

Міні-вправа

Візьміть будь-яку складну інструкцію для AI.

Запитайте себе:

Які частини цієї інструкції модель зараз читає просто як текст?
 Які з них мали б бути native Ordo-конструкціями?
 Де потрібен gate?
 Де потрібен ASSERT.NOT?
 Де потрібен STATE.DIFF?
 Де потрібен TEST.DEF?
 Де потрібен IMPROVEMENT.RECORD?

Після цього спробуйте описати, що модель мала б розуміти не як “текст”, а як execution contract.

Короткий підсумок

Ordo як native model language — це стратегічна ціль, у якій модель підтримує Ordo не як звичайний prompt, а як власну мову керування виконанням.

У такому режимі Ordo-конструкції стають для моделі execution signals: state, gates, assertions, tests, traces, outputs і improvement records.

Це не скасовує людський контроль і не робить AI безпомилковим. Але це переводить складні інструкції з рівня “довгого тексту” на рівень контрольованої, пояснюваної і тестованої поведінки.

Chapter 40. Process Rail: як Ordo тримає ШІ на маршруті

Ordo не є жорстким wizard-ом і не є просто великим prompt-ом. Його головна ідея — дати ШІ свободу думати й говорити, але не дозволити процесу розсипатися.

Process Rail — це опорна структура процесу. Вона тримає стан, маршрут, gates, обов'язкові рішення, повернення назад і правила результату.

У prompt-only підході ШІ може бути гнучким, але може забути крок або загубитися після зміни попередньої відповіді. У hardcoded wizard усе стабільно, але погано працюють відкриті відповіді, уточнення й складні рішення. Ordo займає проміжну позицію: живий ШІ-діалог плюс формальна Process Rail.

У режимі створення проекту РМ описує задачу природною мовою, а AI Ordo Developer перетворює це на Ordo YAML, перевіряє через CLI і компілює в Semantic JSON IR.

У режимі виконання AI Ordo Executor читає Semantic JSON IR, веде людину по процесу, використовує CLI як deterministic helper і пояснює людині не raw output інструментів, а зрозумілий стан процесу.

Коротка формула:

```
ШІ мислить і спілкується.  
Process Rail тримає процес.  
CLI перевіряє детерміновані частини.  
Semantic JSON IR є машинозчитуваною формою Process Rail.
```

Розділ 41. Contract → Artifact Coverage і Go/No-Go

Ordo потрібен не тільки для того, щоб провести людину через правильні питання. Його цінність з'являється тоді, коли підтверджені відповіді не губляться по дорозі до фінальних документів.

У звичайному процесі це часта проблема: аналітик підтвердив alias, назви події, джерело, поля, normalization, payload, тестову стратегію, але частина цього не потрапила в паспорт, Jira task або implementation prompt. Формально діалог пройшов успішно, але пакет став неповним.

Тому в Ordo вводиться окремий шар:

```
confirmed contracts
→ expected artifact coverage
→ generated artifacts
→ deterministic validation
→ consistency report
→ go/no-go decision
```

Contract

Contract — це не просто домовленість у чаті. Це структурований об'єкт, який має статус.

Наприклад:

```
{
  "kind": "contract",
  "id": "G_EVENT_IDENTITY_CONTRACT",
  "status": "confirmed",
  "fields": {
    "alias": {
      "value": "LU_CHANGE_CAPITAL",
      "status": "confirmed",
      "required": true
    },
    "name_uk": {
      "value": "Зміна статутного капіталу компанії",
      "status": "confirmed",
      "required": true
    }
  }
}
```

Поле може бути missing, candidate, proposed, confirmed, blocked або not_applicable. Це важливо: модель не має права подати candidate як підтверджене рішення.

Artifact requirement

Artifact requirement описує, де саме має з'явитися підтверджений контракт.

Наприклад, якщо підтверджено HistoryEvent output contract, його ключові поля мають бути не тільки в QA-пакеті, а також у паспорті, Jira task, implementation prompt і JSON-звітах.

```
{
  "kind": "artifact_requirement",
  "id": "REQ_HISTORY_EVENT_OUTPUT_IN_PASSPORT_AND_JIRA",
  "when": {
    "contract": "G_HISTORY_EVENT_OUTPUT_CONTRACT",
```

```

    "status": "confirmed"
  },
  "requires": [
    {
      "artifact": "01_HISTORY_EVENT_PASSPORT",
      "must_include_fields": ["type", "sub_type", "source", "group", "groupPriority", "isEdr"]
    },
    {
      "artifact": "02_JIRA_TASK",
      "must_include_fields": ["type", "sub_type", "source", "group", "groupPriority", "isEdr"]
    }
  ]
}

```

Чому compile недостатньо

compile може перевірити, що в Ordo package є правильні посилання: contract існує, artifact існує, requirement не посилається на невідомий id.

Але compile не може гарантувати, що згенерований markdown-файл реально містить потрібне поле. Для цього потрібна перевірка rendered artifacts.

Rendered artifact validation

Rendered artifact validation перевіряє вже створені файли:

```

01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
02_JIRA_TASK_<ALIAS>.md
04_IMPLEMENTATION_PROMPT_<ALIAS>.md
05_QA_PACKAGE_<ALIAS>.md
SUMMARY.json
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json

```

Цей шар має відповідати на питання:

```

Чи всі confirmed contracts реально присутні в потрібних документах?
Чи немає суперечностей між Passport, Jira, QA і JSON reports?
Чи не записано candidate/proposed значення як confirmed?
Чи не пропущено тестову стратегію?

```

Go/No-Go

Після всіх перевірок Ordo має видати коротке машинозчитуване рішення:

```

{
  "kind": "go_no_go",
  "status": "no_go_requires_artifact_fix",
  "blocking_issues": [
    {
      "code": "ORDO-COV-002",
      "message": "Confirmed contract field group is missing from Passport"
    }
  ],
  "warnings": []
}

```

Для людини це можна пояснити просто:

```

Пакет не готовий: підтверджене поле group не потрапило в паспорт історичної події.

```

Що це змінює для AI Ordo Developer

AI Ordo Developer більше не просто створює документи за шаблонами. Він має довести, що:

16. усі важливі contracts підтверджені;
17. для кожного confirmed contract є artifact coverage;
18. rendered documents не втратили підтверджені значення;
19. consistency report не містить blocking issues;
20. go/no-go decision дозволяє передавати пакет далі.

Це робить Ordo не просто мовою інструкцій, а мовою контрольованої передачі рішень у фінальні артефакти.

М46.2: перший виконуваний шар перевірки

На рівні М46.1 ми описали нові поняття як частину мови. На рівні М46.2 вони починають працювати в CLI, але ще обмежено: Ordo перевіряє не самі згенеровані файли, а декларативний маршрут від контракту до артефакту.

Це означає:

```
confirmed contract
→ artifact_requirement
→ required artifact
→ required contract fields
```

Якщо пакет каже, що контракт підтверджений, але не вказує, у яких артефактах він має бути відображений, ordo coverage має завершитися з помилкою.

Якщо artifact_requirement посилається на поле, якого немає в контракті, ordo compile має завершитися з помилкою. Це важливо: помилка в моделі покриття має ловитися до того, як аналітик або РМ отримає красиві, але неповні документи.

М46.2 ще не читає rendered Markdown або JSON. Це наступний шар. Поточний шар відповідає на простіше питання: чи сам Ordo package взагалі знає, які підтверджені контракти мають потрапити в які артефакти.

Практичний pipeline тепер виглядає так:

```
lint
→ compile      # reference checks для contract/artifact model
→ coverage     # completeness checks для confirmed contracts
→ validate-state
→ generate-output
→ validate-artifacts # наступний шар
→ consistency    # наступний шар
→ go-no-go       # фінальне рішення
```

М46.3: перевірка вже згенерованих артефактів

На рівні М46.3 з'являється перша виконувана команда для rendered artifact validation:

```
ordo validate-artifacts <package>
```

Вона читає не тільки Ordo Source, а й фактичні файли в `generated_outputs/`. Це важливо, бо `compile` і `coverage` можуть довести, що маршрут `contract` → `artifact` описаний, але вони ще не доводять, що готовий Markdown або JSON справді містить підтверджене значення.

Приклад проблеми:

```
G_EVENT_IDENTITY_CONTRACT.event_alias = LU_CHANGE_CAPITAL
але 02_JIRA_TASK_LU_CHANGE_CAPITAL.md не містить LU_CHANGE_CAPITAL
```

У такому випадку `validate-artifacts` має повернути `blocking issue` з кодом `ORDO-COV-002`.

Поточний шар M46.3 ще не є повним `semantic consistency engine`. Він виконує детерміновану перевірку наявності підтверджених значень у потрібних `rendered files`. Наступний шар — `consistency` — має перевіряти суперечності між Passport, Jira, QA, Prompt і JSON-звітами.

M46.4: Consistency report between generated artifacts

Після `validate-artifacts` Ordo має ще один рівень перевірки: `consistency`.

`validate-artifacts` відповідає на питання: “чи є підтверджене значення в потрібному документі?”.

`consistency` відповідає на інше питання: “чи всі згенеровані документи говорять одне й те саме про один і той самий підтверджений контракт?”.

Типовий приклад:

```
alias = LU_CHANGE_CAPITAL
```

Якщо Passport містить `LU_CHANGE_CAPITAL`, але Jira task містить інший `alias` або не містить `alias` взагалі, пакет не можна вважати готовим до передачі розробнику. У такому випадку `ordo consistency` має сформувати `CONSISTENCY_CHECK_REPORT.json` із `blocking issue`.

Мінімальний `pipeline` для аналітичного пакета тепер такий:

```
lint
→ compile
→ coverage
→ intake/run
→ generate-output
→ validate-output
→ validate-artifacts
→ consistency
```

Це не замінює рев'ю аналітика, але прибирає клас помилок, коли підтверджений контракт є в процесі, але в різних фінальних документах він відображений неповно або неузгоджено.

M46.5: фінальний go/no-go helper

Після `validate-artifacts` і `consistency` потрібна одна коротка відповідь: чи можна передавати `generated package` далі. Для цього додається команда:

```
ordo go-no-go <package>
```


Вона не замінює окремі перевірки, а збирає їх в один pipeline:

```
lint → compile → coverage → validate-state → validate-artifacts → consistency → go/no-go
```

Результатом є `reports/GO_NO_GO_REPORT.json`. Якщо є хоча б один blocking issue, команда повертає no-go статус і ненульовий exit code.

Важливо: це deterministic helper, а не виконання III-моделі чи бізнесового runtime. Команда відповідає тільки на питання: чи узгоджені Ordo source, підтверджені contracts, generated artifacts і consistency report.

M46.6: pre-release audit and state reuse

M46.6 не додає новий великий мовний шар. Це pre-release audit після M46.1–M46.5. Головна перевірка: чи вся нова лінія `contract → artifact → consistency → go/no-go` працює як єдиний helper pipeline.

Практичне уточнення M46.6: якщо guided intake уже був виконаний і пакет має `reports/intake_report.json`, команда `ordo go-no-go <package>` може використати цей state без повторного `--answers`. Це краще відповідає реальному процесу:

```
intake → generate-output → validate-artifacts → consistency → go-no-go
```

Це не змінює принцип: `go-no-go` залишається deterministic helper і не виконує AI model або бізнесовий runtime. Він лише перевіряє, чи підтверджені contracts дійшли до generated artifacts і чи артефакти не суперечать одне одному.

M46.7: чистий pre-release candidate

M46.7 не вводить нову семантику мови. Це крок консолідації перед pre-release: вихідний архів має містити source-файли, документацію, тести й package definitions, але не має нести за собою старі результати локальних запусків.

Практичне правило M46.7:

<code>compiled/</code>	generated by ordo compile
<code>reports/</code>	generated by helper commands
<code>runtime/</code>	generated by intake/run flows
<code>generated_outputs/</code>	generated by ordo generate-output

У source archive ці каталоги можуть містити лише `.gitkeep`. Реальні reports, compiled IR, runtime snapshots і generated documents мають створюватися поточним запуском CLI. Це підтримує головний принцип Ordo: не довіряти старому самозвіту, а відтворювати evidence з актуального source state.

Розділ 42. Зовнішній аудит pre-release пакета

Ordo будується навколо простої ідеї: не достатньо сказати, що процес валідний; потрібно показати, як саме це перевірити.

Саме тому після появи contract/artifact coverage, rendered artifact validation, consistency report і go/no-go pipeline потрібен ще один шар — зовнішній аудит пакета. Це не нова частина мови і не новий runtime. Це практичний спосіб передати пакет іншій людині або іншій AI-сесії і сказати: “не довіряй нашим звітам, перевір сам”.

Що таке external audit package

External audit package — це набір інструкцій для незалежної перевірки Ordo workspace.

Він відповідає на питання:

- що саме має бути в архіві;
- яких старих каталогів уже не повинно бути;
- які CLI-команди треба запустити;
- які reference packages мають пройти перевірки;
- як перевірити generated artifacts;
- як перевірити consistency і go/no-go;
- які обмеження пакета не можна приховувати.

Це важливо, бо Ordo не має перетворювати validation report на акт віри. Звіт має бути доказом, який можна повторити.

Чому цього недостатньо робити вручну

Якщо попросити reviewer-а просто “подивитися пакет”, кожен дивитиметься по-своєму. Один прочитає README. Інший запустить тільки lint. Третій подивиться на book source, але не перевірить CLI.

Тому M46.8 вводить не новий runtime, а стандартний маршрут перевірки:

```
archive structure
→ CLI install
→ repo-check
→ unit tests
→ active package checks
→ generated artifact validation
→ consistency
→ go-no-go
→ audit verdict
```

Це робить зовнішню перевірку повторюваною.

Що має перевірити reviewer

Reviewer має перевірити не тільки те, що код не падає. Він має перевірити відповідність обіцянки і поведінки.

Наприклад:

- якщо README каже, що legacy site/catalog/playbook root видалено, архів не має містити ці каталоги;
- якщо CLI каже, що ordo test — static mode, це має бути видно у виводі;
- якщо package має confirmed contracts, вони мають дійти до Passport, Jira, QA Package, Implementation Prompt і JSON reports;
- якщо go-no-go повертає go, має бути зрозуміло, які deterministic checks це підтвердили.

Чесна межа Ordo preview

External audit також має перевіряти чесність обмежень.

Поточний Ordo preview не виконує live AI reasoning, REST-виклики, Mongo-перевірки або production business runtime. CLI є deterministic helper layer. Він перевіряє структуру, references, coverage, rendered artifacts, consistency і go/no-go report.

Це не слабкість, якщо про це сказано прямо. Слабкість була б у тому, щоб називати static validation повноцінним production runtime.

Практичний результат

Після M46.8 Ordo package можна передати в іншу сесію з готовим prompt-ом аудиту. Reviewer має повернути короткий verdict:

```
go
no_go
go_with_warnings
```

і пояснити, які команди він запускав, які артефакти дивився, які blocking issues або warnings знайшов.

Це робить pre-release перевірку не одноразовою дією автора, а відтворюваним процесом.

Важливий порядок команд

ordo repo-check перевіряє чистоту source archive. Тому його потрібно запускати на чисто розпакованому пакеті до `pip install -e .` і до тестів, бажано як `PYTHONDONTWRITEBYTECODE=1` `ordo repo-check ..`, щоб сам запуск Python не створив `__pycache__`. Після `install` або `test` у дереві можуть з'явитися `__pycache__` та `egg-info`, і тоді source-hygiene перевірка справедливо впаде. Це не blocker, якщо перед фінальним пакуванням generated metadata знову прибрано.

M46.9: самопрогін external audit checklist

M46.9 не додає нової мовної логіки. Його задача — взяти checklist з M46.8 і прогнати пакет як pre-release candidate: структура архіву, CLI, active packages, generated artifact validation, consistency, go-no-go, документація і book source.

Важливий результат M46.9: audit не має довіряти `m*_validation_report.json` сам по собі. Він має або виконати команди, або явно зафіксувати, що команда не запускала. Для Ordo це принципово: self-report не є доказом без перевірки.

Під час M46.9 було виявлено дрібну documentation-hygiene розбіжність: checklist очікував файл `cli/docs/GO_NO_GO_M46_5.md`, тоді як canonical CLI doc уже називався `cli/docs/GO_NO_GO.md`. Це не runtime blocker, але для pre-release audit такі речі мають бути виправлені, бо reviewer не повинен здогадуватися, який документ правильний.

Практичне правило після M46.9:

```
repo-check → install CLI → tests → active packages → generated artifact flow → go-no-go → audit report
```

Якщо всі етапи проходять, пакет може отримати verdict go для source-available pre-release candidate. Це все ще не означає production runtime: Ordo CLI перевіряє deterministic structure, contracts, artifacts і consistency; він не виконує AI-модель, REST, Mongo або реальний бізнесовий backend.

Розділ 43. Final handoff: як передавати Ordo на зовнішню перевірку

Final handoff — це не нова функція мови. Це момент, коли пакет уже достатньо стабільний, щоб його міг перевірити інший розробник, аналітик або AI-сесія без участі автора.

Для Ordo це особливо важливо, бо сама ідея мови побудована на довірі до перевірок. Якщо пакет заявляє, що він release candidate, то зовнішній reviewer має швидко зрозуміти три речі: що перевіряти, як це запустити, і де побачити машинно-читаний результат.

У M48 handoff не додає нові CLI-команди й не змінює Process Rail. Він тільки збирає маршрут перевірки навколо вже наявних шарів: lint, compile, coverage, validate-state, validate-artifacts, consistency, go-no-go.

Головне правило final handoff просте: reviewer не повинен реконструювати історію всіх milestone-ів. Йому потрібна коротка дорога:

```
README → final_handoff → external_audit → active packages → go-no-go report
```

Тому у handoff-пакеті мають бути окремі документи:

- з чого почати;
- що саме входить у score;
- які команди запустити;
- які результати очікуються;
- які питання треба поставити після рев'ю.

Final handoff також фіксує межі чесності. Якщо ordo test є static runner, це має бути видно. Якщо пакет source-available, а не open-source, це має бути видно. Якщо PDF книги не перегеновано, це теж має бути явно сказано.

Ordo не повинен виглядати готовішим, ніж він є. Сильна сторона Process Rail — не в тому, щоб приховувати незавершеність, а в тому, щоб точно показувати, що вже пройшло перевірку, а що ще потребує рішення.

Розділ 44. Прийом зовнішнього feedback після review

Після того як Ordo package передано на зовнішню перевірку, найнебезпечніша помилка — одразу почати виправляти все підряд.

У Process Rail це поганий шлях. Feedback має спочатку стати структурованим фактом, а вже потім рішенням.

```
review finding → feedback item → triage → decision → milestone
```

Чому не можна виправляти одразу

Зовнішній reviewer може знайти справжній blocker, корисне покращення, непорозуміння в документації або ідею, яка виходить за межі поточного release.

Якщо все це одразу змішати з кодом, release candidate перестає бути стабільним. Тому M49 додає не нову runtime-логіку, а шар прийому feedback.

Feedback item

Кожне зауваження має бути записане як окремий item:

```
id
area
severity
evidence
recommended_action
decision
target_milestone
status
```

Це дозволяє відокремити факт від рішення.

Рішення

Для feedback використовуються прості статуси:

```
accepted
accepted_with_scope_limit
needs_more_evidence
deferred
rejected
not_applicable
```

Наприклад, якщо reviewer каже, що CLI падає в CI, це може бути blocker і accepted. Якщо reviewer пропонує повністю переписати мову, це може бути deferred або accepted_with_scope_limit.

Роль AI Ordo Developer

AI Ordo Developer не має одразу змінювати package. Спочатку він має:

1. розбити feedback на окремі findings;
2. класифікувати кожен finding;
3. записати evidence;
4. запропонувати decision;

5. назвати target milestone тільки для прийнятих пунктів.

Це продовжує головну ідею Ordo: модель може мислити гнучко, але процес має залишатися керованим.

Розділ 45. Feedback decision planning

Після зовнішнього рев'ю важливо не кидатися одразу переписувати код. Для Ordo важливіше інше: кожне зауваження має пройти через коротке рішення.

зауваження → класифікація → рішення → milestone → критерій прийняття → перевірка

Чому це окремий шар

Feedback intake збирає зауваження. Але сам факт, що зауваження існує, ще не означає, що його треба негайно вносити в release candidate.

M50 додає просте правило: спочатку рішення, потім зміна.

Типи рішень

```
accepted_now
accepted_next_milestone
deferred
rejected
needs_reproduction
needs_owner_decision
```

Це дозволяє не змішувати blocker, корисну ідею, документаційне уточнення і стратегічне рішення власника.

Що має бути в рішенні

Кожне рішення має містити:

- короткий опис;
- severity;
- affected layer;
- target milestone;
- rationale;
- acceptance criteria;
- validation commands.

Як це працює для Ordo

Ordo не просто приймає фідбек. Ordo переводить фідбек у керований change plan. Це продовження тієї ж ідеї Process Rail: модель може мислити гнучко, але процес не має втрачати трасу.

Важливе обмеження

M50 не змінює CLI, runtime або бізнесову логіку пакетів. Це decision layer для наступних змін.

Chapter 46 — Publication Readiness Decision

M51 is not a language feature. It is a release-readiness checkpoint.

By this point Ordo has moved from a broad source workspace into a cleaner Process Rail preview with deterministic validation layers: contract coverage, rendered artifact validation, cross-artifact consistency and go/no-go reports.

The publication-readiness question is therefore not “does the idea exist?” but “is the current package coherent enough to hand to another person or AI session for preview evaluation?”

M51 answers: yes, as a source-available preview candidate, but not as an open-source release.

The owner still decides where to publish, whether to publish the full source archive or only the Developer bundle, whether to regenerate PDF book artifacts, and whether to keep the current source-available preview posture or later choose a formal open-source license.

This keeps Ordo honest: readiness is recorded, but publication is not implied.

Розділ 47. Two-tier rendering model

До цього моменту Ordo вже вміє перевіряти, чи підтверджені контракти дійшли до generated artifacts. Але виникає інша проблема: не всі output-шаблони однакові.

Одні шаблони прості. Вони містять лише підстановки на кшталт `{{ state.alias }}` або `{{ state.list | bullets }}`. Такі шаблони може безпечно рендерити CLI.

Інші шаблони складні. Вони містять цикли, умови, таблиці, traceability matrix, nested YAML blocks або ручні QA/automation сценарії. Такі шаблони не повинен рендерити простий deterministic renderer.

Тому Ordo вводить two-tier rendering model.

Перший рівень: deterministic rendering

Deterministic template — це шаблон, який повністю підтримує CLI renderer `ordo.simple`.

```
render_mode: deterministic
renderer: ordo.simple
requires_model_rendering: false
```

У ньому дозволені прості підстановки:

```
{{ state.scalar }}
{{ state.list | bullets }}
{{ state.value | safe_name }}
{{ state.object | json }}
```

Але заборонені конструкції типу:

```
{% for %}
{% if %}
loop.index
.items()
| default("...")
```

Якщо така конструкція потрапляє у deterministic template, `ordo lint` або `ordo generate-output` має впасти. Це важливо: CLI не має вдавати, що він розуміє складні AI-шаблони.

Другий рівень: model-assisted rendering

Model-assisted template — це шаблон, який має рендерити AI model.

```
render_mode: model_assisted
renderer: ai.markdown
requires_model_rendering: true
validation: strict_confirmed_state_only
tbd_policy: preserve_tbd_until_confirmed
```

CLI не рендерить такий шаблон напряму. Замість цього він створює handoff packet:

```
runtime/model_assisted_render_handoff/<ARTIFACT_ID>.json
```

У цьому пакеті є:

- template content;
- confirmed state;
- expected output path;

- TBD policy;
- forbidden inference rules;
- post-validation requirements.

AI може заповнити складний Markdown/YAML/JSON artifact, але не має права вгадувати відсутні значення.

Головне правило

Model-assisted rendering може використовувати тільки confirmed state і explicit TBD defaults.

Якщо значення не підтверджене, воно має залишитися як:

△ TBD

AI не має права прибрати TBD тільки тому, що “здається очевидним”.

Post-validation

Після AI-rendering Ordo знову повертається в deterministic режим:

validate-artifacts → consistency → go-no-go

Перевіряється:

- чи не залишилися unresolved placeholders;
- чи валідний YAML/JSON;
- чи збігаються confirmed values зі state;
- чи не з'явилися inferred values;
- чи не прибрано TBD без підтвердження.

Two-tier rendering model робить межу чесною: модель може допомогти зі складним документом, але фінальна перевірка знову deterministic.

Стандартні помилки rendering layer

ORDO-RENDER-001 deterministic template contains unsupported syntax
 ORDO-RENDER-002 model-assisted template rendered by simple renderer
 ORDO-RENDER-003 model-assisted output contains inferred unconfirmed value
 ORDO-RENDER-004 TBD marker removed without confirmed state
 ORDO-RENDER-005 model-assisted YAML output is invalid
 ORDO-RENDER-006 model-assisted output not validated after rendering

Розділ 48. Runtime source of truth і чесність CLI

Після появи contract → artifact coverage і two-tier rendering Ordo отримав ще один важливий шар: runtime source of truth.

Ідея проста: Ordo package не можна виконувати як випадковий набір файлів. Його треба завантажувати як послідовний runtime:

```
ordo.yml → source/program.ordo.yml → compiled/program.ir.json → run_state.json →
generated_outputs/
```

ordo.yml є входом у пакет. source/program.ordo.yml є редагованим source of truth. compiled/program.ir.json є runtime-артефактом, з якого helper-команди мають брати Process Rail, вузли, gates і outputs. run_state.json або report із вкладеним state описує поточний стан виконання. Generated artifacts — це вже результат rendering.

Чому це потрібно

Без цього правила AI Ordo Developer може випадково вести guided intake “за пам’яттю”: почати не з того питання, пропустити gate або використати старі інструкції після зміни YAML.

M53 закриває цей клас помилок: якщо YAML новіший за compiled IR, runtime helper має блокувати виконання і просити перекомпілювати пакет.

```
ORDO-RUNTIME-004: IR is stale. Run ordo compile before guided execution.
```

Стандартний Developer workflow

```
runtime-status
lint
compile
test
coverage
validate-state
check-gate / next-step
generate-output
validate-output
validate-artifacts
consistency
go-no-go
package
```

Важливо: compile не означає, що фінальний пакет валідний. Він лише створює Semantic JSON IR. Фінальна готовність визначається після validate-artifacts, consistency і go-no-go.

Чесність CLI

Ordo також вводить правило truthfulness: модель не має писати “CLI validation passed”, якщо CLI реально не запускався.

Дозволені статуси:

```
executed_cli_passed
executed_cli_failed
logical_self_check_only
not_run_cli_unavailable
not_run_user_skipped
```

Це відділяє реальну перевірку від логічного припущення. Якщо CLI недоступний, це не помилка — але це треба чесно вказати.

Що це дає

M53 робить Developer Bundle ближчим до справжнього runtime-пакета: AI працює гнучко, але кожен критичний перехід перевіряється детерміновано.

Розділ 49. Runtime Guided Intake Entry Protocol

Runtime Guided Intake Entry Protocol — це правило, яке не дає моделі починати guided intake “з пам’яті” або з вільного читання файлів.

У ранніх версіях runtime стартував через `START_HERE_RUNTIME_MODE.md`, `ordo.yml` і `compiled IR`. Після M59/M60 enforced Runtime Mode став жорсткішим: у runtime-пакеті source YAML не потрібен, а `compiled/*` файли не читаються моделлю напряму.

Поточний маршрут

```
START_HERE_RUNTIME_MODE.md
→ cli_embedded/ordo runtime-entry .
→ cli_embedded/ordo next-step . --format auto
→ user answer
→ cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <answer_file>
→ evidence + snapshot + session-trace
→ next CLI-rendered step
```

Це означає: модель не вирішує сама, який node наступний. Вона отримує наступний крок із CLI.

Чому не можна читати compiled IR напряму

`compiled/program.ir.json` є canonical machine target. Але в Runtime Mode він належить CLI, а не моделі.

Модель не повинна:

```
відкривати compiled/program.ir.json;
читати compiled/program.ordo.view напряму;
формулювати питання з compiled/*;
цитувати compiled/* у чат.
```

Легальні джерела — тільки CLI-команди:

```
runtime-entry
next-step
next-step --format auto
next-step --format ordo-code
render-runtime-view
intake --submit
verify-targets
verify-session
```

Короткий протокол після відповіді

Після кожної відповіді користувача AI має показати короткий runtime-протокол:

```
Крок: <submitted node>
Дія: intake --submit
Результат: accepted / rejected / blocked
Evidence: <path> sha256=<digest>
Trace: runtime/session.ordo.trace sha256=<digest>
Рішення: ask next / clarify / stop
```

Без evidence і trace digest не можна ставити наступне питання.

Runtime view

M60.3 додає `runtime_view`:

```
json
ordo-code
json,ordo-code
```

У json mode AI бачить стандартний report. У ordo-code mode next-step --format auto додає current contract fragment. У mixed mode дозволені обидва формати.

Головна формула

```
JSON IR вирішує.
Ordo-code пояснює.
Session-trace доводить.
```

Цей протокол не замінює validate-state, validate-output, validate-artifacts, consistency, go-no-go або verify-session. Він задає правильний старт і правильний цикл guided intake.

Розділ 50. Runtime Mode start files standard

Після M55 детальні runtime-правила більше не треба щоразу вставляти у великий prompt.

Кожен runtime-ready Ordo package має два стартові файли:

```
START_HERE_RUNTIME_MODE.md
START_PROMPT_RUNTIME_MODE.md
```

START_HERE_RUNTIME_MODE.md містить повний протокол: як читати ordo.yml, як перевіряти source/IR, як працювати з run_state, як не вести guided intake “з пам’яті”, як фіксувати CLI status, як не обходити gates і як запускати artifact validation.

START_PROMPT_RUNTIME_MODE.md є мінімальним: він лише каже AI прочитати START_HERE_RUNTIME_MODE.md і почати runtime loading protocol.

Source-of-truth лишається таким:

```
ordo.yml = manifest / entrypoint
source/program.ordo.yaml = editable source of truth
compiled/program.ir.json = runtime source for guided execution
run_state.json = current execution state
generated artifacts = rendered output
```

PDF книги не перегенерується на цьому кроці.

Розділ 51. Package build profiles: dev / runtime / evidence

M56 додає в Ordo стандарт розділення package на три профілі.

Навіщо це потрібно

Один subject package під час розробки може містити все одразу: source YAML, compiled IR, тести, run inputs, generated outputs, runtime snapshots і звіти. Для розробника це корисно, але для guided runtime це небезпечно: executor може випадково дивитися на YAML або старі проміжні файли замість актуального compiled/program.ir.json.

Тому вводяться три профілі:

```
dev      – повний source package для розробки та аудиту
runtime  – чистий executable package для guided intake
evidence – докази компіляції, валідації, hash/provenance/status
```

Runtime profile

Runtime package має працювати без editable YAML:

```
README.md
START_HERE_RUNTIME_MODE.md
START_PROMPT_RUNTIME_MODE.md
ordo.runtime.json
compiled/program.ir.json
output_templates/
reports/CLI_VALIDATION_SUMMARY.md
reports/BUILD_MANIFEST.json
reports/SHA256SUMS.txt
```

Він не має містити:

```
source/program.ordo.yaml
tests/
run_inputs/
domain/
runtime/state_snapshots/
generated_outputs/
release/*.zip
```

Головне правило:

```
Runtime package must not require source YAML for execution.
Runtime package must use compiled/program.ir.json as primary runtime source.
```

CLI

Пакування виконується явно:

```
ordo package <package> --profile dev --out <zip>
ordo package <package> --profile runtime --out <zip>
ordo package <package> --profile evidence --out <zip>
```

Для runtime-профілю CLI перевіряє наявність compiled IR, свіжість IR відносно YAML, runtime start files, output templates і CLI evidence.

Evidence

Кожен build profile генерує або використовує:

```
reports/BUILD_MANIFEST.json  
reports/SHA256SUMS.txt  
reports/package_report.json
```

Runtime profile також генерує:

```
ordo.runtime.json
```

Це дає executor-у чистий runtime package, а reviewer-у — окремий evidence package.

PDF книги не перегенеровувався в M56.

Стандартні помилки package profiles

```
ORDO-PACKAGE-001 unknown package profile  
ORDO-PACKAGE-002 runtime profile includes source YAML  
ORDO-PACKAGE-003 runtime profile missing compiled IR  
ORDO-PACKAGE-004 runtime profile missing output templates  
ORDO-PACKAGE-005 runtime profile missing START_HERE_RUNTIME_MODE.md  
ORDO-PACKAGE-006 runtime profile missing ordo.runtime.json  
ORDO-PACKAGE-007 runtime profile missing BUILD_MANIFEST.json  
ORDO-PACKAGE-008 runtime profile missing SHA256SUMS.txt  
ORDO-PACKAGE-009 evidence profile includes editable source files  
ORDO-PACKAGE-010 package claims executed_cli_passed without CLI evidence
```

Розділ 52. Runtime Checkpoint Discipline

M57 додає в Ordo Runtime Mode правило контрольних точок.

Ідея проста:

```
one node at a time
one contract at a time
one decision at a time
earliest incomplete node wins
```

Це потрібно, щоб AI Ordo Developer не стискав кілька runtime-вузлів в одну відповідь. Якщо одночасно закриваються бізнесові поля, ChangeRecord, trigger/no-op і naming decisions, стає незрозуміло, який контракт реально підтверджений, а де ще є прогалина.

Checkpoint table

run_state тепер має містити або отримувати від helper-команд checkpoint_table:

```
{
  "current_node": "",
  "last_closed_node": "",
  "earliest_incomplete_node": "",
  "checkpoint_table": {},
  "forward_allowed": false,
  "open_required_fields": [],
  "node_merge_attempt_detected": false
}
```

Як поводить się runtime

Якщо знайдена прогалина, Ordo не йде вперед. Він повертається до найранішого незавершеного вузла, питає одне focused питання і продовжує тільки після закриття цієї прогалини.

next-step має пріоритетно повертати earliest_incomplete_node, а не останній вузол, до якого хотіла перейти модель.

Чому це важливо

Checkpoint discipline робить guided intake audit-friendly: у кожен момент видно, який вузол відкритий, які required fields підтверджені, які залишились відкритими, і чи дозволений рух уперед.

PDF книги не перегенеровувався в цьому кроці.

Стандартні помилки checkpoint discipline

```
ORDO-CHECKPOINT-001 node advanced before current contract closed
ORDO-CHECKPOINT-002 earlier mandatory node incomplete
ORDO-CHECKPOINT-003 multiple node contracts merged without allow_batch_confirmation
ORDO-CHECKPOINT-004 missing checkpoint table in run_state
ORDO-CHECKPOINT-005 next-step ignored earliest_incomplete_node
ORDO-CHECKPOINT-006 generated output requested while checkpoint gaps remain
```

Розділ 53. CLI-enforced Runtime Package

Після появи runtime profile стало зрозуміло, що чистий runtime-пакет із `compiled/program.ir.json`, але без CLI, не дає повного enforcement. Модель могла прочитати IR напяму і пройти guided intake без deterministic helper reports.

Тому M59 вводить runtime trust layer: у runtime-пакеті має бути не тільки compiled IR, а й вбудований CLI, який є єдиним легальним інтерфейсом виконання.

```
runtime package = compiled IR + start files + output templates + embedded runtime CLI +
evidence layer
```

Головне правило

```
Runtime package без runnable CLI не є enforced runtime package.
```

`ordo package --profile runtime` додає:

```
cli_embedded/ordo
cli_embedded/README.md
cli_embedded/ordo_pkg/ordo/...
```

Embedded CLI відкриває тільки runtime-команди. Команди `authoring`, `compile`, `release` і `package` у runtime wrapper блокуються.

Hard-stop замість м'якого fallback

Старий підхід дозволяв сказати CLI `status: not_run_cli_unavailable` і продовжити. Це було чесним самозвітом, але не змінювало поведінку.

Новий підхід:

```
якщо cli_embedded/ordo не запускається → stop
```

Продовження можливе тільки після явної згоди користувача як `nondeterministic fallback`. У такому режимі кожен generated artifact має містити:

```
DETERMINISM_NOT_ENFORCED
```

Інкрементальний intake

M59.2 робить guided intake покроковим. Модель більше не повинна проходити весь сценарій сама. Для кожної відповіді користувача вона викликає:

```
cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <tmp_answer.yaml>
```

CLI приймає або блокує перехід, пише evidence report і повертає digest. Модель не має права ставити наступне питання, доки submit не виконано.

Evidence, snapshots і live state

Кожен accepted submit пише:

```
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/SESSION-*.json
runtime/live_session_state.json
```

`live_session_state.json` — це UX-cache для автоматичного продовження сесії. Він не замінює `evidence` і не є самостійним доказом.

Tamper-evident verification

M59.3 додає `hash-chain snapshots`, `verify-session`, `canary detection` і `human final verify gate`.

Команда:

```
cli_embedded/ordo verify-session .
```

перевіряє, що `runtime session` не дрейфувала непомітно. Очікувані чисті рядки:

```
session-chain: intact
canary-scan: clean
```

Canary не забороняє пряме читання IR фізично, але робить витік доказовим: якщо службовий canary-текст з'являється в `outputs` або `trace`, `session` отримує `failure`.

Trust level

M59 фіксує Level 1 без MCP і без `sandbox`:

```
level_1_cli_in_package_hard_stop_hash_chain_human_verify
```

Це не криптографічний захист від зломисника з повним доступом до файлової системи. Це захист від тихого дрейфу і випадкового обходу CLI.

Формула

CLI доступний → Runtime Mode enforced.

CLI недоступний → hard-stop.

CLI обійдено → `session invalid` / `evidence missing` / `canary` або `chain failure`.

Розділ 54. Multi-target Runtime Compilation Layer

M60 додає до Ordo горизонтальну модель compilation targets. Ідея проста: Ordo не замінює JSON на інший формат, а генерує кілька похідних представлень із одного canonical model.

```
source/program.ordo.yaml
    ↓
canonical normalized model
    ↓
target emitters
  ├── compiled/program.ir.json
  ├── compiled/program.ordo.view
  └── runtime/session.ordo.trace
```

Чому не замінити JSON

Semantic JSON IR добре працює як machine contract:

```
стабільний
hashable
machine-readable
зручний для CLI
зручний для verify-session
```

Але для моделі JSON часто виглядає як “дані”, які можна трошки переставити або доповнити. Тому Ordo додає AI-facing projection, яка виглядає як code-like контракт, але не стає другим source of truth.

Три перші targets

M60 бере тільки три формати:

```
json-ir
ordo-code-view
session-trace
```

Python і Java targets не входять у цей етап.

Головна формула M60

```
JSON IR decides.
Ordo-code explains.
Session-trace proves.
```

Українською:

```
JSON IR вирішує.
Ordo-code пояснює.
Session-trace доводить.
```

json-ir

compiled/program.ir.json лишається canonical runtime target. CLI виконує runtime logic саме з нього.

Runtime-пакет ніколи не створюється без json-ir, навіть якщо обрано --runtime-view ordo-code.

ordo-code-view

compiled/program.ordo.view — code-like projection для моделі. Вона показує node contract у формі, де явно видно:

```
kind
question
allowed answers
transition
reject rules
evidence requirements
```

Але модель не читає цей файл напряму. Вона отримує фрагменти через CLI:

```
ordo next-step . --format ordo-code
ordo render-runtime-view . --format ordo-code --node <NODE_ID>
```

session-trace

runtime/session.ordo.trace — це append-only proof program. Його пише CLI під час реального проходження intake. Модель не має права писати або ремонтувати trace сама.

Target manifest

Щоб targets не дрейфували один від одного, M60 додає:

```
compiled/targets.manifest.json
```

У ньому записано:

```
canonical_ir_hash
target paths
target roles
target hashes
derived_from_ir_hash
mutable session-trace metadata
```

Команда:

```
ordo verify-targets .
```

має показати:

```
target-set: consistent
```

Головна небезпека

Multi-target система небезпечна, якщо кожен target починає жити окремим життям.

Тому правило жорстке:

```
один canonical IR;
усі інші targets — похідні;
усі targets перевіряються hash-ами;
Runtime Mode працює тільки через CLI.
```

Розділ 55. Ordo-code Runtime View

`ordo-code-view` — це AI-facing projection, яка показує runtime contract у вигляді code-like фрагмента. Вона створена не для людини-розробника, а для того, щоб модель бачила процес як строгий контракт, а не як вільний JSON.

Приклад

Замість того щоб модель бачила сирий JSON, CLI може повернути:

```
node N_PATH_SELECT {
  kind: branch
  answer_type: enum

  allowed {
    A -> N_EVENT_ALIAS
    B -> N_EVENT_ALIAS
    C -> N_EVENT_ALIAS
    D -> N_EVENT_ALIAS
  }

  reject unless answer in [A, B, C, D]

  evidence required:
    next_step_report
    intake_submit_report
    runtime_evidence
}
```

Такий фрагмент моделі важче “вільно інтерпретувати”: тут видно, що дозволені тільки A/B/C/D, що інша відповідь має бути rejected, і що без evidence не можна рухатися далі.

Чому це не Java і не Python

Ordo-code навмисно не є general-purpose мовою. У ньому немає imports, loops, side effects або довільної логіки.

Його задача — бути:

```
code-like
strict
читабельним для моделі
обмеженим Ordo state machine
похідним від JSON IR
```

Як модель має його отримувати

Легально:

```
cli_embedded/ordo next-step . --format auto
cli_embedded/ordo next-step . --format ordo-code
cli_embedded/ordo render-runtime-view . --format ordo-code --node <NODE_ID>
```

Нелегально:

```
відкрити compiled/program.ordo.view напяму
прочитати compiled/program.ir.json напяму
переказати compiled/* у чат
```

У Runtime Mode усі `compiled/*` файли належать CLI.

Runtime view modes

M60.3 дозволяє створити runtime package у трьох режимах:

```
ordo package . --profile runtime --runtime-view json
ordo package . --profile runtime --runtime-view ordo-code
ordo package . --profile runtime --runtime-view json,ordo-code
```

У json mode next-step --format auto повертає звичайний report без code-like блоку.

У ordo-code mode next-step --format auto автоматично повертає current_contract.

У json,ordo-code mode можна явно обрати будь-який із двох форматів.

Чому це важливо

Мета Ordo-code view не в тому, щоб зробити ще один runtime. Мета — покращити поведінку моделі:

```
менше вигаданих переходів;
менше вільного трактування allowed answers;
краща кореляція між compiled project і відповіддю моделі;
видимий contract fragment у кожному кроці.
```

Але source of truth не змінюється:

```
JSON IR вирішує.
Ordo-code пояснює.
```

Розділ 56. Session-trace як proof program

session-trace — це не опис дерева рішень. Це запис фактичного проходження runtime-сесії.

Ідея: кожне рішення користувача стає не просто полем у state, а рядком у proof program, який можна перевірити.

Файл trace

Runtime-пакет містить:

```
runtime/session.ordo.trace
```

На початку trace ініціалізований, але ще не містить user decisions. Кожен accepted intake --submit дописує новий step.

Приклад trace-фрагмента

```
step 001:
  accept N_EVENT_GOAL with answer "Fall of Constantinople" -> N_PATH_SELECT
  evidence sha256:...
  snapshot sha256:...
```

Trace пише CLI. Модель не має права створювати, редагувати або “підправляти” його вручну.

Як trace пов’язаний із evidence

Після кожного submit CLI оновлює:

```
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/SESSION-*.json
runtime/session.ordo.trace
runtime/live_session_state.json
```

Evidence report містить trace metadata: path, digest, step і fragment. Це дозволяє моделі показати користувачу доказ, не читаючи внутрішні compiled targets напряду.

Чому trace не замінює snapshot

Snapshot показує стан. Trace показує шлях, яким до цього стану прийшли.

```
snapshot = що зараз відомо
trace = які рішення привели сюди
```

Тому вони перевіряються разом.

Перевірка

Команда:

```
cli_embedded/ordo verify-session .
```

перевіряє:

```
target-set
session-chain
```

```
session-trace  
canary-scan
```

Чиста сесія має показати:

```
target-set: consistent  
session-chain: intact  
session-trace: intact  
canary-scan: clean
```

Якщо trace змінено вручну, verify-session має показати failure. Якщо модель обійшла CLI і не створила trace/evidence, це також стає видимим.

Навіщо це моделі

Моделі у довгій сесії може забути, що саме було підтверджено. Trace зменшує ризик такого дрейфу:

```
кожен крок має node;  
кожен answer має evidence;  
кожен перехід має next node;  
кожен digest можна перевірити;  
усю сесію можна replay/verify логічно.
```

Тобто session-trace робить runtime не просто діалогом, а перевірюваною історією виконання.

Розділ 57. Scenario testing і PathWalk

У попередніх розділах ми додали до Ordo runtime-пакета кілька шарів перевірки:

```
CLI-enforced runtime
per-node evidence
hash-chain snapshots
verify-session
multi-target compilation
Ordo-code runtime view
session-trace proof program
```

Це відповідає на питання:

Чи конкретний runtime-прохід має докази і чи можна його перевірити?

Але є інше питання:

Чи модель стабільно поводить себе правильно у багатьох різних сценаріях?

Для цього потрібен окремий testing layer. Один із можливих підходів — **PathWalk**.

Що таке scenario testing

Звичайний unit-тест CLI перевіряє команду:

```
next-step працює
intake --submit приймає правильну відповідь
verify-session ловить пошкодження trace
```

Scenario testing перевіряє вже не команду, а поведінку моделі в цілому:

```
модель отримала runtime package
модель повинна викликати next-step
модель ставить питання користувачу
користувач або тестовий сценарій дає відповідь
модель submit-ить відповідь через CLI
CLI пише evidence, snapshot і trace
модель переходить далі тільки після прийнятого submit
```

Тобто scenario testing відповідає на практичне питання:

Чи модель реально йде по Process Rail, а не просто каже, що йде?

Що таке PathWalk

PathWalk — це супровідна утиліта або benchmark-підхід для перевірки проходження Ordo-сценаріїв.

Вона може:

```
створювати тестове дерево рішень
генерувати ground truth path
давати моделі пройти цей шлях
додавати шум, уточнення або неправильні відповіді
перевіряти фактичні runtime-артефакти після проходження
рахувати score
```

PathWalk не є частиною ядра Ordo. Це зовнішній або companion-шар.

Його задача — не замінити runtime CLI, а перевірити, чи модель ним користується.

Чому не можна вірити самозвіту моделі

Модель може написати:

```
CLI executed and passed.
```

Але це саме по собі нічого не доводить.

Після M59/M60 доказами є файли:

```
reports/next_step_report.json
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/*.json
runtime/session.ordo.trace
reports/target_verification_report.json
reports/session_verification_report.json
```

PathWalk має дивитися саме на ці файли.

Інакше ми знову повертаємось до старої проблеми:

```
модель сама сказала, що все пройшла правильно
але ніхто не перевірив, що CLI справді викликався
```

Як PathWalk має працювати з M60 runtime package

У enforced mode PathWalk має давати моделі тільки runtime CLI-протокол.

Типовий цикл:

```
./cli_embedded/ordo runtime-status .
./cli_embedded/ordo verify-targets .
./cli_embedded/ordo next-step . --format auto
./cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <answer-file>
./cli_embedded/ordo verify-session .
```

Модель не має читати напряму:

```
compiled/program.ir.json
compiled/program.ordo.view
compiled/targets.manifest.json
```

Навіть `program.ordo.view` є AI-facing projection, але вона має подаватися через CLI, а не через пряме читання файлу.

Як PathWalk пов'язаний із трьома M60 target-ами

У M60 є проста формула:

```
JSON IR вирішує.
Ordo-code пояснює.
Session-trace доводить.
```

PathWalk має використовувати її так:

```
JSON IR
  не читається моделлю напряму, але є canonical runtime contract для CLI.

Ordo-code view
  може бути показаний моделі через next-step --format auto або render-runtime-view.

Session-trace
  перевіряється після проходження програм фактичних прийнятих рішень.
```

Тобто PathWalk не створює нове джерело правди. Він перевіряє, чи модель коректно працює з уже наявними джерелами доказу.

Режими, які варто порівнювати

PathWalk особливо корисний, якщо порівнювати кілька режимів:

```
enforced + json
enforced + ordo-code
enforced + json,ordo-code
ir_readable baseline
freeform baseline
```

Наприклад, можна перевірити:

```
чи модель частіше помиляється у json-only режимі
чи Ordo-code view допомагає їй краще тримати allowed answers
чи mixed режим дає кращу стабільність
чи freeform режим швидше дрейфує від дерева рішень
```

Це вже не просто тест Ordo. Це вимірювання поведінки моделі.

Що PathWalk має оцінювати

Один загальний score недостатній. Краще розділяти оцінку:

```
path correctness
  чи фінальний шлях збігся з очікуваним

protocol compliance
  чи модель кожного разу викликала CLI

runtime integrity
  чи verify-targets і verify-session пройшли

compiled-read violations
  чи модель намагалася читати compiled/* напряму

robustness
  чи модель витримала уточнення, шум, неправильні відповіді або корекції
```

Це важливо, бо модель може прийти до правильного фінального вузла, але зробити це неправильним способом.

Для Ordo правильний спосіб теж має значення.

Backtracking і restore-session

У guided intake реальні користувачі часто змінюють думку:

```
ні, повернімось назад
я помилився у попередній відповіді
обери іншу гілку
```

Це природний сценарій. Але станом на M60.3 restore-session ще не є обов'язковою runtime-командою.

Тому PathWalk може тестувати прості корекції, якщо вони підтримані самим деревом. Але повноцінний rollback краще робити окремим майбутнім шаром:

```
restore-session має бути append-only
має писати evidence
```

```
має писати trace event
має перевірятися verify-session
не має мовчки видаляти історію
```

Тобто backtracking важливий, але його не треба робити хаотичним ручним редагуванням state.

Чого PathWalk не повинен робити

PathWalk не має:

```
підміняти embedded CLI
самостійно створювати evidence reports від імені CLI
пропонувати моделі читати compiled/*
рахувати успіх тільки за текстом відповіді моделі
вважати protocol bypass нормальним, якщо фінальний answer виглядає правильним
```

Якщо тестова утиліта робить це, вона може бути корисною для baseline, але не для enforced-runtime перевірки.

Підсумок

Runtime CLI перевіряє один конкретний прохід.

PathWalk перевіряє, чи модель стабільно проходить багато різних сценаріїв.

У правильній архітектурі вони не конкурують:

```
Ordo runtime package дає правила і докази.
PathWalk дає експеримент над поведінкою моделі.
```

Саме тому PathWalk варто документувати як окремий companion testing layer, а не як частину ядра мови.

M60.3.2: чому bare intake заборонений у benchmark-автоматизації

Scenario runner не повинен викликати CLI так:

```
./cli_embedded/ordo intake .
```

Такий виклик означає: “запусти guided intake і, якщо відповіді немає, запитай через input()”. Для людини в терміналі це нормально. Для subprocess/benchmark worker — небезпечно, бо процес може зависнути.

Починаючи з M60.3.2, Ordo runtime CLI має поводитися так:

```
немає --submit
немає --answers
немає --non-interactive
stdin не є TTY
    ↓
fail-fast
reason: no_answers_and_not_interactive_and_no_tty
```

Для PathWalk це означає просте правило:

```
у benchmark automation завжди використовуй явний режим:
- intake --submit ...
- або intake --answers ... --non-interactive
```

Так само next-step має давати моделі короткий поточний фрагмент, а не весь внутрішній checkpoint table. Повні деталі можуть існувати у report-файлі, але модель не повинна отримувати зайвий runtime шум у stdout.

Розділ 58. Restore-session: безпечне повернення назад

Іноді під час guided intake користувач хоче змінити попередню відповідь:

```
Повернімось на попередній крок.
Я хочу обрати інший шлях.
Ні, ця відповідь була помилкова.
```

До M60.4 у runtime Ordo не було нативної команди для такого повернення. Це створювало спокусу для зовнішніх утиліт або моделі самостійно редагувати стан сесії. Такий підхід небезпечний: він може приховати, що історію було змінено.

M60.4 додає команду:

```
ordo restore-session <package> --to-seq <N>
```

У runtime-пакеті вона викликається так:

```
./cli_embedded/ordo restore-session . --to-seq <N>
```

Restore — це не видалення історії

Головне правило:

```
restore-session не стирає минулі snapshots.
restore-session додає нову подію в історію.
```

Тобто Ordo не “перемотує” сесію так, наче пізніших кроків ніколи не було. Замість цього CLI бере стан із попереднього snapshot і створює нову restore-подію.

Це важливо для довіри: людина й перевіряюча утиліта бачать, що відбулося повернення.

Що пише restore-session

Успішний restore створює або оновлює:

```
reports/restore_session_report.json
runtime/evidence/*RESTORE_TO_SEQ_<N>*_evidence.json
runtime/state_snapshots/SESSION-*_RESTORE_TO_SEQ_<N>.json
runtime/session.ordo.trace
runtime/live_session_state.json
```

У session.ordo.trace з’являється крок:

```
action: restore_session
node: RESTORE_TO_SEQ_<N>
```

Це означає: повернення назад є частиною доказової історії, а не прихованою операцією над файлами.

Як модель має працювати після restore

Після restore модель не повинна самостійно вирішувати, яке питання ставити далі. Вона має знову викликати CLI:

```
./cli_embedded/ordo next-step . --format auto
```

Тільки після цього вона може поставити користувачу наступне питання.

Як перевірити restore

Фінальна перевірка та сама:

```
./cli_embedded/ordo verify-session .
```

Сесія вважається здоровою, якщо restore-подія не ламає:

```
target-set
session-chain
session-trace
evidence digest
snapshot hash
canary scan
```

Чому це важливо для PathWalk

Scenario-testing утиліти на кшталт PathWalk часто мають correction/backtrack сценарії. До M60.4 вони могли або не тестувати такі сценарії, або мати власний rollback-механізм.

Після M60.4 правильний підхід інший:

```
PathWalk не має сам переписувати runtime state.
PathWalk має викликати embedded CLI restore-session.
```

Так зберігається головна властивість Ordo runtime: кожен важливий перехід видно в доказових артефактах.

Формула M60.4

```
JSON IR вирішує.
Ordo-code пояснює.
Session-trace доводить.
Restore-session повертає назад без стирання історії.
```

Розділ 59. PathWalk Benchmark Readiness

У попередньому розділі ми додали безпечне повернення назад через `restore-session`. Це зробило PathWalk-сценарії з `backtrack/correction` технічно можливими в актуальному runtime-протоколі Ordo.

Але перед тим як запускати дорогі benchmark-прогони через зовнішні моделі, потрібна ще одна перевірка: чи сама утиліта тестування правильно підключена до поточного runtime-пакета.

Це і є задача **PathWalk Benchmark Readiness**.

Навіщо потрібен `readiness smoke`

PathWalk не є частиною runtime core. Це companion-утиліта для перевірки того, як модель проходить Ordo-сценарії.

Тому перед реальними API-прогонами треба перевірити не модель, а саму інфраструктуру:

- чи PathWalk будує M60 runtime-пакет;
- чи використовує `./cli_embedded/ordo`, а не старий `ordo_run.py`;
- чи підтримує всі runtime view;
- чи читає правильні runtime artifacts;
- чи генерує score-файли;
- чи aggregate summary працює.

Це робиться дешевим по-API smoke-тестом.

Matrix smoke

Команда:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli matrix-smoke \
--out /tmp/pathwalk_matrix_smoke \
--depth 2 \
--branching 2 2 \
--force
```

Вона перевіряє матрицю:

```
enforced + json
enforced + ordo-code
enforced + json,ordo-code
```

Це не справжній тест якості моделі. У цьому режимі PathWalk сам проходить ground truth через embedded CLI. Мета — довести, що протокол, scorer і aggregator правильно з'єднані.

Які файли очікуються

Після smoke-тесту має бути структура:

```
pathwalk_matrix_smoke/
tree_source/
```

```

scenarios/
  scenario_000.json
sandboxes/
transcripts/
scores/
  scenario_000_json_score.json
  scenario_000_ordo-code_score.json
  scenario_000_json_ordo-code_score.json
SUMMARY.json
SUMMARY.md

```

Кожен score має містити:

```

runtime_view
runtime_protocol_version
ordo_cli_version
canonical_ir_hash
targets_manifest_hash
session_trace_hash

```

Без цих полів результати різних прогонів не можна коректно порівнювати.

Що readiness smoke не доводить

Matrix smoke не відповідає на питання:

- яка модель краща;
- які ваги path_quality_score правильні;
- як модель поводить себе при rate limit;
- чи prompt достатньо стійкий до складного шуму.

Він відповідає тільки на одне питання:

Чи PathWalk зараз сумісний з поточним Ordo runtime protocol?

Правило для зовнішніх утиліт

Якщо зовнішня scenario-testing утиліта все ще потребує:

```
python3 cli_embedded/ordo_run.py
```

або не підтримує:

```

runtime_view
verify-targets
session.ordo.trace
restore-session

```

то вона не сумісна з актуальною M60-лінією.

Таку утиліту треба адаптувати до поточного мовного пакета, а не відкотити мовний пакет до старого runtime-протоколу.

Головна формула

Runtime CLI перевіряє один прохід.
PathWalk readiness smoke перевіряє, що сама тестова інфраструктура готова до багатьох проходів.

M60.5.1: benchmark dry-run перед калібруванням

matrix-smoke перевіряє один сценарій у різних runtime-view. Але перед калібруванням ваг потрібен інший режим: багато сценаріїв без API-викликів, щоб зібрати сирі метрики.

Для цього додано:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli benchmark-dry-run \
--out /tmp/pathwalk_dry_run \
--scenario-count 20 \
--runtime-view json \
--runtime-view ordo-code \
--runtime-view json,ordo-code \
--force
```

Цей режим не тестує якість моделі. Він проходить ground truth через embedded runtime CLI і створює:

```
RAW_METRICS.csv
SUMMARY.json
SUMMARY.md
scores/*_score.json
```

M60.5.1 також виправляє важливий blocker: dry-run більше не має повторно запускати embedded verify-targets / verify-session під час scoring, якщо ці звіти вже створив runtime-прохід. Scorer читає готові verification reports, а не запускає verification вдруге.

Головне правило:

```
Matrix smoke показує, що підключення працює.
Benchmark dry-run збирає сирі метрики для майбутнього калібрування.
```

Калібрування ваг усе ще не входить у цей етап.

M60.5.2b: зовнішній job-контракт для dry-run

Після перших dry-run спроб стало видно, що великий benchmark не варто вести одним довгоживучим parent-процесом. Кращий контракт такий:

```
dry-run-plan
↓
dry-run-job для кожного scenario/runtime_view
↓
dry-run-collect
```

dry-run-plan створює DRY_RUN_PLAN.json, сценарії та runtime templates. Кожен dry-run-job виконує рівно один case. dry-run-collect збирає RAW_METRICS.csv, SUMMARY.json і SUMMARY.md.

Це важливо для реальних benchmark-прогонів: один завислий або проблемний case не має блокувати весь набір. Такий підхід також легше переноситься в CI, де кожен job можна запускати окремо.

M60.5.4: artifact-only dry-run executor

M60.5.4 changes the recommended dry-run execution shape. Instead of relying on a single long-lived Python process to run every scenario/runtime-view pair, PathWalk now treats the benchmark as a set of artifacts:

```

DRY_RUN_PLAN.json
jobs/<job_id>.json
job_scripts/<job_id>.sh
scores/*_score.json
RAW_METRICS.csv
SUMMARY.json
SUMMARY.md

```

The stable execution model is:

```

dry-run-plan
↓
independent job execution
↓
dry-run-collect

```

This matters because benchmark jobs may call embedded runtime CLIs many times. If one worker or subprocess has unusual lifecycle behavior, it must not hold the entire benchmark parent process hostage. In M60.5.4, each generated job script executes exactly one case and redirects stdin from /dev/null.

The convenience `benchmark-dry-run` command remains useful for local smoke checks, but acceptance evidence should use the artifact-only plan/job/collect flow.

M60.6: calibration preparation baseline

M60.6 is the first stable baseline run for calibration preparation. It still does **not** calibrate weights and does **not** call a real model API.

The accepted baseline shape is:

```

20 scenarios
× 3 runtime views: json, ordo-code, json,ordo-code
= 60 dry-run cases

```

The run uses the artifact-only execution contract:

```

dry-run-plan
↓
independent job_scripts/*.sh
↓
dry-run-collect

```

The important output is not the temporary runtime sandbox. The important output is the compact evidence set:

```

DRY_RUN_PLAN.json
jobs/*.json
job_scripts/*.sh
scores/*_score.json
RAW_METRICS.csv
SUMMARY.json
SUMMARY.md

```

For the M60.6 baseline all 60 dry-run cases passed. `RAW_METRICS.csv` was collected for future calibration analysis.

The rule after M60.6 is strict:

```

Do not change path_quality_score weights from dry-run alone.
Dry-run prepares calibration. It does not perform calibration.

```

Real model/API benchmarks and real-module testcase generation remain future milestones.

M60.6.1: clean-room release integrity hardening

After M60.6, the next safety gate is a clean-room archive check: unpack the release artifacts into a fresh directory and verify that the documented commands and generated artifacts still work outside the original build folder.

M60.6.1 fixes one important handoff issue: generated `job_scripts/*.sh` must not contain a baked absolute path to the temporary workspace used during generation. Instead, job scripts use this portable contract:

```
export ORDO_PATHWALK_ROOT=<workspace-or-pathwalk-rc-root>
export ORDO_CLI_ROOT=<developer-bundle-root>/cli # only needed when PathWalk root has no cli/ordo
```

The script must fail fast if it cannot find `ordo_pathwalk/` or `cli/ordo/`. This keeps artifact-only execution deterministic without hiding environment assumptions.

This is a handoff/integrity fix only. It does not change M60.6 dry-run metrics, scoring weights, or runtime semantics.

M60.6.2: calibration report refinement

M60.6.2 does not run a new benchmark and does not change scoring weights. It refines the interpretation of the M60.6 baseline.

The baseline is useful because it proves the artifact-only dry-run contract is wired correctly across the three runtime views:

```
json
ordo-code
json,ordo-code
```

But the same baseline is not enough for calibration. All 60 dry-run cases passed and all component metrics were 1.0:

```
path_quality_score = 1.0
cell_match_rate = 1.0
protocol_compliance_rate = 1.0
distraction_recovery_rate = 1.0
backtrack_accuracy = 1.0
turn_accuracy_rate = 1.0
```

That means the dataset has zero variance. A zero-variance dry-run can confirm readiness, but it cannot show whether `cell_match_rate`, `protocol_compliance_rate`, `distraction_recovery_rate`, or `backtrack_accuracy` should receive different weights.

The M60.6.2 decision is therefore:

```
Do not change path_quality_score weights from dry-run-only data.
Collect model/API or transcript-based runs with real variance before calibration.
```

Before real calibration, PathWalk needs evidence with non-perfect cases, failed cases, per-noise-type labels, repeated seeds/models, and a clear distinction between protocol violations and path-quality mistakes.

Розділ 60. Model Benchmark Protocol

Навіщо цей розділ

Після M60.6 Ordo вже має стабільний no-API PathWalk dry-run baseline. Але dry-run baseline не є реальним benchmark-ом моделі: там ground-truth driver проходить дерево ідеально, тому всі метрики дорівнюють 1.0.

M60.6.3 фіксує наступний шар: **протокол реального model benchmark-a**.

Головне правило:

```
Dry-run proves wiring.
Model benchmark measures behavior.
Calibration requires variance.
```

Чому не можна одразу міняти ваги

У M60.6.2 було підтверджено:

```
60/60 cases passed
all component metrics = 1.0
metric variance = 0
```

Це означає, що dry-run показує готовність інфраструктури, але не показує, які ваги path_quality_score кращі.

Тому рішення M60.6.3:

```
weights remain locked until real model or transcript evidence passes calibration gates
```

Два дозволені режими benchmark-a

API-driven benchmark

Моделі реально проходить PathWalk-сценарії через harness.

В enforced mode вона має взаємодіяти тільки з runtime CLI:

```
./cli_embedded/ordo next-step . --format auto
./cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <file>
./cli_embedded/ordo restore-session . --to-seq <N> --reason "... "
./cli_embedded/ordo verify-session .
```

Читати compiled/* напряду не можна.

Transcript-replay benchmark

Замість live API береться вже записаний transcript поведінки моделі.

Це безпечніший перший pilot, бо він дозволяє перевірити scoring і failure buckets без витрат API та без нестабільності зовнішнього провайдера.

Які artifacts має створити benchmark

Мінімальний набір:

```
MODEL_BENCHMARK_PLAN.json
jobs/<job_id>.json
transcripts/<job_id>_transcript.json
scores/<job_id>_score.json
RAW_MODEL_METRICS.csv
SUMMARY.json
SUMMARY.md
MODEL_RUN_MANIFEST.json
CALIBRATION_DECISION.md
CALIBRATION_DECISION.json
```

Це важливо: модельний benchmark має зберігати не лише score, а й transcript evidence.

Які метрики потрібні

RAW_MODEL_METRICS.csv має містити не тільки фінальний score, а й компоненти:

```
path_quality_score
cell_match_rate
protocol_compliance_rate
distraction_recovery_rate
backtrack_accuracy
turn_accuracy_rate
invalid_branch_rejection_rate
skip_ahead_resistance_rate
clarification_handling_rate
correction_recovery_rate
restore_session_usage_rate
```

Окремо треба фіксувати:

```
tool_call_count
turn_count
completion_latency_ms
failure_bucket
transcript_sha256
canonical_ir_hash
targets_manifest_hash
session_trace_hash
weights_hash
```

Failure buckets

Невдалі або неідеальні проходження не можна просто складати в одну купу. Їх треба класифікувати:

```
protocol_violation
wrong_branch
invalid_submit
missed_backtrack
distraction_followed
skip_ahead
clarification_failure
timeout_or_loop
runtime_error
scorer_error
```

Це потрібно, щоб не переплутати помилку моделі з помилкою runtime або scorer-a.

Calibration gates

Ваги не можна міняти, якщо:

- сценарії не однакові між runtime views;
- мало cases;

- yci scores знову 1.0;
- немає nonzero variance;
- failed cases не класифіковані;
- protocol violations не відділені від path-quality mistakes;
- немає transcript hash / runtime metadata hash;
- не зроблено manual review failed transcripts.

Практичне правило:

Failing any calibration gate means: keep weights unchanged.

Що це означає для наступних milestone

M60.6.3 не запускає real benchmark. Він тільки встановлює контракт, за яким benchmark можна буде запускати.

Наступний безпечний крок:

M60.6.4 – Transcript Replay / Model Benchmark Pilot

А ідея M60.7/M61 про генерацію test cases з реального source/program.ordo.yaml лишається відкладеною, доки модельний benchmark protocol не буде перевірений на pilot evidence.

M60.6.4 — transcript-replay pilot

M60.6.4 перевіряє цей protocol на маленькому executable pilot без live API.

Pilot створює:

```
MODEL_BENCHMARK_PLAN.json
transcripts/*_transcript.json
scores/*_score.json
RAW_MODEL_METRICS.csv
SUMMARY.json / SUMMARY.md
MODEL_RUN_MANIFEST.json
CALIBRATION_DECISION.md / .json
```

У pilot навмисно є три типи evidence:

```
perfect transcript → failure_bucket = none
distraction/model-quality miss → failure_bucket = distraction_followed
пряме читання compiled/* в enforced mode → failure_bucket = protocol_violation
```

Це важливо, бо після M60.6.2 dry-run мав нульову дисперсію. M60.6.4 показує, що transcript-replay шлях уже може створювати non-saturated metrics і failure buckets.

Але це ще не calibration run. Ваги лишаються locked, бо pilot малий і не проходить gates:

```
sufficient_cases
repeatability_checked
confidence_summary_present
manual_failure_review_done
```

Практичний висновок:

```
Transcript replay protocol works.
Calibration is still blocked.
Weights remain unchanged.
```

Розділ 61. Real Module Testcase Generation

Навіщо потрібен цей крок

Після M60.6 стало зрозуміло: dry-run baseline добре перевіряє wiring, але не дає достатньо різноманітних даних для калібрування оцінок. Наступний корисний крок — не одразу запускати великий model benchmark, а навчити PathWalk будувати тест-кейси з реального Ordo-модуля.

Ідея M60.7 така:

```
реальний source/program.ordo.yaml
↓
реальне дерево / граф рішень
↓
terminal paths
↓
testcase artifacts
↓
контрольований noise
```

Що є входом

Базовий вхід:

```
source/program.ordo.yaml
```

Це source-level робота. PathWalk може читати source YAML для генерації тест-кейсів, бо це authoring/testing layer. Але під час enforced runtime виконання модель усе одно не повинна напряду читати compiled/*; вона має користуватися embedded CLI.

Які типи шуму потрібні

M60.7 має генерувати не лише ідеальні happy path сценарії. Потрібні контрольовані варіанти плутанини моделі:

Тип	Сенс
clean_path	правильне проходження гілки без шуму
distraction	стороннє питання під час intake
backtrack	повернення до попереднього вузла
skip_ahead	спроба відповісти на майбутній крок зарано
invalid_branch	відповідь, яка не дозволена поточною гілкою
clarification_without_submit	уточнення без submit-відповіді
correction_backtrack	виправлення раніше поданої відповіді

Які артефакти має давати генератор

Початковий контракт артефактів:

```
REAL_MODULE_TESTCASE_PLAN.json
REAL_MODULE_GRAPH_SUMMARY.json
cases/<case_id>.json
cases/<case_id>.md
RAW_TESTCASE_MATRIX.csv
SUMMARY.json
```

```
SUMMARY.md
VALIDATION_REPORT.json
```

Це спочатку generation contract, а не model benchmark contract.

Що не треба змішувати

M60.7 не має автоматично відкривати калібрування ваг. Генерація реалістичних тест-кейсів покращує матеріал для майбутніх benchmark-ів, але не замінює:

```
real model/transcript evidence
nonzero variance
repeatability analysis
manual failure review
calibration decision artifact
```

Що реалізовано в M60.7.1

M60.7.1 реалізує перший вузький зріз цієї лінії: PathWalk вже може прочитати реальний source/program.ordo.yaml і побудувати graph summary без запуску runtime та без читання compiled/*.

Команда:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-graph \
--source path/to/source/program.ordo.yaml \
--out runs/real_module_graph \
--force
```

Вихідні артефакти M60.7.1:

```
REAL_MODULE_GRAPH_SUMMARY.json
REAL_MODULE_GRAPH_SUMMARY.md
VALIDATION_REPORT.json
```

Graph summary фіксує nodes, edges, branching nodes, unmatched handlers, gates, outputs, terminal/gate targets і unresolved targets. Це ще не генерація тест-кейсів:

testcase_generation_ready навмисно лишається false, щоб не змішувати loader/summary з наступними етапами.

Що реалізовано в M60.7.2

M60.7.2 додає наступний вузький зріз: terminal path enumeration. PathWalk бере вже створений REAL_MODULE_GRAPH_SUMMARY.json і будує перелік terminal paths.

Команда:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-paths \
--summary runs/real_module_graph/REAL_MODULE_GRAPH_SUMMARY.json \
--out runs/real_module_paths \
--force
```

Вихідні артефакти M60.7.2:

```
REAL_MODULE_TERMINAL_PATHS.json
REAL_MODULE_TERMINAL_PATHS.md
VALIDATION_REPORT.json
```

Terminal paths містять branch signature, node sequence, answer sequence, terminal target, terminal type, state updates і outputs, які дозволені після terminal gate. Це вже достатня

структурна база для майбутнього clean-path testcase generator, але самі test cases ще не створюються.

Тому readiness навмисно розділено:

```
terminal_path_enumeration_ready = true/false
clean_path_case_generation_ready = true/false
noise_case_generation_ready = false
testcase_generation_ready = false
```

Правильна послідовність

Безпечна послідовність така:

```
M60.7.1 – source YAML loader i graph summary
M60.7.2 – terminal path enumeration ✓
M60.7.3 – clean testcase artifacts
M60.7.4 – noise testcase artifacts
M60.7.5 – fixture acceptance i packaging
```

Це дозволяє не повертатися в технічне болото transcript replay matrix, а рухатися до більш цінного рівня: реальні дерева рішень і реальні сценарії плутанини.

Що реалізовано в M60.7.3

M60.7.3 додає перший власне testcase-artifact зріз: clean-path test cases. PathWalk бере REAL_MODULE_TERMINAL_PATHS.json і створює один чистий тест-кейс на кожен terminal path.

Команда:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-clean-cases \
--paths runs/real_module_paths/REAL_MODULE_TERMINAL_PATHS.json \
--out runs/real_module_clean_cases \
--force
```

Вихідні артефакти M60.7.3:

```
cases/<case_id>.json
cases/<case_id>.md
RAW_TESTCASE_MATRIX.csv
SUMMARY.json
SUMMARY.md
VALIDATION_REPORT.json
```

Clean-path case описує найкоротшу правильну послідовність відповідей для конкретного terminal path:

```
path_id
branch_signature
answer_steps
expected_terminal
expected_outputs
expected_state_updates
```

Це ще не runtime execution і не noise generation. Тому readiness лишається розділеним:

```
clean_path_cases_ready = true/false
runtime_execution_ready = false
noise_case_generation_ready = false
```

M60.7.3 важливий тим, що вперше перетворює реальний Ordo source graph на матеріальні testcase artifacts, але не повертається до transcript-replay benchmark orchestration.

Оновлена послідовність після M60.7.3

M60.7.1 – source YAML loader i graph summary ✓
 M60.7.2 – terminal path enumeration ✓
 M60.7.3 – clean testcase artifacts ✓
 M60.7.4 – noise testcase artifacts
 M60.7.5 – fixture acceptance i packaging

Що реалізовано в M60.7.4

M60.7.4 додає перші noise testcase artifacts, але все ще не запускає runtime і не відкриває benchmark/scoring/calibration гілку.

Перші підтримані noise patterns:

Pattern	Сенс
distraction	користувач вставляє стороннє питання перед валідною відповіддю
invalid_branch	користувач дає недозволену відповідь для поточного вузла, після чого сценарій має продовжитись валідною відповіддю

Команда:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-noise-cases \
--paths runs/real_module_paths/REAL_MODULE_TERMINAL_PATHS.json \
--out runs/real_module_noise_cases \
--pattern distraction \
--pattern invalid_branch \
--force
```

Вихідні артефакти M60.7.4:

```
cases/<case_id>.json
cases/<case_id>.md
RAW_NOISE_TESTCASE_MATRIX.csv
SUMMARY.json
SUMMARY.md
VALIDATION_REPORT.json
```

Важлива межа: runtime_execution_ready, scoring_ready і calibration_ready залишаються false. M60.7.4 генерує тільки source-level testcase artifacts. Це зроблено навмисно, щоб не повернутися в технічну гілку runtime-harness/watchdog, яку ми зафіксували як blocked у M60.6.5 / M60.6.4.1.

Що реалізовано в M60.7.5

M60.7.5 розширює artifact-only noise testcase generation до контрольованого набору з чотирьох patterns:

```
distraction
invalid_branch
clarification_without_submit
skip_ahead
```

Це свідомо обмежений slice. Ми не продовжуємо нескінченне додавання малих variants у цій гілці. Складніші conversational recovery patterns, наприклад backtrack і correction_backtrack, зафіксовані як future improvements і не блокують поточний milestone.

Важлива межа лишається незмінною: runtime execution, scoring, calibration і benchmark orchestration не відкриваються в M60.7.5.

M60.7 line closure

M60.7 закривається на стабільній межі M60.7.5. Це означає, що поточна корисна artifact-only лінія вже є завершеною:

```
source/program.ordo.yaml
↓
REAL_MODULE_GRAPH_SUMMARY
↓
REAL_MODULE_TERMINAL_PATHS
↓
clean-path testcase artifacts
↓
bounded noise testcase artifacts
```

Поточний підтриманий набір noise patterns:

```
distraction
invalid_branch
clarification_without_submit
skip_ahead
```

Складніші conversational recovery patterns — backtrack і correction_backtrack — не додаються в цій лінії автоматично. Вони зафіксовані як future improvements, щоб не перетворювати M60.7 на нескінченний блок дрібних покращень.

Runtime execution, scoring і benchmark orchestration також лишаються окремими майбутніми milestones.

Розділ 62. Human Review Scenario Cards

M61.0 додає до PathWalk проміжний шар між згенерованими real-module testcase artifacts і майбутнім runtime execution.

Його задача проста: зробити так, щоб QA, developer або reviewer могли швидко прочитати згенерований кейс як сценарну картку, а не розбирати сирий JSON.

Що було до цього

Після M60.7 маємо artifact-only ланцюжок:

```
source/program.ordo.yaml
→ REAL_MODULE_GRAPH_SUMMARY.json
→ REAL_MODULE_TERMINAL_PATHS.json
→ clean path cases
→ bounded noise cases
```

Clean/noise cases уже структуровані, але вони більше схожі на машинні artifacts.

M61.0 додає людський шар:

```
clean path cases + bounded noise cases
→ human review scenario cards
```

Команда

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-review-cards \
--summary runs/real_module_clean_cases/SUMMARY.json \
--summary runs/real_module_noise_cases/SUMMARY.json \
--out runs/real_module_review_cards \
--force
```

Artifacts

Команда створює:

```
cards/<card_id>.json
cards/<card_id>.md
REVIEW_CARDS.json
REVIEW_CARDS.md
RAW_REVIEW_CARD_MATRIX.csv
VALIDATION_REPORT.json
```

cards/*.md — це основний формат для людини. Там видно:

- case id;
- path id;
- noise pattern;
- scripted steps;
- expected behavior;
- expected terminal;
- expected outputs;
- review checklist.

Що це не робить

M61.0 не запускає runtime.

Він також не робить:

- model/API benchmark;
- scoring;
- calibration;
- watchdog/process-boundary hardening;
- runtime-harness matrix.

Це важлива межа, щоб не повернутися до blocked гілки M60.6.5 / M60.6.4.1.

Readiness

У review-card artifacts readiness розділена явно:

```
review_cards_ready      # ціль M61.0
runtime_execution_ready # false
scoring_ready           # false
calibration_ready       # false
```

Тобто картки готові для людського рев'ю, але не є доказом runtime проходження або якості моделі.

Чому це корисно

Human review cards дають практичну користь без важкого execution infrastructure:

- можна руками переглянути coverage terminal paths;
- можна побачити, як виглядає distraction / invalid_branch / clarification / skip_ahead;
- можна використати картки як QA checklist;
- можна підготуватися до майбутнього runtime execution milestone.

Межа зупинки

M61.0 — це правильний наступний шар після M60.8 handoff, бо він підвищує usability без відкриття runtime orchestration.

Наступні речі лишаються future work:

```
M62.0 Runtime Execution of Generated Testcases
backtrack
correction_backtrack
scoring generated cases
calibration generated cases
watchdog/process-boundary hardening
```

Розділ 63. Visual Graph Generator як companion utility

Visual Graph Generator — це утиліта для візуального огляду Ordo-програм. Вона читає `source/program.ordo.yaml` або сумісний YAML/IR і створює граф у форматах Mermaid, SVG або PNG.

Ця утиліта не є runtime core. Вона не виконує сесію, не викликає модель, не змінює YAML і не стверджує, що бізнес-логіка пройшла runtime validation. Її роль простіша й дуже корисна: показати структуру процесу так, щоб автор, рев'юер або розробник могли її швидко зрозуміти.

Де вона лежить

Починаючи з M61.2, утиліта входить у пакет так:

```
utilities/ordo_visual_graph_generator/
```

Поруч із нею залишається PathWalk:

```
ordo_pathwalk/
```

Ці дві утиліти не треба зливати в одну. PathWalk відповідає за тестові та review artifacts. Visual Graph Generator відповідає за візуальне пояснення дерева.

Базове використання

Mermaid-граф:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \
  source/program.ordo.yaml \
  --format mmd \
  --out runs/visual_graph/program.mmd
```

SVG-граф, якщо встановлено Graphviz:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \
  source/program.ordo.yaml \
  --format svg \
  --out runs/visual_graph/program.svg
```

Типовий author workflow

1. Автор пише або отримує `source/program.ordo.yaml`.
2. Visual Graph Generator показує дерево процесу.
3. Автор перевіряє вузли, переходи, gates, artifacts і terminal branches.
4. PathWalk генерує terminal paths, clean cases, bounded noise cases і review cards.
5. Visual Graph annotation overlay може підсвітити проблемні або нові елементи графа.

Annotation overlay

Окремий режим annotation overlay дозволяє підсвічувати елементи графа й додавати коментарі. Це корисно під час рев'ю: можна показати не тільки шлях, а й будь-який вузол, gate, state-поле, output або edge, який потребує уваги.

Межа відповідальності

Visual Graph Generator має залишатися read-only utility:

Ordo YAML/IR → graph artifacts

Він не повинен ставати runtime runner-ом, scorer-ом або benchmark harness. Якщо нам знадобиться автоматичне виконання згенерованих cases, це має бути окремий milestone, а не приховане розширення graph utility.

Розділ 64. Спільний маршрут companion utilities

M61.3 фіксує простий практичний маршрут використання утиліт, які йдуть поруч з Ordo. Це не runtime layer і не benchmark runner. Це author/reviewer layer: спочатку подивитися дерево, потім згенерувати path/case/card artifacts, а потім за потреби підсвітити коментарі на графі.

Чому потрібен окремий маршрут

Після M61.2 у пакеті є дві різні companion utilities:

```
Visual Graph Generator
→ показує дерево Ordo YAML/IR як Mermaid/SVG/PNG

PathWalk
→ створює graph summary, terminal paths, clean/noise cases, review cards
```

Якщо їх просто покласти поруч без маршруту, користувач пакета не одразу розуміє, з чого починати. M61.3 відповідає на це питання.

Рекомендований порядок

```
source/program.ordo.yaml
→ Visual Graph Generator: подивитися дерево
→ PathWalk real-module-graph: отримати структурний summary
→ PathWalk real-module-paths: побачити terminal paths
→ PathWalk real-module-clean-cases: створити clean-path cases
→ PathWalk real-module-noise-cases: створити bounded-noise cases
→ PathWalk real-module-review-cards: створити readable scenario cards
→ Visual Graph annotation overlay: підсвітити коментарі або проблемні місця
```

Що дивитися спочатку

Для автора або рев'юера найкращий перший артефакт — це граф:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py source/program.ordo.yaml --
format svg --out runs/companion_review/full_graph.svg
```

Якщо Graphviz недоступний, можна створити Mermaid:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py source/program.ordo.yaml --
format mmd --out runs/companion_review/full_graph.mmd
```

Далі — PathWalk artifacts

Після візуального огляду можна створювати структурні review artifacts:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-graph --source
source/program.ordo.yaml --out runs/companion_review/graph --force
```

Потім terminal paths:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-paths --summary
runs/companion_review/graph/REAL_MODULE_GRAPH_SUMMARY.json --out runs/companion_review/paths
--force
```

Потім clean/noise cases і human review cards.

Що це не доводить

Цей маршрут не доводить, що модель виконала testcase. Він не запускає runtime і не score-ить поведінку моделі. Він дає людині зрозумілі artifacts для аналізу.

Це принципова межа:

```
visual/review artifacts ≠ runtime execution results
```

Стабільна межа

На M61.3 companion utility layer має завершений практичний вигляд:

```
Visual Graph Generator + PathWalk = author/reviewer toolkit
```

Runtime execution of generated testcases лишається окремим майбутнім milestone, а не частиною цього маршруту.

Розділ 73. Як APF почав створювати сам себе

Цей розділ — практична історія з розробки `ordo.applied_project_factory`. Він показує не лише готовий результат, а й спосіб, у який складний Ordo-процес може поступово виникати через власне застосування.

1. Чому цей проєкт не можна було просто “написати”

Applied Project Factory починався не як готовий модуль і не як звичайний набір інструкцій. На старті була простіша, але глибша задача: зрозуміти, чи можемо ми створити процес, який допомагає будувати інші процеси.

Звичайний підхід виглядав би так: сісти й одразу описати повну інструкцію роботи — які питання ставити користувачу, як будувати дерево рішень, як створювати YAML, як перевіряти результат, як збирати пакет. Але в цьому підході є прихована проблема. Щоб добре описати такий процес, уже треба мати сам спосіб мислення про нього. Потрібно знати, які гілки будуть, де будуть кінцеві точки, як працюватимуть результати, шаблони, перевірки й передача готового пакета. На початку цього знання ще не було.

Тому APF не можна було просто написати “згори вниз” як готову специфікацію. Ми поступово намацували його форму. Спочатку було зрозуміло тільки одне: майбутній модуль має не просто генерувати текст, а проводити користувача через створення нового прикладного процесу так, щоб результат був придатний до виконання й перевірки.

Тобто це мав бути не довідник і не порада, а процес, який можна проходити:

вхід → питання → рішення → гілки → результати → шаблони → перевірка → передача готового пакета

Але як побудувати такий процес, якщо сам процес ще тільки створюється? Саме тут і з’явився головний переломний момент.

2. Головний перелом: ми створили частину процесу і почали нею користуватися

Найважливіше рішення в історії APF було не технічним, а методологічним. Ми зрозуміли, що не зобов’язані спочатку повністю завершити весь Applied Project Factory, а вже потім ним користуватися. Можна зробити інакше: створити одну життєздатну частину процесу, а потім використати її як робочий інструмент для створення решти.

Такою частиною стала гілка вільної форми — майбутня **гілка 3: вільний діалог**.

Її сенс був простий: користувач не мусить одразу давати готове дерево, YAML або формальну структуру. Він може пояснювати задум природною мовою: що хоче створити, які варіанти бачить, які документи мають виходити, які рішення має приймати процес. Модель у такому режимі не повинна губити ці ідеї. Вона має витягувати з діалогу:

- можливі питання;
- можливі гілки;

- можливі результати;
- підказки для майбутніх шаблонів;
- відкриті питання;
- припущення;
- чернетку майбутнього дерева.

Коли ця гілка стала достатньо зрозумілою, ми використали її не як майбутню частину готового APF, а як поточний робочий механізм. Ми буквально почали створювати APF через APF-подібний режим: говорили вільно про те, що потрібно додати, модель витягувала структуру, ми підтверджували рішення, після цього ці рішення ставали частиною самого процесу.

Це було схоже на відомий образ “витягнути самого себе за волосся з води”. Спочатку немає всього механізму. Але є маленька робоча частина. Вона ще не ідеальна, але вже достатня, щоб допомагати будувати наступну частину. Потім наступна частина підсилює першу. І так процес поступово починає сам себе стабілізувати.

Саме тут APF перестав бути просто об’єктом проектування. Він став інструментом проектування самого себе.

Цей момент важливий для всієї книги, бо він показує одну з найсильніших властивостей добре організованої мови процесів: вона може бути корисною ще до фінального випуску. Не потрібно чекати повної завершеності, щоб почати отримувати користь. Достатньо мати стабільний фрагмент, який можна виконувати, переглядати, уточнювати і знову застосовувати.

3. Як вільний діалог перетворився на каркас процесу

На початку вільний діалог міг виглядати занадто м’яким для чіткого процесу. Адже якщо користувач просто говорить, то де гарантія, що результат буде структурним? Де межа між ідеєю, припущенням і затвердженим вузлом дерева?

Цю проблему ми вирішили не заборонаю вільної форми, а введенням проміжних станів.

Вільний діалог не стає одразу затвердженим YAML. Він проходить через послідовність:

- вільний діалог
- витягування структури
- чернетка структури
- перегляд
- приведення до нормальної форми
- затверджений фрагмент процесу

Це дало нам дуже зручну модель роботи. Користувач міг мислити природно, не перемикаючись у формат YAML або формальних таблиць. Модель, зі свого боку, не мала права мовчки перетворювати сказане на фінальний контракт. Вона спочатку показувала витягнуту структуру, а ми її погоджували або виправляли.

Поступово з цього з’явилась повна логіка гілки 3:

- користувач говорить вільно
- модель витягує можливу структуру
- відділяє підтверджене від непідтвердженого

- показує чернетку процесу
- заводить результат у спільний шлях перегляду

Саме ця гілка дала нам практичний спосіб рухатися далі. Вона дозволила не застрягнути в спробі одразу описати ідеальний процес. Ми могли почати з нечіткого бачення, але кожен крок перетворював це бачення на більш точну структуру.

З цього моменту APF почав рости не як один великий документ, а як живий процес із перевіркою кожного нового фрагмента.

4. Інші гілки з'явилися вже навколо цього ядра

Коли стало зрозуміло, що вільний діалог може бути робочим входом у процес, ми побачили, що одного входу все одно замало. Реальні користувачі приходять у різних станах готовності.

Хтось має тільки ідею. Хтось має вже приблизну схему. Хтось хоче побудувати дерево з нуля. Хтось не створює новий процес, а виправляє вже існуючий. Тому APF отримав чотири стартові гілки:

1. Доменна модель + дерево рішень
2. Ручне дерево рішень
3. Вільний діалог
4. Коригування існуючого процесу

Але важливо, що вони не були незалежними “чотирма різними процесами”. Ми поступово прийшли до архітектури, де гілки мають різні входи, але далі сходяться в спільні частини.

Гілка 1 стала основним шляхом поступового створення процесу: модель допомагає будувати доменну модель, питання, вузли, гілки, кінцеві шляхи.

Гілка 2 стала перехідником для ручного дерева: якщо користувач уже має дерево в будь-якому людському форматі, APF не змушує його переписувати все в YAML. Спочатку дерево приводиться до нормальної форми, потім проходить перегляд.

Гілка 3 залишилась вільною формою, але вже не як хаотичне накидання ідей, а як контрольований шлях витягування структури.

Гілка 4 стала шляхом для покращення існуючого процесу. Це було необхідно, бо після створення APF ми одразу почали бачити: справжнє життя модуля — це не тільки створення, а й виправлення, точкові правки, адаптація, перевірка сумісності.

Ці гілки разом дали APF гнучкість. Але стабільність з'явилася тільки тоді, коли ми винесли спільні частини в окремі спільні блоки.

5. Кінцева точка перестала бути просто кінцем

Один із наступних великих кроків був пов'язаний із кінцевими точками процесу.

На ранньому етапі легко подумати, що кінцева точка — це просто кінець шляху в дереві. Користувач відповів на питання, ми дійшли до фінального вузла, отже гілка завершена. Але для APF цього було недостатньо.

Процес, який створює прикладний модуль або робочу інструкцію, не може завершуватись словами “ми дійшли до кінця”. Він має відповісти:

- що створюється на цьому кінцевому шляху?
- це вже наявний результат, новий артефакт, відсутність результату чи відкладене рішення?
- чи є шаблон?
- чи зрозуміло, які поля стану потрібні?
- чи можна показати приклад заповнення?
- чи користувач підтвердив прив’язку результату до цього шляху?

Так з’явився спільний підпроцес для результатів і шаблонів.

Ми не стали робити логіку результатів окремо в кожній стартовій гілці. Це було б швидко, але погано для майбутньої стабільності. Натомість усі гілки після своїх специфічних кроків заводяться в один спільний механізм:

```
виявлено кінцеву точку
→ вибір політики результату
→ перевірка стану шаблону
→ перегляд прикладу заповнення
→ підтвердження або правка користувачем
→ прив’язка результату до кінцевої точки
→ перевірка готовності кінцевого шляху
```

Це рішення дуже змінило якість процесу. До нього кінцевий шлях міг бути напівготовим: дерево ніби завершене, але незрозуміло, який документ має вийти. Після цього рішення кінцевий шлях став справжнім контрактом: шлях не готовий, доки не зрозумілий його результат.

Особливо важливим стало розділення між “результат не потрібен” і “рішення про результат відкладене”. Якщо результат не потрібен, це окреме явне рішення. Якщо рішення відкладене, воно не зникає — воно стає незавершеним пунктом і повертається на фінальній перевірці.

Це прибрало одну з найнебезпечніших форм нечіткості: коли “ще не вирішили” випадково починає виглядати як “нічого не потрібно”.

6. Перегляд шаблонів зменшив кількість пізніх зауважень

Ще одна складність була пов’язана з шаблонами. Можна домовитися, що на кінцевому шляху створюється документ, але якщо немає шаблону, то різні виконавці створять різні документи. Для APF це було неприйнятно.

Тому ми зафіксували правило: активний створюваний артефакт має мати шаблон або явно зафіксоване тимчасове обмеження. Але й цього було мало. Шаблон потрібно не тільки мати — його потрібно побачити в роботі.

Так з'явилася ідея прикладу заповнення. Модель не просто показує структуру шаблону, а генерує приклад майбутнього документа з тестовими або умовними значеннями. Завдяки цьому користувач бачить не абстрактну схему, а майже реальний результат.

Для великих документів ми прийшли до перегляду через файл: краще дати користувачу файл або пакет для перегляду, ніж змушувати оцінювати великий документ прямо в чаті. Це зробило перегляд ближчим до реальної роботи з книгою, документацією або файлами пакета.

Після цього зауважень стало менше не тому, що ми стали менше перевіряти, а тому, що помилки почали проявлятися раніше. Якщо шаблон неповний, це видно до передачі готового пакета. Якщо обов'язковий розділ не має джерела даних, це видно до генерації пакета. Якщо артефакт зайвий, його можна видалити до фінальної прив'язки.

У цьому місці APF став набагато практичнішим. Він перестав покладатися на обіцянку “потім згенеруємо документ” і почав вимагати: покажи форму документа зараз, перевір її зараз, погодь її зараз або зафіксуй, що вона ще не завершена.

7. Передача готового пакета стала окремою дисципліною

Після того як стартові гілки й шлях результатів/шаблонів стабілізувалися, залишалася ще одна небезпека: кожна гілка могла б мати власний фінальний шлях. Це швидко призвело б до розходження правил.

Тому ми винесли фінальну частину в спільний шлях перевірки й передачі готового пакета:

```
погодження створення source YAML
→ створення source YAML
→ мінімальна перевірка
→ рішення щодо повної перевірки
→ перегляд результату перевірки
→ цикл виправлень
→ фінальна перевірка незавершених пунктів
→ формування пакета передачі
→ фінальна передача
```

Цей фінальний шлях зробив завершення процесу однаковим для всіх стартових гілок.

Особливо важливим було чесне розділення станів перевірки. Мінімальна перевірка означає, що структура не зламана на базовому рівні. Але це не означає, що модуль готовий до випуску. Повна перевірка дивиться значно ширше: тести, покриття, стан, результати, артефакти, узгодженість, рішення “можна / не можна передавати далі”.

Ми окремо зафіксували принцип:

```
пропущена повна перевірка = обмеження, а не успішне проходження
```

Це коротке правило захищає процес від хибного “зеленого” статусу. Якщо перевірку не запускали, це не успіх і не провал. Це обмеження, яке має бути видимим.

Так само незавершені артефакти, відкладені рішення щодо результатів, відкриті виправлення або провалена перевірка не можуть тихо пройти у фінальну передачу.

Вони або виправляються, або видаляються з активного обсягу, або явно блокують передачу.

8. Як процес доростав через ітерації

APF розвивався не одним великим стрибком, а серією ітерацій. На кожній ітерації ми закривали не абстрактну “якість”, а конкретну проблему, яка виникала під час створення самого процесу.

Спочатку ми стабілізували окремі стартові гілки. Потім виявили, що логіка результатів і шаблонів не повинна дублюватися, і винесли її в спільний підпроцес. Потім так само винесли перевірку й передачу готового пакета. Після цього зробили загальний перегляд усього дерева, щоб перевірити, чи всі гілки сходяться правильно і чи немає сирітських вузлів, тупиків або дублювання логіки.

Ключові етапи виглядали так:

- створити базову логіку APF;
- додати режим коригування існуючого процесу;
- стабілізувати поступове створення дерева;
- оформити ручне дерево як перехідник;
- стабілізувати витягування структури з вільного діалогу;
- винести шлях результатів і шаблонів у спільний блок;
- винести перевірку й передачу готового пакета у спільний фінальний шлях;
- перевірити цілісність усього дерева;
- виконати повну перевірку;
- адаптувати модуль до стану release candidate.

На початку зауваження були концептуальні: процес ще міг бути нечітким, гілки могли дублювати логіку, кінцева точка могла бути недостатньо визначена. Наприкінці зауваження стали переважно технічними: службові дані, звіти, профіль перевірки, узгодженість пакета, посилання на версії.

Це важлива ознака стабілізації. Коли основна логіка ще нестійка, кожне покращення може змінювати форму всього процесу. Коли процес дозріває, покращення стають точковими. Ми вже не перепроєктовували APF, а доводили його до release candidate як стабільний модуль.

9. Чому наприкінці майже не залишилось змістовних зауважень

Найцікавіше в цій роботі — не те, що ми створили багато файлів або пройшли перевірку. Найцікавіше — як змінився характер помилок.

На початку ми постійно уточнювали саму природу процесу:

- що таке кінцева точка?
- коли результат вважається погодженим?
- чи можна мати гілку без шаблону?
- як вільний діалог стає структурою?
- що робити з незавершеним рішенням?
- де закінчується стартова гілка і починається спільний шлях?

Це були питання архітектури.

Після введення спільного підпроцесу для результатів і шаблонів та спільного фінального шляху перевірки більшість таких питань зникла. Процес отримав зрозумілі “рейки”. Нові гілки вже не могли довільно завершуватися, бо всі вони мали пройти ті самі перевірки. Результат не міг загубитися, бо прив’язка до кінцевої точки вимагала явного рішення. Шаблон не міг бути неявним, бо пакет перегляду мав його показати. Перевірка не могла бути декоративною, бо фінальна передача дивилася на реальні блокери.

Тому в кінці ми вже займалися не тим, “яким має бути APF”, а тим, “чи все в пакеті синхронізовано з тим APF, який ми вже погодили”.

Це і є нормальний перехід від проектування до стану кандидата на випуск.

10. Головний урок для книги

Історія APF важлива не тільки як історія одного модуля. Вона показує спосіб створення складних процесів у мовному пакеті.

Найціннішим виявився не фінальний архів і не конкретний набір вузлів. Найціннішим був спосіб, у який ми дійшли до стабільного результату:

1. не намагатися одразу написати весь процес ідеально;
2. створити одну життєздатну гілку;
3. почати використовувати її для створення самого процесу;
4. перетворювати вільні ідеї на чернетку структури;
5. погоджувати кожен фрагмент;
6. виносити повторювану логіку в спільні підпроцеси;
7. відрізнати незавершене від затвердженого;
8. не називати передачу готовою без перевірки;
9. доводити процес точковими ітераціями, а не постійним перепроєктуванням.

Гілка вільного діалогу стала не просто одним із режимів APF. Вона стала стартовим двигуном, завдяки якому APF зміг вирости. Ми спочатку створили механізм для перетворення нечіткої розмови на структуру, а потім використали цей механізм, щоб структурувати сам APF.

Це схоже на побудову мосту, коли перша маленька секція вже дозволяє перенести матеріали для наступної. Міст ще не завершений, але він уже допомагає будувати себе далі.

Саме тому APF вийшов не як статична інструкція, а як живий, виконуваний процес. Він народився через власне застосування. І це, мабуть, найважливіше знання, яке варто зберегти в книзі: іноді складний процес не треба повністю вигадувати наперед. Достатньо створити перший працюючий фрагмент, почати ним користуватися і дозволити йому дисципліновано добудувати решту.

Розділ 74. EXECUTION_TRACE — повна історія виконання процесу

EXECUTION_TRACE — це повноцінний елемент мови Ordo, який зберігає фактичну історію одного запуску процесу. Це не звичайний лог і не приховані міркування моделі. Це структурований, перевірюваний журнал того, які вузли, шляхи, дії, рішення, gates, зміни стану та outputs реально виникли під час виконання.

Основне правило:

один RUN → один основний EXECUTION_TRACE

Trace зберігається як append-only послідовність подій. Уже записана подія не переписується; виправлення або уточнення додається новою подією. Завдяки цьому trace можна використовувати для audit, debug, replay, regression testing, аналізу роботи аналітика та покращення playbook-a.

Головна структура

```
execution_trace:
  id: trace.history_event.001
  version: "1.0"
  run:
    run_id: run.history_event.001
    process_id: history_event.guided_intake
    process_version: "1.42"
    execution_mode: normal
    runtime_mode: chat_internal
    trace_source: hybrid
  status: running
  started_at: "2026-07-10T14:00:00+03:00"
  actor:
    actor_type: analyst
  source:
    entry_point: start
    input_refs: []
  capture_level: standard
  events: []
  replay:
    replayable: false
    replay_mode: deterministic
    required_inputs_preserved: false
  integrity:
    event_count: 0
    sequence_complete: true
```

Що фіксує одна подія

Кожна подія має номер послідовності, стабільний ID, тип, час, виконавця, payload і outcome. За потреби вона також посилається на активний node/path/phase, state before/after, decision, gate, output або батьківську подію.

Рівні деталізації

- minimal — lifecycle запуску, path, gates та outputs;
- standard — також inputs, decisions і state diffs;
- full — також actions, validations, warnings і checkpoints;
- audit — повна доказовість, actor attribution та integrity chain.

Стандартне значення — standard.

Replay

Trace може підтримувати чотири режими: `deterministic`, `re_evaluate`, `simulation`, `audit_only`. Прапорець `replayable: true` дозволений лише тоді, коли потрібні `inputs` збережені або безпечно `referenced`, а для зовнішніх залежностей визначено `strategy`.

Безпека

Trace не зберігає паролі, токени, секрети, повні конфіденційні документи або приватний `chain of thought`. Для чутливих значень використовуються `redaction`, `secure reference` або `hash`. Для рішень моделі зберігається короткий `reason code` та `evidence references`.

Відмінність від TRACE.LOG

`TRACE.LOG` — окреме діагностичне повідомлення. `EXECUTION_TRACE` — повний `canonical artifact` усього запуску. Багато `TRACE.LOG` можуть бути частиною одного `EXECUTION_TRACE`, але не замінюють його.

Legacy-сумісність

Старий блок `trace`: залишається сумісним представленням. Нові Ordo-програми повинні використовувати `execution_trace`. Старі поля перетворюються адаптером на `canonical trace events`.

Як компілятор і runtime працюють з EXECUTION_TRACE

Компілятор не записує історію виконання. Він перетворює блок `execution_trace`: на нормалізовану інструкцію `EXECUTION_TRACE.DEF`: чи увімкнений `trace`, який рівень деталізації використати, куди записувати файл і який `replay`-режим дозволений.

Runtime перед першим кроком створює `runtime/execution_trace.json`, після кожної змістовної дії додає незмінну подію, а при завершенні додає `terminal event`, фінальний стан і `checksum`. Після `terminal status` нові події додавати заборонено.

Рівень `minimal` зберігає лише каркас проходження; `standard` додає відповіді, рішення і зміни стану; `full` — технічні дії та попередження; `audit` — усі дозвалені типи подій. Формат файла в усіх випадках однаковий.

Replay може бути детермінованим, із повторним обчисленням правил, симуляційним без `side effects` або лише аудиторським. Секрети автоматично редагуються, а внутрішній `chain-of-thought` моделі не потрапляє у `trace`.

Розділ 75. Перевірка, replay і практичне використання EXECUTION_TRACE

Після того як EXECUTION_TRACE став елементом мови та отримав runtime-семантику, його потрібно перевіряти так само суворо, як gates, state і generated outputs.

Що потрібно тестувати

Мінімальний набір перевірок складається з шести шарів:

1. canonical YAML-прикладі читаються і відповідають закритому каталогу значень;
2. компілятор створює рівно один EXECUTION_TRACE.DEF;
3. runtime створює append-only trace, правильно нумерує події та забороняє запис після terminal status;
4. integrity validation знаходить пропуски sequence, неправильний event_count і зміну checksum;
5. replay режими правильно керують повторним обчисленням і side effects;
6. секрети редагуються до запису на диск.

Як читати trace

Людина читає trace зверху вниз як хронологію виконання. Для кожної події потрібно дивитися:

- де вона сталася (location);
- хто її виконав (actor);
- які дані були використані (payload);
- чи змінився state (state_effect);
- з яким gate, decision або output вона пов'язана (correlation);
- чим завершилася (outcome).

Чотири способи replay

deterministic повторює збережені inputs і рішення. re_evaluate залишає inputs, але заново обчислює decisions та gates. simulation забороняє зовнішні side effects. audit_only нічого не виконує і лише відтворює історію для перевірки.

Для аналізу поведінки аналітика або навчання достатньо audit_only. Для regression testing найчастіше потрібні deterministic і simulation.

Як виявляється зміна trace

Після terminal event runtime обчислює checksum canonical представлення trace. Якщо хтось змінить payload, sequence, event count або іншу частину історії, повторно обчислений checksum не збігатиметься. Це не цифровий підпис і не заміна захищеного сховища, але це надійний контроль випадкової або непогдженної зміни artifact-a.

Що не входить у trace

EXECUTION_TRACE не зберігає приватний chain-of-thought моделі. Він зберігає тільки observable decision summary, reason code, evidence references і фактичні переходи процесу. Паролі, токени та інші секрети мають бути redacted або замінені secure reference до серіалізації.

Практичне правило

Trace корисний лише тоді, коли його можна перевірити, безпечно зберігати і відтворювати за чітко визначеним режимом.

Додаток А. Словник Ordo

Цей словник пояснює базові терміни Ordo простими словами. Він не замінює повну специфікацію мови, але допомагає швидко згадати сенс основних понять.

AI Agent Programming Language

Мова для опису поведінки AI-агентів: їхніх ролей, дій, правил, станів, перевірок, outputs і зупинок.

Ordo належить саме до цього класу мов. Вона не є мовою навчання моделей. Вона є мовою керування поведінкою моделей і агентів.

Ordo

Мова перетворення людських інструкцій на керований execution contract для AI-моделі.

Коротко:

```
Ordo = intent → contract → context → state → path → steps → gates → result → handoff
```

Ordo Source

Людинозрозумілий запис Ordo-програми.

Ordo Source може бути написаний у форматі YAML, markdown-like structure або іншому читабельному форматі.

Його задача — бути зрозумілим для автора playbook-а, аналітика або архітектора.

Ordo IR

Проміжне представлення Ordo-програми, яке зручніше для виконання моделлю або runner-ом.

Найкращий поточний формат для IR — Semantic JSON IR.

Semantic JSON IR

JSON-представлення Ordo-програми, у якому кожен елемент має семантичний сенс: entry, node, gate, state, output, assertion, test, trace тощо.

Це не просто JSON-версія markdown. Це виконувана карта процесу.

Compact Opcode IR

Майбутній компактний формат Ordo IR, де інструкції записуються як короткі ор-коди.

Наприклад:

```
ENTRY.DEF
NODE.DEF
```

```
GATE.REPORT
ASSERT.NOT
```

Цей формат може бути корисним для native model support.

Intent

Намір користувача.

Відповідає на питання:

```
Що користувач насправді хоче зробити?
```

Intent не завжди дорівнює буквальному тексту повідомлення користувача.

Contract

Погоджені умови виконання.

Contract визначає:

```
що робимо;
що не робимо;
що треба підтвердити;
де модель має зупинитися;
який результат очікується.
```

Context

Дані, правила, обмеження і ситуація, у межах яких виконується Ordo-програма.

Context може включати документи, приклади, domain rules, user preferences, source rows, попередні рішення.

State

Поточний стан виконання.

State відповідає на питання:

```
Що вже відомо?
Що підтверджено?
Що ще очікує рішення?
Який path обрано?
Які gates пройдено?
```

Entry

Точка входу в Ordo-програму.

Entry визначає, з чого починається процес, які дані очікуються і який перший крок має виконати модель.

Node

Окремий вузол процесу.

Node може бути питанням, рішенням, перевіркою, кроком збору даних або переходом до іншого node.

Path

Обраний маршрут виконання.

Path визначає, який сценарій застосовується до конкретного input.

Step

Конкретна дія в межах path або node.

Наприклад: поставити питання, оновити state, перевірити gate, сформувати output.

Gate

Контрольна точка, яку треба пройти перед переходом далі.

Gate — це не порада. Це умова виконання.

Blocking gate

Gate, який зупиняє процес, якщо умова не виконана.

Наприклад: не можна створювати фінальний архів без self-check.

ASSERT.NOT

Негативна перевірка, яка явно забороняє певну дію або стан.

Наприклад:

```
не створювати фінальний output до підтвердження контракту
```

Status semantics

Правила значення статусів.

Status semantics визначає, що означає статус: чи можна виконувати дію, чи треба чекати, чи потрібне людське рішення, чи це по-ор.

Output

Результат, який має створити Ordo-програма.

Output може бути відповіддю, документом, архівом, JSON, тестовим пакетом, handoff note або іншим артефактом.

Handoff

Передача результату людині, іншій системі або наступному процесу.

Handoff має містити не тільки файл або текст, а й стан, обмеження, warnings, validation result і next steps.

FREEFORM

Контрольована зона для інструкцій, які ще не варто або неможливо повністю формалізувати.

FREEFORM не має бути смітником для всіх складних правил. Він має мати binding, reason, scope і coverage.

FREEFORM.COVERAGE

Оцінка того, яка частина інструкцій залишилася у FREEFORM і наскільки вона контрольована.

Ordo Core

Базова частина мови, яка містить універсальні конструкції: intent, contract, state, node, output, gates, assertions, trace тощо.

Ordo Profile

Набір спеціалізованих правил для певного типу роботи.

Наприклад: документаційний profile, QA profile, approval profile.

Domain Pack

Пакет правил для конкретної предметної області.

Наприклад: History Event Domain Pack або Monitoring Event Domain Pack.

Ordo Library

Reusable пакет готових Ordo-конструкцій, який можна підключити до поточної Ordo-програми.

Бібліотека може містити gates, tests, templates, profiles, execution patterns або domain rules.

include

Механізм підключення бібліотеки або її частини до Ordo-програми.

import

Механізм імпорту конкретних exports із бібліотеки.

use

Механізм фактичного використання підключеної конструкції в поточному процесі.

namespace

Простір імен, який запобігає конфліктам між однаковими назвами gates, nodes, templates або rules.

alias

Локальна коротка назва для підключеної бібліотеки або конструкції.

Debug mode

Режим запуску, у якому Ordo-програма формує повний trace виконання.

Trace

Журнал виконання, який показує, що сталося під час run.

Decision log

Журнал рішень моделі або runner-а.

Пояснює, чому було обрано певний path, node або дію.

State snapshot

Знімок state у конкретний момент виконання.

State diff

Різниця між двома state snapshots.

Gate report

Звіт про проходження або провал gates.

Knowledge trace

Список знань, правил або джерел, які були використані при виконанні.

Test case

Опис тестового сценарію для Ordo-програми.

Fixture

Вхідні дані для test case.

Expected behavior

Очікувана поведінка Ordo-програми: path, state, output, gate status, no-op або заборонена дія.

Regression suite

Набір тестів, який перевіряє, що нова зміна не зламала стару поведінку.

Coverage report

Звіт про те, які paths, gates, nodes, outputs, FREEFORM blocks і rules покриті тестами.

Feedback capture

Механізм фіксації зауважень користувача під час роботи з Ordo-програмою.

Improvement record

Структурований запис про проблему або покращення.

Має містити affected unit, severity, root cause, proposed patch і suggested tests.

Human approval

Людське підтвердження перед застосуванням важливої зміни.

Ordo не повинна автоматично змінювати playbook, library або compiler rules без такого підтвердження.

Helper-runner

Зовнішній виконавчий шар, який допомагає моделі виконувати Ordo-програму: зберігає state, перевіряє gates, пише trace, запускає tests.

Native model language

Режим, у якому модель підтримує Ordo як власну мову виконання, а не просто читає її як текст.

Оновлення словника для Ordo v0.12

gate.method

Обов'язкове поле gate, яке визначає спосіб перевірки контрольної точки.

Допустимі значення:

```
mechanical
self_verification
self_consistency
human
```

gate.method потрібен, щоб не змішувати механічні перевірки з семантичними судженнями моделі або людськими рішеннями.

trust_class

Клас довіри до результату перевірки.

Приклади:

```
deterministic
model_judgment
repeated_model_judgment
human_decision
```

trust_class допомагає читачу gate report зрозуміти, що означає passed: код поражував, модель оцінила, кілька проходів погодилися, або людина прийняла рішення.

mechanical gate

Gate, який можна перевірити детерміновано кодом або runner-ом.

Наприклад: кількість пунктів, наявність поля, формат дати, закриття code block.

self_verification gate

Gate, який перевіряється моделлю через окремий evidence-протокол.

Це не дає абсолютної гарантії, але робить семантичну перевірку явною і трасованою.

self_consistency gate

Gate, який перевіряється кількома незалежними проходами моделі з агрегацією результатів.

Використовується для критичних семантичних рішень, де один модельний прохід недостатній.

human gate

Gate, який завершується рішенням людини.

Модель може підготувати дані, але не може самостійно прийняти бізнесове, юридичне або незворотне рішення.

trace_source

Поле debug trace, яке показує джерело довіри до trace.

Допустимі значення:

```
model_self_report
runtime_enforced
hybrid
```

trace_source потрібен, щоб не плутати пояснювальний самозвіт моделі зі справжнім runtime log.

execution_mode

Режим виконання Ordo-програми.

Базові значення v0.12:

```
full_runtime
chat_internal
freeform_only
```

`full_runtime` дає найвищий рівень контролю, `chat_internal` є проміжним режимом, а `freeform_only` є найслабшим і найближчим до дисциплінованого `prompt-a`.

ASSERTION

Канонічний примітив Ordo для правила, яке має бути виконане або заборонене.

`ASSERT.NOT`, `negative gate` і `EXPECT.NOT` розглядаються як проєкції або скорочення від `ASSERTION`.

ASSERTION projection

Автоматичне розгортання одного `ASSERTION` у `runtime`-перевірку, `test expectation` і `debug violation record`.

Це зменшує ризик, що правило є в `playbook`, але забуте в `regression suite`.

on_unmatched_input

Правило вузла `NODE.DEF`, яке визначає, що робити, якщо відповідь користувача не відповідає жодному `allowed_answer`.

Зазвичай запускає `CLARIFY.REQUEST`, а після вичерпання спроб може переводити процес до людини.

CLARIFY.REQUEST

Окремий тип дії для контрольованого уточнення відповіді користувача.

Це не звичайне питання, а спеціальний вихід із ситуації, коли `input` не вклався в дерево рішень.

node_coverage_gap

Клас проблеми у `feedback loop`, який означає, що вузол не покриває реальні відповіді користувачів.

Якщо такі випадки повторюються, `playbook` потрібно розширити або уточнити.

control_level

Рівень строгості Ordo-програми.

Допустимі значення:

```
light
standard
strict
```

light підходить для простих задач, standard — для звичайних Ordo-програм, strict — для критичних процесів із mandatory gates, debug trace і regression coverage.

namespaced ID

Повний ідентифікатор Ordo-одиниці у compiled IR.

Наприклад:

```
domain_pack.history_event.G_CONTRACT_CONFIRMED
library.contract_first.G_NO_FINAL_OUTPUT
```

У Source можна використовувати короткі локальні назви, але у trace, gate report і improvement record потрібні повні namespaced IDs.

layer priority

Правило пріоритету шарів Ordo при конфліктах.

Базова ієрархія:

```
Core > active Profile > active Domain Pack > explicitly included Libraries > controlled
FREEFORM
```

Нижчий шар не може мовчки перекрити вищий.

explicit override

Явний дозвіл на заміну або уточнення правила з іншого шару.

Override має містити target, reason і, для критичних випадків, human approval.

FREEFORM.maturity

Стан зрілості FREEFORM-блоку.

Базові значення:

```
stable
volatile
candidate_for_formalization
```

FREEFORM.maturity допомагає відстежувати, чи не настав час перетворити повторюваний FREEFORM-фрагмент на gate, assertion, test або domain rule.

incident_threshold

Поріг кількості інцидентів для FREEFORM-блоку.

Коли incident_count >= incident_threshold, Ordo має рекомендувати формалізацію через FREEFORM_FORMALIZATION_RECOMMENDED.

Process Rail

Опорна структура процесу, яка допомагає ІІІ вести гнучкий діалог, але не втрачати state, route, gates, backtracking і output readiness.

AI Ordo Developer

Роль ІІІ у режимі створення Ordo-проекту. РМ описує задачу природною мовою, а AI Ordo Developer проектує YAML, запускає helper checks, компілює Semantic JSON IR і пояснює РМ-у стан проекту.

AI Ordo Executor

Роль ІІІ у режимі виконання готового Ordo-проекту. AI Ordo Executor читає Semantic JSON IR як Process Rail, веде людину по процесу, обробляє відхилення й використовує deterministic helper tools.

Deterministic Helper

Інструмент або CLI-команда, яка перевіряє механічну частину процесу: syntax, state, gates, next step, diff або validation. Deterministic Helper не замінює ІІІ й не говорить із користувачем напряду.

Contract → Artifact Coverage

Contract — структуровано підтверджене або запропоноване рішення процесу зі статусом поля.

Artifact requirement — правило, яке описує, у яких generated artifacts має з'явитися confirmed contract або його поля.

Rendered artifact assertion — перевірка вже згенерованого Markdown/JSON/YAML-файлу, а не лише шаблону.

Go/No-Go decision — машинозчитуване рішення, чи можна передавати пакет далі, чи він блокується через confirmation/artifact/template/runner issue.

json-ir

Canonical machine target Ordo runtime. У поточній реалізації це `compiled/program.ir.json`. CLI виконує runtime logic саме з нього.

ordo-code-view

AI-facing code-like projection Ordo-програми. Вона пояснює node contracts моделі, але не є source of truth.

session-trace

Append-only proof program runtime-сесії. Його пише CLI після кожного accepted intake -- submit.

targets.manifest.json

Manifest, який фіксує runtime targets, їхні полі, hashes і зв'язок із canonical IR hash.

runtime_view

Package-level режим, який визначає, який AI-facing формат embedded CLI має показувати за замовчуванням: json, ordo-code або json, ordo-code.

verify-targets

CLI-команда, яка перевіряє, що target files відповідають compiled/targets.manifest.json і не дрейфували від canonical IR.

verify-session

CLI-команда фінальної runtime-перевірки. Вона перевіряє target-set, session-chain, session-trace і canary scan.

scenario testing

Перевірка поведінки моделі на повному runtime-сценарії, а не лише перевірка окремих CLI-команд.

PathWalk

Companion-утиліта або benchmark-підхід для тестування того, чи модель коректно проходить Ordo decision path через CLI-enforced runtime protocol. PathWalk не є обов'язковою частиною runtime-пакета.

model_access_mode

Режим доступу моделі під час scenario testing. Наприклад: enforced, ir_readable, freeform.

protocol_compliance

Оцінка того, чи модель дотримувалась runtime-протоколу: використовувала embedded CLI, не читала compiled/* напямую, показувала evidence/digest і не переходила до наступного вузла без accepted submit.

path correctness

Оцінка того, чи фактично прийнятий CLI шлях збігся з очікуваним сценарним ground truth.

Restore-session

Runtime-команда Ordo M60.4 для безпечного повернення до попереднього snapshot. Restore не видаляє історію, а додає окрему append-only подію з evidence report, state snapshot, session trace step і оновленням live session state.

PathWalk matrix smoke

No-API compatibility smoke for the PathWalk scenario-testing utility. It checks that PathWalk can build and score current M60 runtime packages across `json, ordo-code`, and `json, ordo-code` runtime views before real model/API benchmark runs.

Benchmark readiness

A pre-benchmark condition: the testing utility can generate score files, aggregate summaries, and runtime metadata against the current Ordo runtime protocol without using legacy launchers or direct `compiled/*` reads.

EXECUTION_TRACE

Повний append-only артефакт історії одного запуску Ordo-процесу. Містить `ordered trace events`, посилання на `state`, `decisions`, `gates` і `outputs`, а також `replay` та `integrity metadata`. Не є приватним `chain of thought`.

Додаток В. Каталог ор-кодів

Цей додаток містить базовий каталог ор-кодів Ordo. Він не є фінальною повною специфікацією, але показує мінімальний набір конструкцій, які потрібні для опису, виконання, перевірки і покращення Ordo-програм.

1. Core ор-коди

INTENT.DEF

Описує намір користувача або процесу.

Використовується для фіксації того, що саме має бути досягнуто.

CONTRACT.DEF

Описує execution contract: що дозволено, що заборонено, які підтвердження потрібні, який результат очікується.

CONTEXT.DEF

Описує контекст виконання: джерела, документи, дані, обмеження, domain rules.

STATE.SCHEMA

Описує структуру state, яку Ordo-програма має підтримувати під час виконання.

STATE.SET

Встановлює або оновлює значення в state.

STATE.GET

Читає значення зі state.

ENTRY.DEF

Описує точку входу в процес.

NODE.DEF

Описує вузол процесу: питання, рішення, дію або перехід.

PATH.DEF

Описує можливий маршрут виконання.

PATH.SELECT

Фіксує вибір конкретного path.

STEP.DEF

Описує окремий крок у процесі.

OUTPUT.DEF

Описує очікуваний output.

HANDOFF.DEF

Описує правила передачі результату людині, системі або наступному процесу.

2. Gate і assertion оп-коди**GATE.DEF**

Описує контрольну точку.

GATE.EVAL

Оцінює, чи пройдений gate.

GATE.REPORT

Формує звіт про проходження gates.

ASSERT.REQUIRE

Вимагає наявності певної умови.

ASSERT.NOT

Забороняє певну дію, стан або output.

ASSERT.EQUAL

Перевіряє рівність очікуваного і фактичного значення.

ASSERT.EXISTS

Перевіряє, що потрібне значення існує.

ASSERT.MISSING

Перевіряє, що значення відсутнє, якщо саме це очікується.

3. Status оп-коди**STATUS.DEF**

Описує допустимий статус.

STATUS.SEMANTICS

Описує значення статусу для execution behavior.

STATUS.MAP

Мапить зовнішній статус на внутрішній статус Ordo.

STATUS.EVAL

Оцінює поточний статус і визначає дозволена поведінку.

4. FREEFORM оп-коди**FREEFORM.DEF**

Описує контрольований FREEFORM-блок.

FREEFORM.BIND

Прив'язує FREEFORM до конкретного node, gate, domain rule, output або library.

FREEFORM.REASON

Пояснює, чому ця частина залишена у FREEFORM.

FREEFORM.COVERAGE

Оцінює, наскільки FREEFORM-блок покритий тестами, gates або trace.

FREEFORM.MIGRATE.SUGGEST

Пропонує формалізувати частину FREEFORM у structured Ordo-конструкції.

5. Profile і domain оп-коди

PROFILE.DEF

Описує Ordo Profile.

PROFILE.BIND

Підключає profile до Ordo-програми.

DOMAIN.DEF

Описує domain pack.

DOMAIN.RULE

Описує предметне правило.

DOMAIN.VOCAB

Описує domain vocabulary.

DOMAIN.GATE

Описує domain-specific gate.

DOMAIN.OUTPUT

Описує domain-specific output.

6. Library оп-коди

LIB.DEF

Описує Ordo Library.

LIB.INCLUDE

Підключає бібліотеку до поточної Ordo-програми.

LIB.IMPORT

Імпортує конкретні exports із бібліотеки.

LIB.USE

Фіксує використання підключеної library-конструкції.

LIB.EXPORT

Описує, які конструкції бібліотека надає назовні.

LIB.NAMESPACE

Описує namespace бібліотеки.

LIB.ALIAS

Встановлює локальний alias для бібліотеки або export.

LIB.VERSION.REQUIRE

Фіксує вимогу до версії бібліотеки.

LIB.COMPAT.CHECK

Перевіряє сумісність бібліотеки з версією Ordo, profile або domain pack.

LIB.CONFLICT.DETECT

Виявляє конфлікти між підключеними бібліотеками або локальними правилами.

LIB.CONFLICT.RESOLVE

Описує явне рішення для конфлікту.

LIB.OVERRIDE.ALLOW

Дозволяє явний override.

LIB.OVERRIDE.DENY

Забороняє override.

7. Debug оп-коди

DEBUG.MODE

Вмикає debug mode або описує параметри debug execution.

TRACE.LOG

Фіксує execution trace.

DECISION.LOG

Фіксує рішення, прийняті під час виконання.

PATH.EXPLAIN

Пояснює, чому обрано або відхилено певний path.

STATE.SNAPSHOT

Фіксує знімок state.

STATE.DIFF

Фіксує різницю між двома state snapshots.

KNOWLEDGE.TRACE

Фіксує знання, документи, rules або examples, які були використані.

RUN.DRY

Описує dry run без створення фінального output або зовнішньої зміни.

RUN.REPLAY

Описує replay попереднього run.

FAILURE.EXPLAIN

Пояснює причину помилки або блокування виконання.

8. Test op-коди

TEST.DEF

Описує test case.

FIXTURE.DEF

Описує вхідні дані для test case.

EXPECT.PATH

Описує очікуваний path.

EXPECT.STATE

Описує очікуваний state або state diff.

EXPECT.OUTPUT

Описує очікуваний output.

EXPECT.NOOP

Описує сценарій, де дія не має виконуватися.

EXPECT.GATE

Описує очікуваний gate status.

EXPECT.NOT

Описує те, що не повинно статися під час test run.

REGRESSION.SUITE

Описує набір regression tests.

COVERAGE.REPORT

Описує покриття paths, gates, nodes, outputs, FREEFORM blocks, libraries і domain rules.

9. Feedback i improvement ор-коди

FEEDBACK.CAPTURE

Фіксує feedback користувача.

ISSUE.RECORD

Створює запис про проблему.

IMPROVEMENT.RECORD

Створює запис про покращення.

PROBLEM.CLASSIFY

Класифікує проблему.

ROOT_CAUSE.LINK

Прив'язує проблему до ймовірної причини.

AFFECTED.UNIT

Вказує, яку частину Ordo-програми зачіпає проблема або покращення.

Це може бути node, gate, rule, FREEFORM block, library, domain pack, profile або compiler rule.

PATCH.SUGGEST

Пропонує зміну.

TEST.SUGGEST

Пропонує тест, який треба додати.

REGRESSION.ADD

Додає test case до regression suite.

VERSION.NOTE

Фіксує зміст зміни для версії.

CHANGELOG.UPDATE

Оновлює changelog.

LESSON.LEARNED

Фіксує висновок, який треба враховувати в майбутніх версіях.

10. Documentation Runtime ор-коди

DOC.SPLIT

Описує розбиття великого документа на частини.

DOC.CATALOG

Описує каталог доступних документів або секцій

DOC.SELECT

Фіксує вибір релевантних документів для поточного node або path.

DOC.RENDER

Описує створення rendered artifact.

RENDER.VALIDATE

Перевіряє готовий rendered artifact, а не тільки template.

DOC.TRACE

Фіксує, які документи або секції були використані.

11. Approval ор-коди

APPROVAL.REQUIRE

Описує дію, яка потребує людського підтвердження.

APPROVAL.CAPTURE

Фіксує отримане підтвердження.

APPROVAL.BLOCK

Блокує виконання до підтвердження.

APPROVAL.REJECT

Фіксує відхилення дії або зміни.

12. Мінімальний набір для production playbook

Для складного production playbook мінімально потрібні:

```
INTENT.DEF
CONTRACT.DEF
CONTEXT.DEF
STATE.SCHEMA
ENTRY.DEF
NODE.DEF
PATH.DEF
GATE.DEF
ASSERT.NOT
STATUS.SEMANTICS
OUTPUT.DEF
HANDOFF.DEF
TRACE.LOG
GATE.REPORT
TEST.DEF
FIXTURE.DEF
EXPECT.PATH
EXPECT.GATE
EXPECT.OUTPUT
REGRESSION.SUITE
COVERAGE.REPORT
FEEDBACK.CAPTURE
IMPROVEMENT.RECORD
```

Якщо playbook використовує бібліотеки, додатково потрібні:

```
LIB.INCLUDE
LIB.VERSION.REQUIRE
LIB.COMPAT.CHECK
LIB.CONFLICT.DETECT
LIB.OVERRIDE.ALLOW / LIB.OVERRIDE.DENY
```

Якщо `playbook` має значну частину у `FREEFORM`, додатково потрібні:

```
FREEFORM.DEF
FREEFORM.BIND
FREEFORM.COVERAGE
FREEFORM.MIGRATE.SUGGEST
```

Короткий підсумок

Ор-коди Ordo — це не просто назви команд. Це спосіб зробити складну інструкцію видимою для виконання, перевірки, трасування і покращення.

Каталог ор-кодів потрібен для того, щоб Ordo-програми можна було не тільки читати, а й компілювати, тестувати, порівнювати, дебажити і поступово переводити до `native model support`.

12. Конструкції Ordo v0.12: reliability, trust semantics і execution modes

GATE.METHOD

Описує спосіб перевірки `gate`.

Допустимі значення:

```
mechanical
self_verification
self_consistency
human
```

`Gate` без `method` у `v0.12` має вважатися помилкою компіляції.

TRUST.CLASS

Описує клас довіри до результату перевірки.

Приклади:

```
deterministic
model_judgment
repeated_model_judgment
human_decision
```

TRACE.SOURCE

Фіксує джерело довіри до `execution trace`.

Допустимі значення:

```
model_self_report
runtime_enforced
hybrid
```

EXECUTION.MODE

Фіксує режим виконання програми.

Допустимі значення:

```
full_runtime
chat_internal
freeform_only
```

ASSERTION.DEF

Описує канонічне правило, яке має бути виконане або заборонене.

Приклад:

```
assertion:
  id: A_NO_FINAL_OUTPUT_BEFORE_APPROVAL
  polarity: not
  condition: final_output_before_approval
  phase: [runtime, test]
  severity: block
  on_fail: STOP
```

ASSERTION.PROJECT

Розгортає ASSERTION у runtime-перевірку, test expectation і debug violation record.

CLARIFY.REQUEST

Окремий ор-код для контрольованого уточнення, коли input користувача не відповідає жодному allowed answer.

NAMESPACE.RESOLVE

Розгортає локальні IDs у повні namespaced IDs у compiled IR.

VERSION.REQUIRE

Фіксує вимогу до версії Core, Profile, Domain Pack або Library.

LAYER.CONFLICT.CHECK

Перевіряє конфлікти між Core, Profile, Domain Pack, Libraries і FREEFORM.

OVERRIDE.DEF

Описує явний override правила з іншого шару.

Override має містити target, reason і approval policy.

CONTROL_LEVEL.DEF

Описує рівень строгості Ordo-програми:

```
light
standard
strict
```

FREEFORM.MATURITY

Фіксує стан зрілості FREEFORM-блоку:

```
stable
volatile
candidate_for_formalization
```

FREEFORM.INCIDENT_COUNT

Рахує кількість feedback/improvement інцидентів, пов'язаних із конкретним FREEFORM-блоком.

FREEFORM.FORMALIZATION.WARNING

Попереджає, що FREEFORM-блок перевищив incident threshold і має бути розглянутий для формалізації.

G_NO_UNRESOLVED_LAYER_CONFLICT

Системний gate, який блокує виконання, якщо знайдено конфлікт між шарами без явного OVERRIDE.DEF.

Рекомендований метод:

```
method: mechanical
trust_class: deterministic
```

Contract and Artifact Coverage

Op / construct	Призначення
CONTRACT.DEF	Описує підтверджений або запропонований контракт процесу.
CONTRACT.FIELD	Описує поле контракту та його статус.
CONTRACT.STATUS	Вказує missing, candidate, proposed, confirmed, blocked, not_applicable.
ARTIFACT.DEF	Описує generated artifact, який має бути створений.
ARTIFACT.REQUIREMENT	Зв'язує confirmed contract fields з required artifacts.
COVERAGE.RULE	Описує deterministic coverage policy.
RENDERED_ARTIFACT.ASSERT	Перевіряє вже згенерований файл.
CONSISTENCY.REPORT	Фіксує cross-artifact consistency result.
GO_NO_GO.DECISION	Дає фінальне machine-readable рішення.

M46.2 contract/artifact оп-коди

- `CONTRACT.INSTANCE` — декларація конкретного process contract з полями та статусами.
- `ARTIFACT.DEF` — декларація очікуваного generated artifact.
- `ARTIFACT.REQUIREMENT` — правило, яке мапить поля підтвердженого контракту в артефакти.
- `COVERAGE.RULE` — політика перевірки contract → artifact coverage.
- `RENDERED_ARTIFACT.ASSERT` — майбутня runtime/assertion-проекція для перевірки rendered files.
- `GO_NO_GO.DECISION` — машинозчитуване рішення щодо готовності пакета.

EXECUTION_TRACE.DEF

Оголошує canonical execution-trace artifact для одного run. Визначає metadata, capture level, replay policy, integrity contract і ordered event stream.

TRACE.EVENT.APPEND

Зарезервований ор-код для append-only додавання canonical trace event. Фактична compiler/runtime семантика вводиться в наступній частині інтеграції.

Додаток С. Приклади програм

Навіщо потрібен цей додаток

У попередніх розділах Ordo розглядалася як мова керування поведінкою моделі: з intent, contract, state, nodes, gates, outputs, debug trace, tests, libraries і improvement loop.

Цей додаток показує, як усе це може виглядати у вигляді невеликих Ordo-програм.

Це не фінальний синтаксис мови. Це навчальні приклади, які допомагають побачити головну ідею: Ordo-програма описує не просто те, що модель має написати, а те, як вона має вести процес.

Приклад 1. Найменша Ordo-програма

Завдання

Користувач хоче отримати коротку відповідь клієнту про затримку доставки.

Звичайний prompt міг би виглядати так:

Напиши ввічливу відповідь клієнту про затримку доставки.

У Ordo це краще розкласти на intent, contract, context, gates і output.

```

ordo:
  version: "0.11"

intent:
  id: "INTENT_DELAY_RESPONSE"
  goal: "Підготувати коротку ввічливу відповідь клієнту про затримку доставки."

contract:
  output_type: "customer_message"
  language: "uk"
  tone: "polite"
  max_length: "short"
  must_not:
    - "вигадувати точну дату доставки, якщо її немає в context"
    - "звинувачувати клієнта"
    - "обіцяти компенсацію без підтвердження"

context:
  known:
    delay_reason: "логістична затримка"
    exact_new_date: null

state:
  fields:
    delay_reason_confirmed: true
    exact_new_date_available: false

nodes:
  - id: "N_CHECK_DATE"
    type: "decision"
    question: "Чи є підтверджена нова дата доставки?"
    when:
      field: "exact_new_date_available"
      equals: false
    next: "N_GENERATE_WITHOUT_DATE"
  - id: "N_GENERATE_WITHOUT_DATE"
    type: "output"

```

```

    instruction: "Сформуї коротке повідомлення без точної нової дати."

  gates:
    - id: "G_NO_FAKE_DATE"
      type: "ASSERT.NOT"
      rule: "output must not contain invented delivery date"

  output:
    id: "OUT_CUSTOMER_MESSAGE"
    type: "chat_message"

```

Що тут важливо

Ordo не просто просить модель написати текст. Вона явно каже:

- що відомо;
- чого не відомо;
- що не можна вигадувати;
- яку перевірку пройти перед результатом;
- який тип output потрібен.

Приклад 2. Та сама програма у Semantic JSON IR

Ordo Source може компілюватися в Semantic JSON IR.

```

{
  "ordo_version": "0.11",
  "program_id": "CUSTOMER_DELAY_RESPONSE",
  "ops": [
    {
      "op": "INTENT.DEF",
      "id": "INTENT_DELAY_RESPONSE",
      "goal": "Підготувати коротку ввічливу відповідь клієнту про затримку доставки."
    },
    {
      "op": "CONTRACT.DEF",
      "id": "CONTRACT_OUTPUT",
      "output_type": "customer_message",
      "language": "uk",
      "tone": "polite",
      "must_not": [
        "invent_delivery_date",
        "blame_customer",
        "promise_compensation_without_confirmation"
      ]
    },
    {
      "op": "STATE.SCHEMA",
      "id": "STATE_MAIN",
      "fields": {
        "delay_reason_confirmed": "boolean",
        "exact_new_date_available": "boolean"
      }
    },
    {
      "op": "NODE.DEF",
      "id": "N_CHECK_DATE",
      "node_type": "decision",
      "condition": {
        "field": "exact_new_date_available",
        "equals": false
      },
      "next": "N_GENERATE_WITHOUT_DATE"
    },
    {
      "op": "ASSERT.NOT",
      "id": "G_NO_FAKE_DATE",
      "target": "OUT_CUSTOMER_MESSAGE",

```

```

    "rule": "output_contains_invented_delivery_date"
  },
  {
    "op": "OUTPUT.DEF",
    "id": "OUT_CUSTOMER_MESSAGE",
    "output_type": "chat_message"
  }
]
}

```

Що показує приклад

Semantic JSON IR не обов'язково красивий для людини. Його завдання інше: бути достатньо структурованим, щоб модель, helper-runner або майбутній native Ordo executor могли виконувати його послідовно.

Приклад 3. Gate як зупинка перед фінальним результатом

Сценарій

Користувач просить створити фінальний пакет, але approval ще не отриманий.

```

gates:
- id: "G_PRE_FINAL_APPROVAL"
  type: "APPROVAL.REQUIRE"
  required_before:
  - "OUTPUT.FINAL"
  approval_source: "human_user"
  status_on_missing: "blocked"

assertions:
- id: "A_NO_FINAL_WITHOUT_APPROVAL"
  type: "ASSERT.NOT"
  rule: "final_output_created_before_G_PRE_FINAL_APPROVAL_passed"

```

Очікувана поведінка:

Модель не створює фінальний пакет.
Модель пояснює, який gate блокує дію.
Модель ставить питання або просить підтвердження.

Погана поведінка:

Модель усе одно створює пакет, а потім пише: "я припустила, що підтвердження є".

У Ordo така поведінка має вважатися violation.

Приклад 4. Debug run

Той самий процес можна запустити в debug mode.

```

run:
  mode: "debug"
  trace_source: "model_self_report"
  input: "Зроби фінальний пакет одразу"

trace:
  selected_path:
    id: "PACKAGE_GENERATION"
    reason: "user requested final package"

  gates:
    - id: "G_PRE_FINAL_APPROVAL"
      status: "blocked"
      reason: "human approval is missing"

```

```

decisions:
  - id: "D_BLOCK_FINAL_OUTPUT"
    reason: "required gate is blocked"
    result: "final output not created"

violations: []

output:
  final_package_created: false
  response_type: "gate_block_message"

```

Debug run потрібен не для користувацького фінального тексту, а для пояснення поведінки Ordo-програми.

Приклад 5. Test case для gate

```

test:
  id: "TC_NO_FINAL_PACKAGE_WITHOUT_APPROVAL"
  method: human
  trust_class: human_decision
  mode: "test"

fixture:
  user_message: "Зроби фінальний пакет одразу"
  state:
    pre_final_approval: false

expect:
  path:
    selected: "PACKAGE_GENERATION"

  gate:
    id: "G_PRE_FINAL_APPROVAL"
    status: "blocked"

  output:
    final_package_created: false

  not_allowed:
    - "create_archive"
    - "mark_package_ready"

```

Цей test case перевіряє не красу відповіді, а правильну поведінку.

Приклад 6. No-op test

Но-оп сценарії дуже важливі. Іноді правильна дія — не створювати нову подію, не змінювати state і не генерувати зайвий output.

```

test:
  id: "TC_NOOP_SAME_VALUE"
  mode: "test"

fixture:
  before:
    field: "status"
    value: "ACTIVE"
  after:
    field: "status"
    value: "ACTIVE"

expect:
  noop: true
  no_new_event: true
  no_change_record: true
  explanation_required: true

```

У звичайному prompt-підході модель часто намагається “щось зробити”, навіть коли зміни немає. Ordo має вміти явно сказати: правильний результат — по-ор.

Приклад 7. Підключення бібліотеки

Ordo-програма може підключати готові правила повторного використання.

```
libraries:
  include:
    - id: "ordo.validation.contract_first"
      version: "0.1"
      as: "contract_first"

    - id: "ordo.validation.rendered_artifact"
      version: "0.1"
      as: "render_validation"

  use:
    - "contract_first.required_contract_gate"
    - "render_validation.rendered_output_check"
```

Це означає, що програма не дублює стандартні gates, а явно використовує готові перевірені конструкції.

Правило:

Бібліотека не повинна підключатися неявно.
Версія має бути зафіксована.
Конфлікти мають бути явними.
Override має бути дозволений окремо.

Приклад 8. Conflict між бібліотеками

```
libraries:
  include:
    - id: "ordo.status.strict"
      version: "0.1"
      as: "strict_status"

    - id: "ordo.status.legacy"
      version: "0.1"
      as: "legacy_status"
```

Якщо обидві бібліотеки визначають різну семантику одного статусу, Ordo має зупинитися.

```
conflict:
  id: "CONFLICT_STATUS_READY"
  type: "STATUS.SEMANTICS_CONFLICT"
  symbol: "ready_for_first_run"
  sources:
    - "strict_status"
    - "legacy_status"
  resolution_required: true
```

Погана поведінка:

Модель мовчки обрала одну з версій.

Правильна поведінка:

Модель повідомляє про conflict і просить resolution rule або human decision.

Приклад 9. Feedback & Improvement Loop

Користувач під час роботи каже:

Ти поставив питання про output занадто рано. Спочатку потрібно було підтвердити source field.

Ordo має створити improvement record.

```
feedback_capture:
  id: "FB-001"
  source: "user_feedback"
  message: "Ти поставив питання про output занадто рано. Спочатку потрібно було підтвердити source field."

improvement_record:
  id: "IR-001"
  classification:
    type: "wrong_question_order"
    severity: "medium"

  affected_unit:
    kind: "node"
    id: "N_OUTPUT_DETAILS"
    owner:
      layer: "domain_pack"
      name: "history_event"

  root_cause_hypothesis:
    - "NODE order allows output collection before source field confirmation"

  proposed_patch:
    - "move N_OUTPUT_DETAILS after G_SOURCE_FIELD_CONFIRMED"
    - "add gate before output details question"

  suggested_tests:
    - id: "TC_SOURCE_FIELD_BEFORE_OUTPUT_DETAILS"
      expected:
        required_order:
          - "G_SOURCE_FIELD_CONFIRMED"
          - "N_OUTPUT_DETAILS"

  approval:
    required: true
    status: "pending"
```

Цей приклад показує, що feedback в Ordo не губиться в чаті. Він стає структурованим кандидатом на покращення.

Приклад 10. Малий guided intake

```
entry:
  id: "ENTRY_NEW_EVENT"
  goal: "Почати guided intake нової історичної події."

state:
  fields:
    event_alias: null
    source_field: null
    old_value: null
    new_value: null
    path_confirmed: false

nodes:
  - id: "N_SELECT_PATH"
    type: "question"
    ask: "Який тип зміни потрібно описати?"
    writes:
      - "path_candidate"
```

```

- id: "N_CONFIRM_SOURCE_FIELD"
  type: "question"
  ask: "Яке source field змінюється?"
  writes:
    - "source_field"

- id: "N_COLLECT_VALUES"
  type: "question"
  ask: "Які old/new values потрібно зафіксувати?"
  requires:
    - "source_field"
  writes:
    - "old_value"
    - "new_value"

gates:
- id: "G_SOURCE_FIELD_CONFIRMED"
  type: "APPROVAL.REQUIRE"
  required_before:
    - "N_COLLECT_VALUES"

- id: "G_NO_PACKAGE_BEFORE_CONTRACT"
  type: "ASSERT.NOT"
  rule: "final_package_created_before_required_contract_fields_confirmed"

```

Тут guided intake — це не просто діалог. Це керований процес, де кожне питання має місце, writes, requirements і gates.

Приклад 11. FREEFORM з binding

```

freeform:
  id: "FF_DOMAIN_EDGE_CASES"
  type: "controlled_freeform"
  purpose: "Описати складні доменні edge cases, які ще не формалізовані в окремі gates."
  binding:
    used_by:
      - "N_SELECT_PATH"
      - "G_DOMAIN_REVIEW"
    must_not_override:
      - "ASSERT.NOT"
      - "APPROVAL.REQUIRE"
      - "STATUS.SEMANTICS"
  content: |
    У деяких випадках зміна може виглядати як оновлення поля, але фактично бути появою
    зовнішньої події.
    Якщо є ExternalHistoryEvent, потрібно перейти до path A4, а не трактувати запис як простий
    field delta.

```

FREEFORM тут не є “смітником для правил”. Він має purpose, binding і межі впливу.

Приклад 12. Coverage report

```

coverage_report:
  program: "history_event_playbook"

  paths:
    total: 5
    covered: 4
    uncovered:
      - "A4"

  gates:
    total: 14
    covered: 12
    uncovered:
      - "G_EXTERNAL_EVENT_NORMALIZED"
      - "G_RENDERED_ARTIFACT_VALIDATED"

```

```

freeform:
  total_blocks: 5
  covered_by_tests: 3
  untested:
    - "FF_DOMAIN_EDGE_CASES"
    - "FF_EXAMPLES_LEGACY"

libraries:
  included: 2
  compatibility_checked: 2
  conflicts: 0

```

Coverage report потрібен для чесної відповіді на питання: наскільки ця Ordo-програма справді контрольована тестами й трасуванням?

Короткий підсумок

Приклади в цьому додатку показують головну різницю між prompt-ом і Ordo-програмою.

Prompt зазвичай каже моделі:

Зроби ось це.

Ordo каже:

Ось мета.
 Ось контракт.
 Ось state.
 Ось допустимі шляхи.
 Ось gates.
 Ось заборонені дії.
 Ось output.
 Ось debug trace.
 Ось tests.
 Ось improvement loop.

Саме тому Ordo можна не тільки читати, а й виконувати, дебажити, тестувати, покращувати і поступово переносити до native model support.

Додаток D. Чеклисти

Цей додаток містить практичні чеклисти для роботи з Ordo-програмами. Їх можна використовувати під час створення нового playbook-a, ревізії існуючої інструкції, підготовки domain pack-a, підключення бібліотек, дебагу, тестування або фінальної перевірки перед передачею в роботу.

Чеклист в Ordo — це не заміна gates і не заміна тестів. Це допоміжний інструмент для автора, аналітика або reviewer-a, який допомагає не пропустити важливі частини процесу.

D.1. Чеклист готовності ідеї до Ordo

Перед тим як перетворювати інструкцію на Ordo-програму, потрібно зрозуміти, чи є в ній керований процес.

- ☐ Є зрозумілий intent.
- ☐ Є очікуваний результат.
- ☐ Є межі відповідальності моделі.
- ☐ Є речі, які модель не має робити.
- ☐ Є точки, де потрібне підтвердження людини.
- ☐ Є варіанти шляху виконання.
- ☐ Є дані, які потрібно зібрати перед дією.
- ☐ Є правила, які не можна пропускати.
- ☐ Є фінальний handoff або output.
- ☐ Є ризик помилки, який виправдовує формалізацію.

Якщо більшість пунктів порожні, можливо, це ще не Ordo-програма, а звичайне коротке прохання до моделі.

Якщо більшість пунктів заповнені, це хороший кандидат для Ordo.

D.2. Чеклист Intent

Intent має відповідати на питання: що користувач насправді хоче отримати?

- ☐ Intent сформульований одним або кількома простими реченнями.
- ☐ Intent не змішаний з implementation details.
- ☐ Intent не містить непідтверджених припущень.
- ☐ Intent відділений від output format.
- ☐ Intent не підміняється назвою задачі.
- ☐ Intent не описує випадковий приклад замість загальної мети.
- ☐ Intent зрозумілий людині без читання всієї програми.

Поганий intent:

Зробити як у попередньому прикладі.

Кращий intent:

Провести користувача через керований збір даних для створення аналітичного пакета історичної події.

D.3. Чеклист Contract

Contract визначає, що саме має бути підтверджено до початку або до критичних дій.

- ☐ Відомо, які поля обов'язкові.
- ☐ Відомо, які рішення має підтвердити користувач.
- ☐ Відомо, що можна вважати confirmed.

- [] Відомо, що не можна вважати confirmed.
- [] Є правила для неповного contract.
- [] Є gate, який блокує подальше виконання без contract.
- [] Contract не формується мовчки з припущень моделі.
- [] Contract зберігається у state.
- [] Зміни contract фіксуються як state diff.

Критичне правило:

Якщо contract не підтверджений, модель не має поводитися так, ніби він підтверджений.

D.4. Чеклист Context

Context — це дані, документи, правила, приклади й обмеження, які потрібні для правильного виконання.

- [] Вказано, які документи є джерелами правил.
- [] Вказано, які дані є вхідними.
- [] Вказано, які приклади є лише прикладами.
- [] Вказано, які частини context мають вищий пріоритет.
- [] Вказано, що робити при конфлікті джерел.
- [] Вказано, які дані не можна вигадувати.
- [] Вказано, які джерела можна цитувати або використовувати як evidence.
- [] Context не змішаний із state.

D.5. Чеклист State

State показує, що вже відомо, що підтверджено, що очікує рішення і що заборонено.

- [] State має явну схему.
- [] State містить confirmed і pending значення.
- [] State не змішує факт і припущення.
- [] State оновлюється після кожного важливого кроку.
- [] Є state snapshot для debug mode.
- [] Є state diff для зміни значень.
- [] Не використовується прихована пам'ять замість state.
- [] Критичні рішення не губляться в тексті відповіді.

Приклад корисного state:

```
state:
  contract:
    alias:
      value: "LU_CHANGE_STATUS"
      status: "confirmed"

  approvals:
    pre_archive:
      status: "pending"

  execution:
    selected_path: "A1"
```

D.6. Чеклист Entry і Node

Entry і Node відповідають за керований діалог.

- [] Є стартовий ENTRY.
- [] Кожен NODE має зрозумілу мету.
- [] Кожен NODE ставить тільки потрібні питання.
- [] NODE не забігає наперед.
- [] NODE має умови входу.
- [] NODE має умови виходу.
- [] NODE оновлює state.
- [] NODE знає, куди перейти далі.

- [] Є правила для невизначеної відповіді користувача.
- [] Є правила для зміни попереднього рішення.

Погано:

Спитай у користувача все, що потрібно.

Краще:

```
node:
  id: "N_COLLECT_ALIAS"
  asks:
    - "Підтверди alias події."
  writes:
    - "state.contract.alias"
  next:
    when_confirmed: "N_COLLECT_SOURCE_FIELD"
```

D.7. Чеклист Path

Path потрібен там, де один і той самий intent може виконуватися різними шляхами.

- [] Усі основні paths перелічені.
- [] Для кожного path є умови вибору.
- [] Для кожного path є reason.
- [] Є список rejected paths у debug mode.
- [] Є правило, що робити при неоднозначності.
- [] Є gate або user approval для ризикового path.
- [] Path не обирається за інтуїцією моделі.
- [] Path selection можна протестувати.

D.8. Чеклист Gate

Gate — це контрольна точка, а не порада.

- [] Gate має id.
- [] Gate має умову проходження.
- [] Gate має статус.
- [] Gate має evidence.
- [] Gate має наслідок при fail/block.
- [] Відомо, чи gate blocking.
- [] Gate не можна пройти без потрібних даних.
- [] Gate не ховається у FREEFORM.
- [] Gate відображається у debug trace.
- [] Gate має тест або coverage.

Критичне правило:

Якщо gate є blocking, модель не має права продовжити дію, яку gate блокує.

D.9. Чеклист ASSERT.NOT

Негативні перевірки потрібні, щоб заборонити небезпечні дії.

- [] Є список дій, які модель не має робити.
- [] Для кожної забороненої дії є ASSERT.NOT.
- [] ASSERT.NOT прив'язаний до конкретного ризику.
- [] ASSERT.NOT перевіряється перед output.
- [] ASSERT.NOT має тест.
- [] ASSERT.NOT не замінений м'якою рекомендацією.

Приклади:

- [] Не створювати фінальний архів до approval.
- [] Не вигадувати source row.

- [] Не трактувати example як confirmed rule.
- [] Не змінювати contract без підтвердження.

D.10. Чеклист Output

Output має бути явним і перевірюваним.

- [] Вказано тип output.
- [] Вказано формат output.
- [] Вказано обов'язкові частини output.
- [] Вказано, що не має входити в output.
- [] Вказано, які gates мають бути пройдені перед output.
- [] Output можна перевірити через validation.
- [] Output має handoff note або next action.
- [] Output не створюється раніше дозволеного моменту.

D.11. Чеклист FREEFORM

FREEFORM потрібен для складних смислових частин, але він має бути контрольованим.

- [] У FREEFORM є id.
- [] Є reason, чому ця частина не формалізована.
- [] Є purpose.
- [] Є binding.
- [] Вказано, де FREEFORM використовується.
- [] Вказано, що FREEFORM не може override.
- [] Є coverage або план coverage.
- [] Немає прихованих gates усередині FREEFORM.
- [] Немає прихованих статусів усередині FREEFORM.
- [] Немає правил, які краще винести в Core/Profile/Domain Pack.

D.12. Чеклист Debug

Debug mode має пояснювати виконання, а не просто дублювати відповідь.

- [] Є run_id.
- [] Є input snapshot.
- [] Є selected path.
- [] Є rejected paths with reasons.
- [] Є decision log.
- [] Є state snapshots.
- [] Є state diffs.
- [] Є gate report.
- [] Є knowledge trace.
- [] Є warnings.
- [] Є violations.
- [] Є failure explanation, якщо процес зламався.

Debug trace має відповідати на питання:

Чому модель зробила саме це?

D.13. Чеклист Test

Тести мають перевіряти поведінку Ordo-програми, а не тільки фінальний текст.

- [] Є TEST.DEF.
- [] Є FIXTURE.DEF.
- [] Є expected path.
- [] Є expected state.
- [] Є expected gates.
- [] Є expected output.
- [] Є EXPECT.NOT для заборонених дій.
- [] Є по-ор тести.
- [] Є негативні тести.

- [] Є тести для edge cases.
- [] Є тести для FREEFORM-backed рішень.

D.14. Чеклист Regression Suite

Regression suite потрібен перед змінами.

- [] Є набір базових сценаріїв.
- [] Є сценарії для кожного path.
- [] Є сценарії для blocking gates.
- [] Є сценарії для no-op.
- [] Є сценарії для помилкових або неповних input.
- [] Є сценарії для бібліотек.
- [] Є сценарії для domain pack.
- [] Regression suite запускається перед release.
- [] Результат regression suite фіксується в report.

D.15. Чеклист Coverage

Coverage показує, наскільки Ordo-програма справді покрита контролем.

- [] Покриті всі основні paths.
- [] Покриті всі blocking gates.
- [] Покриті всі критичні ASSERT.NOT.
- [] Покриті всі output types.
- [] Покриті всі status transitions.
- [] Покриті no-op сценарії.
- [] Покриті critical FREEFORM blocks.
- [] Покриті imported libraries.
- [] Є список uncovered areas.
- [] Є план покращення coverage.

D.16. Чеклист Feedback & Improvement Loop

Коли користувач вказує на проблему, її потрібно перетворити на structured record.

- [] Feedback captured.
- [] Вказано original user message.
- [] Проблема класифіковано.
- [] Вказано severity.
- [] Вказано affected unit.
- [] Є root cause hypothesis.
- [] Є proposed patch.
- [] Є suggested test.
- [] Є required approval.
- [] Є version note або changelog item.
- [] Є regression test після виправлення.

Feedback не має губитися в чаті.

D.17. Чеклист бібліотек Ordo

Для підключених бібліотек потрібно контролювати версії, namespace і конфлікти.

- [] Бібліотека підключена явно.
- [] Версія зафіксована.
- [] Є alias або namespace.
- [] Вказано, які exports використовуються.
- [] Виконано compatibility check.
- [] Перевірено конфлікти.
- [] Override дозволений тільки явно.
- [] Немає implicit imports.
- [] Є trust level.
- [] Бібліотека покрита тестами або має власний test pack.

Погано:

```
include:
- "ordo.qa"
```

Краще:

```
include:
- library: "ordo.qa.manual_runbook"
  version: "0.1"
  as: "manual_qa"
```

D.18. Чеклист Domain Pack

Domain Pack має описувати доменну логіку, а не випадковий набір прикладів.

- [] ∈ domain vocabulary.
- [] ∈ domain-specific paths.
- [] ∈ domain-specific gates.
- [] ∈ domain-specific statuses.
- [] ∈ domain-specific output templates.
- [] ∈ mapping до Ordo Core.
- [] ∈ controlled FREEFORM для складних edge cases.
- [] ∈ domain tests.
- [] ∈ coverage report.
- [] ∈ improvement loop.

D.19. Чеклист міграції старого playbook-а в Ordo

- [] Старий playbook розбитий на логічні частини.
- [] Визначено intent.
- [] Визначено contract.
- [] Визначено decision tree.
- [] Визначено paths.
- [] Визначено nodes.
- [] Визначено gates.
- [] Визначено statuses.
- [] Визначено outputs.
- [] Визначено handoff.
- [] Визначено FREEFORM blocks.
- [] Визначено libraries, які можна підключити.
- [] Побудовано Semantic JSON IR.
- [] Створено test cases.
- [] Створено coverage report.
- [] Проведено consistency check.

D.20. Чеклист перед фінальним handoff

Перед передачею Ordo-програми або пакета в роботу потрібно перевірити:

- [] Усі mandatory sections присутні.
- [] Немає невіршених blocking gates.
- [] Немає pending contract fields.
- [] Немає unresolved conflicts.
- [] Немає implicit assumptions.
- [] Немає hidden rules у FREEFORM.
- [] Debug trace можна отримати.
- [] Regression suite пройдений або чесно вказано, що він ще не створений.
- [] Coverage report сформований.
- [] Improvement records враховані або перенесені в backlog.
- [] Version note оновлений.
- [] Handoff note написаний.

Короткий підсумок

Чеклисти не роблять Ordo-програму правильною автоматично. Але вони допомагають автору не пропустити ключові речі:

```
intent
contract
context
state
path
node
gate
assertion
output
debug
test
coverage
feedback
library
domain pack
handoff
```

Якщо Ordo-програма проходить ці чеклисти, її вже можна не тільки читати, а й підтримувати, тестувати, покращувати і передавати іншим людям або системам.

D.22. Чеклист Ordo v0.12 Reliability

Цей чеклист використовується після оновлення програми або playbook-а до Ordo v0.12.

```
[ ] Кожен gate має обов'язкове поле method.
[ ] Для кожного gate вказано trust_class.
[ ] Mechanical gates не змішані з semantic model-judgment gates.
[ ] Gates з method: self_verification мають evidence protocol.
[ ] Критичні semantic gates мають generator/critic або self_consistency pattern.
[ ] Gate report показує method і trust_class.
[ ] Execution trace має trace_source.
[ ] Ordo-програма має execution_mode.
[ ] У документації не створено враження, що model_self_report доповнює runtime_enforced log.
[ ] ASSERT.NOT описаний як shortcut/projection від ASSERTION.
[ ] Критичні заборони оформлені як ASSERTION з phase: [runtime, test].
[ ] EXPECT.NOT не дублює вручну ASSERT.NOT, а походить із ASSERTION projection.
[ ] NODE має on_unmatched_input або явно описаний fallback.
[ ] Для unmatched input передбачено CLARIFY.REQUEST.
[ ] Feedback class node_coverage_gap використовується для повторюваних непокритих відповідей.
[ ] Program має control_level: light / standard / strict.
[ ] Strict-програма має regression coverage для mandatory gates і assertions.
[ ] У trace, gate_report і improvement_record використовуються namespaced IDs.
[ ] Includes мають version.
[ ] Layer priority визначений і не порушений.
[ ] Override є явним і має reason.
[ ] Немає unresolved layer conflicts.
[ ] FREEFORM-блоки мають maturity.
[ ] FREEFORM incident_count і incident_threshold визначені там, де блок ризиковий.
[ ] FREEFORM з перевищенням threshold має formalization warning або improvement record.
```

D.23. Чеклист gate.method

```
[ ] method: mechanical використовується тільки для детермінованих перевірок.
[ ] method: self_verification використовується для семантичних суджень моделі з evidence.
[ ] method: self_consistency використовується для критичних повторюваних модельних перевірок.
[ ] method: human використовується для людських рішень.
[ ] У прикладах немає GATE.CHECK без method.
[ ] У gate_report немає status: passed без пояснення method/trust_class.
```

D.24. Чеклист `execution_mode`

- [] `full_runtime` використовується тільки там, де `runner` реально контролює переходи.
- [] `chat_internal` чесно описаний як проміжний режим.
- [] `freeform_only` не подається як повноцінне контрольоване виконання.
- [] У `chat_internal` явно описано, що точка запуску `gate` не примусова без зовнішнього `runtime`.
- [] Для `chat_internal` описано `state_backing` або інший спосіб зменшити `drift state`.
- [] Для `strict`-процесів `freeform_only` не використовується без окремого `warning`.

D.25. Чеклист ASSERTION

- [] Заборони описані через `ASSERTION`.
- [] У кожного `ASSERTION` є `polarity`.
- [] У кожного `ASSERTION` є `phase`.
- [] `Blocking assertion` має `severity: block`.
- [] `Runtime projection` створює `ASSERT.NOT` або `gate`.
- [] `Test projection` створює `EXPECT.NOT`.
- [] `Debug projection` створює `violation record`.
- [] `Regression suite` перевіряє критичні `assertions`.

D.26. Чеклист `namespace/version/layer priority`

- [] `Source` може мати локальні `IDs`, але `IR` має повні `namespaced IDs`.
- [] `Uci improvement records` посилаються на `namespaced IDs`.
- [] `Uci libraries` мають `version`.
- [] `Floating version` не використовується для `strict`-процесів.
- [] `Core` не перекривається нижчим шаром без `explicit override`.
- [] `Domain Pack` не мовчки змінює `Profile`.
- [] `Library` не мовчки змінює `Domain Pack`.
- [] `FREEFORM` не може перекрити формальне правило.
- [] `G_NO_UNRESOLVED_LAYER_CONFLICT` присутній для складних програм.

Додаток Е. Антипатерни

Антипатерн — це підхід, який здається зручним або швидким, але в реальній роботі робить Ordo-програму менш керованою, менш перевірюваною або менш безпечною.

Цей додаток не замінює повну перевірку playbook-а. Його зручно використовувати як швидкий список типових помилок перед тим, як віддавати Ordo-програму в роботу, публікувати library або збирати фінальний package.

Е.1. Великий prompt замість Ordo-програми

Ознака

Уся логіка описана одним великим текстом без явних intent, contract, state, node, gate, output і handoff.

Чому це проблема

Модель може виконати частину інструкції, пропустити важливий gate, переплутати приклад із правилом або перейти до фінального результату раніше, ніж процес справді готовий.

Як виправити

Розбити інструкцію на явні частини:

```
intent → contract → context → state → path → steps → gates → result → handoff
```

Е.2. Контракт прихований у тексті

Ознака

У тексті є фрази на кшталт “очевидно”, “за потреби”, “якщо все готово”, але не визначено, хто і як підтверджує готовність.

Чому це проблема

Модель починає сама вирішувати, що вже підтверджено, навіть якщо користувач цього не підтверджував.

Як виправити

Оформити контракт явно через CONTRACT.DEF, ANSWER.REGISTRY, APPROVAL.REQUIRE і blocking gates.

Е.3. Gate як порада

Ознака

Gate описаний у стилі “бажано перевірити”, але після failed gate процес усе одно може рухатися далі.

Чому це проблема

Такий gate не контролює виконання. Він лише прикрашає документацію.

Як виправити

Для критичних перевірок використовувати blocking gate:

```
if gate.failed → stop / ask / repair / no handoff
```

Е.4. Відсутність ASSERT.NOT

Ознака

Ordo-програма описує, що потрібно зробити, але не описує, чого робити не можна.

Чому це проблема

Модель може створити зайвий файл, придумати відсутнє значення, пропустити approval або згенерувати фінальний package у ситуації, де мала зупинитися.

Як виправити

Додати негативні перевірки:

```
ASSERT.NOT final_output_before_approval
ASSERT.NOT invented_source_row
ASSERT.NOT hidden_required_gate_inside_freeform
```

Е.5. FREEFORM як смітник

Ознака

У FREEFORM кладуть усе, що було складно формалізувати: gates, правила, status semantics, approval logic, templates, заборони.

Чому це проблема

FREEFORM перестає бути контрольованою лазівкою і стає місцем, де ховається справжня логіка виконання.

Як виправити

У FREEFORM залишати тільки те, що справді не варто формалізувати на цьому етапі. Критичні правила переносити в GATE.DEF, ASSERT.NOT, STATUS.SEMANTICS, OUTPUT.DEF або library.

Е.6. Невидимі імпорти бібліотек

Ознака

Ordo-програма користується правилами або шаблонами з library, але це не зазначено через include, import або use.

Чому це проблема

Незрозуміло, звідки взялося правило, яка версія бібліотеки використана і хто відповідає за зміну поведінки.

Як виправити

Усі бібліотеки підключати явно:

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
    as: "contract_first"
```

Е.7. Override без дозволу

Ознака

Profile, Domain Pack або Library мовчки переписує gate, status або output rule, визначений в іншому шарі.

Чому це проблема

Зміна поведінки стає невидимою. Старі тести можуть ламатися без очевидної причини.

Як виправити

Будь-який override має бути явним, із причиною і trace:

```
override:
  allow:
    - target: "G_PRE_ARCHIVE_APPROVAL"
      reason: "domain pack requires stricter approval gate"
```

Е.8. Тестування тільки фінального тексту

Ознака

Тести перевіряють лише те, що модель видала в кінці, але не перевіряють path, state, gates, no-op і заборонені дії.

Чому це проблема

Модель може отримати правильний результат неправильним шляхом. У production playbook-ах це небезпечно.

Як виправити

Тестувати поведінку:

```
EXPECT.PATH
EXPECT.STATE
EXPECT.GATE
EXPECT.OUTPUT
EXPECT.NOOP
EXPECT.NOT
```

E.9. Відсутність debug trace

Ознака

Коли Ordo-програма поводить ся неправильно, немає запису, який path був обраний, які paths відхилені, які gates пройдені, які знання використані.

Чому це проблема

Неможливо зрозуміти, що саме зламалося: інструкція, domain rule, library, fixture, compiler або поведінка моделі.

Як виправити

Для складних Ordo-програм запускати debug або test mode і зберігати:

```
TRACE.LOG
DECISION.LOG
PATH.EXPLAIN
STATE.DIFF
GATE.REPORT
KNOWLEDGE.TRACE
```

E.10. Feedback губиться в чаті

Ознака

Користувач вказує на проблему або покращення, але це залишається тільки в діалозі й не перетворюється на structured improvement record.

Чому це проблема

Одна й та сама проблема повторюється. Інструкції покращуються випадково, без backlog, тестів і changelog.

Як виправити

Використовувати Feedback & Improvement Loop:

```
FEEDBACK.CAPTURE
→ ISSUE.RECORD
→ ROOT_CAUSE.LINK
→ PATCH.SUGGEST
→ TEST.SUGGEST
→ human approval
→ regression run
```

E.11. All-in-one як source of truth

Ознака

Великий зібраний markdown-файл редагують напряду, а окремі source-файли залишаються застарілими.

Чому це проблема

Збірка перестає бути відтворюваною. Різні версії документації починають суперечити одна одній.

Як виправити

Вважати source-of-truth окремі документи:

section files / chapter files / domain pack files → all-in-one → PDF / archive

Е.12. Перевірка шаблону замість готового артефакту

Ознака

Перевірено template або markdown-source, але не перевірено фактичний PDF, архів, Jira task або інший зібраний output.

Чому це проблема

Помилка може з'явитися саме на етапі render або package generation: зламане посилання, відсутній файл, неправильний порядок, незакритий кодовий блок, некоректна нумерація.

Як виправити

Додавати RENDER_VALIDATE і перевіряти фактичний результат перед handoff.

Е.13. “Модель сама здогадається”

Ознака

Автор не описує явно path selection, gates, outputs або exceptions, бо вважає, що модель “і так зрозуміє”.

Чому це проблема

Чим складніший процес, тим дорожчі неправильні припущення моделі.

Як виправити

Усе, що має значення для виконання, має бути явно представлене у Source або IR. Якщо щось не формалізовано, це має бути контрольований FREEFORM, а не мовчазне припущення.

Е.14. Занадто великий Core

Ознака

У Core починають додавати domain-specific правила, шаблони конкретних документів або правила окремого продукту.

Чому це проблема

Core втрачає універсальність. Мова стає важкою для підтримки й погано переноситься між доменами.

Як виправити

Тримати Core мінімальним, а спеціалізацію виносити у Profiles, Domain Packs і Libraries.

Е.15. Немає межі відповідальності між моделлю і runner-ом

Ознака

Незрозуміло, що саме вирішує модель, що контролює helper-runner, а що має підтвердити людина.

Чому це проблема

Гейт може бути пропущений, output може бути створений без approval, а trace не покаже, хто саме дозволив перехід.

Як виправити

Явно розділити відповідальність:

```
model → semantic work
runner → process control / gates / state / tests
human → approval / governance / final decisions
```

Е.16. Короткий чек перед handoff

Перед тим як віддавати Ordo-програму або package, варто швидко перевірити:

- Чи є явний intent?
- Чи є contract?
- Чи не сховані gates у FREEFORM?
- Чи є blocking gates перед фінальним output?
- Чи є ASSERT.NOT для заборонених дій?
- Чи всі libraries підключені явно?
- Чи є debug/test режим для складної логіки?
- Чи є regression suite?
- Чи feedback перетворюється на improvement records?
- Чи перевірено фактичний rendered artifact?

Головне правило:

Якщо поведінку не можна пояснити, перевірити і відтворити, це ще не Ordo-програма, а лише добре написаний prompt.

Е.12. False deterministic gate

Ознака

Gate виглядає як звичайна контрольна точка, але не має method, або semantic judgement подано так, ніби це mechanical check.

Чому це проблема

Користувач бачить `status: passed` і не розуміє, що один gate був перевірений кодом, а інший — оцінений моделлю.

Як виправити

Кожен gate має мати `method` і `trust_class`:

```
gate:
  id: G_NO_UNSUPPORTED_FACTS
  method: self_verification
  trust_class: model_judgment
```

Е.13. Gate без `method`

Ознака

У програмі є `GATE.DEF` або `GATE.CHECK`, але не вказано спосіб перевірки.

Чому це проблема

Автор playbook-a не зробив явний вибір між `mechanical`, `model judgement` і `human decision`.

Як виправити

У v0.12 відсутній `gate.method` має бути `compilation error`.

Е.14. Trace без `trace_source`

Ознака

Debug trace показує `selected path`, `state diff` і `gate report`, але не вказує, чи це `runtime log`, `hybrid trace` або `model self-report`.

Чому це проблема

Trace може створити завищену довіру до пояснення моделі.

Як виправити

Додавати:

```
trace_source: model_self_report | runtime_enforced | hybrid
```

Е.15. Chat-only режим подано як `full runtime`

Ознака

Процес виконується в чаті без зовнішнього runner-a, але документація описує gates так, ніби вони примусово виконуються кодом.

Чому це проблема

У `chat_internal` код може перевірити `gate`, але модель усе ще вирішує, коли саме запустити перевірку.

Як виправити

Явно вказувати:

```
execution_mode: chat_internal
```

і не плутати його з:

```
execution_mode: full_runtime
```

E.16. Заборона продубльована вручну у трьох місцях

Ознака

Одна й та сама заборона окремо описана як `ASSERT.NOT`, `negative gate` і `EXPECT.NOT`.

Чому це проблема

Рано чи пізно один із трьох записів буде оновлений, а інші — ні.

Як виправити

Описувати заборону один раз як `ASSERTION`, а далі розгортати її через `projection`.

E.17. Відсутній fallback для `unmatched input`

Ознака

`NODE.DEF` має `allowed_answers`, але не має `on_unmatched_input`.

Чому це проблема

Коли користувач відповідає неочікувано, модель повертається до імпровізації.

Як виправити

Додати:

```
on_unmatched_input:
  action: CLARIFY.REQUEST
  strategy: rephrase_and_narrow
  max_attempts: 2
```

E.18. Локальні IDs у `feedback records`

Ознака

`Improvement record` посилається на `G1` або `N2`, але не вказує, з якого шару або бібліотеки цей ID.

Чому це проблема

У складній системі G1 може існувати в Core, Profile, Domain Pack і Library одночасно.

Як виправити

У feedback, trace і reports використовувати тільки повні namespaced IDs.

E.19. Floating library version

Ознака

Library підключена без версії або з надто широкою версією для strict-процесу.

Чому це проблема

Поведінка програми може змінитися без зміни source-файлу.

Як виправити

Фіксувати version policy і не використовувати floating versions у critical / strict workflows.

E.20. FREEFORM ніколи не дозріває

Ознака

Один і той самий FREEFORM-блок постійно породжує feedback incidents, але не формалізується.

Чому це проблема

FREEFORM перетворюється на постійну сіру зону.

Як виправити

Додати maturity lifecycle:

```

freeform:
  maturity: candidate_for_formalization
  incident_count: 3
  incident_threshold: 3

```

і створити FREEFORM_FORMALIZATION_RECOMMENDED.

Додаток F. Практичний довідник ор-кодів і YAML-атрибутів

Цей додаток доповнює короткий каталог ор-кодів із додатка В. Там зручно швидко подивитися назви конструкцій. Тут інша мета: дати практичний довідник для людини, яка відкриває *.ordo.yaml, program.ir.json або намагається написати власний playbook.

Головне правило цього додатка:

назва поля сама по собі ще нічого не пояснює;
для практичного використання треба знати тип поля, допустимі значення,
значення кожного варіанта і те, як поле читає compiler / runtime / validator.

Якщо значення поля є фіксованим enum, його не можна вигадувати довільно. Якщо значення є convention string, його можна вводити в межах пакета, але воно має бути описане в policy docs або package docs. Якщо значення є free text, воно допомагає людині й моделі, але не повинно використовуватися як deterministic runtime condition без окремого правила.

F.1. Типи значень у YAML

Перед тим як читати окремі ор-коди, треба розрізняти типи атрибутів.

strict enum

Поле має закритий перелік дозволених значень.

Приклад:

```
control_level: strict
```

Дозволені значення:

```
light
standard
strict
```

Практичне правило: не додавайте нове значення без оновлення schema / compiler / validator. Якщо написати very_strict, людина може зрозуміти намір, але мова не зобов'язана це прийняти.

convention string

Поле є рядком, але його значення має бути описане як домовленість пакета.

Приклад:

```
resume_policy: return_to_current_node_after_deviation
```

Це не звичайний людський опис. Якщо runtime або AI-layer використовує це поле для поведінки, значення має бути пояснене в policy docs. Без цього воно перетворюється на магічний рядок.

free text

Поле містить людський опис.

Приклад:

```
description: "Користувач описує процес природною мовою."
```

Таке поле можна писати вільніше. Але compiler не має робити deterministic routing тільки на основі free text. Для routing потрібні gates, allowed answers, policies або інші структурні поля.

reference / id

Поле посилається на інший об'єкт.

Приклад:

```
entry: N_START
next: N_REVIEW
artifact: A_SUMMARY
```

Практичне правило: referenced id має існувати. Інакше граф може мати dead link.

path

Поле містить шлях до файлу або шаблон шляху.

Приклад:

```
path_pattern: "reports/{run_id}/summary.md"
```

Практичне правило: шлях не є просто текстом. Validator може перевіряти наявність файлу, розширення, формат або allowed output directory.

state selector

Поле посилається на частину state.

Приклад:

```
field: G_EVENT_IDENTITY_CONTRACT.alias
```

Практичне правило: selector має відповідати state schema або contract field. Інакше output може бути згенерований з неіснуючих даних.

F.2. Program і metadata

PROGRAM.DEF / верхній рівень програми

Призначення: описує програму як цілісний модуль: ідентичність, версію, сумісність і режим виконання.

Де живе: верхній рівень source YAML або header semantic JSON IR.

Поля:

```
program_id / id
  required: yes
  type: id string
  meaning: стабільний ідентифікатор програми або модуля.
  example: ordo.applied_project_factory

ordo_version
  required: yes
```

```

type: version string
meaning: версія мовного пакета Ordo, з якою сумісна програма.
example: "0.12"

control_level
  required: yes
  type: strict enum
  values: light, standard, strict
  meaning: рівень дисципліни runtime / validation.

execution_mode
  required: yes
  type: strict enum
  values: full_runtime, chat_internal, freeform_only
  meaning: спосіб виконання програми.

version
  required: recommended
  type: version string
  meaning: версія самого прикладного модуля.

module_id
  required: recommended for packages
  type: id string
  meaning: package/module identity y standard applied modules.

compatibility
  required: recommended
  type: object / freeform policy
  meaning: додаткові обмеження сумісності з runtime, CLI або schema.

```

Мінімальний приклад:

```

program_id: ordo.applied_project_factory
ordo_version: "0.12"
control_level: standard
execution_mode: full_runtime
version: "0.1.0-rc.1"

```

Типова помилка: вказати version модуля, але не вказати ordo_version або execution_mode. Тоді незрозуміло, яким runtime-профілем це виконувати.

control_level

Тип: strict enum.

Дозволені значення:

```

light
  Мінімальна дисципліна. Добре для ранніх чернеток, але слабше для audit / handoff.

standard
  Нормальний режим для більшості прикладних процесів.

strict
  Найжорсткіший режим. Потрібен там, де важливі gates, trace, validation і release discipline.

```

Як це читає runtime: рівень контролю може впливати на те, чи дозволені freeform-відхилення, чи обов'язкові gates, наскільки суворо трактуються warnings.

Типова помилка: поставити strict, але залишити ключові рішення без gates або без confirmed state.

execution_mode

Тип: strict enum.

Дозволені значення:

```
full_runtime
    Очікується повний runtime / CLI-guided execution.

chat_internal
    Процес може виконуватися як дисциплінований чат із внутрішнім state.

freeform_only
    Структура мінімальна. Підходить для чернеток, але не для release-grade процесів.
```

Як це читає compiler/runtime: визначає очікувану форму виконання і набір обов'язкових артефактів. full_runtime має мати entry point, nodes, gates, state і validation artifacts.

F.3. Interaction, Process Rail і conversation semantics

INTERACTION.MODEL

Призначення: описує ролі людини, AI-моделі та deterministic helper/CLI layer.

Де живе: runtime model docs, IR ops або package policy.

Поля:

```
op
  required: yes in IR ops
  type: strict opcode string
  value: INTERACTION.MODEL

id
  required: yes
  type: id string
  meaning: ідентифікатор interaction model.

roles / interaction_roles
  required: yes
  type: object
  meaning: хто за що відповідає.

raw_tool_output_policy
  required: recommended
  type: convention string / enum-like string
  common values: summarize_for_human, hide_raw_output, expose_on_request
  meaning: чи показувати користувачу сирий output tools або людське пояснення.

human_responsibility
  required: recommended
  type: free text / list
  meaning: рішення, які має підтвердити людина.

ai_responsibility
  required: recommended
  type: free text / list
  meaning: що може робити модель: вести процес, пояснювати, пропонувати next step.

cli_responsibility
  required: recommended
  type: free text / list
  meaning: deterministic checks, validation, compile, tests.
```

Приклад:

```
op: INTERACTION.MODEL
id: interaction_model
```

```
roles:
  human: confirms_content_decisions
  ai: guided_process_driver
  cli: deterministic_validator
raw_tool_output_policy: summarize_for_human
```

Типова помилка: дозволити AI самостійно приймати змістові рішення, які мають бути confirmed by human.

PROCESS_RAIL.DEF / process_rail

Призначення: задає дисципліну руху по процесу: чи можна відхилитися, як повертатися, чи дозволений backtracking.

Де живе: process_rail у semantic JSON IR або окрема оп-операція.

Поля:

```
rail_id
  required: recommended
  type: id string
  meaning: ідентифікатор rail policy.

state_tracking
  required: recommended
  type: strict enum
  values:
    required: runtime має вести state явно.
    optional: state може бути частковим або допоміжним.

allow_deviation
  required: recommended
  type: boolean
  meaning: чи може користувач тимчасово відійти від поточного node.

require_resume_after_deviation
  required: recommended
  type: boolean
  meaning: чи треба повернути користувача до перерваного node після відхилення.

backtracking
  required: recommended
  type: strict enum
  values:
    enabled: назад можна повертатися.
    disabled: назад повертатися не можна.
    restricted: назад можна, але за правилами invalidation / review.
```

Приклад:

```
process_rail:
  rail_id: guided_intake_rail
  state_tracking: required
  allow_deviation: true
  require_resume_after_deviation: true
  backtracking: restricted
```

Типова помилка: дозволити backtracking, але не описати, які dependent state fields треба скидати або переглядати.

CONVERSATION.SEMANTICS

Призначення: описує, як класифікувати людський ввід під час виконання процесу: відповідь на поточний node, уточнення, відхилення, повернення назад, нову вимогу.

Де живе: conversation_semantics у IR або runtime policy docs.

Поля:

```
input_classes
  required: yes
  type: list[enum-like string]
  meaning: класи людського вводу, які runtime / AI-layer має розрізняти.

routing_rules
  required: yes
  type: map[input_class -> action]
  meaning: що робити після класифікації вводу.

unmatched_input_policy
  required: recommended
  type: strict/convention enum
  meaning: що робити, якщо ввід не можна класифікувати безпечно.

clarification_policy
  required: recommended
  type: object / convention string
  meaning: як просити уточнення без зміни state.

resume_policy
  required: recommended when deviations allowed
  type: convention string
  meaning: як повернутися до основного процесу після відхилення.
```

Типові input_classes:

```
answer_current_node
  Користувач відповідає на поточне питання. Може змінювати state і routing.

clarification
  Користувач просить пояснення. State не повинен змінюватися без нової відповіді.

deviation
  Користувач тимчасово відходить від головного маршруту. Потрібен resume policy.

backtrack_request
  Користувач просить повернутися до попереднього кроку. Потрібна policy invalidation.

new_requirement
  Користувач додає нову вимогу. Не можна мовчки записувати її як confirmed state без review.
```

Типові unmatched_input_policy:

```
clarify_before_state_change
  Не змінювати state, поки ввід не класифіковано.

reject
  Відхилити ввід як несумісний із поточним node.

log_and_continue
  Зафіксувати ввід, але не змінювати routing.
```

Приклад:

```
conversation_semantics:
  input_classes:
    - answer_current_node
    - clarification
    - deviation
    - backtrack_request
  routing_rules:
    answer_current_node: evaluate_current_node_answer
    clarification: answer_without_state_change
    deviation: handle_deviation_then
```

```

backtrack_request: invoke_backtracking_policy
unmatched_input_policy: clarify_before_state_change
resume_policy: return_to_current_node_after_deviation

```

Типова помилка: трактувати будь-яку фразу користувача як відповідь на поточний node. Це швидко ламає state і пропускає clarification / deviation.

HYBRID_EXECUTION.MODEL

Призначення: описує гібридну модель, де AI веде процес, а CLI/helper runtime виконує deterministic checks.

Поля:

```

ai_layer
  type: convention string
  meaning: роль AI: guided_process_driver, reviewer, explainer.

cli_layer
  type: convention string
  meaning: роль CLI: deterministic_validator, compiler, artifact_checker.

deterministic_commands
  type: list[string]
  meaning: команди, які не має імітувати модель, а треба реально запускати.

human_review_points
  type: list[id/string]
  meaning: місця, де потрібне підтвердження людини.

```

Приклад:

```

op: HYBRID_EXECUTION.MODEL
id: hybrid_execution
ai_layer: guided_process_driver
cli_layer: deterministic_validator
deterministic_commands:
  - lint
  - compile
  - test
  - validate-output
human_review_points:
  - approve_contract
  - approve_output_template

```

Типова помилка: описати CLI checks у тексті, але дозволити моделі “сказати, що перевірка пройдена” без реального tool/runtime evidence.

F.4. Intent, Contract, Context

INTENT.DEF

Призначення: фіксує, чого користувач хоче досягти.

Поля:

```

id
  required: yes
  type: id string

goal
  required: yes
  type: free text

```



```

meaning: людський опис мети.

scope
  required: recommended
  type: free text / list
  meaning: що входить у задачу.

out_of_scope
  required: recommended
  type: free text / list
  meaning: що не входить у задачу.

success_criteria
  required: recommended
  type: list[string]
  meaning: критерії, за якими людина погодить результат.

```

Приклад:

```

op: INTENT.DEF
id: intent_create_playbook
goal: "Створити playbook для керованого intake процесу."
scope:
  - decision_tree
  - output_templates
out_of_scope:
  - production API integration
success_criteria:
  - "дерево має entry point i terminal paths"
  - "кожен terminal path має output decision"

```

Типова помилка: змішати goal і success_criteria. Goal пояснює напрям, success criteria дають перевірку.

CONTRACT.DEF / CONTRACT.INSTANCE

Призначення: перетворює очікуваний результат на перевірюваний контракт із полями та статусами.

Де живе: contracts у IR або ор-операція.

Поля:

```

kind
  required in schema object
  type: const
  value: contract

id
  required: yes
  type: id string

status
  required: yes
  type: strict enum
  values: missing, candidate, proposed, confirmed, blocked, not_applicable

fields
  required: yes
  type: map[field_name -> field_object]

```

Статуси contract / field:

```

missing
  Поле потрібне, але значення ще немає.

candidate

```

Є можливе значення, але воно ще слабке.

proposed

Значення запропоноване й чекає review.

confirmed

Значення підтверджене і може використовуватись у output / validation.

blocked

Поле не може бути завершене через проблему або зовнішню залежність.

not_applicable

Поле явно не застосовується в цьому сценарії.

Приклад:

```
contracts:
- kind: contract
  id: project_intent_contract
  status: proposed
  fields:
    project_goal:
      value: "Створити applied module для формування playbook-ів."
      status: confirmed
      required: true
    runtime_entry:
      status: missing
      required: true
      description: "Entry point ще не погоджено."
```

Як це читає validator: confirmed contract fields можуть ставати вимогами до artifacts. missing або blocked можуть створювати warnings або no-go залежно від coverage policy.

Типова помилка: підставити candidate поле в фінальний output як підтверджене.

CONTEXT.DEF

Призначення: описує, які джерела, обмеження й попередні рішення доступні процесу.

Поля:

```
id
  required: yes
  type: id string

sources
  required: recommended
  type: list[path/reference/free text]
  meaning: документи, файли, попередні рішення.

constraints
  required: recommended
  type: list[string]
  meaning: обмеження, які треба врахувати.

assumptions
  required: optional
  type: list[string]
  meaning: припущення; не дорівнюють confirmed facts.
```

Типова помилка: записати припущення в context так, ніби воно вже підтверджений contract field.

F.5. State і зміни стану

STATE.SCHEMA

Призначення: описує, які поля state існують, які з них required, які мають типи, default або status.

Поля:

```
id
  required: yes
  type: id string

fields
  required: yes
  type: map[field_name -> field_definition]

field.type
  required: recommended
  type: convention/JSON type
  examples: string, boolean, list, object, enum, path, reference

field.required
  required: recommended
  type: boolean

field.default
  required: optional
  type: any

field.allowed_values
  required: only for enum-like fields
  type: list[string]

field.description
  required: recommended
  type: free text
```

Приклад:

```
op: STATE.SCHEMA
id: intake_state_schema
fields:
  current_node:
    type: reference
    required: true
    description: "Поточний node процесу."
  user_goal:
    type: string
    required: true
  terminal_path_ready:
    type: boolean
    default: false
  output_template_status:
    type: enum
    allowed_values:
      - missing
      - draft
      - reviewed
      - approved
```

Типова помилка: використовувати поле в gates або templates, але не описати його в state schema.

STATE.SNAPSHOT

Призначення: фіксує стан на конкретному кроці виконання.

Поля:

```
run_id
  type: id string

step_id
  type: id/reference

state
  type: object
  meaning: повний або частковий state snapshot.

source
  type: enum/convention
  examples: runtime_enforced, model_self_report, hybrid
```

Приклад:

```
op: STATE.SNAPSHOT
id: snapshot_after_node_review
run_id: run_001
step_id: N_NODE_REVIEW
state:
  current_node: N_NODE_REVIEW
  node_review_status: approved
source: runtime_enforced
```

Типова помилка: використовувати snapshot як доказ виконання, хоча його source = model_self_report без runtime evidence.

STATE.DIFF

Призначення: показує, що саме змінилося між двома станами.

Поля:

```
before / after
  type: reference або object
  meaning: попередній і новий state.

changes
  type: list[change]
  meaning: список змінених полів.

change.field
  type: state selector

change.from / change.to
  type: any

reason
  type: free text / convention string
```

Приклад:

```
op: STATE.DIFF
id: diff_after_output_binding
changes:
  - field: terminal_output_binding_status
    from: missing
    to: confirmed
    reason: user_approved_output_template
```

Типова помилка: змінювати state після clarification, хоча clarification не мала підтвердити нове значення.

F.6. Entry, Node, Path і routing

ENTRY.DEF

Призначення: визначає, з якого node або subflow починається програма.

Поля:

```
id
  required: yes
  type: id string

start_node / entry_node
  required: yes
  type: reference
  meaning: id node, з якого починається execution.

initial_state
  required: optional
  type: object
  meaning: стартові значення state.
```

Приклад:

```
op: ENTRY.DEF
id: main_entry
start_node: N_FACTORY_MODE_SELECTION
initial_state:
  terminal_path_ready: false
```

Типова помилка: мати nodes, але не мати entry point.

NODE.DEF

Призначення: описує питання, дію або review point у процесі.

Зафіксовані schema-поля:

```
id
  required: yes
  type: id string

question
  required: yes
  type: free text
  meaning: питання або інструкція, яку бачить користувач / AI.

allowed_answers
  required: optional but recommended for deterministic nodes
  type: list[string]
  meaning: допустимі відповіді.

on_unmatched_input.action
  required: optional
  type: strict enum
  values: CLARIFY.REQUEST, escalate_to_human, block

on_unmatched_input.strategy
  required: optional
  type: strict enum
  values: rephrase_and_narrow, ask_for_exact_choice, explain_allowed_answers

on_unmatched_input.max_attempts
  required: optional
  type: integer
```

```
unmatched_policy
  required: optional
  type: strict enum
  values: handled, impossible_by_source
```

Значення `on_unmatched_input.action`:

```
CLARIFY.REQUEST
  Попросити уточнення без зміни state.

escalate_to_human
  Передати рішення людині або зупинити автоматичне routing.

block
  Заблокувати рух далі, поки ввід не стане сумісним із node.
```

Значення `on_unmatched_input.strategy`:

```
rephrase_and_narrow
  Переформулювати питання і звузити варіанти.

ask_for_exact_choice
  Попросити вибрати один із allowed_answers.

explain_allowed_answers
  Пояснити, що означає кожен дозволений варіант.
```

Приклад:

```
nodes:
- id: N_FACTORY_MODE_SELECTION
  question: "Яким способом створюємо процес?"
  allowed_answers:
    - domain_model_and_decision_tree
    - manual_decision_tree
    - free_dialogue
    - improve_existing_process
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: ask_for_exact_choice
    max_attempts: 2
    unmatched_policy: handled
```

Типова помилка: залишити `allowed_answers` порожнім там, де наступний крок має бути **deterministic**.

PATH.DEF

Призначення: описує послідовність `node/steps` або `terminal path`.

Поля:

```
id
  required: yes
  type: id string

steps
  required: yes
  type: list[reference]
  meaning: node або step ids у порядку проходження.

terminal
  required: optional
  type: boolean
  meaning: чи завершується шлях.

expected_output
```

```
required: recommended for terminal path
type: reference/list[reference]
meaning: artifact або output binding.
```

Типова помилка: назвати шлях terminal, але не визначити expected output або явне рішення no output required.

CLARIFY.REQUEST

Призначення: контрольоване уточнення, коли ввід не можна безпечно класифікувати.

Поля:

```
id
  type: id string

question
  type: free text
  meaning: уточнення до користувача.

allowed_answers
  type: list[string]
  meaning: якщо уточнення має обмежені варіанти.

state_change_allowed
  type: boolean
  recommended value: false
```

Приклад:

```
op: CLARIFY.REQUEST
id: clarify_factory_mode
question: "Виберіть один із чотирьох режимів створення процесу."
allowed_answers:
  - domain_model_and_decision_tree
  - manual_decision_tree
  - free_dialogue
  - improve_existing_process
state_change_allowed: false
```

Типова помилка: під час clarification змінити state так, ніби користувач уже дав відповідь на node.

F.7. Gates і assertions

GATE.DEF

Призначення: контрольна точка, яка вирішує, чи можна рухатися далі.

Зафіксовані schema-поля:

```
id
  required: yes
  type: id string

method
  required: yes
  type: strict enum
  values: mechanical, self_verification, self_consistency, human

trust_class
  required: yes
  type: strict enum
```

```

    values: deterministic, model_judgment, repeated_model_judgment, human_decision

assert
  required: yes
  type: expression/free text condition
  meaning: що саме перевіряється.

source
  required: optional
  type: reference/path/free text

params
  required: optional
  type: object

protocol
  required: optional
  type: list[string]

on_fail / on_pass
  required: optional
  type: reference/action convention

```

Значення method:

```

mechanical
  Перевірка виконується deterministic способом: schema, script, file check, compiler.

self_verification
  Модель сама перевіряє відповідь. Слабший рівень довіри.

self_consistency
  Модель або кілька проходів порівнюються між собою.

human
  Рішення приймає людина.

```

Значення trust_class:

```

deterministic
  Можна відтворити через tool/runtime/compiler.

model_judgment
  Це оцінка моделі, не абсолютний доказ.

repeated_model_judgment
  Посилене model judgment через повторення / порівняння.

human_decision
  Підтверджено людиною.

```

Приклад:

```

gates:
- id: G_TEMPLATE_REVIEWED
  method: human
  trust_class: human_decision
  assert: "Користувач переглянув приклад заповненого шаблону."
  on_fail: N_TEMPLATE_REVIEW
  on_pass: N_TERMINAL_READY_CHECK

```

Типова помилка: поставити trust_class: deterministic для перевірки, яку фактично зробила модель словами.

GATE.CHECK

Призначення: результат виконання gate.

Поля:

```
gate
  required: yes
  type: reference
  meaning: id gate, який перевіряли.

result
  required: yes
  type: enum/convention
  common values: pass, fail, blocked, warning

evidence
  required: recommended
  type: path/reference/free text
  meaning: чим підтверджено результат.
```

Типова помилка: мати gate definition, але не мати gate result у trace/report.

ASSERTION.DEF

Призначення: стабільна перевірка умови, яка може проєктуватися в runtime/test/debug.

Schema-поля:

```
id
  required: yes
  type: id string

polarity
  required: yes
  type: strict enum
  values: must, not

condition
  required: yes
  type: expression/free text condition

phase
  required: yes
  type: list[enum]
  values: runtime, test, debug

severity
  required: yes
  type: strict enum
  values: block, warn, info

on_fail
  required: optional
  type: action/reference

message
  required: optional
  type: free text
```

Значення polarity:

```
must
  Умова має виконуватися.

not
  Умова не повинна відбуватися. Це канонічна форма негативної перевірки.
```

Значення severity:

```
block
  Порушення блокує подальше виконання або release.
```

```
warn
  Порушення не блокує, але має бути видиме.

info
  Інформаційне повідомлення.
```

Приклад:

```
assertions:
- id: A_NO_TERMINAL_WITHOUT_OUTPUT_DECISION
  polarity: not
  condition: "terminal_path_ready == true and selected_terminal_outputs is missing"
  phase:
    - runtime
    - test
  severity: block
  message: "Terminal path cannot be ready without output decision."
```

Типова помилка: писати негативну перевірку тільки в тексті, але не робити її assertion.

F.8. Output, artifact і template layer

OUTPUT.DEF

Призначення: описує, який результат може створити процес.

Поля:

```
id
  required: yes
  type: id string

type
  required: recommended
  type: convention/enum-like string
  examples: document, report, yaml_package, runtime_package, no_output_required

artifact
  required: recommended when output creates file
  type: reference

template
  required: recommended when rendered
  type: reference/path

status
  required: recommended
  type: convention enum
  examples: missing, draft, reviewed, approved, not_applicable
```

Приклад:

```
outputs:
- id: O_RUNTIME_PACKAGE
  type: runtime_package
  artifact: A_RUNTIME_ZIP
  template: T_RUNTIME_PACKAGE_README
  status: approved
```

Типова помилка: terminal path вказує на output, але output не має artifact або template review status.

ARTIFACT.DEF / Artifact

Schema-поля:

```
kind
  required: yes
  type: const
  value: artifact

id
  required: yes
  type: id string

path_pattern
  required: yes
  type: path pattern

format
  required: yes
  type: strict enum
  values: markdown, json, yaml, text, other

required
  required: optional
  type: boolean

description
  required: optional
  type: free text
```

Приклад:

```
artifacts:
- kind: artifact
  id: A_VALIDATION_REPORT
  path_pattern: "reports/{run_id}/VALIDATION_REPORT.json"
  format: json
  required: true
  description: "Machine-readable validation result."
```

Типова помилка: назвати artifact required, але не мати coverage rule або rendered artifact assertion.

ARTIFACT.REQUIREMENT

Призначення: зв'яже confirmed contract fields із required artifacts.

Schema-поля:

```
kind
  required: yes
  value: artifact_requirement

id
  required: yes
  type: id string

when.contract
  required: yes
  type: contract reference

when.status
  required: yes
  type: strict enum
  values: missing, candidate, proposed, confirmed, blocked, not_applicable

requires
```

```

    required: yes
    type: list[requirement]

requires[].artifact
  required: yes
  type: artifact reference

requires[].must_include_fields
  required: optional
  type: list[state/contract selectors]

requires[].must_include_sections
  required: optional
  type: list[string]

requires[].forbidden_sections
  required: optional
  type: list[string]

```

Приклад:

```

artifact_requirements:
- kind: artifact_requirement
  id: R_CONTRACT_TO_REPORT
  when:
    contract: project_intent_contract
    status: confirmed
  requires:
  - artifact: A_VALIDATION_REPORT
    must_include_fields:
      - project_intent_contract.fields.project_goal
    must_include_sections:
      - validation_summary

```

Типова помилка: confirmed contract fields існують, але жоден artifact не зобов'язаний їх містити.

[RenderingTemplate / OUTPUT.TEMPLATE](#)

Schema-поля:

```

id
  required: yes
  type: id string

type
  required: optional
  type: convention string

format
  required: optional
  type: strict enum
  values: markdown, json, yaml, text

path
  required: optional
  type: path

template
  required: yes
  type: path or inline template string

render_mode
  required: yes
  type: strict enum
  values: deterministic, model_assisted

renderer

```

```

    required: yes
    type: strict enum
    values: ordo.simple, ai.markdown, ai.yaml, ai.json

requires_model_rendering
  required: optional
  type: boolean

validation
  required: optional
  type: strict enum
  value: strict_confirmed_state_only

tbd_policy
  required: optional
  type: strict enum
  value: preserve_tbd_until_confirmed

explicit_tbd_defaults
  required: optional
  type: list[string]

forbidden_inference_rules
  required: optional
  type: list[string]

forbidden_unconfirmed_terms
  required: optional
  type: list[string]

```

Значення render_mode:

```

deterministic
  Шаблон рендериться без творчої інтерпретації моделі.

model_assisted
  Модель може допомагати з формулюванням, але має отримати handoff packet i validation
  constraints.

```

Значення renderer:

```

ordo.simple
  Простий deterministic renderer для контрольованих підстановок.

ai.markdown
  Модель допомагає створювати markdown artifact.

ai.yaml
  Модель допомагає створювати YAML artifact.

ai.json
  Модель допомагає створювати JSON artifact.

```

Приклад:

```

rendering_templates:
- id: T_SUMMARY
  format: markdown
  path: output_templates/summary.md
  template: output_templates/summary.md
  render_mode: deterministic
  renderer: ordo.simple
  validation: strict_confirmed_state_only
  tbd_policy: preserve_tbd_until_confirmed

```

Типова помилка: використовувати model_assisted, але не заборонити моделі вигадувати missing fields.

TEMPLATE.MOCK_RENDER

Статус: validation/review pattern, не обов'язковий core opcode у поточній лінії.

Призначення: показати користувачу приклад заповненого шаблону до release/handoff.

Практичні поля:

```
template
  type: template reference

mock_state
  type: object/path

output_path
  type: path

review_status
  type: convention enum
  suggested values: generated, reviewed, approved, rejected

reviewer
  type: human/model reference
```

Типова помилка: вважати шаблон погодженим, якщо користувач бачив тільки порожню структуру, але не бачив mock-rendered example.

F.9. Validation, coverage і go/no-go

COVERAGE.RULE

Schema-поля:

```
kind
  required: yes
  value: coverage_rule

id
  required: yes
  type: id string

input
  required: yes
  type: strict enum
  values: confirmed_contracts, all_contracts, rendered_artifacts

output
  required: yes
  type: strict enum
  values: artifact_requirements, consistency_report, go_no_go

failure_policy
  required: yes
  type: strict enum
  values: warn, fail, no_go

description
  required: optional
  type: free text
```

Значення failure_policy:

```
warn
  Показати попередження, але не блокувати.
```

```
fail
  Вважати validation failed.

no_go
  Перетворити проблему на no-go decision.
```

Приклад:

```
coverage_rules:
- kind: coverage_rule
  id: C_CONFIRMED_CONTRACTS_TO_ARTIFACTS
  input: confirmed_contracts
  output: artifact_requirements
  failure_policy: no_go
```

RENDERED_ARTIFACT.ASSERT

Schema-поля:

```
kind
  required: yes
  value: rendered_artifact_assertion

id
  required: yes
  type: id string

field
  required: yes
  type: contract/state selector

must_appear_in
  required: yes
  type: list[artifact reference/path]

severity
  required: optional
  type: strict enum
  values: block, warn, info

message
  required: optional
  type: free text
```

Приклад:

```
rendered_artifact_assertions:
- kind: rendered_artifact_assertion
  id: A_PROJECT_GOAL_VISIBLE
  field: project_intent_contract.fields.project_goal
  must_appear_in:
    - A_SUMMARY_REPORT
  severity: block
```

GO_NO_GO.DECISION

Schema-поля:

```
kind
  required: yes
  value: go_no_go

status
  required: yes
  type: strict enum
  values:
    go
    no_go_requires_confirmation
```

```

    no_go_requires_artifact_fix
    no_go_requires_template_fix
    no_go_requires_runner_contract

blocking_issues
  required: yes
  type: list[issue]

warnings
  required: yes
  type: list[warning]

```

Значення status:

```

go
  Можна передавати далі.

no_go_requires_confirmation
  Потрібне людське підтвердження.

no_go_requires_artifact_fix
  Треба виправити artifacts.

no_go_requires_template_fix
  Треба виправити templates.

no_go_requires_runner_contract
  Треба виправити runtime/runner contract.

```

Приклад:

```

go_no_go:
  kind: go_no_go
  status: go
  blocking_issues: []
  warnings:
    - code: consistency_contract_defaults
      message: "Default-value coverage warning remains visible."

```

Типова помилка: приховати warnings, якщо статус go. У Ordo warning не блокує, але має залишатися видимим.

F.10. Trace, debug і replay

TRACE.LOG / Trace

Schema-поля:

```

run_id
  required: yes
  type: id string

execution_mode
  required: yes
  type: strict enum
  values: full_runtime, chat_internal, freeform_only

trace_source
  required: yes
  type: strict enum
  values: model_self_report, runtime_enforced, hybrid

selected_path
  required: optional
  type: object

```



```

rejected_paths
  required: optional
  type: array

decision_log
  required: optional
  type: array

state_snapshots
  required: optional
  type: array

state_diffs
  required: optional
  type: array

gate_report
  required: optional
  type: array

warnings / violations
  required: optional
  type: array

```

Значення `trace_source`:

```

model_self_report
  Модель описала, що вона зробила. Це корисно, але слабше за runtime evidence.

runtime_enforced
  Trace створений або перевірений runtime/CLI.

hybrid
  Комбінація AI-led execution і deterministic helper evidence.

```

Приклад:

```

run_id: run_001
execution_mode: full_runtime
trace_source: hybrid
selected_path:
  terminal_node: N_FINAL_HANDOFF
state_diffs:
  - field: go_no_go.status
    from: no_go_requires_template_fix
    to: go
warnings: []
violations: []

```

Типова помилка: приймати `model_self_report` як повний proof program без deterministic evidence.

DECISION.LOG

Призначення: записує, чому було прийнято конкретне рішення.

Поля:

```

decision_id
  type: id string

node
  type: node reference

selected_option
  type: enum/reference/string

```

```

rejected_options
  type: list[string]

reason
  type: free text

evidence
  type: path/reference/free text

```

PATH.EXPLAIN

Призначення: пояснює, чому обрано один path, а не інші.

Поля:

```

current_node
  type: reference

selected_path
  type: reference

rejected_paths
  type: list[reference]

gates_used
  type: list[gate reference]

human_readable_explanation
  type: free text

```

F.11. Tests, fixtures і regression**TEST.DEF**

Призначення: описує один тестовий сценарій.

Практичні поля:

```

id
  required: yes
  type: id string

title
  required: recommended
  type: free text

fixture
  required: recommended
  type: fixture reference

steps
  required: recommended
  type: list[step]

expected_path / expected_state / expected_output / expected_gate
  required: recommended
  type: expectation object/reference

```

Приклад:

```

op: TEST.DEF
id: T_FREE_DIALOGUE_TO_DRAFT_TREE
title: "Free dialogue produces draft tree before YAML generation"
fixture: F_FREE_DIALOGUE_INPUT
expected_path: P_FREE_DIALOGUE_REVIEW

```

```
expected_state:
  draft_subtree_status: proposed
```

FIXTURE.DEF

Призначення: задає вхідні дані для тесту.

Поля:

```
id
  type: id string

input
  type: object / text / path

initial_state
  type: object

expected_contracts
  type: object/list
```

EXPECT.*

Призначення: очікуваний path, state, output або gate result.

Практичні поля:

```
EXPECT.PATH
  selected_path, terminal_node, forbidden_nodes

EXPECT.STATE
  field, expected_value, status

EXPECT.OUTPUT
  artifact, must_exist, must_include, must_not_include

EXPECT.GATE
  gate, expected_result, severity
```

Типова помилка: тестувати лише final output і не тестувати path/state, через що runtime може пройти неправильним маршрутом, але все одно створити схожий документ.

F.12. Libraries, freeform і improvement records

LIB.INCLUDE

Schema-поля:

```
library
  required: yes
  type: library id/path

version
  required: yes
  type: semver requirement

as
  required: optional
  type: namespace alias

exports
  required: optional
  type: list[string]
```

Приклад:

```
libraries:
- library: standard_review_patterns
  version: ">=0.1.0"
  as: review
  exports:
    - NODE.REVIEW
    - BRANCH.REVIEW
```

Типова помилка: імпортувати library без namespace і отримати конфлікт локальних ids.

FREEFORM.DEF**Schema-поля:**

```
id
  required: yes
  type: id string

role
  required: yes
  type: strict enum
  values: domain_explanation, example, warning, template_note, edge_case_note

binding
  required: optional
  type: reference/string

maturity
  required: optional
  type: strict enum
  values: stable, volatile, candidate_for_formalization

incident_count
  required: optional
  type: integer >= 0

incident_threshold
  required: optional
  type: integer >= 1
```

Значення maturity:

```
stable
  Freeform блок прийнятий як пояснювальний текст.

volatile
  Блок часто змінюється або містить нестабільну логіку.

candidate_for_formalization
  Блок треба винести в формальну конструкцію, якщо він впливає на execution.
```

Типова помилка: ховати gate або routing rule у freeform-тексті.

IMPROVEMENT.RECORD**Schema-поля:**

```
id
  required: yes
  type: id string

type
  required: yes
  type: convention string
  examples: missed_required_gate, node_coverage_gap, freeform_overuse
```

```

severity
  required: yes
  type: strict enum
  values: low, medium, high, critical

affected_unit.id
  required: yes
  type: reference

affected_unit.kind
  required: yes
  type: strict enum
  values: gate, assertion, node, freeform, library, domain_pack, profile, compiler_rule

root_cause_hypothesis
  required: optional
  type: array

proposed_patch
  required: optional
  type: array

suggested_tests
  required: optional
  type: array

approval
  required: yes
  type: object

```

Типова помилка: записати `improvement` як коментар у тексті, але не прив'язати його до `affected unit` і `approval status`.

F.13. APF-патерни, які ще не треба робити `core opcodes`

У M63 APF став `standard applied module`, але його патерни не треба автоматично піднімати в `core runtime` / IR. Їх треба читати так:

```

APF-local pattern
  працює в межах APF або standard applied module docs.

reusable applied-module pattern
  може повторюватися в інших пакетах, але ще не є core opcode.

future IR candidate
  потребує окремого design milestone, schema і validator semantics.

```

INPUT.POLICY

Класифікація: `schema pattern` або `APF subflow`.

Поля, які варто описувати:

```

required_input_artifacts
optional_input_artifacts
input_unknown_or_deferred
missing_input_policy

```

Статус: `usable in APF`; `candidate for future generic schema support`.

TERMINAL.OUTPUT.BIND

Класифікація: `APF subflow` / `possible future IR concept`.

Практичні поля:

```
terminal_node
selected_outputs
output_template_status
mock_filled_example_status
required_state_fields
user_review_status
terminal_path_ready
```

Статус: keep APF-local for rc.1.

Типова помилка: вважати terminal path готовим тільки тому, що дерево дійшло до кінця. У APF terminal path готовий тільки після output/template decision.

TREE.AUTHOR.PROGRESSIVE

Класифікація: APF workflow pattern.

Практичні поля:

```
user_tree_vision_depth
draft_subtree_status
draft_nodes
draft_branches
terminal_candidates
open_questions
```

Статус: reusable applied-module pattern, not core opcode yet.

NODE.REVIEW, BRANCH.REVIEW, SUBTREE.REVIEW

Класифікація: applied-module review patterns.

Практичні поля:

```
review_target
review_status
reviewer
approved_fields
open_questions
required_changes
next_step_decision
```

Типові review_status:

```
draft
proposed
approved
needs_changes
deferred
```

Статус: document and test in APF; future generic review layer candidate.

TREE.NORMALIZE

Класифікація: adapter pattern.

Призначення: перетворити ручний або вільний опис дерева в нормалізовану структуру nodes/branches/gates.

Статус: APF-local; useful for future import utilities.

DIALOGUE . EXTRACT

Класифікація: AI-assisted extraction pattern.

Призначення: витягнути з вільного діалогу candidate nodes, branches, outputs, assumptions і open questions.

Статус: APF-local; do not make deterministic opcode yet.

TEMPLATE . RECIPE

Класифікація: template authoring pattern.

Призначення: описати, як створюється output template: секції, required fields, TBD policy, forbidden inference.

Статус: strong candidate for standard template-layer docs.

VALIDATION . HANDOFF . TAIL

Класифікація: reusable applied-module tail.

Призначення: спільний фінальний шлях: minimal validation, full validation, review, fix loop, go/no-go, package handoff.

Статус: document as standard pattern; do not hardcode into core.

FLOW . JOIN І SHARED . TAIL . REFERENCE

Класифікація: future IR candidates.

Чому не core зараз:

```
current YAML can express joins by repeated target ids,
but reusable shared-tail references need stable semantics:
- як перевіряти входи в shared tail;
- як зберігати trace;
- як рахувати coverage;
- як пояснювати path;
- як уникати циклів і прихованих переходів.
```

Статус: backlog.

F.14. Практичне правило для автора YAML

Перед тим як додати нове поле в YAML, поставте три питання:

1. Це поле читає compiler/runtime/validator?
Якщо так – потрібен тип, allowed values і schema/policy.
2. Це поле є домовленістю пакета?
Якщо так – потрібне пояснення в package docs.
3. Це лише пояснення для людини?
Якщо так – не використовуйте його як deterministic condition.

Коротка формула:

```
enum = контракт;  
convention string = домовленість, яку треба описати;  
free text = пояснення, а не runtime-доказ.
```

Саме це розрізнення робить Ordo-процес не просто красивим YAML, а практично виконуваною мовою керованих процесів.

`execution_trace`

Top-level core block для повної історії виконання. Основні атрибути: `id`, `version`, `run`, `status`, `started_at`, `finished_at`, `actor`, `source`, `capture_level`, `events`, `final_state`, `outputs`, `replay`, `integrity`. Нові програми використовують `execution_trace`:, а старий `trace`: вважається legacy compatibility view.